GTK 4 for Tool Builders

Building desktop tools with C++17 and GTK 4, one runnable step at a time

Scott Peterman

First edition, October 2026

Contents

Preface **1**
Why GTK 4, and why this way . . . 1
How this book was made . . . 1
What you need . . . 2
Licences . . . 2

How to use this book **3**
One application, built in steps . . . 3
Two tracks in every chapter . . . 3
The parts of a chapter . . . 4
Four ideas that keep coming back . . . 4
Conventions . . . 4

Chapter 0 — Toolchain and the debugger **5**
Concepts . . . 5
How the pieces fit . . . 5
Debug information: yours and GTK's . . . 6
The version floor . . . 6
What each launch configuration is for . . . 7
Build . . . 7
Packages . . . 7
Meson in one page . . . 8
The first meson.build . . . 8
VS Code files . . . 8
Stepping into GTK . . . 10
Review . . . 10
Exercises . . . 11

Part I: Foundations **12**

Chapter 1 — A window that runs **14**
Concepts . . . 14
The application owns the process . . . 14
What keeps the loop running . . . 16
One process, or one per launch . . . 16
Asking a window to close . . . 17
Every callback has the same shape . . . 17
Coming from GTK 3 . . . 18
Build . . . 18
Done when . . . 19
Common mistakes . . . 19
Review . . . 20
Exercises . . . 20

Chapter 2 — GObject: references, signals, ownership **21**
Concepts . . . 21
An object system written in C . . . 21
Reference counting, and the floating reference . . . 22
Destruction in two steps . . . 22
Signals . . . 23
Data attached to objects . . . 24
Build . . . 24
Done when . . . 26
Common mistakes . . . 27
Review . . . 27
Exercises . . . 27

Chapter 3 — Layout and containers **29**
Concepts . . . 29
A tree of containers, each with one rule . . . 29
Measure, then allocate . . . 30
Expand, align, margin, minimum . . . 31
Where layouts go wrong . . . 32
Layout is not ownership . . . 32
Build . . . 32
Done when . . . 34
Common mistakes . . . 35
Review . . . 35
Exercises . . . 36
Isolated lab . . . 36

Checkpoint A — A shell that remembers **39**

Chapter 4 — Keep GTK out of the logic **40**
Concepts . . . 40
Two kinds of code . . . 40
Dependencies point one way . . . 41
The callback is an adapter . . . 41
What makes core code testable . . . 42
How the rest of the book uses the core . . . 42
Build . . . 42
The session model . . . 42
Settings . . . 45
Tests . . . 45
meson.build . . . 46
The window uses the core . . . 47
Done when . . . 47
Common mistakes . . . 48
Review . . . 48
Exercises . . . 48
Isolated lab . . . 49

Part II: Building Tools **53**

Chapter 5 — List views I: models and factories **55**
Concepts . . . 55
Why a list needs a model . . . 55
Four objects, one job each . . . 55
Recycling: a row is not an item . . . 56
GtkListBox or GtkListView . . . 57
References along the chain . . . 57
Defining a GObject type . . . 57
Build . . . 58
The item type . . . 58
AppWindow . . . 59
The model chain . . . 61
The factory . . . 62
Done when . . . 63
Common mistakes . . . 64
Review . . . 64
Exercises . . . 65
Isolated lab . . . 65

Chapter 6 — List views II: trees, expansion, filtering **69**
Concepts . . . 69
A tree is a list that grows . . . 69
The row, the item and the expander . . . 70
When the child function runs . . . 71
Filtering a tree . . . 71
Keeping the user's place . . . 72
Build . . . 72
Rebuilding without losing the user's place . . . 74
Done when . . . 75
Common mistakes . . . 76
Review . . . 76
Exercises . . . 77
Isolated lab . . . 77

Chapter 7 — Actions, menus and shortcuts **81**
Concepts . . . 81
Commands, not callbacks . . . 81
Where an action lives . . . 82
Parameter, state and what the menu draws . . . 83
Menus are models . . . 83
Where shortcuts are caught . . . 83
Saving without thrashing . . . 84
Build . . . 84
One table of shortcuts . . . 84
Installing the actions . . . 85
Stateful actions . . . 86
The menu . . . 86
A save that waits for the burst to end . . . 87
Done when . . . 87
Common mistakes . . . 88
Review . . . 89

Exercises . . . 89
Isolated lab . . . 90

Checkpoint B — Session browser **93**

Chapter 8 — Tabs that own things **94**
Concepts . . . 94
A tab is a workspace . . . 94
Three scopes . . . 94
The page owns the tab . . . 95
Commands find the tab when they run . . . 96
Closing is a lifecycle transition . . . 96
Build . . . 97
Done when . . . 99
Common mistakes . . . 100
Review . . . 100
Exercises . . . 101
Isolated lab . . . 101

Chapter 9 — Event controllers and propagation **104**
Concepts . . . 104
No event handlers, only controllers . . . 104
Where an event goes . . . 105
Stopping an event . . . 106
Choosing a phase . . . 106
What the handler is given . . . 107
Build . . . 107
Done when . . . 108
Common mistakes . . . 109
Review . . . 110
Exercises . . . 110

Chapter 10 — Popovers and dialogs **111**
Concepts . . . 111
Three ways to ask . . . 111
Popovers are children . . . 112
Coordinates belong to a widget . . . 112
The menu names commands; it does not own them . . . 112
Dialogs do not block . . . 113
Tracing one click through the layers . . . 113
Build . . . 115
One helper for every popover menu . . . 115
The two menus . . . 116
A question that does not block . . . 117
Importing a file . . . 118
Done when . . . 118
Common mistakes . . . 119
Review . . . 120
Exercises . . . 120
Isolated lab . . . 121

Checkpoint C — Tabs and menus **124**

Part III: The Terminal **125**

Chapter 11 — The main loop: file descriptors, idles, timers, monitors **127**
Concepts . . . 127
One thread, many sources . . . 127
One iteration . . . 128
Priorities and the redraw . . . 129
A source points at you; you own the source . . . 130
A pseudo-terminal . . . 130
Build . . . 131
PtySession . . . 131
A tab that runs a session . . . 134
Focus, on idle . . . 134
The sessions file reloads itself . . . 135
Done when . . . 135
Common mistakes . . . 136
Review . . . 137
Exercises . . . 137

Chapter 12 — Writing a widget **139**
Concepts . . . 139
A widget answers three questions . . . 139
Size is negotiated, top down . . . 140
Queue, don't draw . . . 140
Text on a grid . . . 141
Properties and signals: the widget's outward face . . . 141
Where the C++ lives . . . 142
Build . . . 142
The instance and the class . . . 143
The cell size . . . 144
Size . . . 145
Drawing . . . 146
Bytes in . . . 147
In the app . . . 148
Done when . . . 148
Common mistakes . . . 149
Review . . . 149
Exercises . . . 150

Chapter 13 — Scrollback, keys and selection in your widget **151**
Concepts . . . 151
Two ways to scroll . . . 151
The adjustment is a contract . . . 151
Keys become bytes . . . 152
Who sees a key first . . . 153
Selection and the clipboard . . . 154
Build . . . 154
Scrollback and the adjustments . . . 154
Keys . . . 155
Selection and the clipboard . . . 156
In the app . . . 157
Done when . . . 158
Common mistakes . . . 159

Review . . . 159
Exercises . . . 160

Chapter 14 — Swapping in gtkpyte **161**
Concepts . . . 161
The same shape, a real engine . . . 161
What an emulator does that the grid did not . . . 162
Two screens . . . 163
Who gets the mouse . . . 163
Vendoring a widget . . . 164
Build . . . 164
Vendoring gtkpyte . . . 164
meson.build . . . 164
The wiring . . . 165
Zoom . . . 166
Session logs . . . 167
Done when . . . 168
Common mistakes . . . 169
Review . . . 170
Exercises . . . 170

Checkpoint D — A usable terminal **171**

Part IV: Production **172**

Chapter 15 — Resources, CSS, themes and icons **174**
Concepts . . . 174
A bundle is a file system inside the binary . . . 174
Three readers . . . 175
The CSS cascade: priority first, then specificity . . . 175
Classes, named colours, and colours in data . . . 176
Symbolic icons . . . 177
How GTK finds an icon by name . . . 177
Where this sits in the book . . . 177
Build . . . 178
The bundle . . . 178
Themes: parsed in the core . . . 178
Themes: read from the bundle, applied to terminals . . . 179
Style and icons . . . 180
The theme menu . . . 181
Done when . . . 182
Common mistakes . . . 183
Review . . . 183
Exercises . . . 183

Chapter 16 — Lifetime, proven **185**
Concepts . . . 185
Two kinds of lifetime bug . . . 185
Dispose, then finalize . . . 186
What a window's teardown promises . . . 186
What ASan does, in enough detail to trust it . . . 187
What fatal-criticals does . . . 188

A checklist for any object that hands out `this` . . . 188
Build . . . 189
An ASan build . . . 189
Who deletes the AppWindow . . . 189
Done when . . . 191
Common mistakes . . . 192
Review . . . 192
Exercises . . . 193

Chapter 17 — Holding the floor, and shipping **194**
Concepts . . . 194
The floor is a machine, not a macro . . . 194
Two ways to stand on the floor . . . 195
Build types . . . 195
The application id, the window and the desktop entry . . . 196
Build . . . 197
On Debian 12 . . . 197
GTK 4.8 in a prefix . . . 197
Install rules . . . 198
Done when . . . 198
Common mistakes . . . 199
Review . . . 200
Exercises . . . 200

Capstone — A second tool **201**

Appendices **202**

Appendix A — GTK 3 to GTK 4 **203**

Appendix B — Debugger recipes **205**

Appendix C — Recall answer key **207**

Appendix D — GObject in one page **208**

Appendix E — Source map **209**

Appendix F — Review answers **210**

Preface

This is a book about building desktop tools with GTK 4: the small, sharp programs that engineers write for themselves and their teams. A session manager, a log viewer, a terminal, a dashboard that sits on a second monitor. It is written for people who already program for a living and want to add a native toolkit to what they can build, not for people learning to program.

The book follows one application from an empty window to a finished, packaged tool. That application is **gtklab**, a small session-manager terminal: a tree of saved sessions on the left, tabs of live terminals on the right, menus, shortcuts, themes and logging. gtklab is a teaching copy of **tmssh**, a GTK 4 terminal that is in daily use, and each chapter closes by pointing at the place in tmssh where the same problem was solved for real.

Why GTK 4, and why this way

GTK 4 is the toolkit underneath most of the Linux desktop, and it is a good fit for tools. It starts fast, it is small, it renders with the GPU, and its C API can be used from any language that can call C. It is also a large and old codebase with its own object system, GObject, and a set of conventions that are rarely written down in one place. Most GTK material online is for GTK 3, and much of it no longer applies.

The difficulty for an experienced developer is rarely the widgets. It is the machinery around them: who owns which object, when a callback can run, why a handler fires on an object that is already being destroyed, why the window stops repainting while a terminal prints. Those are the subjects this book spends its time on. They come back again and again, chapter after chapter, because that is how they are learned.

How this book was made

Every example in this book is real code from the companion repository, and every chapter's code builds and runs. The code blocks are not copied into the manuscript by hand: the build pulls them from the repository's chapter directories, so the book cannot show code that the repository does not contain.

Each chapter's code was built from scratch with no compiler warnings on Ubuntu 24.04 with GTK 4.14.5, its unit tests run, and the running program was driven by scripts that click, type and take the screenshots you will see. Every "Break it" exercise was carried out and its result recorded. The finished application was also built and run against GTK 4.8.3, the oldest version the book supports, so that it works on Debian 12 and on ChromeOS Linux.

Where testing found that a common piece of GTK advice was wrong, or that the first version of a chapter had a bug, the book says so. Several of the most useful lessons in it came from exactly that.

What you need

- A Linux machine with GTK 4.8 or later, or a Mac with GTK 4 from Homebrew. ChromeOS Linux (Crostini) works.
- C++17, and enough C to read a C API. You do not need to know GObject; the book teaches it as it is needed.
- VS Code, which the book uses for building and debugging. Any editor and debugger will do, but the launch configurations are written for VS Code.
- The companion repository: https://github.com/scottpeterman/gtk4_for_tool_builders.

Licences

The text of this book is licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0). The code in the companion repository is licensed under the GNU General Public License, version 3 or later. From Chapter 14, gtklab includes gtkpyte, which is GPL-3.0-or-later; `tests/doctest.h` is MIT.

How to use this book

One application, built in steps

You build gtklab in seventeen chapters. Each chapter adds one idea, and its code compiles and runs when you finish it. Four checkpoints close the parts of the book; they give you a specification and a definition of done, and no step-by-step instructions, so you can find out what you have actually learned.

The companion repository has one directory per chapter, each holding the finished state of that chapter:

Directory	Chapter
`ch01-window` ... `ch03-layout`	Chapters 1 to 3
`ckA-remember`	Checkpoint A
`ch04-core` ... `ch07-actions`	Chapters 4 to 7
`ckB-browser`	Checkpoint B
`ch08-tabs` ... `ch10-popovers`	Chapters 8 to 10
`ckC-tabs`	Checkpoint C
`ch11-mainloop` ... `ch14-gtkpyte`	Chapters 11 to 14 (Chapter 14 is also Checkpoint D)
`ch15-resources` ... `ch17-shipping`	Chapters 15 to 17

Work in your own repository and use these directories to check your work. When you are stuck, compare your source with the reference for the chapter you are on; when you are done, compare it with the next one to see the size of the step ahead.

```
diff -ru ch06-tree/src ch07-actions/src
```

Two tracks in every chapter

Build. You write the chapter's increment in your own copy of gtklab. It compiles and runs at the end of the chapter.

Read. You then read how tmssh solved the same problem, by function name. In VS Code, Ctrl+T (Cmd+T on the Mac) finds a function by name. You read after you build, so that you are comparing, not copying.

The parts of a chapter

Every chapter has the same parts, in the same order:

1. **Recall.** Two or three questions on earlier chapters, answered before you read on. The answers are in the appendix.
2. **The idea.** One concept, explained far enough to use it.
3. **Build.** The increment, with the code.
4. **Done when.** What the running program must do, and a screenshot of it doing it.
5. **Debugger drill.** A breakpoint exercise that makes the idea visible.
6. **Break it.** A deliberate bug, and exactly what GTK does about it: a critical warning, a crash, or nothing at all.
7. **Read in tmssh.** Where the production version lives.

Most chapters also have a short **Common mistakes** section, collected from the mistakes made while the book's code was written and tested.

Four ideas that keep coming back

Four ideas cause most real GTK bugs, so the book does not teach them once and move on. Each is introduced early and comes back in later chapters, and the Recall questions are drawn from them, spaced so that each returns after a short gap, then a longer one.

Strand	Introduced	Comes back in	Tested in
Lifetime and ownership	Chapter 2	5, 8, 10, 12	Chapter 16
The main loop	Chapter 1	7, 10, 11	Chapter 14
Model and widget	Chapter 4	5, 6, 12	Chapter 15
Input propagation	Chapter 9	10, 13	Chapter 14

Conventions

Code blocks in this book are taken from the repository when the book is built. A small grey line above each block names the file it came from. Shell commands are written one per line. Shortcuts are written as they are pressed on Linux; on the Mac, gtklab uses the same Ctrl+Shift shortcuts.

Function names in the text refer to the code, not to line numbers, so that they survive edits. The debug output quoted in the book comes from `g_debug` lines in the code, which print when the program runs with `G_MESSAGES_DEBUG=gtklab`.

Chapter 0 — Toolchain and the debugger

By the end of this chapter, pressing F5 in VS Code builds gtklab without optimisation, launches it, and stops at a breakpoint, in four configurations: plain, with the GTK Inspector, stopping on GTK critical warnings, and under gdb with GTK's own source. Nothing in this chapter is GTK code. It is the workbench every later chapter uses, and it is worth getting right before there is anything to debug.

In this chapter

- Trace a build from `meson.build` to a binary the debugger can read, and say what each tool in the chain contributes.
- Explain how the version-floor macros turn "runs on GTK 4.8" into a compile error rather than a bug report.
- Choose the right launch configuration for a problem: a crash, a layout question, a GTK warning, or a step into GTK itself.
- Get symbols and source for GTK on demand with debuginfod.

Concepts

How the pieces fit

Five tools touch every build, and each does one job.

Meson reads `meson.build`, which says what to build: which sources, which libraries, which flags. It does not compile anything. `meson setup` turns that description into a build directory.

pkg-config is how Meson finds a library. `dependency('gtk4')` asks pkg-config for the flags GTK needs, and pkg-config answers from a `.pc` file installed with `libgtk-4-dev`: more than a dozen `-I` paths for GTK and the libraries under it (Pango, GLib, Cairo, HarfBuzz), and the matching `-l` libraries. If `pkg-config --cflags gtk4` fails, so does `meson setup`, and the fix is the development package, not `meson.build`.

Ninja does the compiling. Meson writes `build.ninja` into the build directory, and `meson compile` runs Ninja on it, which rebuilds only what changed. You configure once and compile many times.

Meson also writes **`compile_commands.json`** beside `build.ninja`: the exact compiler command for every source file. VS Code's IntelliSense reads it, so the editor sees the same include paths and the same `-D` flags as the compiler. Without it the editor does not know where GTK's headers are, and reports errors the compiler never sees.

The **debugger** (CodeLLDB, or gdb through the C/C++ extension) needs one more thing from the build: debug information, the tables that map machine addresses back to source lines and variables.



Figure 1: From `meson.build` to a breakpoint. Meson asks pkg-config where GTK is, writes the Ninja file and the compile commands, and the debug build gives the debugger its line tables.

A build directory holds one configuration. `build-asan` (Chapter 16) and `build-floor` (Chapter 17) sit beside `build-debug` without interfering, because nothing is generated in the source tree.

Debug information: yours and GTK's

`--buildtype=debug` compiles with `-O0 -g`: no optimisation, full debug information. You can check it in `compile_commands.json`, where both flags appear in every command. Without optimisation every line of your code exists in the binary in order and every local variable has a home, so stepping and printing behave the way the source reads. A release build (`-O3`, no `-g`) is faster and tells the debugger almost nothing.

The packaged GTK is built optimised and its debug information ships separately, so out of the box the debugger shows GTK's internal frames as `??? () at libgtk-4.so.1` (exported functions such as `g_signal_emit` keep their names) and steps over `gtk_*` calls. That is enough for most bugs, which are in your frame. When you need to see what GTK did with your arguments, a debuginfod server supplies GTK's debug information and source on demand.

The version floor

GTK's headers mark every function with the version that introduced it and the version that deprecated it. Four macros tell the headers which window of versions you mean, and gtklab sets them to GTK 4.8 and GLib 2.74, the versions Debian 12 ships:

Macro	Effect
`GDK_VERSION_MAX_ALLOWED=4_8`	Calling anything newer than 4.8 is flagged as deprecated.
`GDK_VERSION_MIN_REQUIRED=4_8`	Things deprecated *after* 4.8 are not flagged, because 4.8 still needs them.
`GLIB_VERSION_MAX_ALLOWED`, `GLIB_VERSION_MIN_REQUIRED`	The same pair for GLib, at 2.74.
`-Werror=deprecated-declarations`	Turns either kind of flag into a compile error.

The first row is the promise: call a GTK 4.10 function by accident and the build fails on your machine, not on someone's Debian 12 jump host. The second row is easy to overlook and just as useful. `gtk_widget_show` is deprecated in GTK 4.10; compiled against GTK 4.14 with no floor, a call to it is an error under `-Werror=deprecated-declarations`. With the floor it compiles cleanly, because on 4.8 it is a normal function. The floor keeps the compiler from pushing you toward API your oldest users do not have.

What each launch configuration is for

Every configuration builds first, then runs the same binary with a different debugger or environment:

Configuration	Debugger	Adds	Reach for it when
gtklab	CodeLLDB	nothing	stepping your own code
gtklab + GTK Inspector	CodeLLDB	`GTK_DEBUG=interactive`	a layout, CSS or action question
gtklab: stop on GTK criticals	CodeLLDB	`G_DEBUG=fatal-criticals`	GTK printed a `CRITICAL`
gtklab (gdb)	gdb	`G_DEBUG=fatal-criticals`	you need to step into GTK

`GTK_DEBUG=interactive` opens the **GTK Inspector** in a second window. It shows the live widget tree, every property of the selected widget (and lets you change them), the CSS that applies to it, its event controllers, its actions, and the resources compiled into the program. For learning GTK it is the single most useful tool there is, and several chapters' drills use it.

`G_DEBUG=fatal-criticals` turns a GTK `CRITICAL` warning into an abort. A critical warning means a function was called with arguments that break its contract: a null where an object was required, an object of the wrong type, a widget that already has a parent. Without the variable the warning scrolls past and the program carries on in a bad state; with it, the debugger stops with the offending call on the stack.

From Chapter 4 the chapter directories add three more configurations: **gtklab (scratch config)**, which keeps settings and logs under `.debug/` in the project and turns on the book's debug output; **unit tests**, which runs the core library's tests; and **gtklab (ASan)**, which runs the AddressSanitizer build. The scratch configuration sets a third variable, `G_MESSAGES_DEBUG=gtklab`, which prints the `g_debug` lines in gtklab's code. gtklab logs under its own domain, `gtklab`, which `meson.build` sets with `-DG_LOG_DOMAIN="gtklab"` from Chapter 5, so you see gtklab's debug output without GTK's.

Build

Packages

On Debian 12, Ubuntu 23.04 or later, or ChromeOS Linux (Ubuntu 22.04 ships GTK 4.6, below the floor):

```
sudo apt install build-essential meson ninja-build pkg-config libgtk-4-dev gdb
```

On the Mac, with Homebrew:

```
brew install meson ninja pkg-config gtk4
```

The Mac has lldb with the Xcode command-line tools; gdb is Linux only.

In VS Code, install three extensions: **CodeLLDB** (the debugger the launch configurations use), **C/C++** (IntelliSense and the gdb configuration) and **Meson**. The repository recommends them, so VS Code offers to install them when you open a chapter directory.

Meson in one page

GTK and nearly everything around it builds with Meson, and so does gtklab. If you know CMake, the mapping is short: `project()` declares the project, `dependency()` finds a library through pkg-config, `executable()` and `static_library()` declare targets, and a separate build directory holds everything generated. From a chapter directory:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/gtklab
```

The first meson.build

ch01-window/meson.build

```
project('gtklab', 'cpp',
  version: '0.1.0',
  default_options: ['cpp_std=c++17', 'warning_level=2'])

gtk = dependency('gtk4', version: '>= 4.8')

gtk_floor_args = [
  '-DGDK_VERSION_MIN_REQUIRED=GDK_VERSION_4_8',
  '-DGDK_VERSION_MAX_ALLOWED=GDK_VERSION_4_8',
  '-DGLIB_VERSION_MIN_REQUIRED=GLIB_VERSION_2_74',
  '-DGLIB_VERSION_MAX_ALLOWED=GLIB_VERSION_2_74',
  '-Werror=deprecated-declarations',
]
add_project_arguments(gtk_floor_args, language: 'cpp')

executable('gtklab', 'src/main.cpp', dependencies: gtk)
```

`gtk_floor_args` is the version floor from the Concepts section. You can watch it work. This is what calling `gtk_file_dialog_new`, which arrived in GTK 4.10, produces with these flags:

```
error: 'GtkFileDialog* gtk_file_dialog_new()' is deprecated: Not available before 4.10
```

VS Code files

Each chapter directory has a `.vscode/` folder with four files. The first recommends the extensions:

ch01-window/.vscode/extensions.json

```
{
  "recommendations": [
    "vadimcn.vscode-lldb",
    "ms-vscode.cpptools",
    "mesonbuild.mesonbuild"
  ]
}
```

The second points IntelliSense at the compile commands:

ch01-window/.vscode/settings.json

```
{
  "C_Cpp.default.compileCommands": "${workspaceFolder}/build-debug/compile_commands.json"
}
```

The third defines the build tasks. `build-debug` configures the build directory the first time and compiles every time; `build-asan` is used from Chapter 16.

ch01-window/.vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build-debug",
      "type": "shell",
      "command": "[ -d build-debug ] || meson setup build-debug --buildtype=debug; meson compile -C
      ↪  build-debug",
      "problemMatcher": ["$gcc"],
      "group": { "kind": "build", "isDefault": true }
    },
    {
      "label": "build-asan",
      "type": "shell",
      "command": "[ -d build-asan ] || meson setup build-asan --buildtype=debug -Db_sanitize=address; meson
      ↪  compile -C build-asan",
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

The fourth is the one you will use most. Every configuration builds first through `preLaunchTask`, so F5 always runs current code.

ch01-window/.vscode/launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "gtklab",
      "type": "lldb",
      "request": "launch",
      "program": "${workspaceFolder}/build-debug/gtklab",
      "cwd": "${workspaceFolder}",
      "preLaunchTask": "build-debug"
    },
    {
      "name": "gtklab + GTK Inspector",
      "type": "lldb",
      "request": "launch",
      "program": "${workspaceFolder}/build-debug/gtklab",
      "cwd": "${workspaceFolder}",
      "env": {
        "GTK_DEBUG": "interactive"
      },
      "preLaunchTask": "build-debug"
    },
    {
      "name": "gtklab: stop on GTK criticals",
      "type": "lldb",
      "request": "launch",
      "program": "${workspaceFolder}/build-debug/gtklab",
      "cwd": "${workspaceFolder}",
      "env": {
        "G_DEBUG": "fatal-criticals"
      },
      "preLaunchTask": "build-debug"
    },
```

```
    {
      "name": "gtklab (gdb)",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/build-debug/gtklab",
      "cwd": "${workspaceFolder}",
      "MIMode": "gdb",
      "environment": [
        {
          "name": "G_DEBUG",
          "value": "fatal-criticals"
        }
      ],
      "setupCommands": [
        {
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ],
      "preLaunchTask": "build-debug"
    }
  ]
}
```

Stepping into GTK

On Debian, Ubuntu and ChromeOS, gdb can download GTK's symbols and source on demand from a debuginfod server. Set this before you start VS Code, then use the **gtklab (gdb)** configuration:

```
export DEBUGINFOD_URLS="https://debuginfod.debian.net"
```

The first step into a GTK function pauses while the files download. On Ubuntu, use `https://debuginfod.ubuntu.com` instead.

Debugger drill

Once Chapter 1's `main.cpp` exists, run each launch configuration once. In **gtklab (gdb)**, set a breakpoint on `gtk_window_present` and step in. You should land in GTK's `gtkwindow.c`. If you get disassembly instead, debuginfod is not reaching the debugger: check that the variable was set in the shell that started VS Code.

Read in tmssh

`scripts/build.sh`: one script that configures, builds and tests, with flags for a release build (`--release`), a clean build (`--clean`), installing the toolchain (`--deps`), installing to `~/.local` (`--install`) and running afterwards (`--run`). Note the package list it installs; Chapter 5 explains why `librsvg2-common` is on it. Then `gtk_floor_args` in its `meson.build`, the same version floor as gtklab's.

Review

1. `meson setup` fails because it cannot find the `gtk4` dependency. Which tool actually failed, and what do you install?

2. Why does IntelliSense need `compile_commands.json` when the compiler already builds the code correctly?
3. You call a GTK 4.12 function and the build fails with "is deprecated". Nothing about the function is deprecated. Which macro produced the message, and what is it protecting?
4. Why does gtklab set `GDK_VERSION_MIN_REQUIRED` as well as `GDK_VERSION_MAX_ALLOWED`? What would happen to a `gtk_widget_show` call without it?
5. In a stack from the plain **gtklab** configuration, GTK's frames show as `???`. Is something misconfigured? What would you change, and when is it worth doing?
6. GTK printed a `CRITICAL` line and the program kept running. Which configuration do you run next, and what does it give you that the printed line does not?
7. Why keep `build-debug`, `build-asan` and `build-floor` as separate directories rather than reconfiguring one?

Questions for deeper thought. Your tool has to run on the oldest machine in your fleet, not the newest. Which other dependencies of a tool you build would benefit from a floor like GTK's, and how would you enforce it at compile time? If a colleague reports a bug you cannot reproduce, which of this chapter's configurations would you ask them to run, and what output would you ask for?

Exercises

1. **A fatal-warnings configuration.** In `ch01-window/.vscode/launch.json`, copy **gtklab: stop on GTK criticals** to a new configuration named **gtklab: stop on warnings** with `G_DEBUG=fatal-warnings`. Done when it appears in the Run and Debug list and launches the window.
2. **Compare build types.** In `ch01-window`, run `meson setup build-release --buildtype=release` and compile. Done when you can point at the difference between the two `compile_commands.json` files: `-O0 -g` in one, `-O3` and no `-g` in the other.
3. **Watch the floor.** In `on_activate` of `ch01-window/src/main.cpp`, add a call to `gtk_file_dialog_new()` and build. Done when you have seen the "Not available before 4.10" error, removed the call, and the build is clean again.
4. **Read pkg-config's answer.** Run `pkg-config --cflags gtk4` and find each `-I` path in the command for `main.cpp` in `build-debug/compile_commands.json`. Done when you can say which library each of three paths belongs to.

Part I: Foundations

Part I builds an empty but correctly made window: an application that starts and stops properly, widgets whose ownership you can explain, a layout shaped like tmssh's, and settings that survive a restart. There is very little to look at yet. What you get instead is the ground every later chapter stands on: the application lifecycle, GObject's reference counting and signals, the container model, and a core library that keeps GTK out of the logic.

Chapter 1 — A window that runs

You start with the smallest GTK 4 program that behaves like an application: a window that opens, can be resized, and reports its size when it closes. It is twenty-five lines, and every one of them is doing something you will meet again.

In this chapter

- Write the smallest correct GTK 4 program: an application, an `activate` handler and one window.
- Explain who runs the main loop, what keeps it running and what ends it.
- Choose between a unique application and one process per launch, and say what each does on a second launch.
- Intercept a window closing with `close-request`.
- Read any GTK call stack by recognising the shape every callback shares.

Concepts

The application owns the process

In GTK 4 a program is a `GtkApplication`. It parses the command line, runs the main loop, and tells you when it is time to create windows by emitting the `activate` signal. You never call a "main" function of GTK's yourself; `g_application_run` runs the loop and returns only when the application quits. Your `main` is three lines of setup and a hand-over.

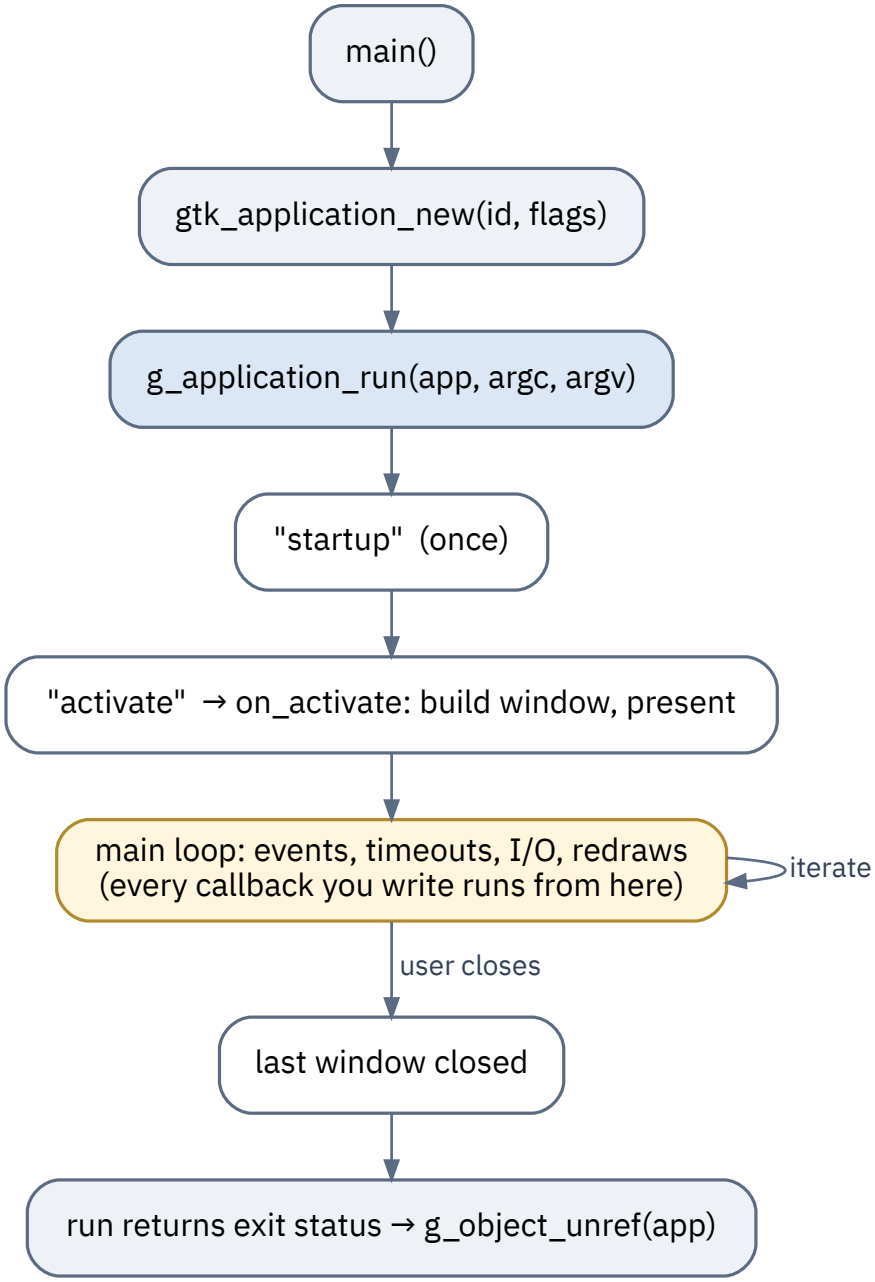


Figure 1: What runs when: `main` hands control to the application, which emits `activate` once the main loop is running.

An application emits three signals over its life, in a fixed order:

Signal	When	What belongs there
`startup`	once, before anything else, in the process that will run the program	application-wide setup that must exist before any window

Signal	When	What belongs there
`activate`	when the program is launched with no files to open, after `startup`	create and present a window
`shutdown`	once, after the main loop ends	application-wide teardown

Connecting a handler to each and closing the window prints them in that order: `startup`, `activate`, the `close-request` line, `shutdown`, and then `g_application_run` returns. gtklab only needs `activate`. The other two matter once there are things that outlive a window.

`activate` is also the first moment GTK is ready. `main` runs before the main loop and before the display connection exists, so nothing that draws can be built there.

What keeps the loop running

The application quits when it has no windows left. A window made with `gtk_application_window_new(app)` is registered with the application when it is created, so it keeps the application alive until it closes. Two windows keep it alive until both have closed. A window that was never registered keeps nothing alive, which is why the first Break it below is so short.

This rule is worth holding on to because it decides when your cleanup runs. Everything after `g_application_run` in `main`, including the final `g_object_unref(app)`, happens after the last window is gone.

One process, or one per launch

The application id, `io.github.gtklab`, names the application on the desktop and on D-Bus. By default GTK makes an application unique: a second launch registers the same id, finds the first process holding it, asks it to activate, and exits. Measured with the default flags under a D-Bus session: the second launch printed nothing and returned 0, and the first process received a second `activate`, so it opened a second window.

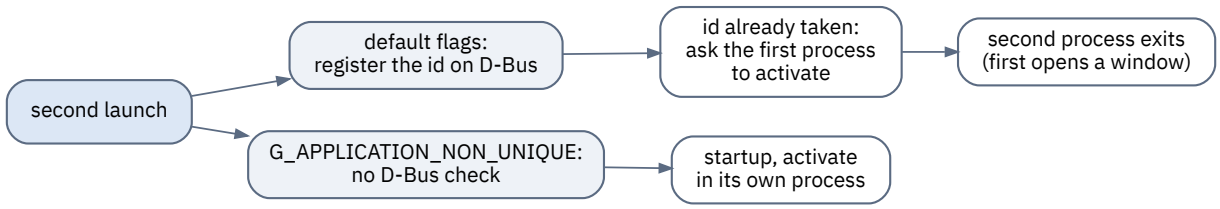


Figure 2: What a second launch does. With default flags the first process does the work; with `G_APPLICATION_NON_UNIQUE` each launch is independent.

That is the right behaviour for a document editor and the wrong one for a terminal you may want to run twice with different settings, so gtklab passes `G_APPLICATION_NON_UNIQUE` and every launch is its own process. tmssh does the same. The trade-off is real: a unique application can share one session list and one settings file safely, because only one process ever writes them. A non-unique one has to tolerate another copy saving the same file; Chapter 4's atomic save at least guarantees that file is never left half-written.

Asking a window to close

A window that is asked to close, by the window manager's close button or by `gtk_window_close`, emits `close-request` first. A handler can return `TRUE` to keep the window open (to ask "save changes?", for example) or `FALSE` to let it close. When every handler returns `FALSE`, GTK destroys the window, and if it was the last one the application quits. gtklab uses the signal to report the window's size; from Checkpoint A it saves it.

Returning a value from a signal handler is unusual in GTK. Most handlers return `void`, because the signal is only a notification. `close-request` returns a `gboolean` because it asks a question, and the answer changes what GTK does next.

Every callback has the same shape

After `activate`, nothing in your program runs on its own. Every line you write runs because the main loop took an event from a source (input, a timer, a file descriptor becoming readable, a frame to draw), turned it into a signal, and the signal called your handler. When the handler returns, control goes back to the loop.

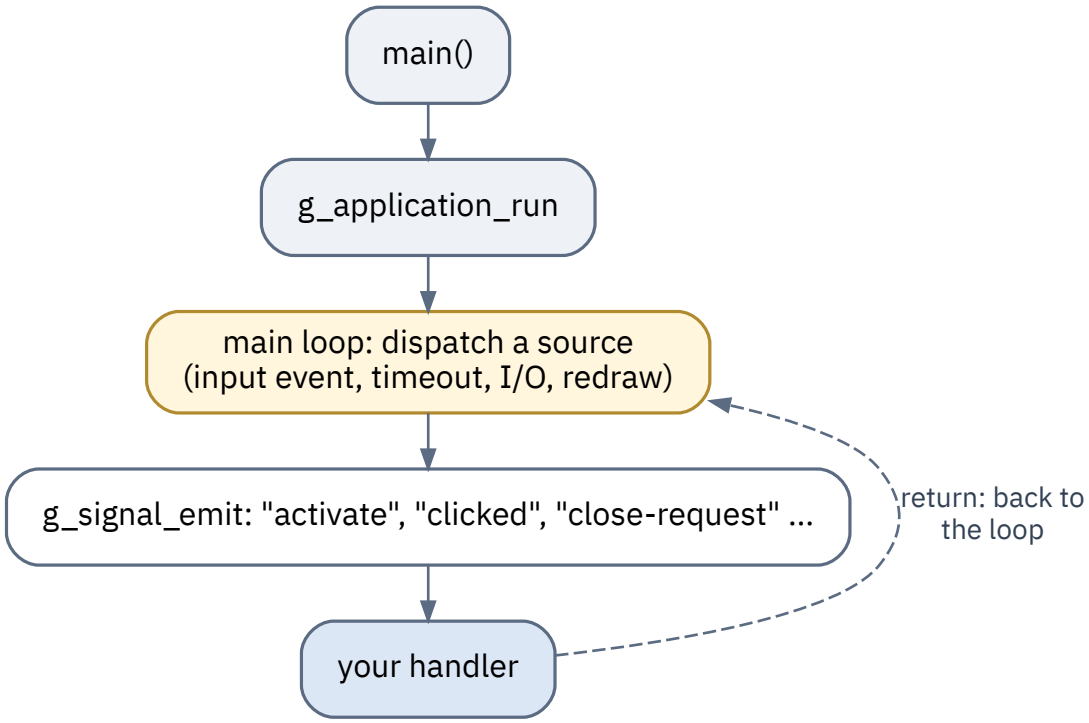


Figure 3: The shape of every GTK callback: your handler runs inside a signal emission, inside the main loop, inside `g_application_run`.

Two consequences follow. A handler that takes a long time freezes the window, because the loop that would redraw it is waiting for the handler to return; Chapter 11 shows how to wait on slow things, a child process or a file, without blocking it. And a crash inside a handler shows this same stack, with your function at the top and GTK's frames below, which is the first thing to recognise when you read one.

Coming from GTK 3

If you have written GTK 3, a handful of habits have to go:

GTK 3	GTK 4
`gtk_init` then `gtk_main`	`GtkApplication` and `g_application_run`; `gtk_main` is gone
`gtk_widget_show_all`	nothing: widgets are visible by default
`delete-event` on a window	`close-request`
`gtk_container_add`	one call per container type (Chapter 3)

Build

`src/main.cpp`, complete:

ch01-window/src/main.cpp

```
#include <gtk/gtk.h>

static gboolean on_close_request(GtkWindow *win, gpointer) {
    int w = 0, h = 0;
    gtk_window_get_default_size(win, &w, &h);
    g_print("closing at %dx%d\n", w, h);
    return FALSE;  // FALSE lets the window close
}

static void on_activate(GtkApplication *app, gpointer) {
    GtkWidget *win = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(win), "gtklab");
    gtk_window_set_default_size(GTK_WINDOW(win), 900, 600);
    g_signal_connect(win, "close-request", G_CALLBACK(on_close_request), nullptr);
    gtk_window_present(GTK_WINDOW(win));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(on_activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

Three details are easy to miss.

`gtk_window_get_default_size` returns the size the window has now, not the size it was created with. GTK 4 updates a window's default size as the user resizes it, which is what makes it the right value to save.

`gtk_window_present` shows the window and asks the window manager to raise it. GTK 4 widgets are visible by default, so there is no `gtk_widget_show_all`; a window is the one widget you still present.

The application is unreferenced after `g_application_run` returns. It was created with one reference, which `main` owns.

Done when

The window opens, you resize it and close it, and the terminal prints the new size. In the recorded run the window was resized to 1100 by 700 and closed, and the program printed `closing at 1100x700`.

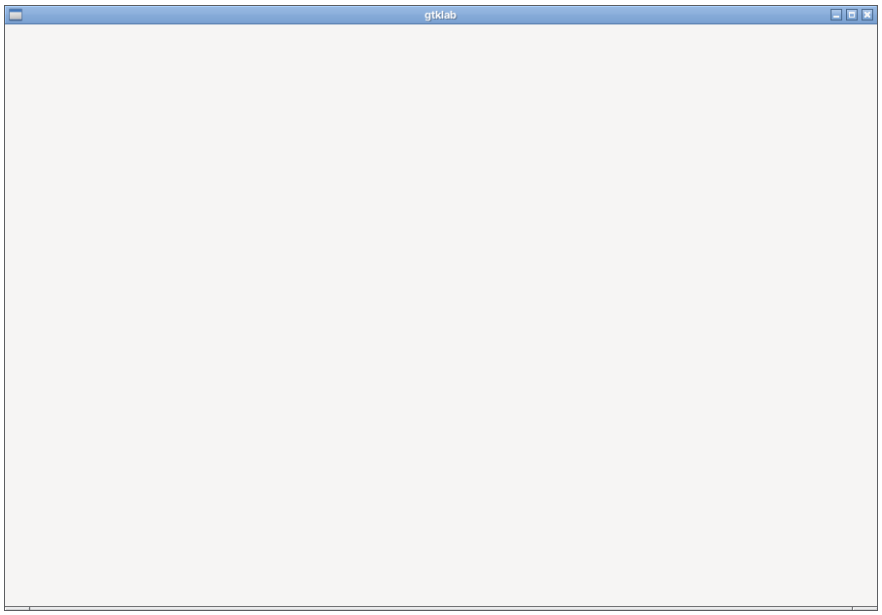


Figure 4: Chapter 1: an empty window, opened by `on_activate`.

Debugger drill

Set a breakpoint in `on_activate` and press F5. When it stops, read the call stack from the bottom up: `main`, then `g_application_run`, then the main loop dispatching a source, then a signal emission, then your handler. That shape, your code called from a signal emitted from the main loop, is the shape of every GTK callback you will ever debug. Set a second breakpoint in `on_close_request` and close the window to see the same shape again.

Break it

Replace `gtk_application_window_new(app)` with `gtk_window_new()` and run. The program exits at once with status 0. The window was created, but it was never added to the application, so as soon as `activate` returned the application had no windows and quit. (`gtk_window_set_application` would add it, which is all `gtk_application_window_new` does differently, plus a menu bar that gtklab does not use.)

Common mistakes

Writing a GTK 3 main. `gtk_init` followed by `gtk_main` is GTK 3. `gtk_main` no longer exists. Use `GtkApplication` and `g_application_run`.

Building windows before `activate`. `main` runs before the main loop and before the display is open. Create windows in `activate`, never in `main`.

Leaving the default flags on a tool you run twice. Without `G_APPLICATION_NON_UNIQUE` the second launch raises the first window and exits, which looks like a bug when you wanted a second instance.

Read in tmssh

`src/main.cpp`: `main` parses tmssh's own options before the application runs, and `on_activate` creates the window. Then `on_close_request` in `src/app_window.cpp`, which saves settings where gtklab prints.

Review

1. `main` creates the application and calls `g_application_run`, yet your window code lives in `on_activate`. Why can it not live in `main`?
2. Name everything that keeps gtklab's process running after `activate` returns. What is the one event that ends it?
3. A user launches gtklab twice. Describe what happens with `G_APPLICATION_NON_UNIQUE`, and what would happen without it.
4. You want a tool where a second launch opens a new window in the existing process. Which flag choice gives you that, and what does `on_activate` have to tolerate as a result?
5. Why does a `close-request` handler return a value when a `clicked` handler does not? What does GTK do if your handler returns `FALSE`?
6. A handler runs a five-second ping before returning. What does the user see during those five seconds, and why?
7. You call `g_print` after `g_application_run` returns. Is any window still open at that point? Explain from the rule that keeps the loop running.

Questions for deeper thought. For a network tool, is it better to have one process holding every open session or one process per window? Think about crash isolation, shared settings and memory. What would your tool need to do differently if a second launch had to hand a host name to an already-running instance?

Exercises

1. **Startup and shutdown.** In `main` in `ch01-window/src/main.cpp`, connect handlers to the application's `startup` and `shutdown` signals that print their names. Done when one run prints `startup`, then the closing size, then `shutdown`.
2. **Confirm before closing.** Change `on_close_request` so that the first close is refused (return `TRUE` and print "close again to quit") and the second is allowed. Done when the window survives one click on the close button and closes on the second.
3. **Two windows.** Make `on_activate` create and present two windows, each with its own title. Done when closing either leaves the program running and closing the second ends it with both sizes printed.
4. **See uniqueness.** Change the flags in `main` to `G_APPLICATION_DEFAULT_FLAGS` and launch gtklab twice from two terminals. (The older spelling, `G_APPLICATION_FLAGS_NONE`, is deprecated in GLib 2.74, and the version floor rejects it.) Done when the second command returns at once and the first process opens a second window. Put `G_APPLICATION_NON_UNIQUE` back afterwards.

Chapter 2 — GObject: references, signals, ownership

GTK is written in C, and its widgets are objects in GObject, a small object system written in C. You do not need to know much of GObject to use GTK, but you do need to know three things well: how objects are kept alive, how signals call your code, and how to attach your own data to an object so it dies when the object does. Almost every hard bug in a GTK program is one of those three going wrong, and this chapter builds a tiny program that shows all three working, and then breaks them on purpose.

In this chapter

- Tell an owning reference from a borrowed pointer, and say who owns each widget in a window.
- Predict when a widget is destroyed, and what that does to pointers you kept.
- Connect a signal handler that cannot be called with a dead pointer.
- Make a widget own a C++ object, so the object is deleted when the widget is.

Concepts

An object system written in C

GTK's API looks like a collection of C functions, but behind it is a runtime type system. Most of what you handle derives from `GObject`, which supplies four things C does not have: a runtime type, reference-counted lifetime, signals, and named properties. Every widget is a GObject; not every GObject is a widget. The type chains for this chapter's objects, as GLib reports them:

Object	Type chain
a button	`GtkButton` → `GtkWidget` → `GInitiallyUnowned` → `GObject`
a window	`GtkWindow` → `GtkWidget` → `GInitiallyUnowned` → `GObject`
the application	`GtkApplication` → `GApplication` → `GObject`

The compiler knows none of this. A cast macro such as `GTK_BOX(widget)` or `G_OBJECT(button)` is a checked cast at run time: it creates nothing and takes no reference, and if the object is not of that type GLib prints a critical warning naming the type it expected. That check is often the only evidence of a lifetime bug, as the second Break it shows.

Properties are the fourth piece. A property is a named, typed value on an object (a paned's `position`, a stack's `visible-child-name`) that can be read and set by name with `g_object_get` and `g_object_set`.

The GTK Inspector is built on them, which is why Chapter 3's drill can change a paned's position by typing a number.

Reference counting, and the floating reference

Every GObject has a reference count. `g_object_ref` adds one, `g_object_unref` removes one, and when the count reaches zero the object is destroyed. That is the whole rule, and it is the rule for application objects, models and settings objects.

Widgets add one twist, which is what `GInitiallyUnowned` in the table means. A new widget starts with a **floating** reference: a reference that nobody owns yet. The first container you add it to *sinks* that reference and becomes the owner. So this code has no leak and no unref, because the box owns the button and the window owns the box:

```
GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
GtkWidget *button = gtk_button_new_with_label("Go");
gtk_box_append(GTK_BOX(box), button);    // box sinks the button's reference
gtk_window_set_child(GTK_WINDOW(win), box);
```

The consequence is the part that matters: **removing a widget from its parent destroys it**, unless you took a reference of your own first. A pointer you kept to it is then a pointer to freed memory.

Windows are the exception to the parent rule, because they have no parent. A new window is not floating: GTK keeps it in its list of toplevel windows, and closing the window is what releases it.

A pointer in your code says nothing about which of these cases it is. The question to ask of every one is whether it represents a reference you own:

Where the pointer came from	What you hold	What you must do
`gtk_application_new`, `g_key_file_new`	a full reference	unref it when done
`gtk_button_new`, any widget constructor	a floating reference	give it to a parent
`gtk_widget_get_parent`, `g_object_get_data`	a borrowed pointer	nothing; do not keep it past the owner
`user_data` in a handler	whatever you passed at connect time	make sure it outlives the connection

Destruction in two steps

When the last reference goes, GObject destroys an object in two steps. **Dispose** drops the object's references to other objects; for a widget that includes its children, which is how removing a box destroys everything inside it. **Finalize** frees the object itself, and on the way it clears the data attached to the object, calling each destroy function.

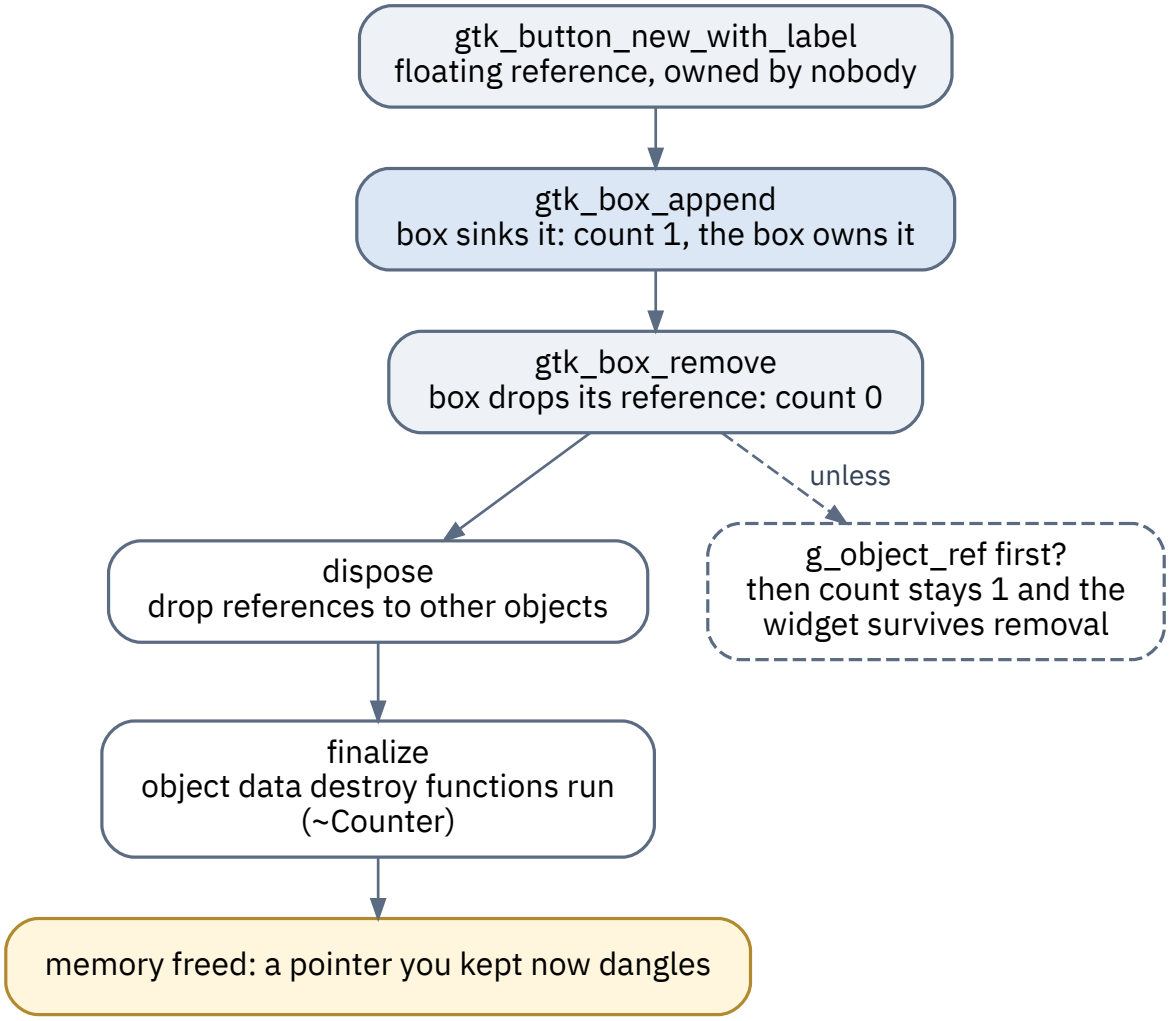


Figure 1: The life of the counter button. Appending sinks its floating reference; removing it drops the count to zero, and finalize runs the destroy function that deletes the `Counter`.

The Debugger drill shows the second step from the inside: `~Counter` is called from `g_datalist_clear` inside `g_object_unref`.

Signals

A signal is a named event on an object: `clicked` on a button, `close-request` on a window. You connect a C function to it with `g_signal_connect(object, "clicked", G_CALLBACK(handler), user_data)`, and when the object emits the signal GObject calls your function with the object, the signal's own arguments, and your `user_data` pointer.

Emission is synchronous. `g_signal_emit` calls every connected handler, in order, before it returns; there is no queue. GObject also holds a reference to the emitting object for the whole emission. In a test where a `clicked` handler dropped the button's last reference, the button was finalized only after the handler returned and before `g_signal_emit` did. The emitter is always alive inside its own handlers; other objects are not.

That `user_data` pointer is where lifetime bugs come from. GObject stores it and hands it back; it does not know what it points at and does not keep it alive. If it points at something that is destroyed before the signal fires, your handler gets a dangling pointer.

`g_signal_connect_object` is the fix for the most common case, where `user_data` is itself a GObject. It ties the connection to that object's lifetime: when the object is destroyed, the handler is disconnected, so it can never be called with a dead pointer. It protects that one object only. Anything else the handler reaches through it still needs its own answer to "who keeps this alive?".

Connection	`user_data` is	Safe when
`g_signal_connect`	anything	`user_data` outlives the emitting object, or you disconnect first
`g_signal_connect_object`	a GObject	always, for that object
`g_signal_connect` with `nullptr`	nothing	always; the handler finds its state on the emitter

The third row is this chapter's counter button: its handler takes no `user_data` at all and finds its state on the object that emitted the signal, which cannot be dead while it is emitting.

Data attached to objects

`g_object_set_data_full(object, "key", pointer, destroy)` attaches your own data to any GObject, with a function to call when the object is destroyed (or the key is replaced). It is how a widget can own a C++ object: attach it, and give `delete` as the destroy function. You will use exactly this in Chapter 8 to make a notebook page own everything behind its tab.

This is the bridge between the two lifetime systems in a GTK program written in C++. C++ objects live by scope and `unique_ptr`; GObjects live by reference count. Attaching a C++ object to a GObject with a destroy function makes the reference count decide when the destructor runs, so you never have to find the right moment to `delete` it yourself.

Build

A counter button that owns a C++ `Counter` struct, and a second button that removes the counter from the window. Replace `main.cpp` with:

ch02-gobject/src/main.cpp

```cpp
#include <gtk/gtk.h>
#include <cstdio>

struct Counter {
    int clicks = 0;
    ~Counter() { g_print("Counter freed after %d clicks\n", clicks); }
};

static void on_count_clicked(GtkButton *button, gpointer) {
    auto *c = static_cast<Counter *>(g_object_get_data(G_OBJECT(button), "counter"));
    c->clicks++;
    char label[32];
    std::snprintf(label, sizeof label, "Clicked %d", c->clicks);
    gtk_button_set_label(button, label);
```

```
}

// user_data is the counter button; g_signal_connect_object disconnects
// this handler when that button is finalised.
static void on_remove_clicked(GtkButton *, gpointer user_data) {
    GtkWidget *count = GTK_WIDGET(user_data);
    GtkWidget *box = gtk_widget_get_parent(count);
    if (box)
        gtk_box_remove(GTK_BOX(box), count);
}

static gboolean on_close_request(GtkWindow *win, gpointer) {
    int w = 0, h = 0;
    gtk_window_get_default_size(win, &w, &h);
    g_print("closing at %dx%d\n", w, h);
    return FALSE;
}

static void on_activate(GtkApplication *app, gpointer) {
    GtkWidget *win = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(win), "gtklab");
    gtk_window_set_default_size(GTK_WINDOW(win), 900, 600);
    g_signal_connect(win, "close-request", G_CALLBACK(on_close_request), nullptr);

    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
    gtk_widget_set_margin_top(box, 12);
    gtk_widget_set_margin_bottom(box, 12);
    gtk_widget_set_margin_start(box, 12);
    gtk_widget_set_margin_end(box, 12);

    GtkWidget *count = gtk_button_new_with_label("Clicked 0");
    g_object_set_data_full(G_OBJECT(count), "counter", new Counter,
                           [](gpointer p) { delete static_cast<Counter *>(p); });
    g_signal_connect(count, "clicked", G_CALLBACK(on_count_clicked), nullptr);

    GtkWidget *remove = gtk_button_new_with_label("Remove counter");
    g_signal_connect_object(remove, "clicked", G_CALLBACK(on_remove_clicked), count,
                            GConnectFlags(0));

    gtk_box_append(GTK_BOX(box), count);
    gtk_box_append(GTK_BOX(box), remove);
    gtk_window_set_child(GTK_WINDOW(win), box);
    gtk_window_present(GTK_WINDOW(win));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(on_activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

Read the two connections side by side. The counter button's handler gets no `user_data`, because it finds its `Counter` on the button itself with `g_object_get_data`. The remove button's handler gets the counter button as `user_data`, and it is connected with `g_signal_connect_object`, so the connection dies with the counter button.

The lambda passed to `g_object_set_data_full` is a capture-less lambda, which converts to a plain function pointer; that is the form a C callback needs. You will see this pattern throughout the book.

Done when

Clicking the counter counts, Remove prints `Counter freed after N clicks`, and clicking Remove a second time does nothing.

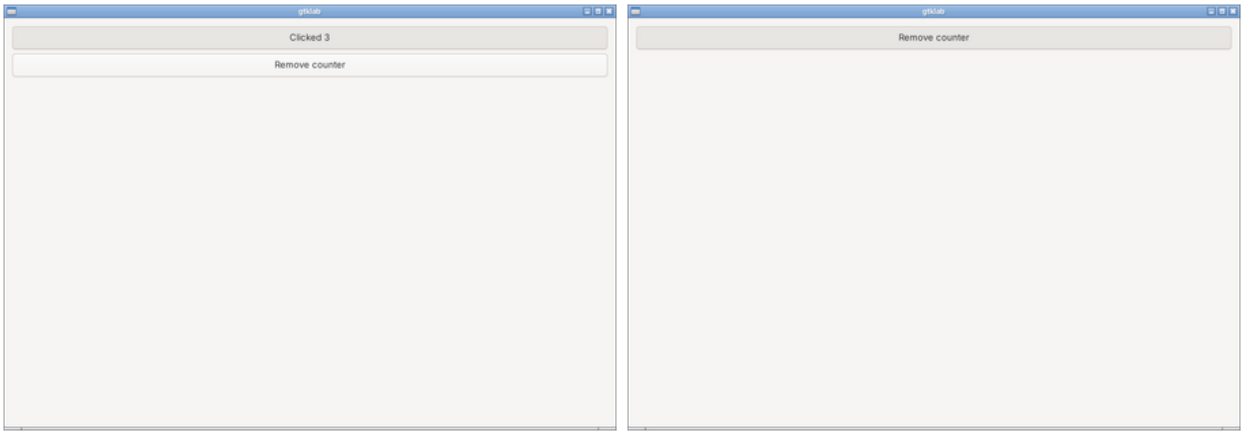


Figure 2: Three clicks, then Remove: the counter button is gone and the `Counter` it owned has been deleted.

Debugger drill

Set a breakpoint in `~Counter` and click Remove. The recorded stack, from the top:

```
#0  Counter::~Counter at ../src/main.cpp:6
#1  operator() at ../src/main.cpp:47
#3  g_datalist_clear at libglib-2.0.so.0
#4  g_object_unref at libgobject-2.0.so.0
 ... GTK and signal emission frames ...
```

Further down are your `on_remove_clicked` and the button's `clicked` signal. Read it upward: the click ran your handler, your handler removed the button from the box, the box dropped the last reference, GObject destroyed the button, and destroying it cleared its data, which called your lambda, which deleted the `Counter`. You never wrote `delete` for it anywhere else.

Break it

1. Append `count` to the box a second time. Run the **stop on GTK criticals** configuration. It stops on `gtk_box_append: assertion 'gtk_widget_get_parent (child) == NULL' failed`, with your line on the stack. A widget has one parent.
2. Change `g_signal_connect_object(...)` to `g_signal_connect(remove, "clicked", G_CALLBACK(on_remove_clicked), count)` and click Remove twice. The second click hands the destroyed button to your handler, and GLib prints `invalid unclassed pointer in cast to 'GtkWidget'`. Then build with AddressSanitizer (Chapter 16 shows how) and try again: ASan says nothing. The freed memory belonged to GLib, which is not built with ASan, so the only evidence is GTK's own type check. Chapter 16 comes back to the two kinds of lifetime bug and the tool for each.

Common mistakes

Keeping a pointer to a child you removed. Removing a widget from its parent destroys it. Take a reference with `g_object_ref` first if you need it afterwards, and drop it with `g_object_unref` when you are done.

Unreferencing a widget you never referenced. The container owns it. An extra `g_object_unref` destroys it while it is still in the window, and the crash comes later, somewhere else.

Passing `this` as `user_data` and forgetting it. Every handler connected with a raw pointer must be disconnected before that pointer dies. Chapter 16 is about the time this goes wrong at quit.

Read in tmssh

`add_tab` in `src/app_window.cpp`: each notebook page owns its `Tab` through `g_object_set_data_full`, exactly like the `Counter` here, except that deleting a `Tab` closes a pty and a log file.

Review

1. `gtk_box_append(box, button)` is followed by no `g_object_unref`. Why is that not a leak, and what would an unref there do?
2. Why does `on_remove_clicked` look up the parent before calling `gtk_box_remove`, and why must it not use `count` after the call?
3. A window has no parent. What keeps it alive, and what releases it?
4. In the Debugger drill, `~Counter` is called from `g_object_unref`. Trace the chain from the click on Remove to that destructor.
5. The counter button's handler is connected with `nullptr` as `user_data`. Why is that the safest of the three connection styles?
6. `g_signal_connect_object` protects the remove button's handler. Exactly which pointer does it protect, and what would it not protect?
7. With plain `g_signal_connect`, a second click on Remove produced `invalid unclassed pointer in cast to 'GtkWidget'` and ASan said nothing. Which part of GObject caught the bug, and why could ASan not?
8. You need the counter button again after removing it, to put it in a different box. What do you do before `gtk_box_remove`, and what do you owe afterwards?

Questions for deeper thought. Would you store a device inventory on the list widget that shows it, or in a separate object? What happens to that choice when a background operation finishes after the widget has been removed? If every C++ object in a tool is owned by some GObject through attached data, what does the ownership tree look like, and where would you look first for a leak?

Exercises

1. **Two counters.** In `on_activate` in `ch02-gobject/src/main.cpp`, add a second counter button with its own `Counter` and its own remove button. Done when removing one prints only its own `Counter freed after N clicks` and the other keeps counting.
2. **Keep it alive.** Change `on_remove_clicked` to take a reference with `g_object_ref` before removing the counter, and append it back to the box from a third "Restore" button, then unref it. Done when the count survives a remove and restore, and `~Counter` runs only when the window closes.

3. **Watch the two steps.** In `on_activate`, register a weak reference on the counter button with `g_object_weak_ref`, using a capture-less lambda that prints `weak notify`. Done when Remove prints `weak notify` before `Counter freed after N clicks`, and you can say which of dispose and finalize each line comes from.
4. **A weak pointer.** Store the counter button in a `static GtkWidget *` and register it with `g_object_add_weak_pointer`. Add a button that prints whether the pointer is null. Done when it prints non-null before Remove and null after, with no crash.

Chapter 3 — Layout and containers

This chapter gives gtklab its shape: a header bar, a sidebar for sessions on the left, and on the right either a "No tabs open" hint or a notebook of tabs. Nothing in it does anything yet. The point is to learn how GTK 4 decides where things go and how big they are, because every layout bug you will meet later is one of a handful of mistakes about that.

In this chapter

- Read a window as a tree of containers and predict each container's share of the space.
- Explain the measure-then-allocate negotiation, and use `hexpand`, `vexpand` and alignment to steer it.
- Choose between a box, a paned, a stack and a notebook for a region of a tool's window.
- Diagnose the common layout failures: a collapsed scrolled window, a sidebar dragged out of existence, a stack sized by a page you cannot see.

Concepts

A tree of containers, each with one rule

A GUI toolkit could let you place controls at coordinates. GTK does not. You describe a tree of widgets, each container in it applies one layout rule to its children, and GTK works out positions and sizes again every time the window changes. A sidebar never needs to know where the content area starts; it only takes part in its parent's rule.

A GTK window holds one child. That child is usually a container, which holds children of its own, and so on down to buttons and labels. There is no generic "add" call: each container has its own, because each has its own idea of what a child is.

Container	Its rule	Add a child with	Watch for
`GtkBox`	children in a row or a column	`gtk_box_append`	spacing is between children; margins belong to each child
`GtkPaned`	two children and a draggable divider	`gtk_paned_set_start_child`, `..._end_child`	by default either child can be squeezed to nothing
`GtkStack`	several named children, one visible	`gtk_stack_add_named`	by default sized for its largest page, visible or not
`GtkScrolledWindow`	one child seen through a viewport	`gtk_scrolled_window_set_child`	its natural height is small

Container	Its rule	Add a child with	Watch for
`GtkNotebook`	pages with a tab each	`gtk_notebook_append_page`	the tab row is its own widget, measured like any other
`GtkHeaderBar`	buttons in the title bar	`gtk_header_bar_pack_start` / `..._end`	it goes in the window's titlebar slot, not its child

A window has two slots, not one. `gtk_window_set_titlebar` installs the header bar where the window manager's title bar would be; `gtk_window_set_child` installs the content. gtklab's tree hangs the header bar and the paned off the window side by side:

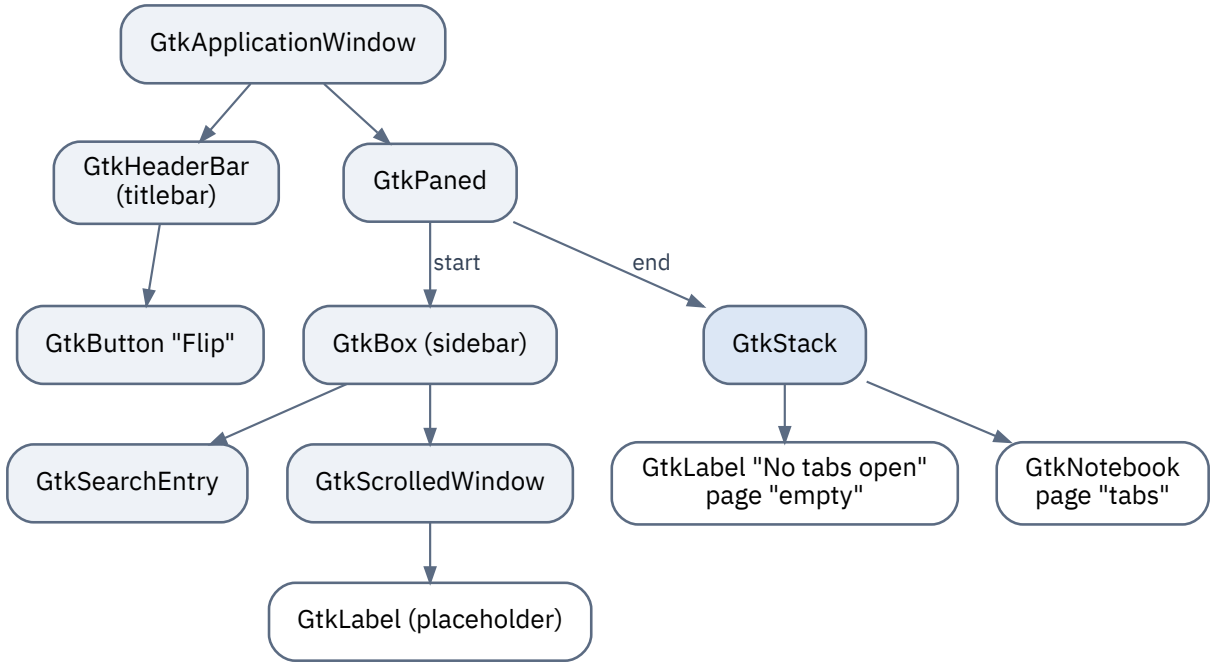


Figure 1: gtklab's widget tree at the end of this chapter. The sidebar is the paned's start child and the stack its end child; the stack shows one of its two pages at a time.

The stack and the notebook are two levels of navigation. The stack chooses what the content area is showing (the empty hint or the tabs); the notebook chooses which tab within it. Pages in a stack are not created and destroyed as you switch: they stay in the tree, hidden, which matters both for sizing and, from Chapter 8, for the state behind each tab.

Measure, then allocate

Sizes are negotiated in two passes. First every widget is **measured**: asked for its minimum and natural size. Then every widget is **allocated** the space its parent decided to give it. Measuring goes up the tree, because a box cannot know its own size until it knows its children's; allocating goes down, because a child cannot know its rectangle until its parent has divided the space. GTK runs both in the layout phase of a frame, so a change you make in a handler shows up on the next frame, not inside the call.

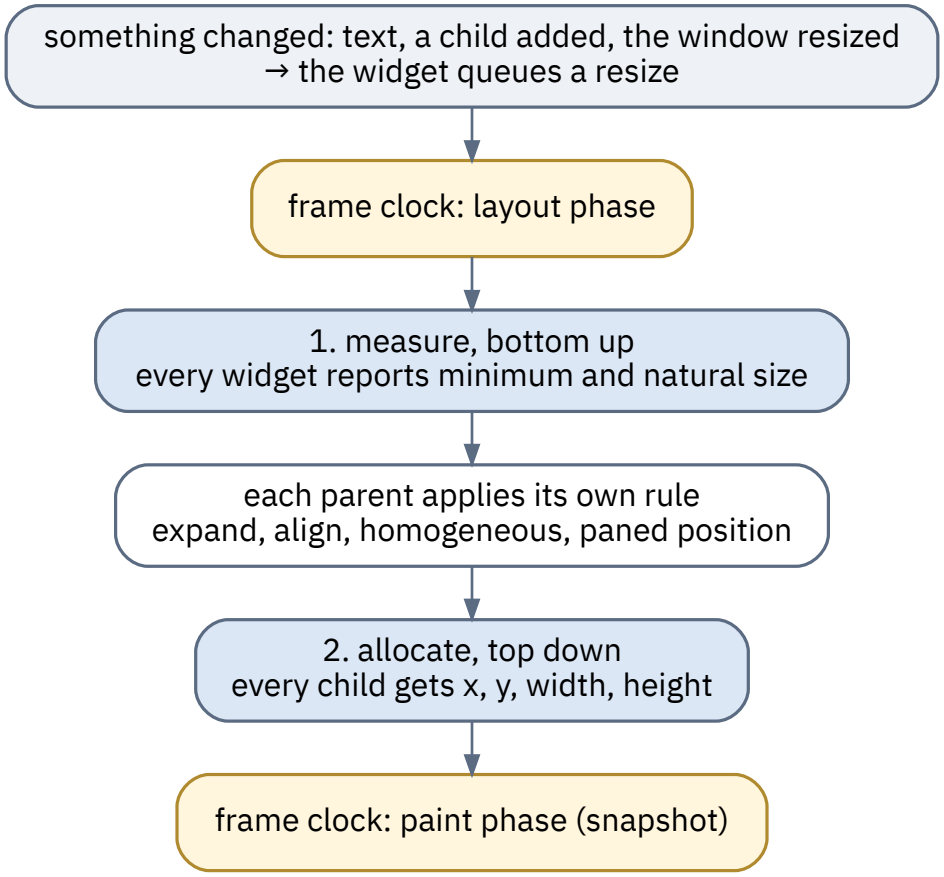


Figure 2: One layout pass. Changes only queue a resize; the frame clock runs measurement and allocation together, then paints.

Minimum and natural are different promises. Minimum is the smallest size at which the widget still works; natural is the size it would like. A window can never be smaller than its child's minimum. Between minimum and natural, and beyond natural, the parent's rule decides.

Expand, align, margin, minimum

Four properties on every widget steer the parent's decisions, and they are easy to confuse:

Property	Answers	Default
`hexpand`, `vexpand`	Should this widget get a share of spare space?	no
`halign`, `valign`	Where does it sit inside the space it got?	`FILL`: use all of it
margins	How much empty space surrounds it?	0
`gtk_widget_set_size_request`	What is its minimum, at least?	its own measurement

A widget gets more than its natural size only if it asks to expand. Expansion propagates upward, so a widget deep in the tree that wants to expand makes its ancestors expand too: a box holding one `hexpand` label reports that it wants to expand, though nobody set `hexpand` on the box. Alignment is separate. It decides what a widget does with more space than it asked for. A label with `valign` set to `START` in a tall space is given only its natural height and placed at the top; with the default `FILL` it would be stretched over the whole height. The sidebar placeholder uses `START`.

A size request is a floor, not a size. It cannot make a widget smaller than it needs, and, as the Isolated lab shows, it does not stop a paned from squeezing the widget anyway.

Where layouts go wrong

`GtkScrolledWindow` is where people get caught most. Its natural height is not its content's: measured with eight lines of text inside, a scrolled window reported the same small value for minimum and natural, because the point of a scrolled window is that its content can be larger than it. Put one in a vertical box without `vexpand`, and it gets that natural height, which is a few pixels. Chapter 5's Break it shows exactly that.

`GtkPaned` defaults to `resize` and `shrink` both on, for both children: both grow when the window grows, and either can be dragged below its minimum size until it disappears. For a sidebar both defaults are wrong, and the Build section turns them off for the start child.

`GtkStack` defaults to homogeneous sizing: it is as big as its biggest page, so it measures every page, including the hidden ones. A hidden page can make the visible one sit in a space too wide for the pane, or, as the Build section shows, make GTK warn about a page nobody can see.

Layout is not ownership

Chapter 2's rule still holds: a container owns its children, and removing a child destroys it. But ownership says how long a widget lives, and layout says where it is and how big. Hiding a widget with `gtk_widget_set_visible(w, FALSE)` changes its layout (it takes no space) and leaves its lifetime alone; removing it ends both. GTK 4 widgets are visible from the moment they are created, so there is no `gtk_widget_show_all`, and `gtk_widget_set_visible(w, TRUE)` brings a hidden widget back.

The Flip button's handler gets the stack as `user_data`, a borrowed pointer. That is safe here for a reason worth naming: the button and the stack are in the same window and are destroyed together. A pointer from one window's widget to another window's would need the care Chapter 2 described.

Build

The layout code is in three functions, and the whole `main.cpp` is short enough to read in full:

ch03-layout/src/main.cpp

```
#include <gtk/gtk.h>

static gboolean on_close_request(GtkWindow *win, gpointer) {
    int w = 0, h = 0;
    gtk_window_get_default_size(win, &w, &h);
    g_print("closing at %dx%d\n", w, h);
    return FALSE;
}
```

```
// Temporary until Chapter 7 turns this into an action.
static void on_flip_clicked(GtkButton *, gpointer user_data) {
    GtkStack *stack = GTK_STACK(user_data);
    const char *now = gtk_stack_get_visible_child_name(stack);
    gtk_stack_set_visible_child_name(stack, g_strcmp0(now, "empty") == 0 ? "tabs" : "empty");
}

static GtkWidget *build_sidebar() {
    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
    gtk_widget_set_margin_top(box, 6);
    gtk_widget_set_margin_bottom(box, 6);
    gtk_widget_set_margin_start(box, 6);
    gtk_widget_set_margin_end(box, 6);

    GtkWidget *search = gtk_search_entry_new();
    gtk_box_append(GTK_BOX(box), search);

    GtkWidget *placeholder = gtk_label_new("Sessions go here (Chapter 5)");
    gtk_widget_add_css_class(placeholder, "dim-label");
    gtk_widget_set_valign(placeholder, GTK_ALIGN_START);

    GtkWidget *scroller = gtk_scrolled_window_new();
    gtk_widget_set_vexpand(scroller, TRUE);
    gtk_scrolled_window_set_child(GTK_SCROLLED_WINDOW(scroller), placeholder);
    gtk_box_append(GTK_BOX(box), scroller);
    return box;
}

static GtkWidget *build_content(GtkWidget **stack_out) {
    GtkWidget *stack = gtk_stack_new();
    gtk_stack_set_hhomogeneous(GTK_STACK(stack), FALSE);  // measure only the visible page
    gtk_stack_set_vhomogeneous(GTK_STACK(stack), FALSE);

    GtkWidget *hint = gtk_label_new("No tabs open");
    gtk_widget_add_css_class(hint, "dim-label");
    gtk_stack_add_named(GTK_STACK(stack), hint, "empty");

    GtkWidget *notebook = gtk_notebook_new();
    gtk_notebook_append_page(GTK_NOTEBOOK(notebook), gtk_label_new("A page"),
                             gtk_label_new("Tab 1"));
    gtk_stack_add_named(GTK_STACK(stack), notebook, "tabs");

    gtk_stack_set_visible_child_name(GTK_STACK(stack), "empty");
    *stack_out = stack;
    return stack;
}

static void on_activate(GtkApplication *app, gpointer) {
    GtkWidget *win = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(win), "gtklab");
    gtk_window_set_default_size(GTK_WINDOW(win), 900, 600);
    g_signal_connect(win, "close-request", G_CALLBACK(on_close_request), nullptr);

    GtkWidget *stack = nullptr;
    GtkWidget *content = build_content(&stack);

    GtkWidget *header = gtk_header_bar_new();
    GtkWidget *flip = gtk_button_new_with_label("Flip");
    g_signal_connect(flip, "clicked", G_CALLBACK(on_flip_clicked), stack);
    gtk_header_bar_pack_start(GTK_HEADER_BAR(header), flip);
    gtk_window_set_titlebar(GTK_WINDOW(win), header);

    GtkWidget *paned = gtk_paned_new(GTK_ORIENTATION_HORIZONTAL);
```

```
    gtk_paned_set_start_child(GTK_PANED(paned), build_sidebar());
    gtk_paned_set_end_child(GTK_PANED(paned), content);
    gtk_paned_set_shrink_start_child(GTK_PANED(paned), FALSE);
    gtk_paned_set_resize_start_child(GTK_PANED(paned), FALSE);
    gtk_paned_set_position(GTK_PANED(paned), 260);

    gtk_window_set_child(GTK_WINDOW(win), paned);
    gtk_window_present(GTK_WINDOW(win));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(on_activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

The paned's two settings for its start child work together. `gtk_paned_set_resize_start_child(paned, FALSE)` means that when the window grows, the extra space goes to the content, not the sidebar. `gtk_paned_set_shrink_start_child(paned, FALSE)` means the divider cannot be dragged past the sidebar's minimum width.

The stack has homogeneous sizing turned off in both directions. By default a stack is as big as its biggest page, so it measures every page even when one is hidden. That sounds harmless, and is, until a page is a notebook with no tabs: GTK 4.14 then logs `GtkGizmo (tabs) reported min height -3, but sizes must be >= 0` every time the last tab closes. Measuring only the visible page avoids it.

The Flip button is temporary scaffolding; Chapter 7 replaces it with an action.

Done when

The window has a header bar with a Flip button, the sidebar holds a search entry, the Flip button switches between the hint and the notebook, and the divider cannot be dragged so far left that the sidebar disappears.

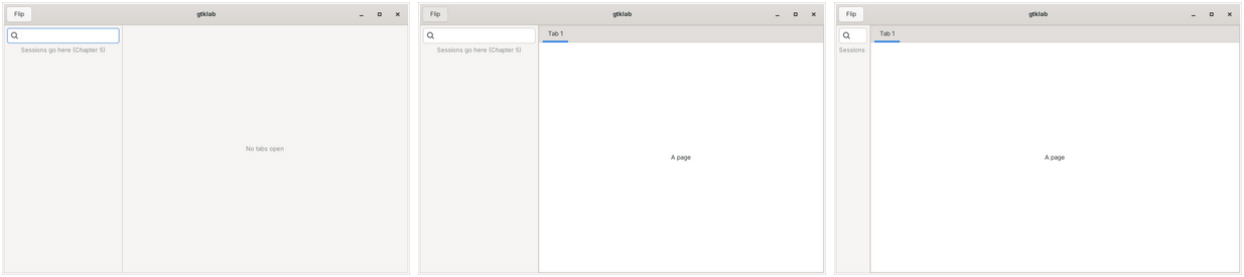


Figure 3: Left: the empty state. Middle: Flip shows the notebook. Right: the divider dragged as far left as it will go; the sidebar stops at its minimum width.

Debugger drill

Run **gtklab + GTK Inspector**. In the Inspector's object tree, find the `GtkPaned` and change its `position` property: the divider moves as you type. Select the `GtkStack` and change `visible-child-name` to `tabs` by hand. Then select the search entry and look at its allocation and its `hexpand` and `vexpand` values. Everything `on_activate` built is there, live.

Break it

Delete the `gtk_paned_set_shrink_start_child(..., FALSE)` line and drag the divider fully left. The sidebar collapses to nothing, search entry and all, and there is no way to tell from the window that it was ever there.

Figure 4: The Break it: with shrinking allowed, the divider swallows the sidebar.

Common mistakes

Looking for `gtk_widget_show_all`. It is gone, and not needed.

Calling `gtk_container_add`. Gone too. Each container has its own call: `gtk_box_append`, `gtk_window_set_child`, `gtk_paned_set_start_child`, `gtk_scrolled_window_set_child`.

A scrolled window without `vexpand`. It gets its natural height, which is tiny.

Setting sizes in pixels. `gtk_widget_set_size_request` sets a minimum, not a size, and fights the user's font size. Prefer expansion and alignment.

Read in tmssh

`build_ui`, which creates the header bar, paned, stack and notebook, and `build_sidebar`, which builds the search entry, an error banner and the session list inside a scrolled window.

Review

1. Why does GTK measure from the bottom of the tree up but allocate from the top down?
2. A label in a vertical box has `vexpand` set and `valign` set to `START`. How much space does it get, and where does its text appear?
3. Nobody calls `gtk_widget_set_vexpand` on gtklab's sidebar box. Why does it still report that it wants spare vertical space?
4. A scrolled window in a vertical box shows a strip a few pixels high. What is its natural height based on, and what is the one-line fix?
5. Why is a stack better than removing the notebook and adding a label in its place when the last tab closes?
6. The stack's hidden page is wider than the pane. With homogeneous sizing on, what happens to the visible page, and why?

7. What do `resize` and `shrink` mean for a paned's start child, and why are the defaults wrong for a sidebar?
8. `gtk_widget_set_size_request(sidebar, 170, -1)` is set, yet the sidebar can still be dragged to nothing. Why does the size request not prevent it?

Questions for deeper thought. A tool's window will be used on a laptop screen and on a wide monitor. Which regions of your layout should absorb extra space, which should hold steady, and which should the user control? When would you let the user hide a region entirely, and how would they get it back if the divider is the only handle?

Exercises

1. **A footer button.** In `build_sidebar` in `ch03-layout/src/main.cpp`, append a "New session" button after the scroller. Done when the button stays pinned to the bottom of the sidebar as the window grows taller.
2. **A third page.** In `build_content`, add a third stack page named `settings` holding a label, and change `on_flip_clicked` to cycle `empty`, `tabs`, `settings`. Done when three clicks of Flip return to the hint.
3. **Hide the sidebar.** In `on_activate`, add a second header-bar button whose handler toggles the sidebar with `gtk_widget_set_visible`. Done when the content takes the full width while the sidebar is hidden and the sidebar comes back at its old width.
4. **Animate the switch.** In `build_content`, call `gtk_stack_set_transition_type` with `GTK_STACK_TRANSITION_TYPE_` Done when Flip slides between the pages instead of cutting.

Isolated lab

gtklab's window has a header bar, a paned, a sidebar and a stack in it, and all of them take part in layout. The lab strips that down to the three decisions this chapter is about: a box that stacks a toggle button over a paned, a paned with a fixed-purpose sidebar, and a stack with two pages. Every container keeps its default settings except the one `vexpand`, so what you see is GTK's default policy, and each experiment changes one line.

labs/lab03-layout/src/main.cpp

```
// Lab 3: layout policy in isolation. A box, a paned and a stack; drag the
// divider, resize the window and flip pages to see who decides the sizes.
#include <gtk/gtk.h>

static void on_toggled(GtkToggleButton *button, gpointer user_data) {
    GtkStack *stack = GTK_STACK(user_data);
    gboolean active = gtk_toggle_button_get_active(button);
    const char *page = active ? "details" : "welcome";
    g_print("Switching to page: %s\n", page);
    gtk_stack_set_visible_child_name(stack, page);
}

static void on_activate(GtkApplication *app, gpointer) {
    GtkWidget *window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "Layout Laboratory");
    gtk_window_set_default_size(GTK_WINDOW(window), 640, 360);

    GtkWidget *root = gtk_box_new(GTK_ORIENTATION_VERTICAL, 12);
    gtk_widget_set_margin_top(root, 16);
    gtk_widget_set_margin_bottom(root, 16);
```

```
    gtk_widget_set_margin_start(root, 16);
    gtk_widget_set_margin_end(root, 16);
    gtk_window_set_child(GTK_WINDOW(window), root);

    GtkWidget *toggle = gtk_toggle_button_new_with_label("Show details");
    gtk_box_append(GTK_BOX(root), toggle);

    GtkWidget *paned = gtk_paned_new(GTK_ORIENTATION_HORIZONTAL);
    gtk_widget_set_vexpand(paned, TRUE);
    gtk_box_append(GTK_BOX(root), paned);

    GtkWidget *sidebar = gtk_label_new("Sidebar: fixed purpose");
    gtk_widget_set_size_request(sidebar, 170, -1);
    gtk_paned_set_start_child(GTK_PANED(paned), sidebar);

    GtkWidget *stack = gtk_stack_new();
    gtk_stack_add_named(GTK_STACK(stack), gtk_label_new("Welcome page"), "welcome");
    gtk_stack_add_named(GTK_STACK(stack), gtk_label_new("Details page"), "details");
    gtk_stack_set_visible_child_name(GTK_STACK(stack), "welcome");
    gtk_paned_set_end_child(GTK_PANED(paned), stack);
    gtk_paned_set_position(GTK_PANED(paned), 190);

    g_signal_connect(toggle, "toggled", G_CALLBACK(on_toggled), stack);
    gtk_window_present(GTK_WINDOW(window));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab.lab03",
                                              G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(on_activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

The sidebar has a 170-pixel size request and the paned starts at 190, so the divider starts just right of the sidebar's minimum. The stack is left homogeneous, unlike gtklab's.

Build and run from `labs/lab03-layout`:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

Or open the directory in VS Code and use the **lab** launch configuration; **lab + GTK Inspector** is the one for this lab. Clicking **Show details** prints `Switching to page: details` and shows the second page.

Stop at	What it shows
`on_activate`, after `gtk_paned_set_position`	the whole tree is built and nothing is on screen: layout has not run yet
`on_toggled`	`active` is the toggle's new state; `user_data` is the stack, a borrowed pointer
`gtk_stack_set_visible_child_name` (gdb, with debuginfod)	GTK changing the visible child and queueing a resize, not laying out on the spot
Inspector: select the `GtkPaned`	its `position`, and the `shrink-start-child` and `resize-start-child` values the lab never set

Experiments:

1. **Drag the divider fully left.** The sidebar collapses to nothing, 170-pixel size request and all, because `shrink-start-child` defaults to on. Then add `gtk_paned_set_shrink_start_child(GTK_PANED(paned), FALSE)` and drag again: the divider stops at the sidebar's minimum.
2. **Remove `gtk_widget_set_vexpand(paned, TRUE)`.** The paned shrinks to a single line under the toggle and the rest of the window is empty. The root box gives spare space only to children that ask for it.
3. **Make a hidden page wide.** Change the details label to a sentence longer than the pane is wide and run without clicking anything. "Welcome page" is no longer centred: it sits right of centre, because the homogeneous stack is allocated the hidden page's minimum width and the pane shows only the left part of it. Add `gtk_stack_set_hhomogeneous(GTK_STACK(stack), FALSE)` and "Welcome page" is centred again.
4. **Resize the window wider.** With the defaults, both sides of the paned grow. Add `gtk_paned_set_resize_start_child(GTK_PANED(paned), FALSE)` and only the stack grows.

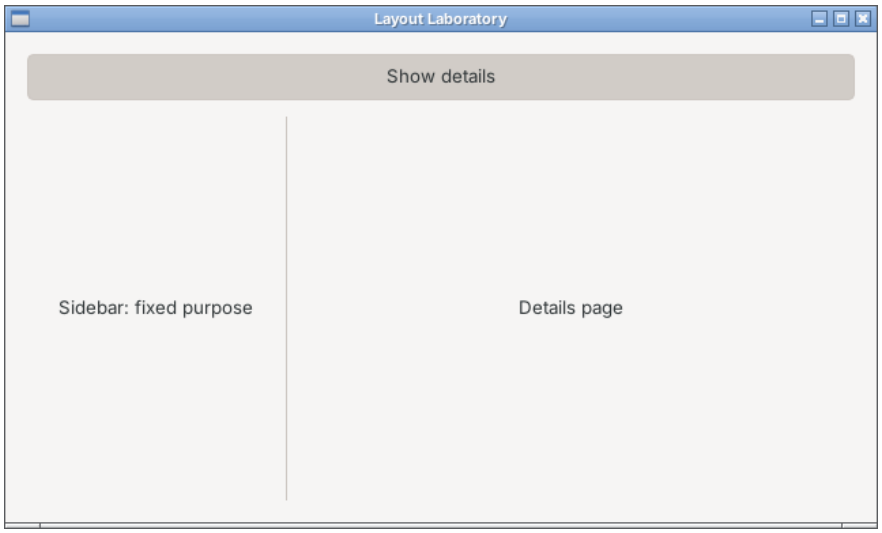


Figure 5: Lab 3 after Show details: the toggle above, the sidebar and the stack's details page below.

Checkpoint A — A shell that remembers

Spec. Chapter 3's window, plus its size saved when it closes and restored when it opens. Store it in a `GKeyFile` at `g_get_user_config_dir()/gtklab/settings.ini`, creating the directory with `g_mkdir_with_parents`. Save in `close-request`; load before `gtk_window_set_default_size`.

Done when. You resize the window, quit and relaunch, and it comes back at the same size. Deleting the settings file brings back 900 by 600 with no warning printed. In the recorded test the window was resized to 1100 by 700, the file read `width=1100` and `height=700`, and the relaunched window measured 1100 by 700; with the file deleted it opened at 900 by 600.

Hints. `g_key_file_get_integer` reports a missing key through its `GError`; treat that as "use the default", not as a failure. When you save, load the existing file first with `G_KEY_FILE_KEEP_COMMENTS`, so saving the size does not erase other settings later chapters add.

Reference. `ckA-remember/src/main.cpp`. Compare it with tmssh's `on_close_request` and the settings load in the `AppWindow` constructor. Chapter 4 moves settings out of GTK code entirely.

Chapter 4 — Keep GTK out of the logic

Before gtklab shows a single session, this chapter writes the code that reads sessions, builds ssh command lines and loads settings, and tests it, without a window. It comes before the list view chapters on purpose. Chapters 5 and 6 display this model; they do not own it.

In this chapter

- Decide, function by function, whether code belongs in a GTK-free core or in the GTK layer.
- Build a core static library that the build itself keeps free of GTK.
- Write GTK callbacks as adapters: turn the event into a core call, then refresh the view from the core.
- Test parsing, command building and settings with doctest, with no window and no main loop.

Concepts

Two kinds of code

A desktop tool has two kinds of code. Some of it needs a display: windows, widgets, the clipboard, drawing. Most of it does not: parsing a file, validating it, turning a session into a command line, clamping a setting to a sane range. Put the second kind in a static library that never includes `gtk/gtk.h`, and three things follow.

The logic gets tests that run in milliseconds, with no window and no main loop, so most bugs are found by a test runner instead of by clicking. The logic can be stepped through in a debugger without GTK on the stack at all. And the same code is ready for the next front end a tool tends to grow: a command-line mode, a script, a scheduled job. None of them should have to open a window to build an ssh command line.

The test for which side a function belongs on is what it needs to know, not how long it is:

Core (no GTK)	GTK layer
parse and validate `sessions.ini`	build the sidebar and the notebook
build the argv for a session	connect signals, decide what a click does
filter sessions by search text	read the search entry's text
load, clamp and save settings	read the window's size in `close-request`
report errors as values	show an error to the user

Each row is one job split at the boundary. The right column asks GTK a question or shows an answer; the left column computes the answer.

Dependencies point one way

The GTK layer depends on the core. The core never depends on the GTK layer, and the build enforces it: the core target is not given GTK, so an accidental `#include <gtk/gtk.h>` in it does not compile. A rule the compiler checks survives deadlines and new contributors; a rule in a comment does not.

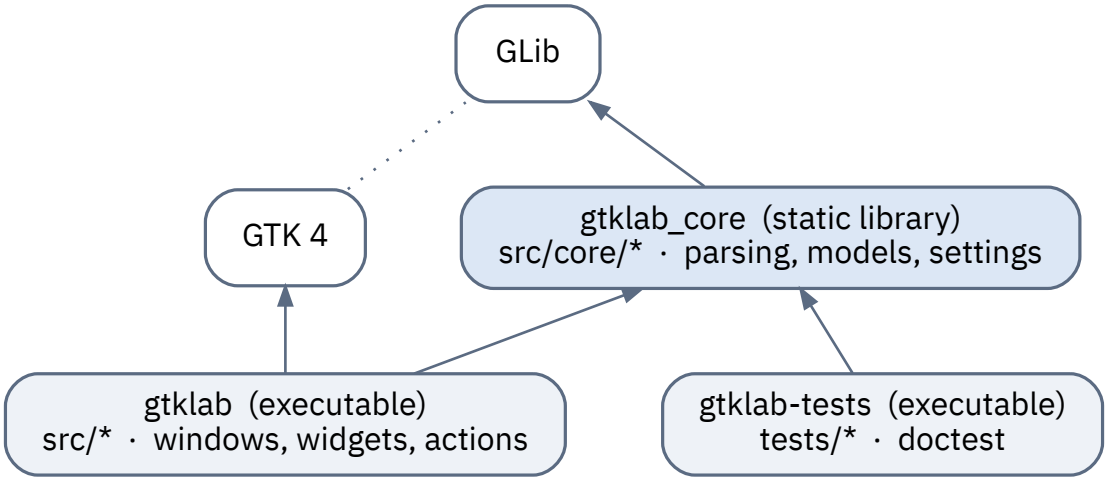


Figure 1: The build targets from this chapter on. The core library depends on GLib only; the application adds GTK; the tests link only the core.

GLib is allowed in the core. It is not a GUI library; it is GTK's portable C runtime, and it brings things a tool needs anyway: `GKeyFile` for INI-style files, `g_file_get_contents` and `g_file_set_contents`, path helpers, and UTF-8 functions. The line is drawn at the display, not at GNOME.

The callback is an adapter

GTK is event-driven, and a callback is how it tells you something happened. The mistake is to let the callback become the place where the work lives: it reads an entry, parses the text, writes a file, updates three labels. Each feature adds a reason to change it, and soon nothing useful can happen without a window.

The alternative is a callback that only translates. It turns the event into a call on the core, then refreshes the view from what the core now holds. It knows both GTK and the core; the core knows nothing of GTK.

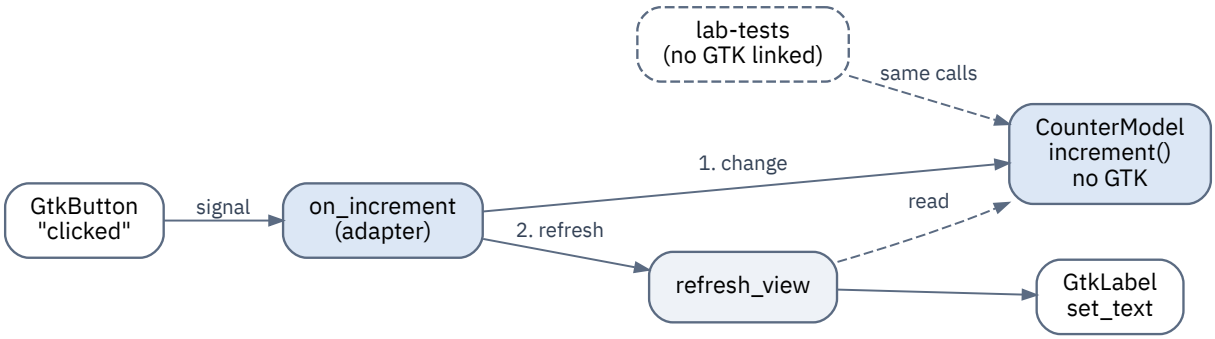


Figure 2: A callback as an adapter, from this chapter's Isolated lab. The click changes the model, then the view is redrawn from the model; the tests call the same model with no GTK at all.

Two kinds of state follow from this. **Model state** (the sessions, the settings, a count) stays meaningful with no window open. **View state** (a label's text, a list row) is a presentation of it. The model is the source of truth; a label is output, never storage. A callback that parses a label to find out what the count was has the arrow backwards.

What makes core code testable

A core function is easy to test when everything it depends on comes in through its parameters and everything it produces goes out through its return value. Hidden inputs (an environment variable, the current directory, the clock) make a test pass on one machine and fail on another; hidden outputs (a dialog, a `g_warning`) make a failure invisible to the test. The Build section shows two places where gtklab's core was shaped by this rule.

Core tests and GUI checks answer different questions. A core test asks whether the behaviour is right: does a session on port 2222 produce `-p 2222`? A check through the window asks whether GTK is wired to that behaviour: does a double-click start the right session? The first kind is cheap, so write many; the second is slow, so write few, and only for the wiring.

How the rest of the book uses the core

Chapters 5 and 6 display the core's `Folder` and `Session` values without owning them: a small GObject wrapper holds a copy so a GTK list can show it. From this chapter on, the window loads and saves `Settings` through the core; the fields later chapters need, such as the font size and the terminal theme, are already declared in this chapter's struct, with their defaults. tmssh has the same shape at a larger size: its core library holds session parsing, the ssh command builder, settings, themes and the session-log cleaner, each with tests.

Build

The session model

The interface, complete:

ch04-core/src/core/session.h

```
// Session model: plain C++ plus GLib's GKeyFile. Never includes GTK.
#pragma once

#include <string>
#include <vector>

struct Session {
    std::string name;
    std::string folder;
    std::string host;
    std::string user;
    int port = 22;
    bool local = false;
};

struct Folder {
    std::string name;
    std::vector<Session> sessions;
};

// Parse a sessions file held in memory. One [group] per session:
//
//   [core1]
//   folder=Core
//   host=10.0.0.1
//   user=admin
//   port=2222
//
//   [localhost]
//   folder=Local
//   local=true
//
// Folders and sessions keep file order. On a parse error, returns an empty
// list and sets `error`.
std::vector<Folder> parse_sessions(const std::string &text, std::string &error);

// Read and parse a sessions file from disk.
std::vector<Folder> load_sessions(const std::string &path, std::string &error);

// The text of a first-run sessions file: one folder with a localhost entry.
std::string default_sessions_text();

// argv for the session: ssh for remote sessions, the user's shell for local.
// `shell` is passed in (normally $SHELL) so tests do not depend on the host.
std::vector<std::string> ssh_argv(const Session &s, const std::string &shell = "/bin/sh");

// Folders filtered to sessions whose name, host or user contains `text`
// (case-insensitive). Empty text returns the input. Folders with no
// matching sessions are dropped.
std::vector<Folder> filter_sessions(const std::vector<Folder> &folders, const std::string &text);
```

Two design choices are worth copying. Errors come back as a string in an out parameter, with an empty result, rather than as exceptions crossing into C callbacks later. And `ssh_argv` takes the shell as a parameter instead of reading `$SHELL` itself, so a test does not depend on the machine running it.

The parser is built on `GKeyFile`. Note how a missing `port` key and a `port` key that is not a number are told apart by the `GError` code:

ch04-core/src/core/session.cpp

```
std::vector<Folder> parse_sessions(const std::string &text, std::string &error) {
    std::vector<Folder> folders;
    error.clear();
```

```
    GKeyFile *kf = g_key_file_new();
    GError *err = nullptr;
    if (!g_key_file_load_from_data(kf, text.c_str(), text.size(), G_KEY_FILE_NONE, &err)) {
        error = err->message;
        g_error_free(err);
        g_key_file_unref(kf);
        return folders;
    }

    gsize n = 0;
    char **groups = g_key_file_get_groups(kf, &n);
    for (gsize i = 0; i < n; i++) {
        Session s;
        s.name = groups[i];
        s.folder = get_string(kf, groups[i], "folder");
        if (s.folder.empty())
            s.folder = "Sessions";
        s.host = get_string(kf, groups[i], "host");
        s.user = get_string(kf, groups[i], "user");
        s.local = g_key_file_get_boolean(kf, groups[i], "local", nullptr);

        GError *perr = nullptr;
        int port = g_key_file_get_integer(kf, groups[i], "port", &perr);
        if (perr == nullptr) {
            if (port < 1 || port > 65535) {
                error = "session '" + s.name + "': port out of range";
                folders.clear();
                break;
            }
            s.port = port;
        } else if (perr->code == G_KEY_FILE_ERROR_INVALID_VALUE) {
            error = "session '" + s.name + "': port is not a number";
            g_error_free(perr);
            folders.clear();
            break;
        } else {
            g_error_free(perr);  // key missing: keep the default 22
        }

        if (!s.local && s.host.empty()) {
            error = "session '" + s.name + "': no host";
            folders.clear();
            break;
        }

        auto it = std::find_if(folders.begin(), folders.end(),
                               [&](const Folder &f) { return f.name == s.folder; });
        if (it == folders.end()) {
            folders.push_back(Folder{s.folder, {}});
            it = folders.end() - 1;
        }
        it->sessions.push_back(std::move(s));
    }
    g_strfreev(groups);
    g_key_file_unref(kf);
    return folders;
}
```

The command builder is the piece most worth testing, because a wrong argument in it produces an ssh that connects to the wrong place:

ch04-core/src/core/session.cpp

```
std::vector<std::string> ssh_argv(const Session &s, const std::string &shell) {
    if (s.local)
        return {shell.empty() ? "/bin/sh" : shell};

    std::vector<std::string> argv{"ssh"};
    if (s.port != 22) {
        argv.push_back("-p");
        argv.push_back(std::to_string(s.port));
    }
    argv.push_back(s.user.empty() ? s.host : s.user + "@" + s.host);
    return argv;
}
```

tmssh reads YAML in PathfinderSSH's format with yaml-cpp. gtklab uses `GKeyFile` so that its core needs nothing beyond GLib; the shape of the code is the same.

Settings

`Settings` replaces Checkpoint A's ad-hoc key file. Later chapters use its fields, and the struct is the only place their defaults live:

ch04-core/src/core/settings.h

```
// App settings: plain C++ plus GLib's GKeyFile. Never includes GTK.
#pragma once

#include <string>

struct Settings {
    int width = 900;
    int height = 600;
    int sidebar_position = 260;
    bool sidebar_visible = true;
    std::string app_variant = "light";     // "light" | "dark"
    std::string terminal_theme = "dark";   // a theme name (Chapter 15)
    int font_size = 11;                    // points (Chapter 14)
};

// $XDG_CONFIG_HOME/gtklab/<name> (normally ~/.config/gtklab/<name>).
std::string config_path(const std::string &name);

// Missing file or missing keys fall back to the defaults above. Values out of
// range are clamped, so a hand-edited file cannot produce a 10x10 window.
Settings parse_settings(const std::string &text);
Settings load_settings(const std::string &path);

std::string settings_to_text(const Settings &s);
bool save_settings(const std::string &path, const Settings &s, std::string &error);
```

Two details in `settings.cpp` matter for a tool people hand-edit. Values are clamped on load, so a settings file that says `width=10` gives a usable window rather than a ten-pixel one. And saving goes through `g_file_set_contents`, which writes a temporary file and renames it over the old one, so a crash in the middle of a save never leaves half a file.

Tests

doctest is a single header. Copy `doctest.h` into `tests/` rather than fetching it when the build is configured: the build then works offline, which matters on a jump host. Eleven tests cover the core; here are

three:

ch04-core/tests/test_session.cpp

```
TEST_CASE("ssh argv: custom port") {
    auto argv = ssh_argv(remote("core1", "admin", 2222));
    CHECK(argv == std::vector<std::string>{"ssh", "-p", "2222", "admin@core1"});
}
```

ch04-core/tests/test_session.cpp

```
TEST_CASE("parse: malformed files return an error, not a crash") {
    std::string err;
    CHECK(parse_sessions("this is not a key file\n", err).empty());
    CHECK_FALSE(err.empty());

    CHECK(parse_sessions("[x]\nhost=h\nport=abc\n", err).empty());
    CHECK(err.find("port") != std::string::npos);

    CHECK(parse_sessions("[x]\nhost=h\nport=70000\n", err).empty());
    CHECK(err.find("range") != std::string::npos);

    CHECK(parse_sessions("[x]\nfolder=F\n", err).empty());
    CHECK(err.find("no host") != std::string::npos);
}
```

ch04-core/tests/test_settings.cpp

```
TEST_CASE("settings: hand-edited nonsense is clamped") {
    Settings s = parse_settings("[window]\nwidth=10\nheight=abc\n[appearance]\napp-variant=purple\n");
    CHECK(s.width == 400);
    CHECK(s.height == 600);
    CHECK(s.app_variant == "light");
}
```

meson.build

ch04-core/meson.build

```
project('gtklab', 'cpp',
  version: '0.1.0',
  default_options: ['cpp_std=c++17', 'warning_level=2'])

gtk = dependency('gtk4', version: '>= 4.8')
glib = dependency('glib-2.0', version: '>= 2.74')

gtk_floor_args = [
  '-DGDK_VERSION_MIN_REQUIRED=GDK_VERSION_4_8',
  '-DGDK_VERSION_MAX_ALLOWED=GDK_VERSION_4_8',
  '-DGLIB_VERSION_MIN_REQUIRED=GLIB_VERSION_2_74',
  '-DGLIB_VERSION_MAX_ALLOWED=GLIB_VERSION_2_74',
  '-Werror=deprecated-declarations',
]
add_project_arguments(gtk_floor_args, language: 'cpp')

inc = include_directories('src')

# The core: no GTK. Everything here is testable without a display.
core = static_library('gtklab_core',
  'src/core/session.cpp',
  'src/core/settings.cpp',
  include_directories: inc,
  dependencies: glib)
```

```
core_dep = declare_dependency(link_with: core,
  include_directories: inc,
  dependencies: glib)

executable('gtklab', 'src/main.cpp', dependencies: [gtk, core_dep])

# doctest is one header, vendored in tests/ (MIT licence).
tests = executable('gtklab-tests',
  'tests/test_main.cpp',
  'tests/test_session.cpp',
  'tests/test_settings.cpp',
  dependencies: core_dep)
test('core', tests)
```

Note that the core library itself is given the include directory, not only the targets that depend on it. `session.cpp` includes `core/session.h`, so the core needs `src/` on its include path too; the first version of this file left it out and failed with `core/settings.h: No such file or directory`.

The window uses the core

`main.cpp` now loads settings through the core, creates a default sessions file on first run, parses it and prints what it found:

ch04-core/src/main.cpp

```cpp
static void ensure_sessions_file(const std::string &path) {
    if (g_file_test(path.c_str(), G_FILE_TEST_EXISTS))
        return;
    char *dir = g_path_get_dirname(path.c_str());
    g_mkdir_with_parents(dir, 0700);
    g_free(dir);
    const std::string text = default_sessions_text();
    g_file_set_contents(path.c_str(), text.c_str(), static_cast<gssize>(text.size()), nullptr);
}
```

Done when

`meson test -C build-debug` passes, and the first run creates `~/.config/gtklab/sessions.ini` with one Local folder holding localhost, and prints `folder Local: 1 sessions`.

Debugger drill

Run the **unit tests** launch configuration. It passes `-tc=*argv*`, so four of the eleven tests run. Set a breakpoint inside `ssh_argv`. When it stops, look at the stack: `main`, doctest's runner, your test, `ssh_argv`. No window, no main loop, no GTK. Compare it with the stack from Chapter 1.

Break it

Add `#include <gtk/gtk.h>` to the top of `session.cpp` and build. The core does not compile: `fatal error: gtk/gtk.h: No such file or directory`. The core target was never given GTK, so the boundary holds by construction. That error is the split doing its job.

Common mistakes

A GTK type in a core header. One `GtkWidget *` in `session.h` and every test needs GTK. Keep core headers to standard C++ and GLib.

Testing through the window. If the only way to check the ssh command is to click a session, every change costs a minute. Test it in the core in a millisecond.

Throwing across C callbacks. An exception that escapes a GTK callback unwinds through C frames that were not built for it. Report errors as values.

Read in tmssh

`tmssh_core` in tmssh's `meson.build`: the library holding `sessions.cpp`, `ssh_command.cpp`, `settings.cpp`, `themes.cpp` and the session-log cleaner. Then `tests/test_sessions.cpp` and `tests/test_ssh_command.cpp`.

Review

1. A function formats a session as `user@host:port` for the sidebar's tooltip. Core or GTK layer? What about the function that sets the tooltip?
2. What stops an `#include <gtk/gtk.h>` from creeping into `session.cpp`, and why is that better than a comment saying "no GTK here"?
3. GLib is allowed in the core and GTK is not. Where exactly is the line drawn, and why there?
4. `ssh_argv` takes the shell as a parameter instead of reading `$SHELL`. What would go wrong in the tests if it read the environment itself?
5. Why does the core report a bad sessions file as a string in an out parameter rather than by throwing?
6. In the lab, `on_increment` changes the model and then calls `refresh_view`. What would be wrong with reading the number back from the label's text and adding one?
7. Which questions can a core test answer that a click-through test cannot, and which can only the click-through answer?
8. Chapter 5 shows sessions in a list. Why does it wrap them in a GObject instead of making `Session` itself a GObject?

Questions for deeper thought. Take a tool you already have that only works with its window open. Which of its functions would move to a core library, and which piece would be hardest to separate? If the same core had to serve a command-line mode tomorrow, what in its interface would you change today?

Exercises

1. **An identity file.** Add an `identity` field to `Session` in `ch04-core/src/core/session.h`, read it in `parse_sessions`, and make `ssh_argv` add `-i <path>` when it is set. Add a test case in `tests/test_session.cpp`. Done when `meson test -C build-debug` passes with the new test and the existing eleven.
2. **Search the folder name.** Change `filter_sessions` so text matching a folder's name keeps every session in that folder. Write the failing test first. Done when the new test passes and "filter: case-insensitive on name, host and user; empty folders dropped" still does.

3. **A clamped setting.** Add a `scrollback` field to `Settings` (default 10000, clamped to 100 to 100000) in `settings.h`, `parse_settings` and `settings_to_text`, and extend "settings: hand-edited nonsense is clamped". Done when a file saying `scrollback=5` loads as 100.
4. **Print the commands.** In `on_activate` in `ch04-core/src/main.cpp`, print each session's `ssh_argv`, joined by spaces, passing `g_getenv("SHELL")` (or an empty string when it is unset). Done when the first run prints your shell's path for the localhost session.

Isolated lab

gtklab's Chapter 4 split is real but busy: two core files, eleven tests, settings on disk. The lab isolates the one idea underneath it, the callback as an adapter, in a program small enough to hold in your head. A `CounterModel` knows how to count and describe itself and nothing else; the window knows how to show a label and two buttons; two callbacks connect them. It is a separate program so that the boundary is the only thing on the screen.

The model, header-only:

labs/lab04-boundary/src/core/counter_model.h

```
#pragma once
// Pure application logic: no GTK, no GLib. The meson target that builds the
// tests is not given GTK, so an #include <gtk/gtk.h> here would not compile.
#include <string>

class CounterModel {
public:
    int value() const { return count_; }
    void increment() { ++count_; }
    void reset() { count_ = 0; }
    std::string description() const { return "Operations recorded: " + std::to_string(count_); }

private:
    int count_ = 0;
};
```

The window. `WindowState` holds the model and a borrowed pointer to the label, and the window owns the state through `g_object_set_data_full`, the pattern from Chapter 2:

labs/lab04-boundary/src/main.cpp

```
// Lab 4: the callback is an adapter. It turns a GTK signal into a model call,
// then refreshes the view from the model. The model knows nothing of GTK.
#include <gtk/gtk.h>
#include <string>

#include "core/counter_model.h"

struct WindowState {
    CounterModel model;
    GtkWidget *label = nullptr;  // borrowed: the window's widget tree owns it
};

static void refresh_view(WindowState *state) {
    std::string text = state->model.description();
    gtk_label_set_text(GTK_LABEL(state->label), text.c_str());  // the label copies the text
}

static void on_increment(GtkButton *, gpointer data) {
    auto *state = static_cast<WindowState *>(data);
    state->model.increment();
```

```
    refresh_view(state);
}

static void on_reset(GtkButton *, gpointer data) {
    auto *state = static_cast<WindowState *>(data);
    state->model.reset();
    refresh_view(state);
}

static void on_activate(GtkApplication *app, gpointer) {
    GtkWidget *window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "Separation Lab");
    gtk_window_set_default_size(GTK_WINDOW(window), 420, 180);

    // The window owns the state: it is deleted when the window is finalized,
    // after its buttons (and their handlers) are gone.
    auto *state = new WindowState;
    g_object_set_data_full(G_OBJECT(window), "state", state,
                           [](gpointer p) { delete static_cast<WindowState *>(p); });

    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 12);
    gtk_widget_set_margin_top(box, 20);
    gtk_widget_set_margin_bottom(box, 20);
    gtk_widget_set_margin_start(box, 20);
    gtk_widget_set_margin_end(box, 20);
    gtk_window_set_child(GTK_WINDOW(window), box);

    state->label = gtk_label_new("");
    gtk_box_append(GTK_BOX(box), state->label);

    GtkWidget *buttons = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    gtk_widget_set_halign(buttons, GTK_ALIGN_CENTER);
    gtk_box_append(GTK_BOX(box), buttons);
    GtkWidget *increment = gtk_button_new_with_label("Increment");
    GtkWidget *reset = gtk_button_new_with_label("Reset");
    gtk_box_append(GTK_BOX(buttons), increment);
    gtk_box_append(GTK_BOX(buttons), reset);

    g_signal_connect(increment, "clicked", G_CALLBACK(on_increment), state);
    g_signal_connect(reset, "clicked", G_CALLBACK(on_reset), state);
    refresh_view(state);
    gtk_window_present(GTK_WINDOW(window));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab.lab04", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(on_activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

The test, which links no GTK:

labs/lab04-boundary/tests/test_counter.cpp

```
// Runs with no display and no GTK: `meson test -C build-debug`.
#include "core/counter_model.h"
#include <cassert>
#include <cstdio>

int main() {
    CounterModel model;
    assert(model.value() == 0);
```

```
    model.increment();
    model.increment();
    assert(model.value() == 2);
    assert(model.description() == "Operations recorded: 2");
    model.reset();
    assert(model.value() == 0);
    std::puts("counter model: all checks passed");
    return 0;
}
```

labs/lab04-boundary/meson.build

```
# The core is header-only here. The test target is given no GTK dependency,
# so the boundary is checked by the compiler.
core_inc = include_directories('src')

executable('lab', 'src/main.cpp', include_directories: core_inc, dependencies: gtk)

test('counter model',
  executable('lab-tests', 'tests/test_counter.cpp', include_directories: core_inc))
```

Build, run and test from `labs/lab04-boundary`:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
meson test -C build-debug
```

In VS Code, **unit tests (no GTK)** runs `lab-tests` under the debugger.

Stop at	What it shows
`on_increment`	the click arriving as a signal; step into `increment()` and the stack below the model is the adapter, then GTK
`refresh_view`	the label's text derived from `model.description()`; `gtk_label_set_text` copies it, so the `std::string` can go
the lambda in `on_activate` that deletes `WindowState`	it runs only when the window closes, called from `g_datalist_clear` inside `g_object_unref`
`main` in `test_counter.cpp` (**unit tests (no GTK)**)	the same `CounterModel` calls with no GTK anywhere on the stack

Experiments:

1. **Break the boundary.** Add `#include <gtk/gtk.h>` to the top of `counter_model.h` and build. `lab` still compiles, because it is given GTK; `lab-tests` fails with `fatal error: gtk/gtk.h: No such file or directory`. The test target is the guard.
2. **Add a rule to the model.** Add `decrement()`, and decide in the model whether the count may go negative. Add a Decrement button whose handler has the same two lines as `on_increment`, and an `assert` in the test. The callback should not need an `if`.
3. **Store state in the view.** Rewrite `on_increment` to parse the number out of the label's text and write it back, without the model. Then ask how a test would check it, and how Reset would work if the label's wording changed.

4. **Watch the state die.** Put a breakpoint on the delete lambda, click Increment three times and close the window. Read the stack from the bottom: the window closing, the window's last reference going, its data cleared, your lambda.



Figure 3: Lab 4 after three clicks on Increment.

Part II: Building Tools

Part II turns the empty shell into a tool. It shows sessions in a list, then in a filterable tree of folders; puts every command behind an action with a menu entry and a keyboard shortcut; opens tabs that own their resources; handles mouse and keyboard input in the right order; and adds context menus and dialogs that do not block. Two checkpoints close it: a session browser, and a tabbed window whose every shortcut is in a menu.

From Chapter 5 on, the window's state lives in a C++ class, `AppWindow`, instead of in globals, and `main.cpp` shrinks to creating it.

Chapter 5 — List views I: models and factories

The sidebar finally shows sessions. This chapter puts every session from Chapter 4's model into a `GtkListView`, which means learning the list widget design that replaced GTK 3's `GtkTreeView`, and writing your first GObject class.

Recall

Which layer owns the session data now (Chapter 4)? What does the floating reference on a new widget mean (Chapter 2)?

In this chapter

- Name the four pieces of a GTK 4 list (model, selection, view, factory) and say which one to look at for a given bug.
- Write a factory that builds row widgets once and refills them as they are recycled.
- Follow every reference in the model chain, and know which constructors take ownership of their argument.
- Define a final GObject type that carries plain C++ data into a `GListModel`.

Concepts

Why a list needs a model

The obvious first sidebar is a box with one label per session. It works for a dozen sessions. Then a session is renamed, the list is filtered, a row is selected, and the question becomes: which copy is the truth? The label's text, the vector in `AppWindow`, or the file on disk? Every feature added to a widget-per-item list has to answer that again.

GTK 4's list widgets answer it once. The data lives in a model. The view shows whatever the model holds and never stores anything of its own. When the model changes, it emits `items-changed` with a position and the number of items removed and added, and that signal is the only way the view learns anything. Chapter 4 already put the sessions in a GTK-free core; the model here is a thin adapter over it, not a second copy of the truth.

Four objects, one job each

GTK 4's list widgets separate four things that GTK 3 tangled together.

The **model** holds the data. It is any object implementing the `GListModel` interface: a list of GObjects that can tell you how many items it has, give you one by position, and announce when items change. GLib's `GListStore` is the general-purpose implementation.

The **selection model** wraps the data model and adds which items are selected. `GtkSingleSelection`, `GtkMultiSelection` and `GtkNoSelection` are the usual choices. It is an adapter over the same items, not a copy of them.

The **view**, `GtkListView`, shows the selection model's items. It never creates a widget per item. It asks a **factory** to create a few row widgets, enough to fill the screen and a margin around it, and then reuses them as rows scroll in and out, refilling each one for whichever item it now shows. That is why a list of a million items is as cheap as a list of a hundred.



Figure 1: The model chain. Data flows right through the models; the view asks the factory for row widgets and binds them to items as they become visible.

Because each piece has one job, a list bug usually lives in one place. A missing session is a model problem: count the items after `fill_list`. The right number of rows with the wrong text is a `bind` problem. Old text flashing while you scroll is an `unbind` problem. The wrong row highlighted is the selection model. Start at the left of the chain and stop at the first piece that is wrong.

Recycling: a row is not an item

`GtkSignalListItemFactory` turns the factory's work into four signals:

- **`setup`**: create the widgets for one row. No item yet.
- **`bind`**: fill an existing row's widgets for an item.
- **`unbind`**: the row is about to show a different item, or none.
- **`teardown`**: the row's widgets are being destroyed.



Figure 2: One row widget over time. It is built once, then bound and unbound for different items as the list scrolls.

The discipline that makes recycling work is simple: build widgets in `setup`, only fill them in `bind`, and in `unbind` let go of anything that refers to the old item. Labels are easy, because the next `bind` overwrites them. A signal connected to the item, a timer, or a pointer cached on the row are not: left behind, they outlive the item's turn in that row and act on the wrong session.

Do not count on `bind` running once per item, either. It runs whenever a row needs refilling, and the isolated lab shows a selection change re-binding two rows whose items had not changed at all. `bind` must be cheap and must produce the same row every time for the same item.

GtkListBox or GtkListView

GTK 4 still has `GtkListBox`, and it is the right tool for some lists.

	`GtkListBox`	`GtkListView`
Row widgets	one per item, made by `create_widget_func` when the item is added	a few, recycled by a factory
Cost of 2,000 items	2,000 rows of widgets	about as many rows as fit on screen, plus a margin
Rows can differ	freely: each row is its own widget	every row comes from one factory, so `bind` must handle each kind
Good for	settings pages, short lists, rows with their own controls	sessions, logs, files, anything that can grow

A session manager's list grows with the user's network, so gtklab uses `GtkListView` from the start.

References along the chain

Every arrow in the model chain is a reference, and the constructors that build the chain take ownership of what you pass them. GTK's documentation marks those parameters "transfer full": the callee now owns the reference you handed over, and you must not release it again. If you want to keep using the object, as `AppWindow` keeps `store_`, take a second reference for yourself before handing one over.

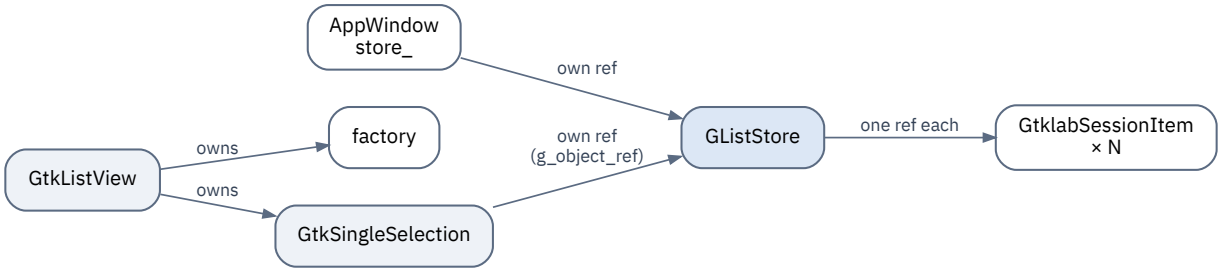


Figure 3: Who holds a reference to what. The window and the selection each hold their own reference to the store; the store holds one to every item.

`g_object_unref` does not mean delete. It gives back one reference; the object is finalized only when the last one goes. That is why appending an item to the store and then unreffing it is correct: the store holds its own reference, and the item lives as long as the store keeps it.

Defining a GObject type

Items in a `GListModel` must be GObjects, and a session is a plain C++ struct. Chapter 4 kept it that way on purpose, so the core could be tested without GTK. It needs a small GObject wrapper at the GTK boundary, which means defining a GObject type of your own. It takes two macros and three functions.

In the header, `G_DECLARE_FINAL_TYPE(GtklabSessionItem, gtklab_session_item, GTKLAB, SESSION_ITEM, GObject)` declares the type: a `GtklabSessionItem` struct, a `GTKLAB_SESSION_ITEM()` cast macro, a `GTKLAB_IS_SESSION_ITEM()` check, and `gtklab_session_item_get_type()`. "Final" means nothing can subclass it, which is what you want for almost every application type.

In the source file, you define the instance struct, whose first member must be the parent type's instance (`GObject parent_instance`), and then `G_DEFINE_FINAL_TYPE(GtklabSessionItem, gtklab_session_item, G_TYPE_OBJECT)` registers the type. That macro expects you to write:

- **`class_init`**, run once per type, which fills in virtual functions such as `finalize`;
- **`init`**, run for every new instance, like a constructor;
- and, if the type holds anything that needs freeing, a **`finalize`** that frees it and then chains up to the parent's `finalize`.

There is one trap for C++. GObject allocates and zero-fills instances itself; it never runs C++ constructors or destructors. So a `std::string` member must be constructed by hand in `init`, with placement `new`, and destroyed by hand in `finalize`. Forget the first and you use an object that was never constructed; forget the second and you leak.

Build

The item type

ch05-listview/src/session_item.h

```
// GtklabSessionItem: a GObject wrapper so the core's plain C++ Session and
// Folder can live in a GListModel. Your first GObject subclass.
#pragma once

#include <glib-object.h>

#include "core/session.h"

G_BEGIN_DECLS

#define GTKLAB_TYPE_SESSION_ITEM (gtklab_session_item_get_type())
G_DECLARE_FINAL_TYPE(GtklabSessionItem, gtklab_session_item, GTKLAB, SESSION_ITEM, GObject)

G_END_DECLS

// Constructors copy their argument; the item owns the copy.
GtklabSessionItem *gtklab_session_item_new_session(const Session &s);
GtklabSessionItem *gtklab_session_item_new_folder(const Folder &f);

bool gtklab_session_item_is_folder(GtklabSessionItem *item);
const Session &gtklab_session_item_session(GtklabSessionItem *item);
const Folder &gtklab_session_item_folder(GtklabSessionItem *item);
```

ch05-listview/src/session_item.cpp

```
#include "session_item.h"

#include <new>

struct _GtklabSessionItem {
    GObject parent_instance;

    // GObject zero-fills the instance and never runs C++ constructors, so
```

```
    // the C++ members are built with placement new in init and destroyed
    // by hand in finalize.
    bool is_folder;
    Session session;
    Folder folder;
};

G_DEFINE_FINAL_TYPE(GtklabSessionItem, gtklab_session_item, G_TYPE_OBJECT)

static void gtklab_session_item_finalize(GObject *object) {
    GtklabSessionItem *self = GTKLAB_SESSION_ITEM(object);
    self->session.~Session();
    self->folder.~Folder();
    G_OBJECT_CLASS(gtklab_session_item_parent_class)->finalize(object);
}

static void gtklab_session_item_class_init(GtklabSessionItemClass *klass) {
    G_OBJECT_CLASS(klass)->finalize = gtklab_session_item_finalize;
}

static void gtklab_session_item_init(GtklabSessionItem *self) {
    self->is_folder = false;
    new (&self->session) Session();
    new (&self->folder) Folder();
}

GtklabSessionItem *gtklab_session_item_new_session(const Session &s) {
    auto *item = GTKLAB_SESSION_ITEM(g_object_new(GTKLAB_TYPE_SESSION_ITEM, nullptr));
    item->session = s;
    return item;
}

GtklabSessionItem *gtklab_session_item_new_folder(const Folder &f) {
    auto *item = GTKLAB_SESSION_ITEM(g_object_new(GTKLAB_TYPE_SESSION_ITEM, nullptr));
    item->is_folder = true;
    item->folder = f;
    return item;
}

bool gtklab_session_item_is_folder(GtklabSessionItem *item) {
    return item->is_folder;
}

const Session &gtklab_session_item_session(GtklabSessionItem *item) {
    return item->session;
}

const Folder &gtklab_session_item_folder(GtklabSessionItem *item) {
    return item->folder;
}
```

AppWindow

From this chapter, the window and everything it owns live in one class:

ch05-listview/src/app_window.h

```
// AppWindow: the window and everything it owns. From Chapter 5 on, the
// window's state lives in this struct instead of globals.
#pragma once

#include <gtk/gtk.h>
```

```
#include <string>
#include <vector>

#include "core/session.h"
#include "core/settings.h"

class AppWindow {
public:
    explicit AppWindow(GtkApplication *app);
    ~AppWindow();

    AppWindow(const AppWindow &) = delete;
    AppWindow &operator=(const AppWindow &) = delete;

    GtkWindow *window() const { return GTK_WINDOW(window_); }

private:
    void build_ui();
    GtkWidget *build_sidebar();
    void load_sessions_file();
    void fill_list();

    static gboolean on_close_request(GtkWindow *win, gpointer self);
    static void on_flip_clicked(GtkButton *, gpointer self);
    static void on_factory_setup(GtkSignalListItemFactory *, GtkListItem *item, gpointer self);
    static void on_factory_bind(GtkSignalListItemFactory *, GtkListItem *item, gpointer self);
    static void on_factory_unbind(GtkSignalListItemFactory *, GtkListItem *item, gpointer self);

    GtkWidget *window_ = nullptr;
    GtkWidget *stack_ = nullptr;
    GtkWidget *notebook_ = nullptr;
    GtkWidget *paned_ = nullptr;
    GtkWidget *list_view_ = nullptr;
    GListStore *store_ = nullptr;  // owned: GtklabSessionItem

    Settings settings_;
    std::string sessions_path_;
    std::vector<Folder> folders_;
};
```

The window owns its `AppWindow` the same way Chapter 2's button owned its `Counter`: through `g_object_set_data_full`. Chapter 16 changes how that ownership is wired, and the reason is one of the most useful lessons in the book.

ch05-listview/src/app_window.cpp

```
AppWindow::AppWindow(GtkApplication *app) {
    settings_ = load_settings(config_path("settings.ini"));
    sessions_path_ = config_path("sessions.ini");

    window_ = gtk_application_window_new(app);
    // The window owns this AppWindow: deleting it when the window is destroyed.
    g_object_set_data_full(G_OBJECT(window_), "app-window", this,
                           [](gpointer p) { delete static_cast<AppWindow *>(p); });

    build_ui();
    load_sessions_file();
    fill_list();
}
```

`main.cpp` now only creates it:

ch05-listview/src/main.cpp

```
static void on_activate(GtkApplication *app, gpointer) {
    // The window owns the AppWindow (see its constructor).
    auto *w = new AppWindow(app);
    gtk_window_present(w->window());
}
```

The model chain

ch05-listview/src/app_window.cpp

```
GtkWidget *AppWindow::build_sidebar() {
    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
    gtk_widget_set_margin_top(box, 6);
    gtk_widget_set_margin_bottom(box, 6);
    gtk_widget_set_margin_start(box, 6);
    gtk_widget_set_margin_end(box, 6);

    gtk_box_append(GTK_BOX(box), gtk_search_entry_new());

    // Model -> selection -> view, with a factory making row widgets.
    store_ = g_list_store_new(GTKLAB_TYPE_SESSION_ITEM);
    // The selection model takes ownership of the model it wraps, so give it
    // a reference of its own; store_ keeps ours for fill_list().
    GtkSingleSelection *selection = gtk_single_selection_new(G_LIST_MODEL(g_object_ref(store_)));

    GtkListItemFactory *factory = gtk_signal_list_item_factory_new();
    g_signal_connect(factory, "setup", G_CALLBACK(on_factory_setup), this);
    g_signal_connect(factory, "bind", G_CALLBACK(on_factory_bind), this);
    g_signal_connect(factory, "unbind", G_CALLBACK(on_factory_unbind), this);

    // gtk_list_view_new takes ownership of both arguments.
    list_view_ = gtk_list_view_new(GTK_SELECTION_MODEL(selection), factory);
    gtk_widget_add_css_class(list_view_, "navigation-sidebar");

    GtkWidget *scroller = gtk_scrolled_window_new();
    gtk_widget_set_vexpand(scroller, TRUE);
    gtk_scrolled_window_set_policy(GTK_SCROLLED_WINDOW(scroller), GTK_POLICY_NEVER,
                                   GTK_POLICY_AUTOMATIC);
    gtk_scrolled_window_set_child(GTK_SCROLLED_WINDOW(scroller), list_view_);
    gtk_box_append(GTK_BOX(box), scroller);
    return box;
}
```

Follow the references. `g_list_store_new` returns a store with one reference, which `store_` keeps. `gtk_single_selection_new` *takes* a reference to the model it wraps, so the window passes it a new one with `g_object_ref`. `gtk_list_view_new` takes ownership of both the selection model and the factory, so neither needs an unref. GTK's documentation marks every such parameter "transfer full"; it is worth reading for every function that takes a model.

Filling the store is a loop of append and unref, because the store keeps its own reference to each item:

ch05-listview/src/app_window.cpp

```
void AppWindow::fill_list() {
    g_list_store_remove_all(store_);
    for (const Folder &f : folders_) {
        for (const Session &s : f.sessions) {
            GtklabSessionItem *item = gtklab_session_item_new_session(s);
            g_list_store_append(store_, item);
            g_object_unref(item);  // the store holds its own reference
        }
```

```
    }
}
```

The factory

`setup` builds the row and stores non-owning pointers to its parts on the row widget, so `bind` can find them without walking the widget tree:

ch05-listview/src/app_window.cpp

```
// setup: build the row widgets once. No item yet; nothing to fill.
void AppWindow::on_factory_setup(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    GtkWidget *icon = gtk_image_new();
    GtkWidget *text = gtk_box_new(GTK_ORIENTATION_VERTICAL, 0);
    GtkWidget *name = gtk_label_new(nullptr);
    GtkWidget *detail = gtk_label_new(nullptr);
    for (GtkWidget *l : {name, detail}) {
        gtk_label_set_xalign(GTK_LABEL(l), 0);
        gtk_label_set_ellipsize(GTK_LABEL(l), PANGO_ELLIPSIZE_END);
    }
    gtk_widget_add_css_class(detail, "dim-label");
    gtk_widget_add_css_class(detail, "caption");
    gtk_box_append(GTK_BOX(text), name);
    gtk_box_append(GTK_BOX(text), detail);
    gtk_box_append(GTK_BOX(row), icon);
    gtk_box_append(GTK_BOX(row), text);

    // Non-owning pointers for bind; the row owns its children.
    g_object_set_data(G_OBJECT(row), "icon", icon);
    g_object_set_data(G_OBJECT(row), "name", name);
    g_object_set_data(G_OBJECT(row), "detail", detail);
    gtk_list_item_set_child(item, row);
}
```

`bind` fills those widgets for one item:

ch05-listview/src/app_window.cpp

```
// bind: fill the existing row widgets for one item. Runs as rows scroll in.
void AppWindow::on_factory_bind(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *row = gtk_list_item_get_child(item);
    auto *si = GTKLAB_SESSION_ITEM(gtk_list_item_get_item(item));
    const Session &s = gtklab_session_item_session(si);
    g_debug("bind %s", s.name.c_str());

    gtk_image_set_from_icon_name(GTK_IMAGE(g_object_get_data(G_OBJECT(row), "icon")),
                                 s.local ? "computer-symbolic" : "network-server-symbolic");
    gtk_label_set_text(GTK_LABEL(g_object_get_data(G_OBJECT(row), "name")), s.name.c_str());

    std::string detail;
    if (s.local) {
        detail = "local";
    } else {
        detail = s.user.empty() ? s.host : s.user + "@" + s.host;
        if (s.port != 22)
            detail += ":" + std::to_string(s.port);
    }
    gtk_label_set_text(GTK_LABEL(g_object_get_data(G_OBJECT(row), "detail")), detail.c_str());
}
```

`unbind` clears them, so a row never shows a stale name for a moment while it is being reused:

ch05-listview/src/app_window.cpp

```
// unbind: the row is about to show a different item (or none). Clear
// anything that refers to the old one.
void AppWindow::on_factory_unbind(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *row = gtk_list_item_get_child(item);
    gtk_label_set_text(GTK_LABEL(g_object_get_data(G_OBJECT(row), "name")), "");
    gtk_label_set_text(GTK_LABEL(g_object_get_data(G_OBJECT(row), "detail")), "");
}
```

Done when

Two thousand generated sessions scroll smoothly. The repository's `tools/gen_sessions.py` writes them:

```
tools/gen_sessions.py 2000 ~/.config/gtklab/sessions.ini
```

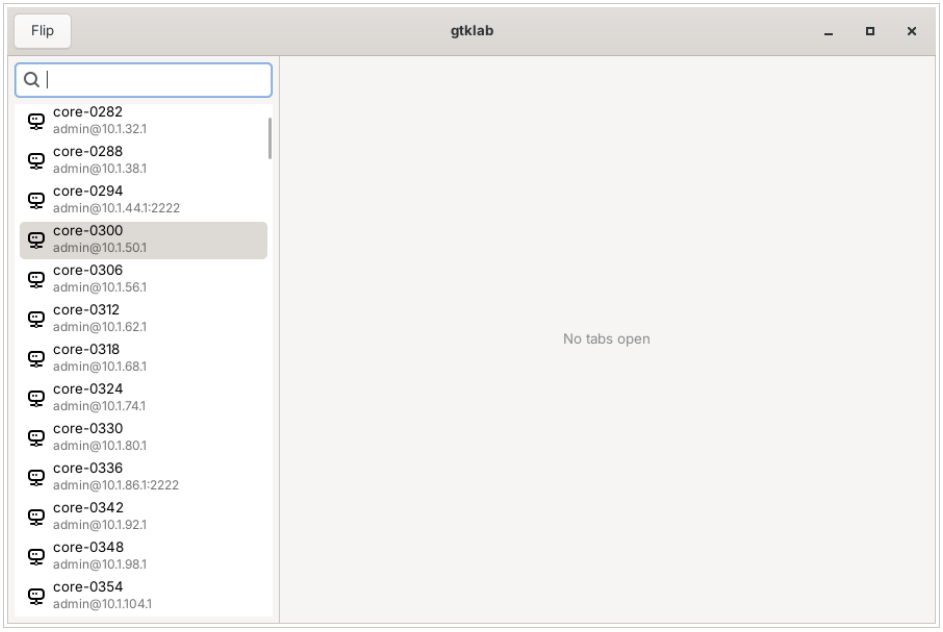


Figure 4: 2,000 sessions in a `GtkListView`. Each row is a recycled widget, re-bound as it scrolls into view.

Debugger drill

Run the **scratch config** launch configuration with 2,000 sessions and count the `bind` lines it prints. In the recorded run there were about 200 at startup, not 2,000: GTK binds the visible rows and a margin around them. After thirty notches of the mouse wheel the total was 376. Those extra binds are the same few dozen row widgets being re-bound to new items. Then set a breakpoint in `on_factory_bind`, scroll one notch, and watch it run once per row that came into view.

Break it

Remove `gtk_widget_set_vexpand(scroller, TRUE)` from `build_sidebar`. With 2,000 sessions, the list shrinks to a single visible row. A scrolled window's natural height is tiny, and nothing asked

for more. Open the Inspector, select the `GtkScrolledWindow` and set `vexpand` back on: the list grows to fill the sidebar.

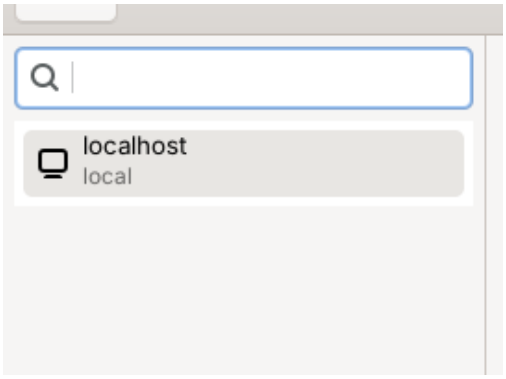


Figure 5: The Break it: without `vexpand`, one row of 2,000.

Common mistakes

Icons that show as a broken image. The symbolic icons here are SVG files, and GTK loads SVG through a gdk-pixbuf loader in a package of its own. On a minimal system without it (`librsvg2-common` on Debian and Ubuntu), every icon renders as a broken-image placeholder. The screenshots for this book showed exactly that until the package was installed, which is also why tmssh's `scripts/build.sh` installs it.

Building widgets in `bind`. It works, slowly, and leaks a widget every time a row is reused. Build in `setup`, fill in `bind`.

Forgetting the unref after `g_list_store_append`. The store takes its own reference; yours leaks.

Handing the store to the selection model without a reference. The selection takes ownership. If the window also unrefs `store_` in its destructor, the store is freed twice.

C++ members without placement new. GObject never runs constructors. A `std::string` in the instance struct that is used before being constructed is undefined behaviour that usually looks fine until it does not.

Read in tmssh

`on_factory_setup`, `on_factory_bind` and `on_factory_unbind` in `src/app_window.cpp`. tmssh's rows show a folder or a session with an icon, a name and a count; the structure is the same as gtklab's.

Review

1. A row shows `core2-den` with `core1-den`'s address underneath. Which of the four pieces do you look at first, and why not the model?
2. What can `setup` do that `bind` must not, and what goes wrong if you build the row's widgets in `bind` instead?
3. `unbind` in gtklab only clears two labels. Give an example of something a row might hold that would make an empty `unbind` a real bug, not a cosmetic one.

4. Why must items in a `GListModel` be GObjects, and why does gtklab wrap `Session` instead of making it a GObject?
5. `build_sidebar` calls `g_object_ref(store_)` before `gtk_single_selection_new`. What happens without it, and when?
6. `fill_list` unrefs each item right after appending it. Why is the item still alive afterwards?
7. In `gtklab_session_item_init`, what would happen if the placement `new` for `session` were removed, and why does the compiler not stop you?
8. When would you choose `GtkListBox` over `GtkListView` in a tool you are writing?

Questions for deeper thought. A log viewer appends a few hundred lines a second to a `GListStore`. What does each append cost the view, and how would you batch them? Your sessions file grows a field the sidebar does not show. Should the GObject wrapper copy it, or hold a pointer back into the core's data, and what does each choice cost when the file is reloaded?

Exercises

1. **Show the selection.** In `build_sidebar` in `ch05-listview`, connect `notify::selected` on the `GtkSingleSelection` and, in the handler, put the selected session's name in the "No tabs open" label (`gtk_single_selection_get_selected_item` returns the item without a new reference). Done when clicking a row changes the label, and you can say why no session was copied to do it.
2. **Folder in the detail line.** Change `on_factory_bind` so remote sessions show `Core · admin@10.0.0.1`, using `Session::folder`. Done when every row shows its folder, and `unbind` still leaves no stale text.
3. **Count the rows.** Add counters for `setup` and `bind` calls (static variables are fine in an exercise) and print them in `on_close_request`. Done when, with 2,000 sessions and some scrolling, you can state how many row widgets existed and how many times they were bound.
4. **One change instead of many.** Rewrite `fill_list` to build a `std::vector<gpointer>` of items and insert them with one `g_list_store_splice`. Connect `items-changed` on `store_` and print its arguments. Done when the old loop prints one line per session and the new one prints a single line.
5. **Watch teardown.** Connect a `teardown` handler in `build_sidebar` that prints a line. Done when you have seen when it runs, and can explain why gtklab does not need one.

Isolated lab

gtklab's list comes with a session parser, a GObject subclass and a window full of other widgets. The lab strips all of that away to show the list pipeline alone: plain `GObjects` with a name attached by `g_object_set_data_full`, a `GListStore`, a `GtkSingleSelection`, a `GtkListView` and a factory whose `bind` and `unbind` print what they do. An "Add session" button appends one item per click. With nothing else on the screen, every line in the terminal is the list talking.

labs/lab05-listmodel/src/main.cpp

```
// Lab 5: the list pipeline in isolation. Plain GObjects in a GListStore,
// a single selection, a list view and a factory; watch setup, bind and unbind.
#include <gtk/gtk.h>

struct Lab {
    GListStore *store = nullptr;
    GtkWidget *status = nullptr;
};
```

```
static void setup(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    gtk_list_item_set_child(item, gtk_label_new(""));
}

static void bind(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *label = gtk_list_item_get_child(item);
    auto *object = G_OBJECT(gtk_list_item_get_item(item));
    const char *name = static_cast<const char *>(g_object_get_data(object, "name"));
    gtk_label_set_text(GTK_LABEL(label), name ? name : "(unnamed)");
    g_print("bind: %s\n", name ? name : "(unnamed)");
}

static void unbind(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    gtk_label_set_text(GTK_LABEL(gtk_list_item_get_child(item)), "");
    g_print("unbind\n");
}

static void add_item(GtkButton *, gpointer data) {
    auto *lab = static_cast<Lab *>(data);
    guint index = g_list_model_get_n_items(G_LIST_MODEL(lab->store)) + 1;
    GObject *object = G_OBJECT(g_object_new(G_TYPE_OBJECT, nullptr));
    char *name = g_strdup_printf("Session %u", index);
    g_object_set_data_full(object, "name", name, g_free);
    g_list_store_append(lab->store, object);
    g_object_unref(object); // Store now holds its own reference.
    gtk_label_set_text(GTK_LABEL(lab->status), "Item appended to model");
}

static void activate(GtkApplication *app, gpointer) {
    auto *lab = new Lab;
    GtkWidget *window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "List Model Debugging Lab");
    gtk_window_set_default_size(GTK_WINDOW(window), 460, 320);
    g_object_set_data_full(G_OBJECT(window), "lab", lab, [](gpointer p) {
        auto *state = static_cast<Lab *>(p);
        g_clear_object(&state->store);
        delete state;
    });

    GtkWidget *column = gtk_box_new(GTK_ORIENTATION_VERTICAL, 8);
    gtk_widget_set_margin_top(column, 12);
    gtk_widget_set_margin_bottom(column, 12);
    gtk_widget_set_margin_start(column, 12);
    gtk_widget_set_margin_end(column, 12);
    gtk_window_set_child(GTK_WINDOW(window), column);

    lab->store = g_list_store_new(G_TYPE_OBJECT);
    GtkSelectionModel *selection = GTK_SELECTION_MODEL(
        gtk_single_selection_new(G_LIST_MODEL(g_object_ref(lab->store))));
    GtkListItemFactory *factory = gtk_signal_list_item_factory_new();
    g_signal_connect(factory, "setup", G_CALLBACK(setup), nullptr);
    g_signal_connect(factory, "bind", G_CALLBACK(bind), nullptr);
    g_signal_connect(factory, "unbind", G_CALLBACK(unbind), nullptr);
    GtkWidget *view = gtk_list_view_new(selection, factory);
    GtkWidget *scroll = gtk_scrolled_window_new();
    gtk_widget_set_vexpand(scroll, TRUE);
    gtk_scrolled_window_set_child(GTK_SCROLLED_WINDOW(scroll), view);
    gtk_box_append(GTK_BOX(column), scroll);

    GtkWidget *button = gtk_button_new_with_label("Add session");
    g_signal_connect(button, "clicked", G_CALLBACK(add_item), lab);
    gtk_box_append(GTK_BOX(column), button);
    lab->status = gtk_label_new("Click Add session and inspect the callbacks");
```

```
    gtk_box_append(GTK_BOX(column), lab->status);
    gtk_window_present(GTK_WINDOW(window));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab.lab05", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(activate), nullptr);
    int result = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return result;
}
```

Build and run from `labs/lab05-listmodel`:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

The launch configurations are the same as every lab's: **lab**, **lab: stop on GTK criticals**, **lab + GTK Inspector**, **lab (gdb)** and **lab (ASan)**. In the recorded run, three clicks printed `bind: Session 1` to `bind: Session 3`, and quitting printed three `unbind` lines.

Stop at	What it shows
`add_item`	the click arriving; step over `g_list_store_append` and the model has one more item before any row exists for it
`bind`	`gtk_list_item_get_item` returning the object that `add_item` created; the label is the one `setup` made
`unbind`	a row giving up its item; at quit, one stop per bound row
`g_list_store_append`	the stack: GLib at the top, your `add_item` below it, the button's `clicked` emission below that

Experiments:

1. **Many items, few rows.** Make `add_item` append 1,000 items per click in a loop. In the recorded run one click produced 205 `bind` lines, not 1,000; thirty wheel notches added 141 more, each one after an `unbind`; quitting printed 206 `unbind` lines. The view never bound more than about two hundred rows.
2. **bind is not once per item.** Print `gtk_list_item_get_position(item)` in `bind`, add three sessions, then click Session 2. In the recorded run the click printed `unbind`, `bind: Session 1 at 0`, `unbind`, `bind: Session 2 at 1`: the selection moved, and both rows were bound again for items that had not changed.
3. **Take the selection away.** Replace `gtk_single_selection_new` with `gtk_no_selection_new`. The rows still appear, the first row is no longer highlighted at startup, and clicking a row printed nothing at all. Only one responsibility changed.
4. **Follow a reference.** Remove `g_object_unref(object)` from `add_item`, build `build-asan`, and run `ASAN_OPTIONS=detect_leaks=1 ./build-asan/lab` from a terminal (the launch configuration turns leak detection off). Add three sessions and quit. The report is long either way, because GTK and its libraries keep some memory until exit; search it for `add_item`. In the recorded run the unchanged lab had no `add_item` frame in its report, and the changed one had leak stacks through

`add_item`, starting at the `g_object_new` line. The store gave its reference back; yours was never given back.

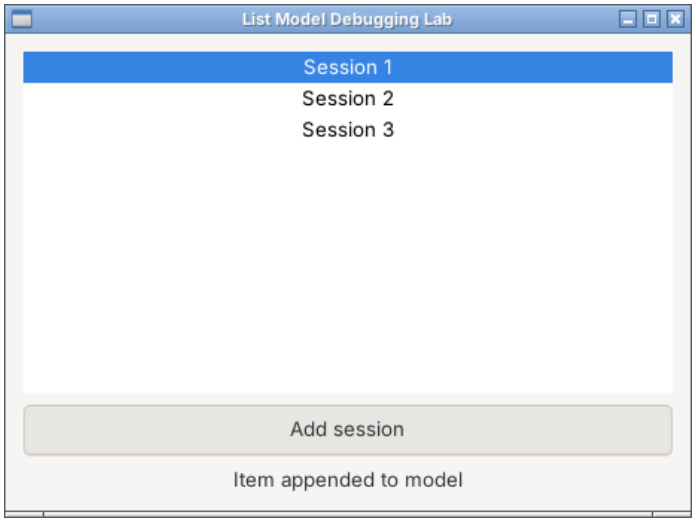


Figure 6: Lab 5 after three clicks on Add session.

Chapter 6 — List views II: trees, expansion, filtering

A flat list of two thousand sessions is not a session manager. This chapter groups sessions into folders that open and close, adds a search box that filters as you type, and makes sure that clearing the search puts the tree back exactly as the user left it. That last requirement is the one that shapes the code.

Recall

What does `setup` do that `bind` must not (Chapter 5)? Why must list items be GObjects? Which file holds the session model, and does it include GTK (Chapter 4)?

In this chapter

- Turn a folder model into an expandable tree with `GtkTreeListModel` and `GtkTreeExpander`, reusing Chapter 5's factory.
- Predict when the child-model function runs, and write one that is cheap and safe to call again.
- Choose where to filter a tree, and filter in the tested core.
- Rebuild a tree without losing which folders the user had open.

Concepts

A tree is a list that grows

GTK 4 has no tree widget. It has `GtkTreeListModel`, a model that turns a tree into a flat list. You give it a root model, here the folders, and a function that returns a model of children for an item, or `NULL` for an item that has none. When a row is expanded, its children are inserted into the flat list after it; when it is collapsed, they are removed. The list view underneath neither knows nor cares that it is showing a tree.

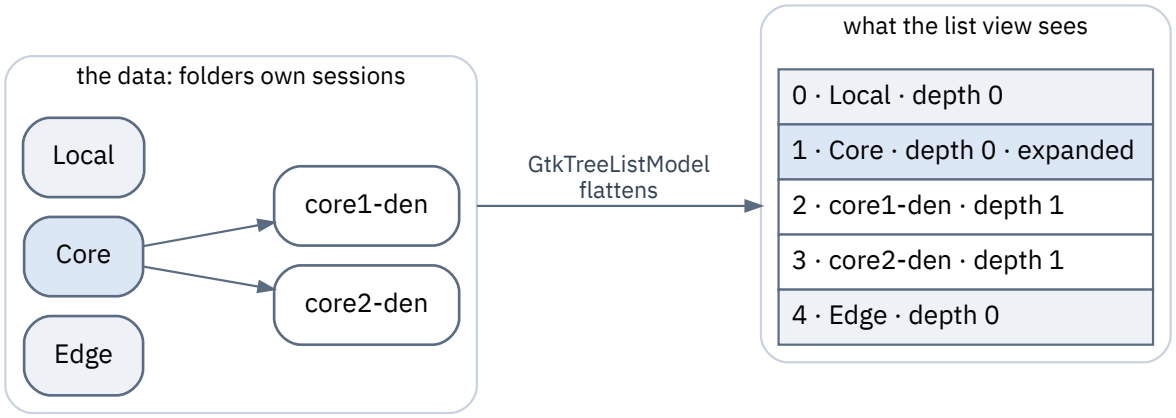


Figure 1: The data is a tree; the view sees a list. Expanding Core inserted its two sessions after it, and Edge moved from position 2 to position 4.

Two consequences follow. Expanding and collapsing change the presentation, not the data: a closed folder still has its sessions, they are just not rows. And a position is not an identity. Opening a folder shifts every row below it, so anything that must survive an expand, a rebuild or a reload (the folders that were open, the session that was selected) has to be remembered by something stable, such as a name, never by a row number.

The row, the item and the expander

Three objects with similar names do three different jobs:

Object	What it is	Question it answers
`GtkTreeListModel`	the model that flattens the tree and tracks expansion	which rows exist right now?
`GtkTreeListRow`	one item of that model, wrapping your item	where is this item: depth, expanded, parent?
`GtkTreeExpander`	a widget in your row	how is it drawn: indent, triangle, click to open

Your own `GtklabSessionItem` is the fourth thing, inside the row. With `passthrough` set to `FALSE`, the list items the factory receives are `GtkTreeListRows`, and you unwrap yours with `gtk_tree_list_row_get_item`, which returns a new reference. The expander gets the row in `bind`, and loses it in `unbind`, so a recycled row widget never shows the previous row's indent or triangle. Chapter 5's discipline is unchanged; the row widget just has one more layer.

With `autoexpand` set to `TRUE`, every folder opens as soon as it appears. gtklab passes `FALSE`: the user decides what is open, and the code below spends most of its effort respecting that.

When the child function runs

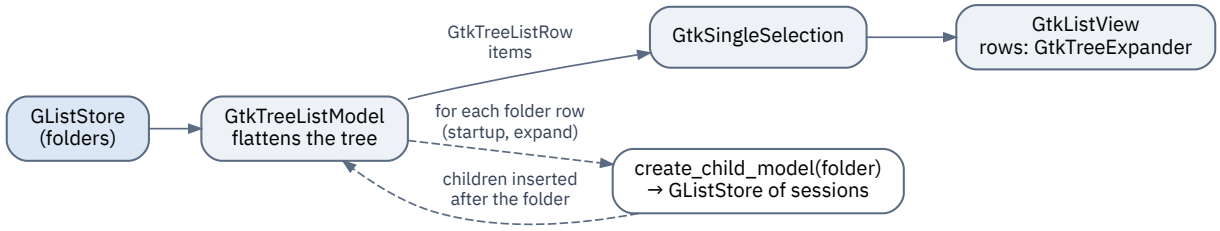


Figure 2: The model chain for a tree. The tree list model calls `create_child_model` for folder rows, and inserts the children after the folder when it is expanded.

The usual guess is that the child function runs when a folder is opened. It runs earlier and more often than that. An expander has to decide whether to draw a triangle before anyone clicks it, so it asks the tree model whether the row can expand, and the tree model answers by calling your function. In GTK 4.8's source, the model built to answer that question is released straight away; expanding the row calls the function again. The isolated lab measures it: both folders' child functions run at startup, before any click, and every leaf's runs as its row is bound. Remove the call that hands the row to the expander and the function is never called at all.

The contract, from GTK's own header: return `NULL` for an item that can never have children, and a model for one that can. An item that has no children yet but may get them later should return an empty model and fill it when they arrive. That is the hook for lazy loading, say a folder of devices read from an inventory API, but the function itself must stay quick. It runs on the main thread in the middle of GTK's own work; a network call in it freezes the window.

What you return is owned by the tree model ("transfer full"). gtklab builds a new store on every call. The lab keeps one store per group and returns a new reference to it. Both are correct. Returning a pointer without a reference of its own is not: the tree model will release a reference that was never its to release.

Filtering a tree

Filtering a flat list is one decision. Filtering a tree is three, because you can cut at three places:

Where you filter	What it can express	What it costs
`GtkFilterListModel` over the flattened rows	only rows that exist; a session in a closed folder is not a row, so it cannot match	a folder whose name does not match hides its children
`GtkFilterListModel` over the root	folders only	sessions are not filtered at all
In the core, then rebuild the root (gtklab)	anything the core can test: name, host or user, folders without a match dropped	a rebuild per change, and the open folders to restore

gtklab takes the third. `filter_sessions` already exists in Chapter 4's core, with tests, so the GTK side only has to replace the root model with its result. The price is that new folder items start closed, which is the problem the rest of the chapter solves.

Keeping the user's place

The tree has two kinds of state. The sessions come from the file. Which folders are open belongs to the user, and nothing in the file records it. A rebuild throws away every `GtkTreeListRow`, and their expanded flags with them, so the window has to carry that state across by name.

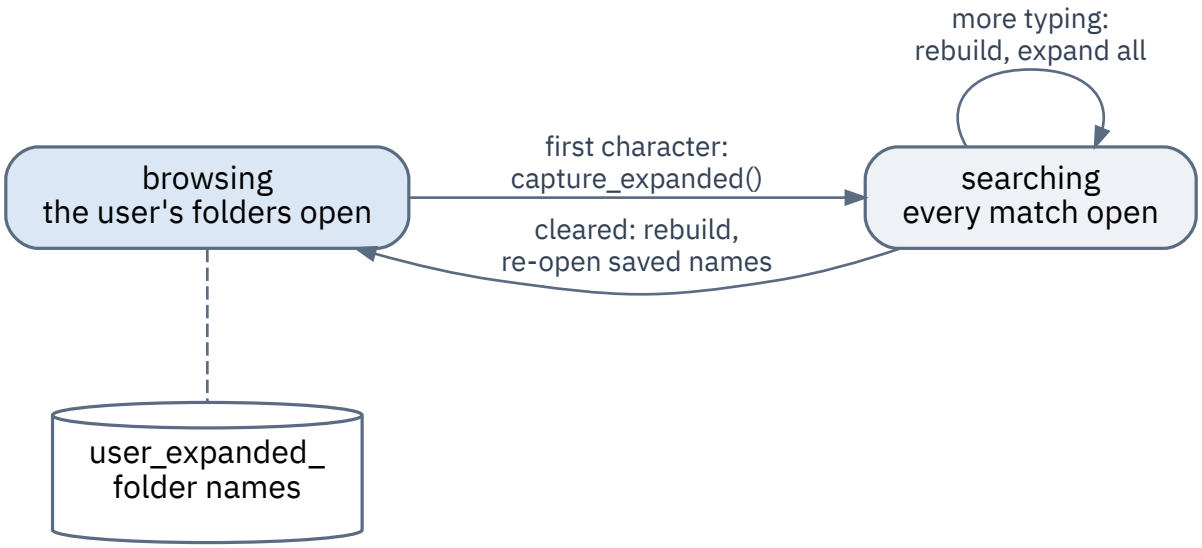


Figure 3: The search cycle. The first character saves the user's open folders; while searching, every match is open; clearing the search re-opens the saved ones.

The design is a small state machine. Browsing, the user's folders are open. The first keystroke of a search records them by name. While searching, every folder with a match is open, because a hidden match is useless. Clearing the search rebuilds the full tree and re-opens the recorded names. Chapter 7's Reload and Checkpoint B reuse the same capture and re-expand steps.

Re-opening is itself an expansion, so the first diagram happens again underneath the code that walks the rows: each folder it opens inserts rows after it, while the walk is still going.

Build

The chain gains a link. `passthrough` is `FALSE`, so the list items are `GtkTreeListRows` wrapping your items; `autoexpand` is `FALSE`, so folders start closed:

ch06-tree/src/app_window.cpp

```
root_ = g_list_store_new(GTKLAB_TYPE_SESSION_ITEM);
tree_model_ = gtk_tree_list_model_new(G_LIST_MODEL(g_object_ref(root_)),
                                      FALSE,  // passthrough: items are GtkTreeListRow
                                      FALSE,  // autoexpand
                                      create_child_model, this, nullptr);
GtkSingleSelection *selection =
    gtk_single_selection_new(G_LIST_MODEL(g_object_ref(tree_model_)));
```

The child function returns a new store of the folder's sessions, or `NULL` for a session:

ch06-tree/src/app_window.cpp

```
// Called by GtkTreeListModel whenever it needs a row's children: for each
// folder row it shows (to decide whether to draw an expander) and again on
// expand. Return a new model each time (the tree model takes ownership), or
// nullptr for a leaf, which gets no expander.
GListModel *AppWindow::create_child_model(gpointer item, gpointer) {
    auto *si = GTKLAB_SESSION_ITEM(item);
    if (!gtklab_session_item_is_folder(si))
        return nullptr;
    const Folder &f = gtklab_session_item_folder(si);
    g_debug("create_child_model %s", f.name.c_str());

    GListStore *children = g_list_store_new(GTKLAB_TYPE_SESSION_ITEM);
    for (const Session &s : f.sessions) {
        GtklabSessionItem *child = gtklab_session_item_new_session(s);
        g_list_store_append(children, child);
        g_object_unref(child);
    }
    return G_LIST_MODEL(children);
}
```

The row widget is Chapter 5's row inside an expander. In `bind`, the item is fetched from the tree row; `gtk_tree_list_row_get_item` returns a new reference, so it is released at the end:

ch06-tree/src/app_window.cpp

```
void AppWindow::on_factory_bind(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *expander = gtk_list_item_get_child(item);
    auto *row = GTK_TREE_LIST_ROW(gtk_list_item_get_item(item));
    gtk_tree_expander_set_list_row(GTK_TREE_EXPANDER(expander), row);

    auto *si = GTKLAB_SESSION_ITEM(gtk_tree_list_row_get_item(row));  // transfer full
    auto *icon = GTK_IMAGE(g_object_get_data(G_OBJECT(expander), "icon"));
    auto *name = GTK_LABEL(g_object_get_data(G_OBJECT(expander), "name"));
    auto *detail = GTK_LABEL(g_object_get_data(G_OBJECT(expander), "detail"));

    if (gtklab_session_item_is_folder(si)) {
        const Folder &f = gtklab_session_item_folder(si);
        g_debug("bind folder %s", f.name.c_str());
        gtk_image_set_from_icon_name(icon, "folder-symbolic");
        gtk_label_set_text(name, f.name.c_str());
        const std::string n = std::to_string(f.sessions.size()) +
                              (f.sessions.size() == 1 ? " session" : " sessions");
        gtk_label_set_text(detail, n.c_str());
    } else {
        const Session &s = gtklab_session_item_session(si);
        g_debug("bind %s", s.name.c_str());
        gtk_image_set_from_icon_name(icon, s.local ? "computer-symbolic" : "network-server-symbolic");
        gtk_label_set_text(name, s.name.c_str());
        std::string d;
        if (s.local) {
            d = "local";
        } else {
            d = s.user.empty() ? s.host : s.user + "@" + s.host;
            if (s.port != 22)
                d += ":" + std::to_string(s.port);
        }
        gtk_label_set_text(detail, d.c_str());
    }
    g_object_unref(si);
}
```

Rebuilding without losing the user's place

Rebuilding the root replaces every folder item, and new items start collapsed. So the window remembers which folders were open before it rebuilds, and opens them again afterwards. Remembering is a walk over the flattened rows:

ch06-tree/src/app_window.cpp

```
// Names of the folders whose rows are expanded now.
std::set<std::string> AppWindow::capture_expanded() const {
    std::set<std::string> names;
    GListModel *rows = G_LIST_MODEL(tree_model_);
    const guint n = g_list_model_get_n_items(rows);
    for (guint i = 0; i < n; i++) {
        auto *row = GTK_TREE_LIST_ROW(g_list_model_get_item(rows, i));
        if (gtk_tree_list_row_get_depth(row) == 0 && gtk_tree_list_row_get_expanded(row)) {
            auto *si = GTKLAB_SESSION_ITEM(gtk_tree_list_row_get_item(row));
            names.insert(gtklab_session_item_folder(si).name);
            g_object_unref(si);
        }
        g_object_unref(row);
    }
    return names;
}
```

The rebuild swaps all the folders in with one `g_list_store_splice`, which sends the view a single change instead of one per folder, then re-expands. Expanding a row inserts rows after it, so the loop re-reads the count each time around:

ch06-tree/src/app_window.cpp

```
// Replace the root with the (filtered) folders, then re-open folders.
void AppWindow::rebuild_tree(const std::set<std::string> &expand, bool expand_all) {
    std::vector<Folder> shown = filter_sessions(folders_, filter_);

    // Build the new items first, then swap them in with one splice: one
    // items-changed signal instead of one per folder.
    std::vector<gpointer> items;
    for (const Folder &f : shown)
        items.push_back(gtklab_session_item_new_folder(f));
    g_list_store_splice(root_, 0, g_list_model_get_n_items(G_LIST_MODEL(root_)),
                        items.data(), static_cast<guint>(items.size()));
    for (gpointer p : items)
        g_object_unref(p);

    // Expanding a row inserts its children after it, so re-read the count
    // each time round instead of caching it.
    GListModel *rows = G_LIST_MODEL(tree_model_);
    for (guint i = 0; i < g_list_model_get_n_items(rows); i++) {
        auto *row = GTK_TREE_LIST_ROW(g_list_model_get_item(rows, i));
        if (gtk_tree_list_row_get_depth(row) == 0) {
            auto *si = GTKLAB_SESSION_ITEM(gtk_tree_list_row_get_item(row));
            if (expand_all || expand.count(gtklab_session_item_folder(si).name))
                gtk_tree_list_row_set_expanded(row, TRUE);
            g_object_unref(si);
        }
        g_object_unref(row);
    }
}
```

The search handler decides what "open" means. When a search starts, it captures the user's tree; while searching, every folder with a match is open; when the search is cleared, the captured tree comes back:

ch06-tree/src/app_window.cpp

```
void AppWindow::on_search_changed(GtkSearchEntry *entry, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    const std::string text = gtk_editable_get_text(GTK_EDITABLE(entry));
    const bool was_filtering = !self->filter_.empty();

    if (!was_filtering && !text.empty())
        self->user_expanded_ = self->capture_expanded();  // remember the user's tree

    self->filter_ = text;
    if (text.empty())
        self->rebuild_tree(self->user_expanded_, false);  // back to exactly what was open
    else
        self->rebuild_tree({}, true);  // show every match
}
```

Activating a row (Enter, or a double-click) toggles a folder or opens a session, which for now prints its command line:

ch06-tree/src/app_window.cpp

```
void AppWindow::on_row_activated(GtkListView *, guint position, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    auto *row = GTK_TREE_LIST_ROW(g_list_model_get_item(G_LIST_MODEL(self->tree_model_), position));
    auto *si = GTKLAB_SESSION_ITEM(gtk_tree_list_row_get_item(row));
    if (gtklab_session_item_is_folder(si))
        gtk_tree_list_row_set_expanded(row, !gtk_tree_list_row_get_expanded(row));
    else
        self->activate_session(gtklab_session_item_session(si));
    g_object_unref(si);
    g_object_unref(row);
}
```

Done when

Typing filters the tree, clearing the search restores exactly the folders that were open, and Enter on a session prints its command. In the recorded run, activating `core1-den` printed `open core1-den: ssh admin@10.0.0.1`.

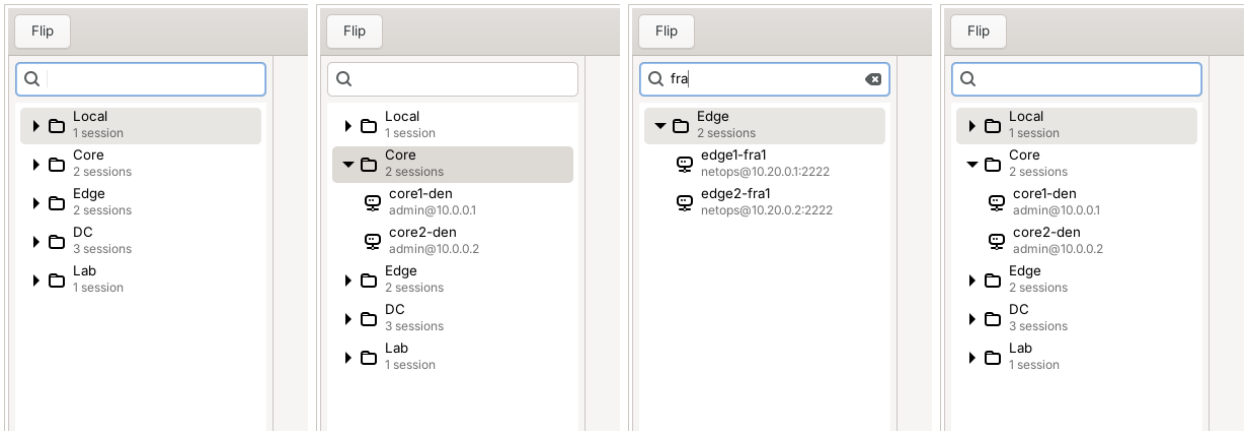


Figure 4: Left to right: folders closed; Core opened; filtered to “fra”, with every match shown; filter cleared, with Core open again as the user left it.

Debugger drill

Set a breakpoint in `create_child_model`, or watch its `g_debug` line in the scratch config, and predict when it will run before you start. The usual guess is "the first time a folder is opened". The measured answer is: for every visible folder at startup, because the expander must know whether a row has children before it can decide to draw a triangle; and again for several folders on each expand and collapse. So the function must be cheap, and every call must hand over a reference the tree can own: gtklab builds a new store each time; lab06 returns a new reference to a store it keeps.

Break it

Pass `TRUE` for `passthrough`. The list items are now your `GtklabSessionItems`, not tree rows, and `bind` casts them to `GtkTreeListRow`. Under **stop on GTK criticals** the program stops at startup with `invalid cast from 'GtklabSessionItem' to 'GtkTreeListRow'`. Passthrough mode exists for views that do not need expanders; with it, the tree list model hands your items straight through.

Common mistakes

Caching the row count while expanding. Every expansion inserts rows. A loop over a count read once before it starts misses rows or reads past the end.

Returning a model you do not own. The tree list model takes ownership of what you return and drops it when it is done. Return a new model, or `g_object_ref` one you keep; a borrowed pointer is freed out from under you.

Filtering the flattened rows. It hides children of folders whose names do not match. Filter the data, then rebuild.

Forgetting the unref after `gtk_tree_list_row_get_item`. It returns a new reference.

Read in tmssh

`build_sidebar`, `create_child_model`, `rebuild_tree` and `capture_expanded`. tmssh's tree is driven by the same capture, rebuild, re-expand cycle.

Review

1. A collapsed folder is not in the flat list at all, yet still has its sessions. Where do they live while it is closed?
2. What does the factory receive as its list item when the model is a `GtkTreeListModel` with `passthrough` set to `FALSE`, and how do you get from it to your session?
3. The lab's child function ran for both folders before anything was clicked. Why, and what does that mean for what the function may do?
4. Why must `unbind` call `gtk_tree_expander_set_list_row` with `nullptr`? What would a recycled row show otherwise?
5. Why does gtklab filter in the core and rebuild, instead of putting a `GtkFilterListModel` over the tree list model?
6. After a rebuild, every folder is closed. Why, and why does gtklab remember open folders by name rather than by row position?
7. `rebuild_tree` re-reads `g_list_model_get_n_items` on every pass of its loop. Describe a sessions file for which a count read once would skip a folder that should be re-opened.

8. While a search is active, the user closes a folder, then clears the search. Which folders are open afterwards, and why?

Questions for deeper thought. A network inventory tool shows sites, devices and interfaces, and the interfaces come from a slow API. What would your child function return for a device, and when would it fill it? If the same device can appear under two sites, what should identify a row across a rebuild? Should the search state survive a restart, and where would it be stored?

Exercises

1. **Keep the selection.** In `rebuild_tree` in `ch06-tree`, note the selected session's name before the splice and select it again afterwards with `gtk_single_selection_set_selected`, finding its new position by walking the rows as `capture_expanded` does. Done when selecting `core1-den` and typing `core` leaves `core1-den` selected.
2. **Matches out of total.** While a filter is active, make a folder's detail line read `1 of 2 sessions` instead of `1 session`. The filtered `Folder` only holds the matches, so look the total up in `folders_` by name. Done when the counts are right while filtering and unchanged without a filter.
3. **Expand all, collapse all.** Add two functions to `AppWindow` that open or close every folder row, modelled on the loop in `rebuild_tree`, and call them from a temporary button. Done when both work with some folders already open, and you can say why collapsing needs no special care about the count.
4. **Show the match.** In `on_factory_bind`, when `filter_` is not empty, set the session's name with `gtk_label_set_markup` and wrap the matching part in `<b>`. The factory callbacks already receive the window as `user_data`; use it. Escape the name with `g_markup_escape_text` first. Done when typing `fra` shows `fra` in bold in `edge1-fra1`, and a session named `a<b` does not break the label.

Isolated lab

In gtklab, the tree is tangled up with filtering, re-expansion and a session model. The lab keeps only the tree: two groups of `GtkStringObject` leaves, each group carrying its child store as object data, a `GtkTreeListModel` with `passthrough` and `autoexpand` both `FALSE`, and a factory whose rows are an expander around a label. Every callback prints, including the child function, so the question "when does it run?" is answered by reading the terminal.

labs/lab06-tree/src/main.cpp

```
// Lab 6: a tree is a list that grows. A GtkTreeListModel over string items,
// with a child model per group; watch when children_for and bind run.
#include <gtk/gtk.h>

// Called by the tree list model whenever it needs to know an item's
// children: for every visible row, so the expander knows whether to draw a
// triangle, and again on expand. Return a new reference, or nullptr for a leaf.
static GListModel *children_for(gpointer item, gpointer) {
    const char *name = gtk_string_object_get_string(GTK_STRING_OBJECT(item));
    GListModel *children = G_LIST_MODEL(g_object_get_data(G_OBJECT(item), "children"));
    g_print("children_for: %s -> %s\n", name, children ? "model" : "leaf");
    if (children == nullptr)
        return nullptr;
    return G_LIST_MODEL(g_object_ref(children));
}

static void setup_row(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *expander = gtk_tree_expander_new();
```

```
    GtkWidget *label = gtk_label_new("");
    gtk_label_set_xalign(GTK_LABEL(label), 0.0f);
    gtk_tree_expander_set_child(GTK_TREE_EXPANDER(expander), label);
    gtk_list_item_set_child(item, expander);
    g_print("setup: row widget created\n");
}

static void bind_row(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkTreeListRow *row = GTK_TREE_LIST_ROW(gtk_list_item_get_item(item));
    GtkStringObject *data = GTK_STRING_OBJECT(gtk_tree_list_row_get_item(row));
    GtkWidget *expander = gtk_list_item_get_child(item);
    GtkWidget *label = gtk_tree_expander_get_child(GTK_TREE_EXPANDER(expander));
    gtk_tree_expander_set_list_row(GTK_TREE_EXPANDER(expander), row);
    gtk_label_set_text(GTK_LABEL(label), gtk_string_object_get_string(data));
    g_print("bind: %s depth=%u\n", gtk_string_object_get_string(data),
            gtk_tree_list_row_get_depth(row));
}

static void unbind_row(GtkSignalListItemFactory *, GtkListItem *item, gpointer) {
    GtkWidget *expander = gtk_list_item_get_child(item);
    gtk_tree_expander_set_list_row(GTK_TREE_EXPANDER(expander), nullptr);
    g_print("unbind: row widget released from item\n");
}

static void add_leaf(GListStore *store, const char *name) {
    GtkStringObject *item = gtk_string_object_new(name);
    g_list_store_append(store, item);
    g_object_unref(item);
}

static void add_group(GListStore *roots, const char *name,
                      const char *first, const char *second) {
    GtkStringObject *group = gtk_string_object_new(name);
    GListStore *children = g_list_store_new(GTK_TYPE_STRING_OBJECT);
    add_leaf(children, first);
    add_leaf(children, second);
    g_object_set_data_full(G_OBJECT(group), "children", children,
                           (GDestroyNotify)g_object_unref);
    g_list_store_append(roots, group);
    g_object_unref(group);
}

static void activate(GtkApplication *app, gpointer) {
    GtkWidget *window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "Tree Model Debugging Lab");
    gtk_window_set_default_size(GTK_WINDOW(window), 540, 360);

    GListStore *roots = g_list_store_new(GTK_TYPE_STRING_OBJECT);
    add_group(roots, "Production", "Router A", "Switch B");
    add_group(roots, "Laboratory", "Firewall C", "Host D");

    GtkTreeListModel *tree = gtk_tree_list_model_new(
        G_LIST_MODEL(roots), FALSE, FALSE, children_for, nullptr, nullptr);  // passthrough, autoexpand
    GtkSingleSelection *selection = gtk_single_selection_new(G_LIST_MODEL(tree));
    GtkListItemFactory *factory = gtk_signal_list_item_factory_new();
    g_signal_connect(factory, "setup", G_CALLBACK(setup_row), nullptr);
    g_signal_connect(factory, "bind", G_CALLBACK(bind_row), nullptr);
    g_signal_connect(factory, "unbind", G_CALLBACK(unbind_row), nullptr);

    GtkWidget *view = gtk_list_view_new(GTK_SELECTION_MODEL(selection), factory);
    GtkWidget *scroll = gtk_scrolled_window_new();
    gtk_scrolled_window_set_child(GTK_SCROLLED_WINDOW(scroll), view);
    gtk_window_set_child(GTK_WINDOW(window), scroll);
```

```
    gtk_window_present(GTK_WINDOW(window));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new("io.github.gtklab.lab06",
                                              G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(activate), nullptr);
    int status = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return status;
}
```

Build and run from `labs/lab06-tree`:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

In the recorded run, `children_for: Production -> model` and `children_for: Laboratory -> model` ran at startup, before any click, each with a `setup` and a `bind`. Clicking Production's expander ran `children_for: Production -> model` again, then a `setup` and `bind` for Router A and Switch B, each preceded by `children_for: ... -> leaf`.

Stop at	What it shows
`children_for` at startup	the stack: `gtk_tree_list_row_is_expandable`, called by `gtk_tree_expander_set_list_row` in your own `bind_row`; the expander asking, not a click
`children_for` after a click	the expand itself; the store returned is the same one, with one more reference
`bind_row`	`gtk_list_item_get_item` is a `GtkTreeListRow`; `depth` is 0 for groups and 1 for leaves
`unbind_row`	the expander letting go of its row before the widget is reused

Experiments:

1. **Open everything.** Pass `TRUE` for `autoexpand`. In the recorded run, `children_for` ran for all six items, groups and leaves, before the first `setup`, and the window opened with both groups expanded.
2. **Leave out the row.** Delete the `gtk_tree_expander_set_list_row` call in `bind_row`. The labels appear without triangles or indent, clicking does nothing, and `children_for` never ran at all: nothing asked.
3. **Passthrough.** Pass `TRUE` for `passthrough` and run **lab: stop on GTK criticals**. It stopped at the first bind with `invalid cast from 'GtkStringObject' to 'GtkTreeListRow'`: the list items are now your strings.
4. **A third level.** In `activate`, after the groups are built, add a group "Rack 1" with "Server E" and "Server F" to Production's child store with `add_group`. Expanding Production then showed Rack 1 at depth 1 with its own triangle, and printed `children_for: Rack 1 -> model`. Nothing in the factory or the view changed.

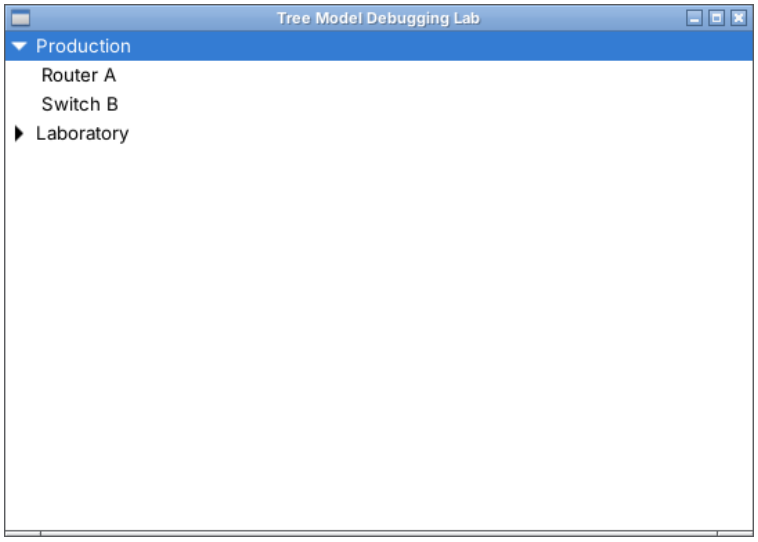


Figure 5: Lab 6 with Production expanded.

Chapter 7 — Actions, menus and shortcuts

Every command in a tool should be reachable three ways: from a menu, from a keyboard shortcut, and sometimes from a button. GTK 4 makes that one piece of code instead of three, by putting every command behind a named **action**. This chapter adds actions for reload, quit, showing the sidebar and choosing light or dark, a menu that names them, and shortcuts that reach them first.

Recall

When does `create_child_model` run (Chapter 6)? What is the order of `setup`, `bind` and `unbind` (Chapter 5)? What survives a rebuild of the tree, and how?

In this chapter

- Put each command behind one named action and reach it from a menu, a shortcut and a button.
- Choose where an action lives (`app.`, `win.` or a widget's own group) and debug a name that does not resolve.
- Model toggles and choices as stateful actions, and know when your code must set the state itself.
- Explain which code sees a key press first, and register shortcuts so they reach the action before the focused widget.
- Save settings once after a burst of changes, with a timeout on the main loop.

Concepts

Commands, not callbacks

Imagine Reload as a menu item, a shortcut and a toolbar button, each with its own handler. Sooner or later one of them forgets to keep the open folders, or to clear the error banner, and the three ways in no longer mean the same thing. The fix is to give the command a name and make every control refer to the name.

A **`GAction`** is a named command, optionally with a parameter and a state. `GSimpleAction` is the implementation you use: you give it a name and a callback. Actions live in an action map, and both `GtkApplication` and `GtkApplicationWindow` are action maps. An action added to the window is addressed as `win.name`; one added to the application, as `app.name`.

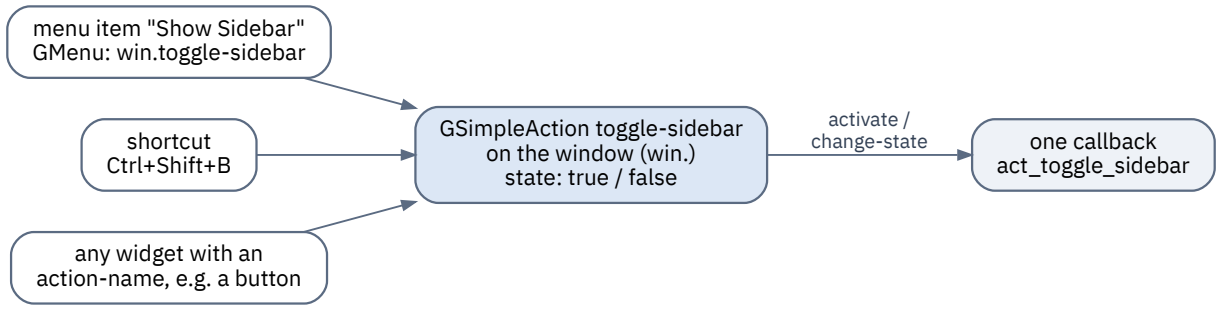


Figure 1: One action, three ways in. Menu items, keyboard shortcuts and buttons all name the same action; the action runs one callback.

The controls do not hold a pointer to your code, only a string. That is what lets the menu be built from data, the shortcut table live in one array, and a button in a later chapter reach `win.reload` without knowing that `AppWindow` exists. It is Chapter 4's boundary again: the control asks for work by name; the action does it.

Where an action lives

A widget with an action name looks for it on itself, then on its parent, and so on up to the window. A `GtkApplicationWindow` answers for the `win.` prefix, and above the window the application answers for `app.`. If nothing answers, the control is made insensitive, and nothing is printed.

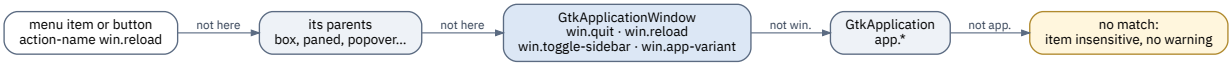


Figure 2: How `win.reload` is found. The name is looked up from the control upwards; a name nobody answers greys the control out, silently.

The prefix is a scope, and choosing it is a design decision:

Added to	Prefix	Lives as long as	Fits
the `GtkApplication`	`app.`	the process	commands that need no window: new window, about
the `GtkApplicationWindow`	`win.`	that window	commands on one window's state: everything in gtklab
any widget, with `gtk_widget_insert_action_group`	your own, e.g. `row.`	that widget	commands for one part of the window, reachable only from inside it

gtklab puts every action on the window, even Quit, because every callback needs the `AppWindow` it receives as user data: the sidebar to hide, the tree to reload, the settings to save. An `app.` action would first have to work out which window it meant. The isolated lab shows the other choice: its `app.refresh` holds a pointer to a label in one window, which is fine until there are two windows.

Parameter, state and what the menu draws

An action can take a **parameter** (what to do this time) and carry a **state** (what is true now). The combination decides how a menu draws the item:

Shape	Parameter	State	Menu draws	gtklab
command	none	none	a plain item	`win.quit`, `win.reload`
toggle	none	boolean	a check mark when true	`win.toggle-sidebar`
choice	string (`"s"`)	string	a radio dot on the item whose target equals the state	`win.app-variant`

A choice is one action with several menu items. Each item names it with a target, `win.app-variant::dark`, and activating that item asks for the state `'dark'`.

Activating and changing state are separate steps. When a `GSimpleAction` has no `activate` handler, activating a boolean toggle asks for the opposite state, and activating a choice asks for the state named by the parameter; either way the request arrives at `change-state`. If you connect `change-state`, the request is all you get: the state changes only if your handler sets it. A short GIO program confirms all three: a handler that forgot to set the state left it unchanged. gtklab relies on this default path, which is why its toggle and choice actions have no `activate` callback. The lab takes the other route: it connects `activate` and flips the state itself.

Enabled is the third property every action has. A disabled action cannot be activated, and every control that names it greys out together, which is how a tool says "not now" once instead of three times.

Menus are models

A **`GMenu`** is not a widget. It is a data model of items, sections and submenus, where each item is a label and the name of an action. A widget such as `GtkMenuButton` or `GtkPopoverMenu` turns that model into something you can click. Because the menu only names actions, the menu item's sensitivity, check mark or radio dot comes from the action itself: disable the action and the item greys out; change its state and the check mark moves. This is Chapter 5's split once more: a model describing what exists, and a widget that draws it.

Where shortcuts are caught

A key press goes to the window first. In the capture phase the window, then each ancestor of the focused widget, gets a chance at it before the focused widget does (Chapter 9 covers phases properly). Shortcuts handled there never reach the widget.

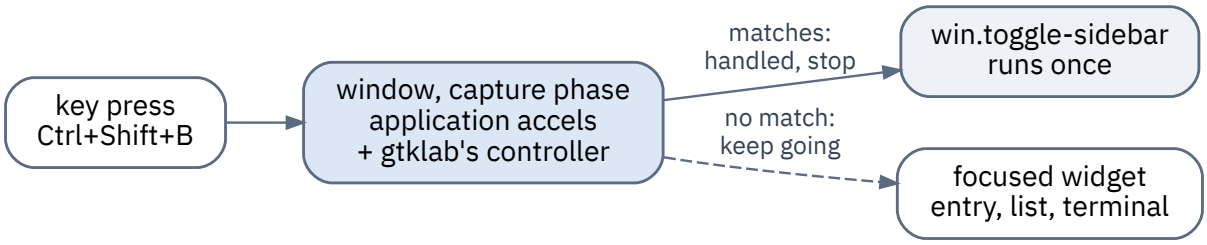


Figure 3: The path of a shortcut. Both of gtklab's registrations sit on the window in the capture phase; a key they match runs its action once and goes no further.

gtklab registers each shortcut twice, from one table. `gtk_application_set_accels_for_action` is what makes menus display "Shift+Ctrl+B" next to the item, and it also makes the key work: GTK puts the application's accelerators on each window in the capture phase. Measured on GTK 4.8.3 and 4.14, a focused widget that swallowed every key never saw Ctrl+R when the action was registered only this way. The `GtkShortcutController` that gtklab adds to the window, also in the capture phase, is the second registration. A key press still runs the action once, because the first controller that handles the key stops it.

What the controller buys is control. It belongs to the window, it is built in your code where you can read it, and it does not have to match what the menu shows: tmssh gives zoom several keys in its controller and shows only one of them in the menu. The ordering matters from Chapter 13, when the focused widget is a terminal that would otherwise turn your shortcut into bytes for the remote end. It also has a cost: a shortcut on a key a widget needs is stolen from that widget. gtklab's shortcuts all use Ctrl+Shift, which leaves plain Ctrl keys such as Ctrl+C to the widget underneath.

Saving without thrashing

A toggle that writes the settings file on every press does three writes for three presses. A debounce writes once: each change restarts a short timer, and only the timer that survives a quiet period runs the save.

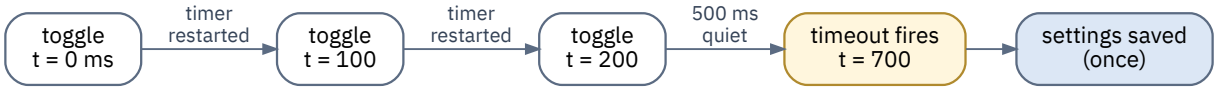


Figure 4: Three toggles, one save. Each change removes the pending timeout and starts a new one; only the last fires.

The timer holds a pointer to the `AppWindow`. That makes it a lifetime question in the Chapter 2 sense: whoever starts a source that points at an object must remove it before the object dies.

Build

One table of shortcuts

ch07-actions/src/app_window.cpp

```cpp
// One table drives the accelerators shown in menus and the capture-phase
// shortcut controller.
struct Accel {
    const char *action;
    const char *trigger;
};
```

ch07-actions/src/app_window.cpp

```cpp
static const Accel kAccels[] = {
    {"win.quit", "<Control><Shift>q"},
    {"win.reload", "<Control><Shift>r"},
    {"win.toggle-sidebar", "<Control><Shift>b"},
};
```

Installing the actions

`GActionEntry` is a table of actions to create in one call. Each entry gives the name, the activate callback, the parameter type (a GVariant type string, `"s"` for a string), the initial state (in GVariant text format, so a string is written `"'light'"` with its own quotes) and the change-state callback:

ch07-actions/src/app_window.cpp

```cpp
void AppWindow::install_actions(GtkApplication *app) {
    static const GActionEntry entries[] = {
        // name, activate, parameter type, initial state, change_state
        {"quit", act_quit, nullptr, nullptr, nullptr, {0, 0, 0}},
        {"reload", act_reload, nullptr, nullptr, nullptr, {0, 0, 0}},
        // A boolean state and no parameter: activating toggles it, then
        // change_state runs with the new value.
        {"toggle-sidebar", nullptr, nullptr, "true", act_toggle_sidebar, {0, 0, 0}},
        // A string parameter and state: a radio group in menus.
        {"app-variant", nullptr, "s", "'light'", act_app_variant, {0, 0, 0}},
    };
    g_action_map_add_action_entries(G_ACTION_MAP(window_), entries, G_N_ELEMENTS(entries), this);

    // Initial states from settings.
    GAction *a = g_action_map_lookup_action(G_ACTION_MAP(window_), "toggle-sidebar");
    g_simple_action_set_state(G_SIMPLE_ACTION(a), g_variant_new_boolean(settings_.sidebar_visible));
    a = g_action_map_lookup_action(G_ACTION_MAP(window_), "app-variant");
    g_simple_action_set_state(G_SIMPLE_ACTION(a), g_variant_new_string(settings_.app_variant.c_str()));

    // 1) Application accelerators: menus show them, and GTK also installs them
    //    on every window in the capture phase, ahead of the focused widget.
    // 2) The same triggers in a capture-phase controller we own and can read.
    GtkEventController *shortcuts = gtk_shortcut_controller_new();
    gtk_event_controller_set_propagation_phase(shortcuts, GTK_PHASE_CAPTURE);
    for (const Accel &acc : kAccels) {
        const char *accels[] = {acc.trigger, nullptr};
        gtk_application_set_accels_for_action(app, acc.action, accels);
        gtk_shortcut_controller_add_shortcut(
            GTK_SHORTCUT_CONTROLLER(shortcuts),
            gtk_shortcut_new(gtk_shortcut_trigger_parse_string(acc.trigger),
                             gtk_named_action_new(acc.action)));
    }
    gtk_widget_add_controller(window_, shortcuts);
}
```

Stateful actions

A `change_state` callback must set the state itself, or nothing changes and the check mark never moves:

ch07-actions/src/app_window.cpp

```
void AppWindow::act_toggle_sidebar(GSimpleAction *action, GVariant *value, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    const bool visible = g_variant_get_boolean(value);
    g_debug("toggle-sidebar -> %d", visible);
    g_simple_action_set_state(action, value);  // change_state must set the state itself
    gtk_widget_set_visible(self->sidebar_, visible);
    self->settings_.sidebar_visible = visible;
    self->save_settings_soon();
}
```

ch07-actions/src/app_window.cpp

```
void AppWindow::act_app_variant(GSimpleAction *action, GVariant *value, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    const char *variant = g_variant_get_string(value, nullptr);
    g_debug("app-variant -> %s", variant);
    g_simple_action_set_state(action, value);
    self->set_app_variant(variant);
    self->settings_.app_variant = variant;
    self->save_settings_soon();
}
```

Dark and light are a GTK setting, not a CSS class. The window asks for the dark variant of the theme:

ch07-actions/src/app_window.cpp

```
void AppWindow::set_app_variant(const std::string &variant) {
    g_object_set(gtk_settings_get_default(), "gtk-application-prefer-dark-theme",
                 variant == "dark", nullptr);
}
```

The menu

ch07-actions/src/app_window.cpp

```
GMenuModel *AppWindow::build_menu() {
    GMenu *menu = g_menu_new();

    GMenu *top = g_menu_new();
    g_menu_append(top, "Reload Sessions", "win.reload");
    g_menu_append(top, "Show Sidebar", "win.toggle-sidebar");
    g_menu_append_section(menu, nullptr, G_MENU_MODEL(top));
    g_object_unref(top);

    GMenu *appearance = g_menu_new();
    g_menu_append(appearance, "Light", "win.app-variant::light");
    g_menu_append(appearance, "Dark", "win.app-variant::dark");
    GMenu *appearance_section = g_menu_new();
    g_menu_append_submenu(appearance_section, "Appearance", G_MENU_MODEL(appearance));
    g_menu_append_section(menu, nullptr, G_MENU_MODEL(appearance_section));
    g_object_unref(appearance);
    g_object_unref(appearance_section);

    GMenu *bottom = g_menu_new();
    g_menu_append(bottom, "Quit", "win.quit");
    g_menu_append_section(menu, nullptr, G_MENU_MODEL(bottom));
    g_object_unref(bottom);
```

```
    return G_MENU_MODEL(menu);  // caller owns the returned reference
}
```

`build_ui` puts it on a menu button in the header bar. `gtk_menu_button_set_menu_model` takes its own reference, so `build_ui` releases the one `build_menu` returned:

ch07-actions/src/app_window.cpp

```
GtkWidget *menu_button = gtk_menu_button_new();
gtk_menu_button_set_icon_name(GTK_MENU_BUTTON(menu_button), "open-menu-symbolic");
GMenuModel *menu = build_menu();
gtk_menu_button_set_menu_model(GTK_MENU_BUTTON(menu_button), menu);  // takes its own ref
g_object_unref(menu);
gtk_header_bar_pack_end(GTK_HEADER_BAR(header), menu_button);
```

The Flip button from Chapter 3 is gone.

A save that waits for the burst to end

Toggling the sidebar three times quickly should write the settings file once, not three times. Every change restarts a 500 ms timer, and only the timer that survives writes. This is the main loop strand coming back: a timeout is a source on the main loop, like the click that triggered it.

ch07-actions/src/app_window.cpp

```
// Every change restarts a 500 ms timer; only the last one in a burst writes.
void AppWindow::save_settings_soon() {
    if (save_source_)
        g_source_remove(save_source_);
    save_source_ = g_timeout_add(500, on_save_timeout, this);
}
```

ch07-actions/src/app_window.cpp

```
gboolean AppWindow::on_save_timeout(gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    self->save_source_ = 0;  // returning G_SOURCE_REMOVE destroys the source
    self->save_settings_now();
    return G_SOURCE_REMOVE;
}
```

A pending timeout holds `this`, so the destructor removes it. Returning `G_SOURCE_REMOVE` destroys the source, which is why `on_save_timeout` forgets its id first: removing a source that has already gone is a critical warning.

Done when

The menu shows each item's shortcut, Ctrl+Shift+B shows and hides the sidebar, Dark sticks across a restart, and three quick toggles write the file once. In the recorded run, three toggles 100 ms apart produced a single `settings saved`.

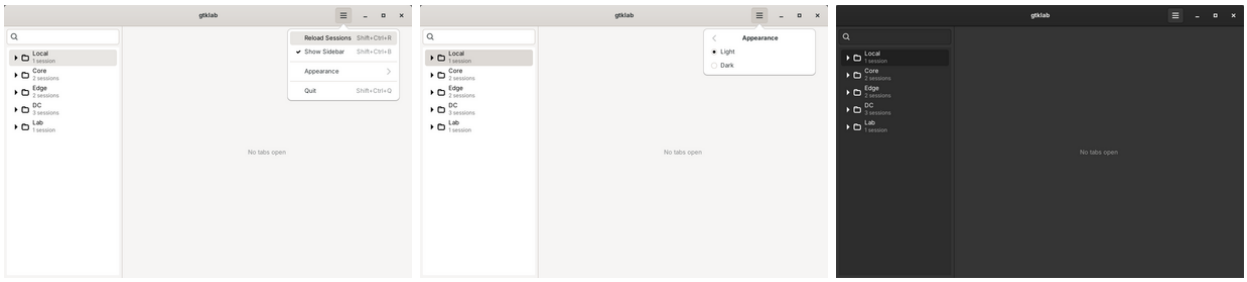


Figure 5: The menu with its shortcuts, the Appearance submenu, and Dark applied.

Debugger drill

Set a breakpoint in `act_app_variant` and choose Dark from the menu. The requested value arrives as the `GVariant` parameter, already a string; the action's current state is still the old one until your code sets it. Step over `g_simple_action_set_state` and open the menu again: the dot has moved.

Break it

In `build_menu`, name the sidebar item's action `toggle-sidebar` without the `win.` prefix. The item turns grey, with no warning of any kind: the menu looked for an action called `toggle-sidebar` in a map that does not exist and concluded it was disabled. When a menu item greys out for no reason, check the prefix first.

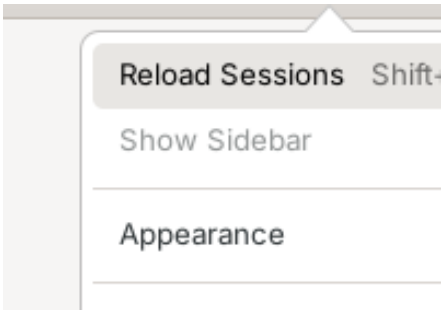


Figure 6: The Break it: Show Sidebar greyed out by a missing prefix.

Common mistakes

Forgetting to set the state in `change_state`. The callback is a request; the state changes only if you change it.

Writing a string state without its quotes. The state is parsed as GVariant text, so a string needs its own quotes: `"'light'"`, not `"light"`.

A shortcut on a key a widget needs. Application accelerators and the capture-phase controller both run on the window before the focused widget sees the key. Bind a plain letter and the search entry can no longer type it; bind Ctrl+C and, from Chapter 13, the terminal never receives it.

A timeout that outlives its object. Any `g_timeout_add` or `g_idle_add` that holds `this` must be removed in the destructor.

Read in tmssh

`install_actions`, including the `accels` table; `build_menu`, whose theme list is built from data; and `save_settings_soon`. Note `act_log` and `sync_log_action` for a check item whose state follows the current tab; gtklab does the same in Chapter 14.

Review

1. What is the difference between an action's parameter and its state? Give one gtklab action that has only a state and one that has both.
2. A menu item is grey and nothing is printed. List the causes you would check, in order.
3. Remove `g_simple_action_set_state` from `act_toggle_sidebar`. Describe what the first and the second Ctrl+Shift+B do to the sidebar and to the menu's check mark.
4. Why is the initial state of `app-variant` written `"'light'"` in the `GActionEntry` table, and what is `"light"` without the inner quotes?
5. Every gtklab action is a `win.` action, Quit included. What would an `app.quit` have to do that `win.quit` does not?
6. gtklab registers each shortcut twice. Which registration makes the menu show the key, which one runs the action, and why does a key press not run it twice?
7. Why does `~AppWindow` remove `save_source_`, and why does `on_save_timeout` set it to 0 before returning `G_SOURCE_REMOVE`?
8. The lab's `activate` adds `app.refresh` with a pointer to its window's status label. What happens to Refresh if the application opens a second window with the same code?

Questions for deeper thought. A tool has a Connect command that only makes sense when a session is selected. Is "disabled" the right answer, or should the command explain why it cannot run? Which commands in a tool you use daily belong to the application, which to a window, and which to one row or tab? A terminal needs Ctrl+C, Ctrl+D and Ctrl+W; which keys are left for the tool's own shortcuts?

Exercises

1. **Collapse all.** Add a `win.collapse-all` action in `install_actions` in `ch07-actions`, a "Collapse All" item in `build_menu`'s top section and `<Control><Shift>k` in `kAccels`. Done when the menu shows the key and both the item and the key close every folder.
2. **Not now.** In `load_sessions_file`, disable `win.reload` when the sessions file could not be read and enable it when it could (`g_simple_action_set_enabled`). Done when, with the file made unreadable, the menu item is grey and Ctrl+Shift+R does nothing.
3. **A button for the same command.** Add a Reload button to the header bar in `build_ui` with `gtk_actionable_set_action_name(..., "win.reload")`. Done when the button, the menu item and Ctrl+Shift+R all stop at one breakpoint in `act_reload`, and the button greys out with the menu item when Exercise 2 disables the action.
4. **Debounce the divider.** Connect `notify::position` on `paned_` to a handler that stores the position in `settings_` and calls `save_settings_soon`. Done when dragging the divider across the window writes the settings file once, after you let go and wait half a second.

Isolated lab

gtklab's actions are wired to a tree, a settings file and a debounced save. The lab keeps only the wiring: one stateless `app.refresh` and one stateful `win.details`, reached from two buttons, a menu and Ctrl+R. It is a separate program so that the one thing on the screen is the claim of this chapter: several controls, one command.

labs/lab07-actions/src/main.cpp

```
// Lab 7: one command, three ways in. A button, a menu item and Ctrl+R all
// activate app.refresh; win.details is a stateful action.
#include <gtk/gtk.h>

static void on_refresh(GSimpleAction *, GVariant *, gpointer data) {
    GtkLabel *status = GTK_LABEL(data);
    gtk_label_set_text(status, "Refreshed through the action");
    g_print("app.refresh activated\n");
}

static void on_toggle(GSimpleAction *action, GVariant *, gpointer data) {
    GVariant *state = g_action_get_state(G_ACTION(action));  // a new reference
    gboolean enabled = !g_variant_get_boolean(state);
    g_variant_unref(state);
    g_simple_action_set_state(action, g_variant_new_boolean(enabled));
    gtk_label_set_text(GTK_LABEL(data), enabled ? "Details: ON" : "Details: OFF");
    g_print("win.details state: %s\n", enabled ? "true" : "false");
}

static void activate(GtkApplication *app, gpointer) {
    GtkWidget *window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "Actions Laboratory");
    gtk_window_set_default_size(GTK_WINDOW(window), 520, 280);

    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 14);
    gtk_widget_set_margin_top(box, 24);
    gtk_widget_set_margin_bottom(box, 24);
    gtk_widget_set_margin_start(box, 24);
    gtk_widget_set_margin_end(box, 24);
    gtk_window_set_child(GTK_WINDOW(window), box);

    GtkWidget *status = gtk_label_new("Choose a command");
    gtk_box_append(GTK_BOX(box), status);

    GSimpleAction *refresh = g_simple_action_new("refresh", nullptr);
    g_signal_connect(refresh, "activate", G_CALLBACK(on_refresh), status);
    g_action_map_add_action(G_ACTION_MAP(app), G_ACTION(refresh));
    g_object_unref(refresh);

    GSimpleAction *details = g_simple_action_new_stateful(
        "details", nullptr, g_variant_new_boolean(FALSE));
    g_signal_connect(details, "activate", G_CALLBACK(on_toggle), status);
    g_action_map_add_action(G_ACTION_MAP(window), G_ACTION(details));
    g_object_unref(details);

    GtkWidget *refresh_button = gtk_button_new_with_label("Refresh");
    gtk_actionable_set_action_name(GTK_ACTIONABLE(refresh_button), "app.refresh");
    gtk_box_append(GTK_BOX(box), refresh_button);

    GtkWidget *details_button = gtk_button_new_with_label("Toggle details");
    gtk_actionable_set_action_name(GTK_ACTIONABLE(details_button), "win.details");
    gtk_box_append(GTK_BOX(box), details_button);
```

```
    GMenu *menu = g_menu_new();
    g_menu_append(menu, "Refresh", "app.refresh");
    g_menu_append(menu, "Toggle details", "win.details");
    GtkWidget *menu_button = gtk_menu_button_new();
    gtk_menu_button_set_label(GTK_MENU_BUTTON(menu_button), "Commands");
    gtk_menu_button_set_menu_model(GTK_MENU_BUTTON(menu_button), G_MENU_MODEL(menu));
    gtk_box_append(GTK_BOX(box), menu_button);
    g_object_unref(menu);

    const char *shortcuts[] = {"<Control>r", nullptr};
    gtk_application_set_accels_for_action(app, "app.refresh", shortcuts);
    gtk_window_present(GTK_WINDOW(window));
}

int main(int argc, char **argv) {
    GtkApplication *app = gtk_application_new(
        "io.github.gtklab.lab07", G_APPLICATION_NON_UNIQUE);
    g_signal_connect(app, "activate", G_CALLBACK(activate), nullptr);
    int result = g_application_run(G_APPLICATION(app), argc, argv);
    g_object_unref(app);
    return result;
}
```

`on_toggle` reads the state with `g_action_get_state`, which returns a new reference; the lab releases it with `g_variant_unref` before setting the new state.

Build and run from `labs/lab07-actions`:

```
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

In the recorded run, the Refresh button and Ctrl+R each printed `app.refresh activated`, Toggle details printed `win.details state: true`, and the menu showed "Refresh Ctrl+R" and a checked "Toggle details".

Stop at	What it shows
`on_refresh`	reached from the button, the menu item and Ctrl+R; run `bt` each time and find where the three stacks join
`on_toggle`	`state` holding the old value; the new one exists only after `g_simple_action_set_state`
`g_simple_action_set_state`	the menu's check mark and the label both follow from this one call

Experiments:

1. **Disable the command.** Call `g_simple_action_set_enabled(refresh, FALSE)` before the unref. In the recorded run the Refresh button and the Refresh menu item were both grey, and Ctrl+R printed nothing.
2. **Drop the prefix.** Set the details button's action name to `details`. The button was grey and the terminal said nothing: no control above it answers an unprefixed name.

3. **Let GIO toggle.** Replace the `activate` connection with a `change-state` handler that calls `g_simple_action_set_state(action, value)` and updates the label from `value`. Clicking Toggle details twice printed `true` then `false`: with no `activate` handler, the action flipped its boolean and asked for the change itself.
4. **Steal a key.** Change the accelerator to plain `r` and add a `GtkEntry` above the buttons. Typing `error` into the entry left `eo` in it and printed `app.refresh activated` three times: the application's accelerators ran before the focused entry saw the key.



Figure 7: Lab 7 after Toggle details.

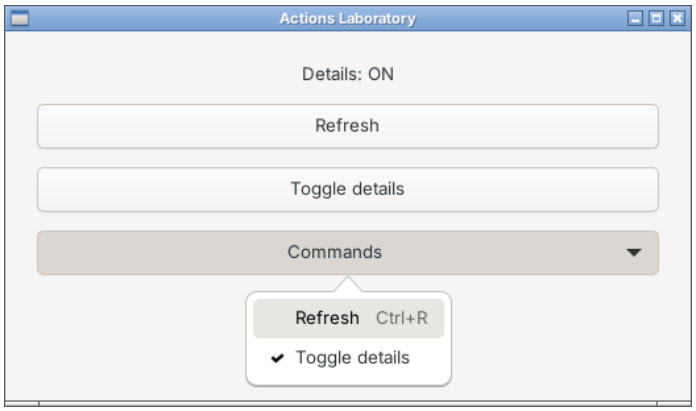


Figure 8: The lab's menu: the shortcut comes from the application's accelerators, the check mark from the action's state.

Checkpoint B — Session browser

Spec. Chapter 7, plus:

- an **Edit Sessions File** action (Ctrl+Shift+E) that opens the sessions file in the user's editor; `gtk_show_uri` is the 4.8 API for it;
- a red banner above the tree when the sessions file is missing or malformed, giving the reason, with the last good tree kept underneath it rather than emptied;
- Reload (Ctrl+Shift+R) that re-reads the file without collapsing open folders.

Done when. Editing the file and pressing Ctrl+Shift+R updates the tree with open folders still open; a `port=abc` line shows the banner; fixing the file and reloading clears it. In the recorded run the banner read "Sessions file: session 'broken': port is not a number".

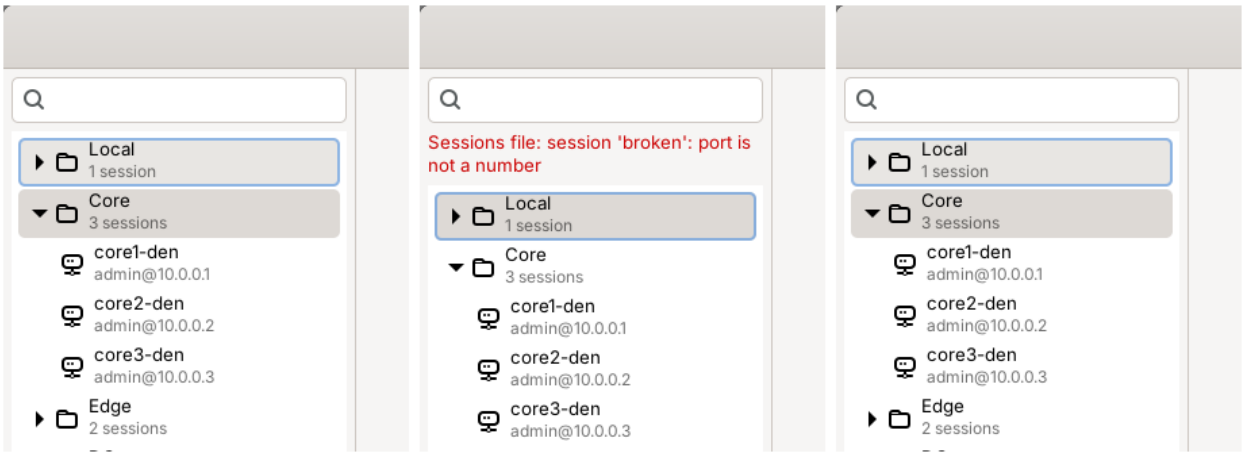


Figure 9: Checkpoint B: a session added and reloaded; a bad port shows the banner over the last good tree; fixed and reloaded.

Hints. The banner is a `GtkLabel` with the `error` style class, which the theme colours red, and wrapping turned on. Hide it with `gtk_widget_set_visible` when the file is good. Keep the last good folders by loading into a temporary and swapping only on success.

Reference. `ckB-browser/`.

Chapter 8 — Tabs that own things

Activating a session now opens a tab. A tab in a terminal application is not just a widget; from Chapter 11 it owns a child process, a pseudo-terminal, and later a log file. This chapter sets up the ownership so that all of that is cleaned up in exactly one place, by closing the tab.

Recall

What does `g_object_set_data_full` guarantee (Chapter 2)? Why does the settings save use a timeout (Chapter 7)? What does the `win.` prefix mean?

In this chapter

- Separate application, window and tab scope, and say which object owns each.
- Make a notebook page own its `Tab`, so the notebook is the only list of tabs.
- Resolve the current tab when a command runs, instead of keeping a pointer to it.
- Trace a tab close from any path to `~Tab`, and say what is still alive at each step.

Concepts

A tab is a workspace

A tab looks like a label over a panel, but in a tool it stands for something running: from Chapter 11 a child process and its pseudo-terminal, later a log file. Two rules follow. Switching tabs changes which workspace you see, never which ones exist: an inactive tab keeps its process, its scrollback and its zoom. And closing a tab must release everything that tab holds, every time, by whatever route it was closed.

Three scopes

Chapter 7 put commands on the window (`win.`) or the application (`app.`). Tabs add a third scope that GTK has no prefix for.

Scope	In gtklab	Lives until	Commands reach it through
Application	`GtkApplication`, which holds its windows	the process exits	`app.` actions
Window	`AppWindow`: settings, the session tree, the notebook, the debounced save	the window is destroyed	`win.` actions

Scope	In gtklab	Lives until	Commands reach it through
Tab	`Tab`: a copy of its `Session`, pointers to its widgets; later its zoom and pty	its page is finalized	the notebook's current page, looked up when the command runs

Keep state at the narrowest scope that needs it. A tab's zoom level belongs to the tab (Chapter 9), not to the window, or switching tabs would carry one tab's zoom into another. The settings belong to the window: `AppWindow` loads them when it is built and saves them when the window closes.

Each `Tab` holds its own copy of the `Session` it was opened from. Reloading the sessions file replaces the window's folders and rebuilds the tree's items, and a tab that pointed into either would be left pointing at freed memory. A copy is a snapshot: the tab keeps the host and user it connected with, whatever the file says now.

The page owns the tab

There are two ways to keep a C++ object per tab. One is a registry: a `std::vector<std::unique_ptr<Tab>>` in the window, next to the notebook, with each entry remembering its page. The other is the one this book uses and tmssh uses: no registry. A `GtkNotebook` page is a widget, and like any widget it is destroyed when it is removed from its parent. So make the page own its `Tab` through `g_object_set_data_full`, exactly as Chapter 2's button owned its `Counter`.

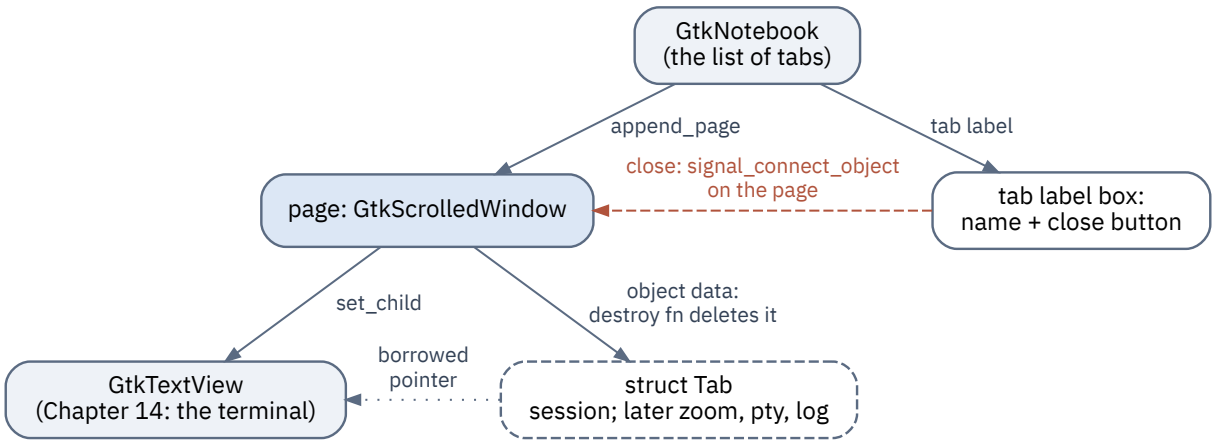


Figure 1: What owns what in a tab. Solid arrows own; the `Tab` hangs off the page as object data. Its widget pointers are borrowed, and the close button's connection is broken if the page goes first.

Question	Registry beside the notebook	Page owns the `Tab`
How many lists of tabs?	Two, kept in sync by hand	One: the notebook
Tab dragged to a new position	The registry's order is now wrong	Nothing to do
Find the `Tab` for a page	Search the registry	`g_object_get_data(page, "tab")`

Question	Registry beside the notebook	Page owns the `Tab`
Who deletes the `Tab`?	Every close path must erase its entry	The page, when it is finalized
Window destroyed with tabs open	Depends on the order the window's members and widgets are torn down	Each page is finalized, each `~Tab` runs

The registry works in a small program. It stops working the first time a close path forgets to erase its entry, or a reorder leaves the indexes wrong. With the page as owner there is nothing to forget: closing a tab becomes "remove the page", and removing the page destroys it, which clears its data, which deletes the `Tab`.

The close button on each tab is connected with `g_signal_connect_object`, with the page as its object. If the page is destroyed some other way first, the connection goes with it, and the button can never call its handler with a dead page.

The `Tab`'s widget pointers are borrowed. The notebook owns the page and the page owns the view; the `Tab` only remembers where they are. That has a consequence for its destructor. A program that put a weak reference on the view and logged the page's teardown printed, in this order:

```
view finalized
page destroy (dispose)
~Tab
```

By the time `~Tab` runs, the view is gone. A tab's destructor releases what GTK does not know about (the pty in Chapter 11, a log file) and never touches its widgets.

Commands find the tab when they run

A command such as Copy or Close Tab acts on "the current tab". Do not keep a `Tab *current_` that you update on every switch; it goes stale the moment a tab closes or a drag reorders the pages. Ask the notebook at the moment the command runs: current page number, then that page, then its `Tab` from the object data. Chapter 10 writes exactly that as `current_tab()`.

There is one trap in the notebook's own signal. Inside a `switch-page` handler, `gtk_notebook_get_current_page` does not yet return the new page. Measured on a three-page notebook, switching to page 2 reported 0 (the old page) inside the handler, and switching away from a page being removed reported -1. Use the page the signal passes you.

Closing is a lifecycle transition

A tab closes in more ways than one: its close button, the Close Tab and Close Other Tabs commands (Chapter 10), its process exiting (Chapter 11), and the window closing with tabs open. All of them end in the same place.

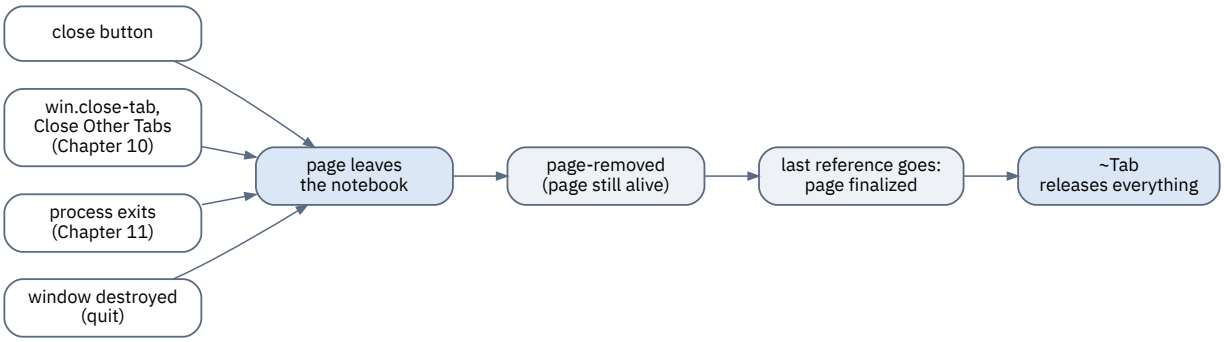


Figure 2: Every close path ends in the same three steps. Cleanup in `~Tab` runs whichever path was taken.

Stage	In gtklab	Where
Ask	Close Tab asks first (Chapter 10)	the action, before anything is removed
Detach	`gtk_notebook_remove_page`	every close path
Update the window	Show "No tabs open" when the last tab goes	`page-removed`
Release	Close the pty, flush the log (Chapter 11 on)	`~Tab`

Closing the current tab of three, the recorded order was: `switch-page` to its neighbour, then `page-removed` with the page still alive, then `~Tab`. That last step happens when the page's last reference goes, not when it leaves the notebook. A test that took an extra reference on a page before removing it saw `page-removed`, then a page with no parent and its `Tab` still attached, and `~Tab` only when the extra reference was dropped. Chapter 10's confirm dialog depends on exactly that.

Inside `~Tab`, the order matters too. C++ destroys members in reverse order of declaration, and tmssh uses that: its `Tab` declares the log before the pty, so the pty is closed first and the log's footer is the last thing written.

Build

`src/tab.h`:

ch08-tabs/src/tab.h

```
// Tab: everything one notebook page owns. The page owns its Tab through
// g_object_set_data_full, so removing the page deletes the Tab.
#pragma once

#include <gtk/gtk.h>

#include "core/session.h"

struct Tab {
    Session session;
    GtkWidget *page = nullptr;   // not owned: the notebook owns the page
    GtkWidget *view = nullptr;   // not owned: the page owns the view
    GtkWidget *label = nullptr;  // not owned: the tab label box owns it
```

```
    explicit Tab(const Session &s) : session(s) {}
    ~Tab() { g_print("~Tab %s\n", session.name.c_str()); }  // Chapter 11: closes the pty here
};
```

`add_tab` builds the page, attaches the `Tab`, and builds the tab label with its close button. The text view is a placeholder until Part III:

ch08-tabs/src/app_window.cpp

```
Tab *AppWindow::add_tab(const Session &s) {
    auto *tab = new Tab(s);

    // Placeholder content until Part IV: a text view showing the command.
    const char *shell = g_getenv("SHELL");
    std::string line = "$";
    for (const std::string &arg : ssh_argv(s, shell ? shell : ""))
        line += " " + arg;
    line += "\n(the terminal arrives in Chapter 14)\n";
    tab->view = gtk_text_view_new();
    gtk_text_view_set_monospace(GTK_TEXT_VIEW(tab->view), TRUE);
    gtk_text_view_set_left_margin(GTK_TEXT_VIEW(tab->view), 8);
    gtk_text_view_set_top_margin(GTK_TEXT_VIEW(tab->view), 8);
    gtk_text_buffer_set_text(gtk_text_view_get_buffer(GTK_TEXT_VIEW(tab->view)), line.c_str(), -1);

    tab->page = gtk_scrolled_window_new();
    gtk_scrolled_window_set_child(GTK_SCROLLED_WINDOW(tab->page), tab->view);

    // The page owns the Tab: removing the page deletes it.
    g_object_set_data_full(G_OBJECT(tab->page), "tab", tab,
                           [](gpointer p) { delete static_cast<Tab *>(p); });

    // Tab label: name + close button. The handler gets the page, and is
    // disconnected automatically if the page goes first (Chapter 2).
    GtkWidget *label_box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 6);
    tab->label = gtk_label_new(s.name.c_str());
    GtkWidget *close = gtk_button_new_from_icon_name("window-close-symbolic");
    gtk_button_set_has_frame(GTK_BUTTON(close), FALSE);
    gtk_widget_set_tooltip_text(close, "Close tab");
    g_signal_connect_object(close, "clicked", G_CALLBACK(on_tab_close_clicked), tab->page,
                            GConnectFlags(0));
    gtk_box_append(GTK_BOX(label_box), tab->label);
    gtk_box_append(GTK_BOX(label_box), close);

    const int n = gtk_notebook_append_page(GTK_NOTEBOOK(notebook_), tab->page, label_box);
    gtk_notebook_set_tab_reorderable(GTK_NOTEBOOK(notebook_), tab->page, TRUE);
    gtk_notebook_set_current_page(GTK_NOTEBOOK(notebook_), n);
    gtk_widget_grab_focus(tab->view);
    return tab;
}
```

The close handler gets the page and finds its notebook. A page's parent is the notebook's internal stack, not the notebook itself, so it asks for the nearest ancestor of the right type:

ch08-tabs/src/app_window.cpp

```
void AppWindow::on_tab_close_clicked(GtkButton *, gpointer page_ptr) {
    GtkWidget *page = GTK_WIDGET(page_ptr);
    // The page's parent is the notebook's internal stack, not the notebook.
    GtkWidget *notebook = gtk_widget_get_ancestor(page, GTK_TYPE_NOTEBOOK);
    if (!notebook)
        return;
    const int n = gtk_notebook_page_num(GTK_NOTEBOOK(notebook), page);
    if (n >= 0)
```

```
        gtk_notebook_remove_page(GTK_NOTEBOOK(notebook), n);
}
```

The window's stack shows the notebook when a page is added, and the "No tabs open" hint when the last one goes:

ch08-tabs/src/app_window.cpp

```
void AppWindow::on_page_added(GtkNotebook *, GtkWidget *, guint, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    gtk_stack_set_visible_child_name(GTK_STACK(self->stack_), "tabs");
}
```

ch08-tabs/src/app_window.cpp

```
void AppWindow::on_page_removed(GtkNotebook *nb, GtkWidget *, guint, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    g_debug("page-removed, %d left", gtk_notebook_get_n_pages(nb));
    if (gtk_notebook_get_n_pages(nb) == 0)
        gtk_stack_set_visible_child_name(GTK_STACK(self->stack_), "empty");
}
```

Activating a session in the tree now calls `add_tab` instead of printing.

Done when

Sessions open as tabs; tabs can be reordered by dragging; the close button removes a tab; and `~Tab` prints once for every tab, whether it was closed or still open at quit.

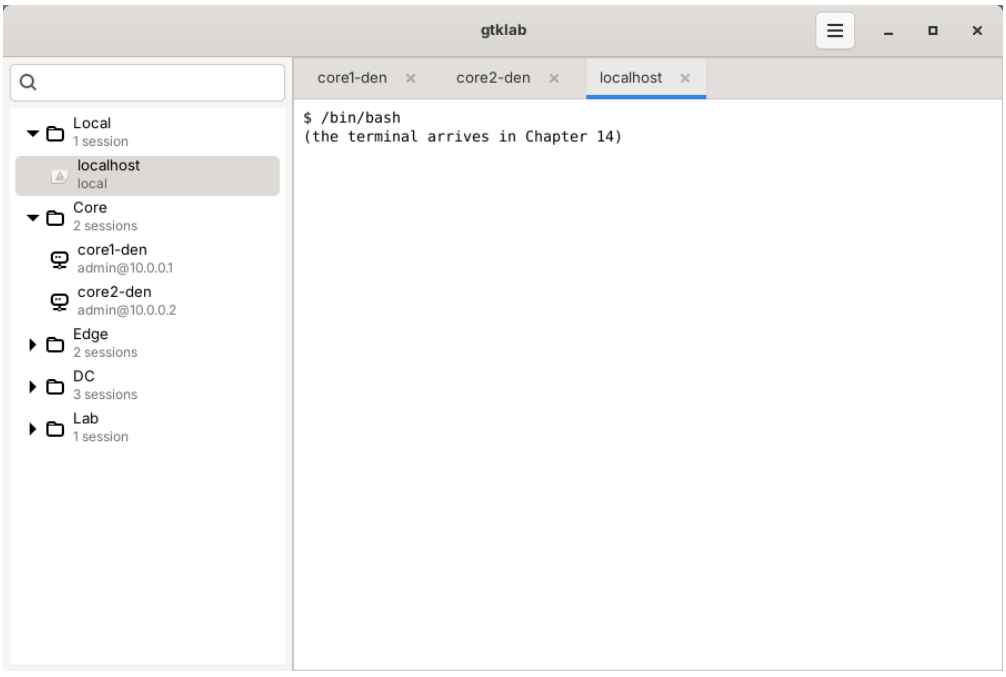


Figure 3: Three tabs opened from the tree.

Debugger drill

Set breakpoints in `~Tab` and in `on_page_removed`. Close a tab with its button. The order is: `page-removed` first, while the page still exists, and then `~Tab` as the page is destroyed.

Break it (keep this one)

You do not have to write this bug; Chapter 8 as written already has it. Open three tabs and quit. The log shows:

```
Gtk-WARNING: Child name 'empty' not found in GtkStack
```

When the window is destroyed, its widgets are torn down. By the time the notebook removes its pages, the stack's "empty" page has already gone, and your `page-removed` handler, which holds `this`, runs against a window that is half dismantled. Here it only produces a warning. Leave it in. Chapter 16 shows what it really is, why the usual memory tool cannot see it, and fixes it the way tmssh does.

Common mistakes

Keeping a list of tabs beside the notebook. The notebook already is the list. A second one has to be kept in sync on every open, close, reorder and drag, and will not be.

Using `gtk_widget_get_parent` to find the notebook. It returns an internal stack.

Cleaning up in the close button's handler. Tabs also close when their process exits, when the window closes, and through Close Other Tabs. Cleanup belongs in the destructor that every path reaches.

Read in tmssh

`add_tab`, the deleter lambda passed to `g_object_set_data_full`, and the `page-removed` handler in `build_ui`.

Review

1. Opening a session gives a tab a copy of its `Session`, not a pointer into the tree's items. What goes wrong with the pointer when the sessions file is reloaded?
2. A tab is dragged from position 3 to position 1. What does the code have to update, and why?
3. Why does Close Tab look up the current tab when it runs instead of reading a `current_` member kept up to date by `switch-page`?
4. Inside a `switch-page` handler, you call `gtk_notebook_get_current_page`. What do you get, and what should you use instead?
5. List the close paths a tab will have by Chapter 11. Why does cleanup belong in `~Tab` rather than in any of them?
6. Something holds an extra reference on a page while the tab is closed. Which of `page-removed` and `~Tab` runs at `remove_page`, and when does the other one run?
7. Why must `~Tab` not touch `tab->view`?
8. The close button is connected with `g_signal_connect_object` and the page as the object. What could happen with a plain `g_signal_connect` and the page as user data?

Questions for deeper thought. A future version detaches a tab into a new window. With the page owning its `Tab`, what has to move, and what would a window-owned registry have made harder? Which

state in a tool you know belongs at tab scope, but is kept at window scope today? If closing a tab had to wait for a remote end to say goodbye, where would that waiting state live, and what would the window show meanwhile?

Exercises

1. **Tab count in the title.** In `ch08-tabs/src/app_window.cpp`, make `on_page_added` and `on_page_removed` set the window title to "gtklab (3 tabs)", or plain "gtklab" with none. Done when the title is right after opening, closing and dragging tabs.
2. **Follow the current tab.** In `build_ui`, connect `switch-page` and set the title to the session name of the page the signal passes. Done when the title follows clicks on tabs and is right after closing the current tab while others remain; then try `gtk_notebook_get_current_page` in the handler and see which title you get.
3. **Focus instead of duplicating.** In `activate_session`, walk the notebook's pages, read each page's `Tab` with `g_object_get_data`, and if one has the same session name, make it current instead of calling `add_tab`. Done when double-clicking an open session switches to its tab, and `~Tab` still prints once per tab at quit.
4. **Watch the order.** In `add_tab`, add a `g_object_weak_ref` on `tab->view` that prints "view gone" with the session name. Done when you can say, from the log, whether the view or the `Tab` goes first on a close and at quit, and why that rules out touching widgets in `~Tab`.

Isolated lab

`labs/lab08-tabs` is the chapter's ownership design with everything else taken away: no session tree, no actions, no settings. Each tab is a `Session` with a counter, a page that owns it, and three buttons that act on the current tab. Small enough to hold in your head, so you can break it on purpose and see exactly what changes.

The `Session` prints when it is created and destroyed:

labs/lab08-tabs/src/main.cpp

```
struct Session {
    int id;
    int counter = 0;
    GtkWidget *label = nullptr;  // borrowed: the page owns it
    explicit Session(int number) : id(number) { g_print("CREATE session %d\n", id); }
    ~Session() { g_print("DESTROY session %d, count=%d\n", id, counter); }
};
```

The page owns it, and commands resolve it from the notebook when they run:

labs/lab08-tabs/src/main.cpp

```
// Resolved at the moment a command runs, from the notebook itself.
static Session *active_session(WindowState *state) {
    GtkNotebook *nb = GTK_NOTEBOOK(state->notebook);
    int index = gtk_notebook_get_current_page(nb);
    return index < 0 ? nullptr : session_for(gtk_notebook_get_nth_page(nb, index));
}
```

labs/lab08-tabs/src/main.cpp

```
static void add_tab(WindowState *state) {
    auto *session = new Session(state->next_id++);
    GtkWidget *page = gtk_box_new(GTK_ORIENTATION_VERTICAL, 12);
    gtk_widget_set_margin_top(page, 28);
    gtk_widget_set_margin_start(page, 28);
    session->label = gtk_label_new("Counter: 0");
    gtk_box_append(GTK_BOX(page), session->label);

    // The page owns the session: it is deleted when the page is destroyed,
    // whether the tab was closed or the window went away.
    g_object_set_data_full(G_OBJECT(page), "session", session,
                           [](gpointer p) { delete static_cast<Session *>(p); });

    std::string name = "Session " + std::to_string(session->id);
    int index = gtk_notebook_append_page(GTK_NOTEBOOK(state->notebook), page,
                                         gtk_label_new(name.c_str()));
    gtk_notebook_set_current_page(GTK_NOTEBOOK(state->notebook), index);
}
```

labs/lab08-tabs/src/main.cpp

```
static void close_active(WindowState *state) {
    int index = gtk_notebook_get_current_page(GTK_NOTEBOOK(state->notebook));
    if (index >= 0)
        gtk_notebook_remove_page(GTK_NOTEBOOK(state->notebook), index);  // destroys page and session
}
```

The notebook's handlers carry the status label, connected with `g_signal_connect_object` so that teardown order at quit cannot matter:

labs/lab08-tabs/src/main.cpp

```
// The notebook's handlers get the status label, not the window state, and
// are connected with g_signal_connect_object (Chapter 2): when the label is
// destroyed, they are disconnected. So whichever of the label and the
// notebook GTK tears down first at quit, no handler touches a dead widget.
// switch-page passes the page being switched to.
static void on_switch(GtkNotebook *nb, GtkWidget *page, guint, gpointer status) {
    show_status(GTK_WIDGET(status), nb, session_for(page));
}
```

labs/lab08-tabs/src/main.cpp

```
static void on_page_removed(GtkNotebook *nb, GtkWidget *, guint, gpointer status) {
    if (gtk_notebook_get_n_pages(nb) == 0)
        show_status(GTK_WIDGET(status), nb, nullptr);
}
```

Build and run it:

```
cd labs/lab08-tabs
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

Or open the folder in VS Code and run the "lab" configuration.

Stop at	What you see
`add_tab`	A new `Session` and page; nothing holds the `Session` except the page's data

Stop at	What you see
`active_session`	The current page number, then the `Session` from that page's data
`on_switch`	`page` is the new page; `gtk_notebook_get_current_page(nb)` is not yet its number (it was -1 when the current tab was being closed)
`close_active`	The index being removed
`on_page_removed`	The page is out of the notebook, but its `Session` is not yet destroyed
`Session::~Session`	Called from `g_datalist_clear` inside `g_object_unref`: the page's last reference just went

Experiments:

1. Increment Session 2 three times, switch to Session 1 and increment once. In the recorded run Session 1 showed count 1 and Session 2 kept 3: the inactive tab was not touched. Quit printed `DESTROY session 1, count=1` and `DESTROY session 2, count=3`.
2. With breakpoints in `on_switch`, `on_page_removed` and `Session::~Session`, close the current tab of two. The recorded order was `on_switch` (to the remaining tab), then `on_page_removed`, then `~Session`.
3. Replace `g_object_set_data_full` in `add_tab` with `g_object_set_data` and no destroy function. Predict what quit prints, then run it.
4. In `close_active`, call `g_object_ref` on the page before removing it and never unref it. Predict which of `on_page_removed` and the `DESTROY` line still appears, then check.

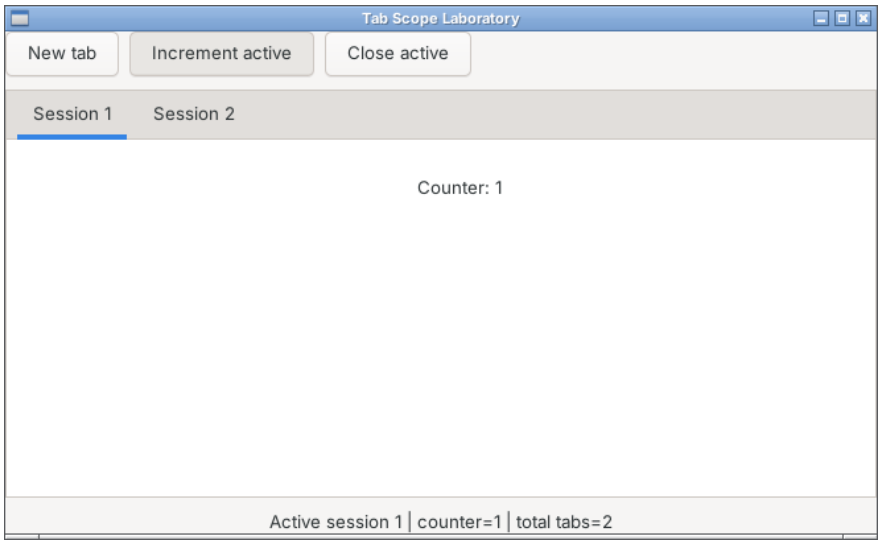


Figure 4: The lab after incrementing Session 1 once: the status line shows the current tab's count and the total.

Chapter 9 — Event controllers and propagation

A right-click on a tab should open gtklab's menu, and Ctrl+wheel should zoom, even though the click and the scroll land on a text view that has its own ideas about both. Getting that right needs the one part of GTK 4 that is most different from GTK 3, and from most other toolkits: how input reaches widgets.

Recall

In what order do `page-removed` and `~Tab` run (Chapter 8)? Which phase did the Chapter 7 shortcut controller use, and why?

In this chapter

- Pick the controller for a kind of input and attach it to the right widget.
- Predict which controllers see an event, and in what order, from the widget tree and each controller's phase.
- Stop an event deliberately: claim or deny a gesture, return `TRUE` or `FALSE` from a discrete handler.
- Give a container first refusal over input without breaking the widgets inside it.

Concepts

No event handlers, only controllers

GTK 3 code handled input by connecting to signals every widget had, `button-press-event`, `key-press-event`, `scroll-event`, and returning `TRUE` to stop the event. A widget that wanted input also had to ask for it with an event mask. GTK 4 removed all of it. A program that looked those signals up on `GtkWidget` got signal id 0 for each one, the 4.8 headers have no `gtk_widget_add_events`, and `GtkWidgetClass` has no event functions to override.

GTK 3	GTK 4
`button-press-event` on the widget	a `GtkGestureClick` added to the widget
`scroll-event`, return `TRUE` to stop	`GtkEventControllerScroll`, return `TRUE` to stop
`key-press-event`	`GtkEventControllerKey`, or a `GtkShortcutController` for shortcuts
an event mask before anything arrives	nothing: a controller gets the input it handles
override a class's event function	add a controller in the widget's init or where it is built

Input is handled by **event controllers** that you attach with `gtk_widget_add_controller`, which takes ownership of the controller. A controller belongs to one widget and dies with it. Each controller handles one kind of input:

Controller	Handles
`GtkGestureClick`	presses and releases, single or multiple clicks
`GtkGestureDrag`	press, move, release, with the offset
`GtkEventControllerKey`	key presses and releases
`GtkEventControllerScroll`	wheel and touchpad scrolling
`GtkEventControllerMotion`	pointer enter, leave and motion
`GtkEventControllerFocus`	keyboard focus in and out
`GtkShortcutController`	keyboard shortcuts mapped to actions

Two defaults are worth knowing because they bite. A new click gesture listens to button 1 only (`gtk_gesture_single_get_button` returned 1); pass 0 to `gtk_gesture_single_set_button` to hear every button. And every controller starts in the bubble phase, including key and shortcut controllers, which is why Chapter 7 had to set the capture phase on its shortcut controller explicitly.

Where an event goes

When an event arrives, GTK first finds its **target**. A pointer event's target is the widget under the pointer; a key event's target is the widget with keyboard focus. Then it walks the chain of ancestors between the window and the target in three phases. **Capture** runs from the window down to the target, giving each ancestor a chance first. **Target** runs controllers on the target itself. **Bubble** runs from the target back up to the window. A controller runs in the phase you set with `gtk_event_controller_set_propagation_phase`.

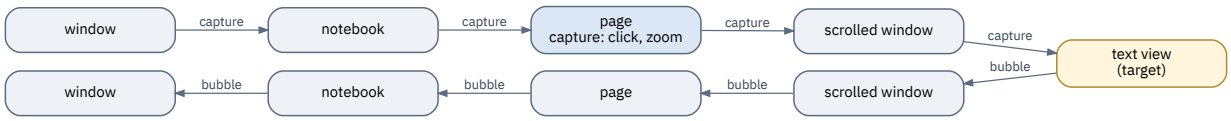


Figure 1: The three phases for a click on the text view inside a tab. Capture-phase controllers on the page run before anything inside it; bubble-phase controllers run only if the event is still unhandled on the way back up.

A test program with a box holding a label, and a click gesture in each of the three phases on both, printed this for one click on the label:

```
pressed: parent capture
pressed: child capture
pressed: child target
pressed: child bubble
pressed: parent bubble
```

The box's target-phase gesture never ran: the box was not the target. A target-phase controller hears only input aimed at its own widget, never at a child.

Stopping an event

How a controller stops an event depends on its kind.

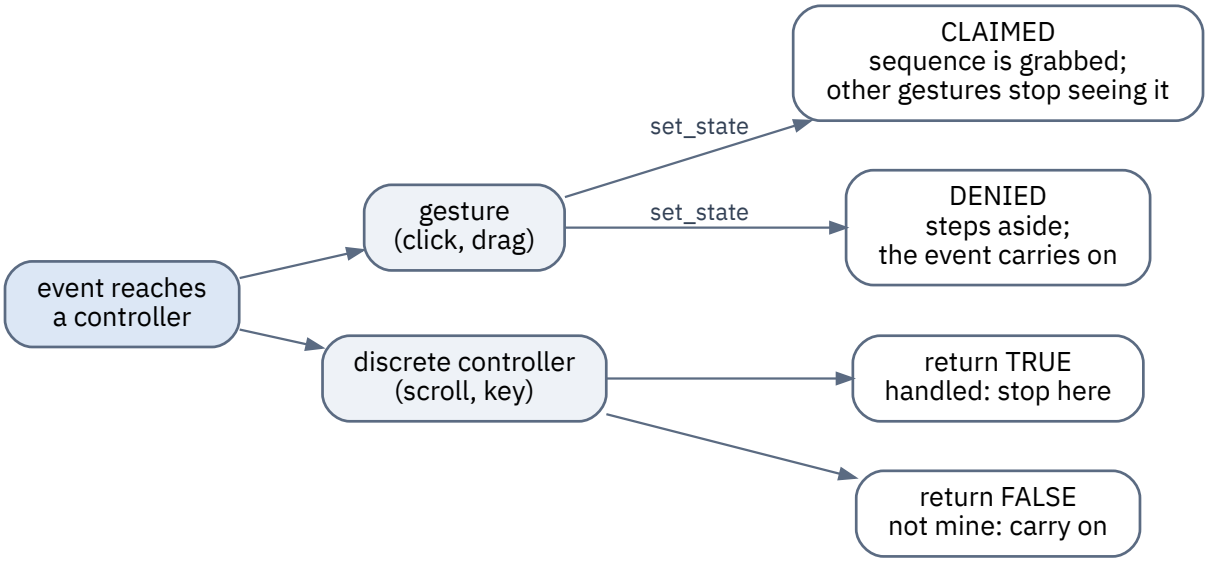


Figure 2: Gestures decide by setting a sequence state; discrete controllers decide by their return value.

A **gesture** decides about the whole press-move-release sequence. Setting its state to `GTK_EVENT_SEQUENCE_CLAIMED` takes the sequence, and no other gesture sees it. `GTK_EVENT_SEQUENCE_DENIED` steps aside, and the event carries on. In the test program, making the label's bubble gesture claim the press removed the last line: the box's bubble gesture never ran. A **discrete** controller such as scroll or key returns a boolean from its signal: `TRUE` means handled, stop here; `FALSE` means not mine, carry on.

So the pattern for "the page gets first refusal" is a controller on the page in the capture phase, which claims what it wants and denies or returns `FALSE` for everything else.

Choosing a phase

You want	Phase	Attach to	In gtklab
First refusal over everything inside a container	capture	the container	the page's click and zoom; the window's shortcuts (Chapter 7)
Input on a widget, unless something inside used it	bubble (the default)	the widget	the text view's logging gesture
Input aimed at this widget only, not its children	target	the widget	not used
A controller switched off for now	none	the widget	not used

Capture is powerful and easy to overuse. A capture controller that claims a little too much breaks every widget inside its container, and the widgets cannot do anything about it. Claim exactly what you handle and let the rest through. That discipline is what Part III needs: a terminal wants almost every key and every mouse button, the application wants a few shortcuts and a right-click menu, and a full-screen program inside the terminal may want the mouse too. Each of those is a controller, a phase and a claim.

What the handler is given

A controller's handler gets the controller, the event's details (button, position, scroll deltas) and your user data. Read the event's modifier keys with `gtk_event_controller_get_current_event_state` inside the handler; it is the state for the event being handled, not whatever the keyboard happens to be doing now.

The user data here is the `Tab`, a plain pointer, with plain `g_signal_connect`. That is safe for the reason Chapter 8 gave: the controllers belong to the page, the page owns the `Tab`, and they go together. A handler whose data had a different owner would need `g_signal_connect_object` or a disconnect, as Chapter 2 showed.

Build

In `add_tab`, two capture-phase controllers on the page and one ordinary bubble-phase gesture on the text view, there only to show when the view gets a click:

ch09-input/src/app_window.cpp

```
// Capture phase on the page: these run before anything inside it.
GtkGesture *click = gtk_gesture_click_new();
gtk_gesture_single_set_button(GTK_GESTURE_SINGLE(click), 0);  // all buttons
gtk_event_controller_set_propagation_phase(GTK_EVENT_CONTROLLER(click), GTK_PHASE_CAPTURE);
g_signal_connect(click, "pressed", G_CALLBACK(on_page_pressed), tab);
gtk_widget_add_controller(tab->page, GTK_EVENT_CONTROLLER(click));

GtkEventController *scroll = gtk_event_controller_scroll_new(GTK_EVENT_CONTROLLER_SCROLL_VERTICAL);
gtk_event_controller_set_propagation_phase(scroll, GTK_PHASE_CAPTURE);
g_signal_connect(scroll, "scroll", G_CALLBACK(on_zoom_scroll), tab);
gtk_widget_add_controller(tab->page, scroll);

// Default (bubble) phase on the text view: runs only if nothing above
// claimed the click.
GtkGesture *view_click = gtk_gesture_click_new();
g_signal_connect(view_click, "pressed", G_CALLBACK(on_view_pressed), tab);
gtk_widget_add_controller(tab->view, GTK_EVENT_CONTROLLER(view_click));
```

The click gesture listens to every button (`set_button(..., 0)`) and decides per press. A right-click is claimed, so the view never sees it; Chapter 10 opens a menu there. Anything else is denied, so the view gets it:

ch09-input/src/app_window.cpp

```
void AppWindow::on_page_pressed(GtkGestureClick *gesture, int, double x, double y, gpointer data) {
    auto *tab = static_cast<Tab *>(data);
    const guint button = gtk_gesture_single_get_current_button(GTK_GESTURE_SINGLE(gesture));
    if (button == GDK_BUTTON_SECONDARY) {
        // Claim: no other gesture sees this click. Chapter 10 shows a menu here.
        gtk_gesture_set_state(GTK_GESTURE(gesture), GTK_EVENT_SEQUENCE_CLAIMED);
        g_debug("page capture: button %u claimed (menu at %.0f,%.0f) [%s]", button, x, y,
```

```
                tab->session.name.c_str());
    } else {
        // Deny: step aside so the widgets inside get the click.
        gtk_gesture_set_state(GTK_GESTURE(gesture), GTK_EVENT_SEQUENCE_DENIED);
        g_debug("page capture: button %u denied", button);
    }
}
```

The scroll handler takes Ctrl+wheel and lets a plain wheel through. The modifier state comes from the event being handled, with `gtk_event_controller_get_current_event_state`:

ch09-input/src/app_window.cpp

```
gboolean AppWindow::on_zoom_scroll(GtkEventControllerScroll *ctl, double, double dy, gpointer data) {
    auto *tab = static_cast<Tab *>(data);
    GdkModifierType mods = gtk_event_controller_get_current_event_state(GTK_EVENT_CONTROLLER(ctl));
    if (!(mods & GDK_CONTROL_MASK)) {
        g_debug("zoom scroll: plain wheel, passing it on");
        return FALSE;  // not ours: let the scrolled window scroll
    }
    tab->zoom = CLAMP(tab->zoom + (dy < 0 ? 10 : -10), 50, 300);
    const std::string text = tab->session.name + " \u00b7 " + std::to_string(tab->zoom) + "%";
    gtk_label_set_text(GTK_LABEL(tab->label), text.c_str());
    g_debug("zoom scroll: ctrl+wheel, zoom %d%%", tab->zoom);
    return TRUE;  // handled: nothing else sees this scroll
}
```

For now the zoom level only shows in the tab label; Chapter 14 turns it into a font size.

Done when

Left-click still places the cursor in the text view, right-click does not open the text view's own menu, the wheel scrolls, and Ctrl+wheel changes the number in the tab label instead of scrolling.

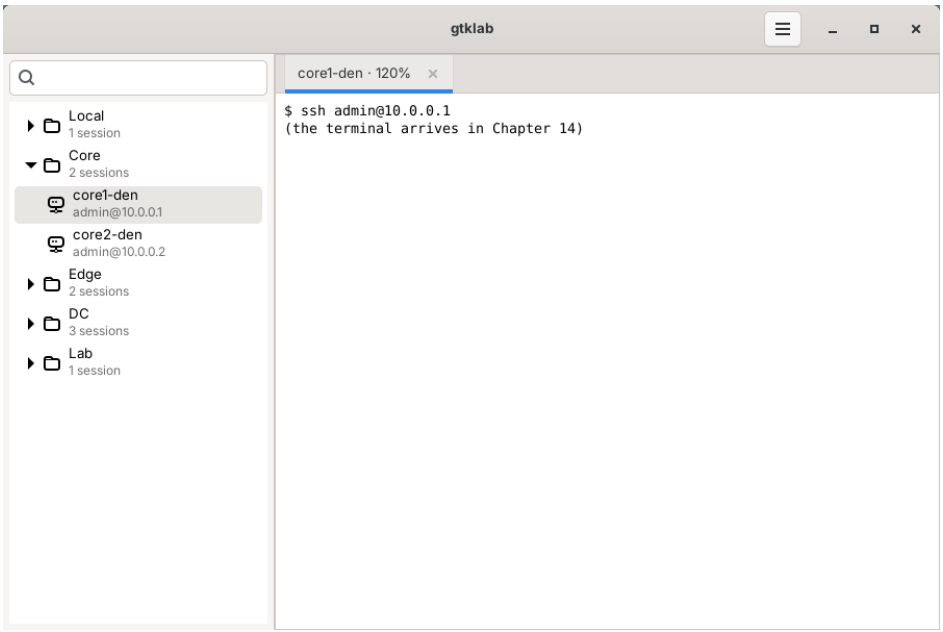


Figure 3: Ctrl+wheel twice over a tab: the label shows 120%, and the text did not scroll.

Debugger drill

Run the scratch config, so the handlers' `g_debug` lines print, and try the four inputs in turn. This is the recorded log:

1. Left-click: `page capture: button 1 denied`, then `text view (bubble): button 1`.
2. Right-click: `page capture: button 3 claimed`. The text view never sees it.
3. Wheel: `zoom scroll: plain wheel, passing it on`, and the text scrolls.
4. Ctrl+wheel: `zoom scroll: ctrl+wheel, zoom 110%`, and nothing scrolls.

That sequence is the propagation model. If you can predict all four lines before you try them, you understand it. Then set breakpoints in the three handlers and repeat, to see the phases in the call stacks.

Break it

Change the click gesture's phase to `GTK_PHASE_BUBBLE` and right-click. The text view's own context menu (Cut, Copy, Paste, Select All, Insert Emoji) appears instead of your handler. The target handled the click in its own phase, before it could bubble up to the page.

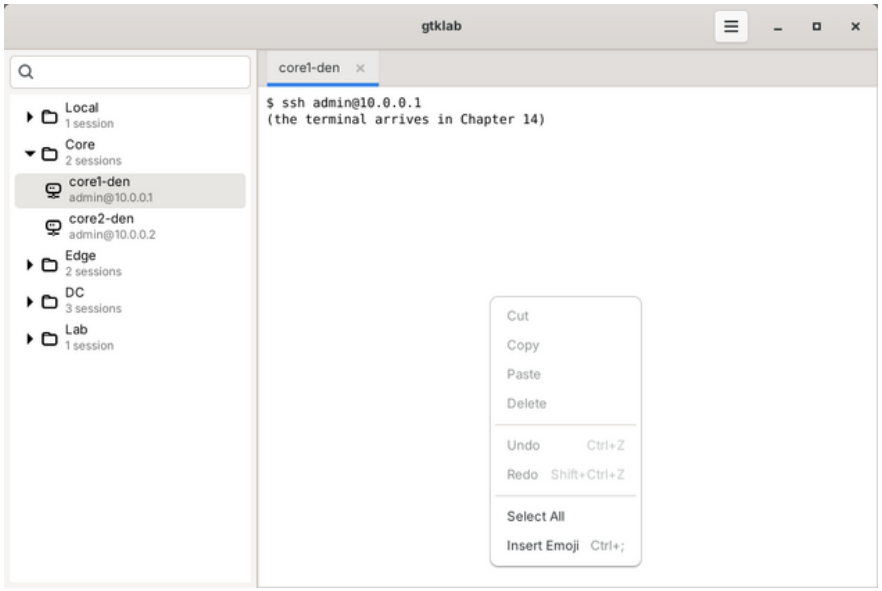


Figure 4: The Break it: in the bubble phase, the text view's built-in menu wins.

Common mistakes

Claiming everything. A capture-phase gesture that claims every press makes the widgets inside the page unusable. Claim only what you handle.

Returning `TRUE` from a scroll handler for events you ignored. The scroll is swallowed and the view never scrolls.

Reading modifiers from a global. Use `gtk_event_controller_get_current_event_state` inside the handler; it is the state for this event.

Adding a controller to a widget that has not been created yet. Add controllers where the widget is built. A controller belongs to one widget.

Read in tmssh

The page controllers in `add_tab`, `on_terminal_pressed` and `on_zoom_scroll`. Then gtkpyte's own controllers in `gtkpyte_terminal_init`, especially `on_report_begin`, a capture-phase drag gesture that claims mouse input when a full-screen program such as vim wants it and denies it so that selection works when no program does, or when you hold Shift.

Review

1. You left-click in a tab's text view. Name the gtklab controllers that see the press, in order, and what each one does with it.
2. The page's click gesture claims a right-click. Why does that stop the text view's own context menu, and why does the same gesture in the bubble phase not?
3. What would the user notice if `on_zoom_scroll` returned `TRUE` for a plain wheel too?
4. A key press while the session tree has focus: which widget is the target, and why could a shortcut controller on the notebook never see it?
5. You add a click gesture to the page in the target phase. Does it hear a click on the text view? Why?
6. Why is it safe to pass the `Tab` as plain user data to the page's controllers?
7. Why does the zoom handler ask the controller for the modifier state instead of tracking Ctrl with a key controller of its own?

Questions for deeper thought. A terminal widget wants every key, and your tool wants Ctrl+Shift+T for a new tab. Which side should win, where does each controller sit, and how would a user type a key your shortcut took? A capture-phase gesture on a container is invisible to whoever writes the widgets inside it; how would you make that policy discoverable to a teammate? GTK 3 let any widget see any event; what did GTK 4 gain by making input something you attach rather than something you override?

Exercises

1. **Shift for the built-in menu.** In `ch09-input/src/app_window.cpp`, change `on_page_pressed` to deny a right-click when Shift is held (read it with `gtk_event_controller_get_current_event_state`). Done when right-click logs `claimed` and Shift+right-click opens the text view's own menu. This is the rule gtkpyte uses for selection.
2. **Ctrl+0 resets the zoom.** In `add_tab`, add a `GtkEventControllerKey` to the page in the capture phase whose `key-pressed` handler returns `TRUE` only for Ctrl+0, setting `tab->zoom` back to 100 and updating the label. Done when Ctrl+0 resets the label to 100% and every other key still types into the view.
3. **Middle-click closes.** In `add_tab`, add a `GtkGestureClick` for button 2 to the tab label box that closes the tab the way `on_tab_close_clicked` does. Done when middle-clicking a tab's label closes it and prints `~Tab`, and middle-clicking inside the text view does not.
4. **Watch the pointer.** Add a `GtkEventControllerMotion` to the page and `g_debug` its `enter` and `leave` signals with the session name. Done when moving the pointer into the text area and out to the sidebar logs one enter and one leave, and switching tabs with the pointer still over the content tells you whether the new page gets an enter.

Chapter 10 — Popovers and dialogs

The right-click that Chapter 9 claimed now opens a menu, the session tree gets one too, closing a tab asks first, and sessions can be imported from a file. All of it is built from two GTK 4 pieces, popovers and dialogs, and both work differently from what GTK 3 code taught a generation of developers.

Recall

What does claiming a gesture sequence do (Chapter 9)? What happens after a scroll handler returns `FALSE`? What owns a tab's `Tab` struct (Chapter 8)?

In this chapter

- Show a context menu where the user clicked, parented to a widget that outlives everything the menu can do, and clean it up afterwards.
- Convert a point from the widget that was clicked to the popover's parent.
- Ask a question with a dialog that does not block, and act on the answer safely when it arrives.
- Follow one right-click through gesture, menu, action, dialog and state change, and log it at each boundary.

Concepts

Three ways to ask

GTK offers several surfaces for asking the user something, and they are not interchangeable.

You need	4.8 API used here	Its answer arrives in	From GTK 4.10
A context menu	`GtkPopoverMenu` over a `GMenu`	the chosen item's action	the same
A yes/no question	`GtkMessageDialog`	the `response` signal	`GtkAlertDialog`
A file to open	`GtkFileChooserNative`	the `response` signal	`GtkFileDialog`

The 4.10 headers mark `gtk_message_dialog_new`, `gtk_dialog_add_button` and `gtk_file_chooser_native_new` deprecated, and `GtkAlertDialog` and `GtkFileDialog` arrive in 4.10. The build's floor flags make 4.8 the version that counts, so the old calls compile cleanly and the new ones do not compile at all. Raise the floor to 4.10 and the deprecations turn into errors under `-Werror=deprecated-declarations`: moving the floor is a port, not a version bump.

Popovers are children

A **popover** is a widget like any other, with a parent. You create it, parent it to a widget with `gtk_widget_set_parent`, tell it where to point with `gtk_popover_set_pointing_to`, and pop it up. It is drawn above everything else, but it lives in the widget tree under its parent, and it dies with its parent. `GtkPopoverMenu` is a popover that renders a `GMenu` model, so a context menu is the same data as Chapter 7's main menu: labels naming actions. Its items show the same accelerators, and grey out when their action is disabled.

Two rules follow from "a popover has a parent". First, parent it to a widget that outlives anything the menu can do. The tab menu has a Close Tab item, so it is parented to the notebook rather than to the tab it was opened from. Second, whoever calls `gtk_widget_set_parent` must call `gtk_widget_unparent`; a parent does not drop a popover on its own. And the moment the popover closes is not the moment to unparent it, as the Break it shows: `closed` comes before the chosen item's action.

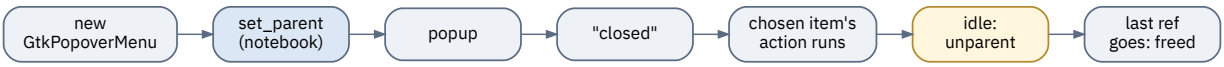


Figure 1: A one-use context menu. The popover is unparented on idle, after the chosen action has run.

	Long-lived popover	One per use
Example	the lab's popover on its button; the menu inside Chapter 7's `GtkMenuButton`	gtklab's tab and row menus
Created	once, with its parent	on each right-click
Unparented	when its parent is destroyed (the menu button does this itself)	on idle after `closed`
Fits when	the anchor and the contents never change	the contents depend on what was clicked

Coordinates belong to a widget

A gesture reports `x, y` in the coordinates of the widget it is attached to. The popover wants a rectangle in its parent's coordinates. Whenever an API takes a point, say whose point it is. `gtk_widget_compute_point(from, to, ...)` converts between any two widgets in the same window. In the recorded run a right-click at (258, 175) in the page was (258, 212) in the notebook: the page sits below the tab strip. Skip the conversion and the menu opens 37 pixels above the pointer. The older `gtk_widget_translate_coordinates` does the same job but is deprecated from GTK 4.12.

The menu names commands; it does not own them

A context menu is a command surface. Its items name actions (`win.copy`, `win.close-tab`), and the actions resolve their target the way Chapter 8 said: they ask the notebook for the current tab when they run. So the tab menu first makes the clicked tab current. That is a contract worth noticing: "current" now means "the one you right-clicked".

The alternative is an action with a parameter that names the target. The row menu does that, because a row is not "current" in any sense the actions know about: its Open item names `win.open-session` with the session's name as the target, the same mechanism as Chapter 7's radio items. Passing an identifier costs a parameter and a lookup; relying on "current" costs a side effect (the tab switches). For tabs the side effect is what a user expects. For rows, which are recycled (Chapter 5), the gesture cannot even remember its session; it keeps its `GtkListItem` and asks it at click time.

Dialogs do not block

A **dialog** is a window. In GTK 3, `gtk_dialog_run` ran a nested main loop and returned the user's answer, so code could read top to bottom. That function is not in GTK 4's headers. A GTK 4 dialog is shown and your function returns; the answer arrives later in the `response` signal, and the code that acts on it lives in that handler. Modal does not mean blocking: a modal dialog stops input to its parent window, but the main loop keeps running, timers fire, and from Chapter 11 the other tabs keep receiving output.

So the state when the question is asked may not be the state when the answer comes. Anything the handler needs must still be alive, and anything it assumes must be checked again. The close confirmation holds a reference to the tab's page, not a `Tab *`. Chapter 8 measured what that buys: with a reference held, a removed page keeps its `Tab` until the reference goes. The handler then checks that the page is still in a notebook before closing it.

Treat every answer except the one you asked for as "no". In a recorded run, pressing Escape on the confirm dialog delivered response -4 (`GTK_RESPONSE_DELETE_EVENT`), neither Accept nor Cancel. `confirm` acts only on `GTK_RESPONSE_ACCEPT`, so Escape, Cancel and the window's close button all leave the tab open.

An import is the same shape with more steps. Choosing a file is only the first: the file is parsed by the Chapter 4 core before anything is replaced, and a malformed file changes nothing.

Step	Question	On failure
Choose	Did the user accept?	Nothing happens
Validate	Does the file parse as sessions?	"Not imported" with the parser's message; the tree is untouched
Store	Can it be copied over the sessions file?	"Not imported" with the file error
Refresh	Reload and rebuild the tree	Expanded folders are kept

Tracing one click through the layers

By now a right-click crosses four layers: an input **controller**, a **popover** that offers commands, an **action** that resolves its target, and a **state change** that may wait for a dialog. They are separate stories. Propagation decides which controller sees the click; it says nothing about which tab Close Tab will close.

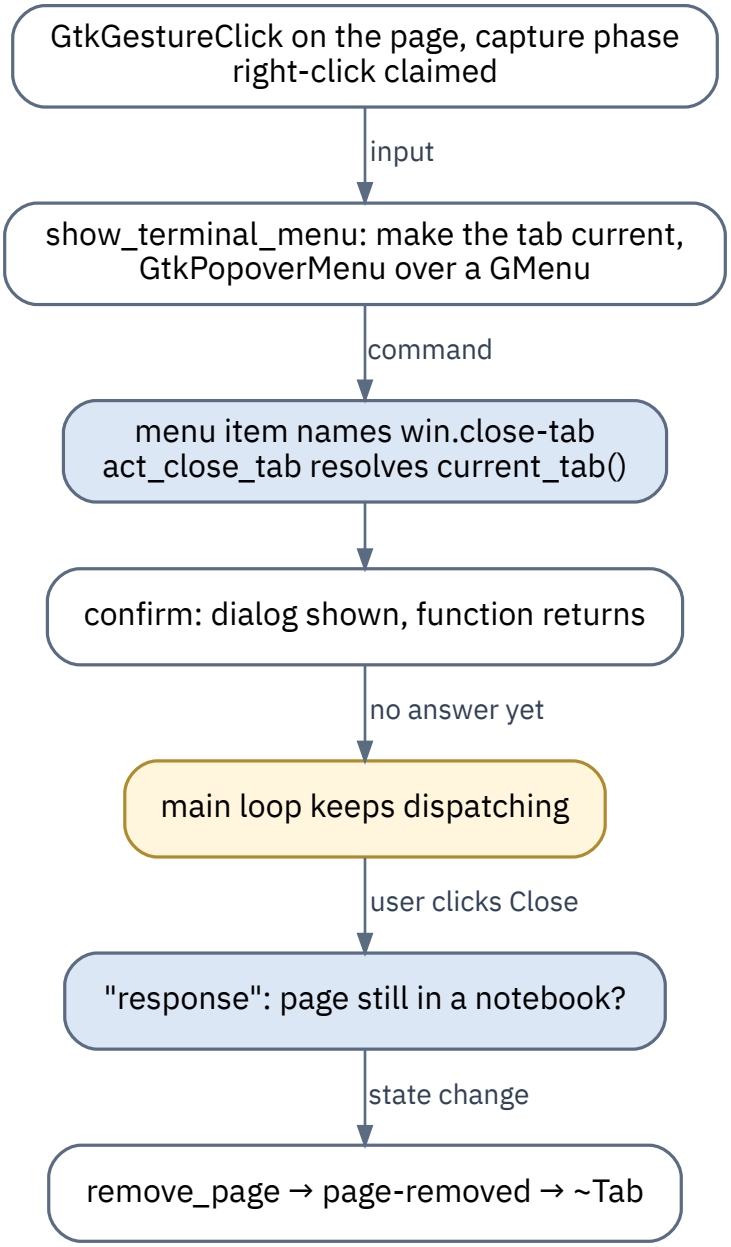


Figure 2: One right-click, from gesture to `~Tab`. The dialog splits it in two: everything below the main loop happens in a later callback.

When something goes wrong (a menu in the wrong place, the wrong tab closed, nothing happening at all) log at the boundaries where responsibility changes, and include the target in every line. This is the recorded log of right-clicking the second of two tabs, choosing Close Tab, and confirming:

```
page capture: button 3 claimed (menu at 258,175) [core2-den]
popover: from (258,175) to parent (258,212)
confirm response -3
page-removed, 1 left
~Tab core2-den
```

-3 is `GTK_RESPONSE_ACCEPT`. About two seconds passed between the second line and the third, while the menu and the dialog were up. There is no line for the action itself, because `act_close_tab` does not log. Set a breakpoint there and one in `on_page_pressed` and compare the call stacks: the action runs from the click on the menu item, a later event, and the removal runs from the dialog's answer, later still. No single call stack contains the whole story, which is why the log has to carry the target.

Build

One helper for every popover menu

ch10-popovers/src/app_window.cpp

```
// A GtkPopoverMenu parented to `parent`, pointing at (x, y) in `from`'s
// coordinates. The parent must outlive anything the menu can do: Close Tab
// destroys the tab page, so the terminal menu hangs off the notebook.
void AppWindow::show_popover(GMenuModel *model, GtkWidget *parent, GtkWidget *from, double x, double y) {
    // gtk_widget_compute_point exists in every GTK 4; the older
    // gtk_widget_translate_coordinates is deprecated from 4.12.
    const graphene_point_t in = GRAPHENE_POINT_INIT(static_cast<float>(x), static_cast<float>(y));
    graphene_point_t out;
    if (!gtk_widget_compute_point(from, parent, &in, &out))
        return;
    const double px = out.x, py = out.y;
    g_debug("popover: from (%.0f,%.0f) to parent (%.0f,%.0f)", x, y, px, py);

    GtkWidget *popover = gtk_popover_menu_new_from_model(model);
    gtk_widget_set_parent(popover, parent);
    gtk_popover_set_has_arrow(GTK_POPOVER(popover), FALSE);
    gtk_widget_set_halign(popover, GTK_ALIGN_START);
    GdkRectangle rect = {static_cast<int>(px), static_cast<int>(py), 1, 1};
    gtk_popover_set_pointing_to(GTK_POPOVER(popover), &rect);
    g_signal_connect(popover, "closed", G_CALLBACK(on_popover_closed), nullptr);
    gtk_popover_popup(GTK_POPOVER(popover));
}
```

Coordinates arrive in the clicked widget's space and must be converted to the parent's. Use `gtk_widget_compute_point`, which exists in every GTK 4. Most code you will find uses `gtk_widget_translate_coordina` which works but is deprecated from GTK 4.12; the first version of this chapter used it too, and it was changed when the deprecation was checked against the headers.

When the popover closes, it is unparented on the next idle, holding a reference so it cannot vanish in between:

ch10-popovers/src/app_window.cpp

```
// "closed" fires before the chosen item's action runs. Unparenting here
// would destroy the menu under its own feet, so do it on idle.
void AppWindow::on_popover_closed(GtkPopover *popover, gpointer) {
    g_idle_add(
        [](gpointer p) -> gboolean {
            GtkWidget *w = GTK_WIDGET(p);
            if (gtk_widget_get_parent(w))
                gtk_widget_unparent(w);
            g_object_unref(w);
            return G_SOURCE_REMOVE;
        },
        g_object_ref(popover));
}
```

The two menus

The tab menu makes the clicked tab current first, because the actions it names (Copy, Close Tab) work on the current tab:

ch10-popovers/src/app_window.cpp

```
void AppWindow::show_terminal_menu(Tab *tab, double x, double y) {
    // Actions work on the current tab, so make the clicked one current.
    const int n = gtk_notebook_page_num(GTK_NOTEBOOK(notebook_), tab->page);
    gtk_notebook_set_current_page(GTK_NOTEBOOK(notebook_), n);

    GMenu *menu = g_menu_new();
    GMenu *edit = g_menu_new();
    g_menu_append(edit, "Copy", "win.copy");
    g_menu_append(edit, "Paste", "win.paste");
    g_menu_append_section(menu, nullptr, G_MENU_MODEL(edit));
    GMenu *tabs = g_menu_new();
    g_menu_append(tabs, "Close Tab", "win.close-tab");
    g_menu_append(tabs, "Close Other Tabs", "win.close-other-tabs");
    g_menu_append_section(menu, nullptr, G_MENU_MODEL(tabs));
    show_popover(G_MENU_MODEL(menu), notebook_, tab->page, x, y);
    g_object_unref(edit);
    g_object_unref(tabs);
    g_object_unref(menu);
}
```

The row menu needs to know which session was clicked. Row widgets are recycled (Chapter 5), so the gesture cannot remember its session when it is created. It keeps its `GtkListItem` instead and asks it at click time:

ch10-popovers/src/app_window.cpp

```
void AppWindow::on_row_pressed(GtkGestureClick *gesture, int, double x, double y, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    auto *item = GTK_LIST_ITEM(g_object_get_data(G_OBJECT(gesture), "list-item"));
    auto *row = GTK_TREE_LIST_ROW(gtk_list_item_get_item(item));
    if (!row)
        return;
    gtk_gesture_set_state(GTK_GESTURE(gesture), GTK_EVENT_SEQUENCE_CLAIMED);
    auto *si = GTKLAB_SESSION_ITEM(gtk_tree_list_row_get_item(row));
    GtkWidget *widget = gtk_event_controller_get_widget(GTK_EVENT_CONTROLLER(gesture));
    if (gtklab_session_item_is_folder(si)) {
        Session none;
        self->show_row_menu(widget, none, x, y);
    } else {
        self->show_row_menu(widget, gtklab_session_item_session(si), x, y);
    }
    g_object_unref(si);
}
```

Its Open item names an action with a target, `win.open-session` with the session's name, the same mechanism as Chapter 7's radio items:

ch10-popovers/src/app_window.cpp

```
void AppWindow::show_row_menu(GtkWidget *row, const Session &s, double x, double y) {
    GMenu *menu = g_menu_new();
    if (!s.name.empty()) {
        GMenuItem *open = g_menu_item_new("Open", nullptr);
        g_menu_item_set_action_and_target_value(open, "win.open-session",
                                                g_variant_new_string(s.name.c_str()));
        g_menu_append_item(menu, open);
        g_object_unref(open);
```

```
    }
    g_menu_append(menu, "Edit Sessions File", "win.open-sessions-file");
    // Row widgets are recycled by the list view; hang the menu off the
    // list view itself.
    show_popover(G_MENU_MODEL(menu), list_view_, row, x, y);
    g_object_unref(menu);
}
```

A question that does not block

`confirm` shows a modal question and returns at once. What to do on "yes" travels with the dialog in a small struct, along with a reference to the object it acts on, which is released when the answer arrives:

ch10-popovers/src/app_window.cpp

```
// A modal question. Nothing blocks: the answer arrives in "response".
void AppWindow::confirm(const std::string &title, const std::string &detail, const char *accept_label,
                        void (*on_accept)(AppWindow *, gpointer), gpointer data) {
    GtkWidget *dialog = gtk_message_dialog_new(GTK_WINDOW(window_), GTK_DIALOG_MODAL,
                                               GTK_MESSAGE_QUESTION, GTK_BUTTONS_NONE, "%s",
                                               title.c_str());
    gtk_message_dialog_format_secondary_text(GTK_MESSAGE_DIALOG(dialog), "%s", detail.c_str());
    gtk_dialog_add_button(GTK_DIALOG(dialog), "Cancel", GTK_RESPONSE_CANCEL);
    GtkWidget *accept = gtk_dialog_add_button(GTK_DIALOG(dialog), accept_label, GTK_RESPONSE_ACCEPT);
    gtk_widget_add_css_class(accept, "destructive-action");
    gtk_dialog_set_default_response(GTK_DIALOG(dialog), GTK_RESPONSE_ACCEPT);

    auto *cd = new ConfirmData{this, on_accept, G_OBJECT(g_object_ref(data))};
    g_signal_connect(dialog, "response",
                     G_CALLBACK(+[](GtkDialog *d, int response, gpointer p) {
                         auto *c = static_cast<ConfirmData *>(p);
                         g_debug("confirm response %d", response);
                         if (response == GTK_RESPONSE_ACCEPT)
                             c->on_accept(c->window, c->object);
                         g_object_unref(c->object);
                         delete c;
                         gtk_window_destroy(GTK_WINDOW(d));
                     }),
                     cd);
    gtk_window_present(GTK_WINDOW(dialog));
}
```

Closing the current tab asks first. The callback holds the page, not the `Tab` pointer, because the tab could be closed some other way while the dialog is up, and checks that the page is still in a notebook before acting:

ch10-popovers/src/app_window.cpp

```
void AppWindow::act_close_tab(GSimpleAction *, GVariant *, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    Tab *tab = self->current_tab();
    if (!tab)
        return;
    // The dialog is asynchronous: hold the page, not the Tab pointer, in case
    // the tab is closed some other way while the dialog is up.
    self->confirm("Close " + tab->session.name + "?", "The session in this tab will end.", "Close",
                  [](AppWindow *w, gpointer page) {
                      if (auto *t = static_cast<Tab *>(g_object_get_data(G_OBJECT(page), "tab")))
                          if (gtk_widget_get_parent(GTK_WIDGET(page)))
                              w->close_tab(t);
                  },
```

```
                tab->page);
}
```

Importing a file

`GtkFileDialog` is the modern file picker, but it arrived in GTK 4.10, and the version floor turns its use into a compile error. The 4.8 API is `GtkFileChooserNative`, which also uses the platform's own picker where there is one. It is a dialog too, so its answer arrives in a signal; the import parses the file before replacing anything:

ch10-popovers/src/app_window.cpp

```
void AppWindow::on_import_response(GtkNativeDialog *dialog, int response, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    if (response == GTK_RESPONSE_ACCEPT) {
        GFile *file = gtk_file_chooser_get_file(GTK_FILE_CHOOSER(dialog));
        char *path = g_file_get_path(file);
        std::string err;
        load_sessions(path ? path : "", err);  // validate before replacing anything
        if (!err.empty()) {
            self->message("Not imported", err);
        } else {
            char *contents = nullptr;
            gsize len = 0;
            GError *gerr = nullptr;
            if (g_file_get_contents(path, &contents, &len, &gerr) &&
                g_file_set_contents(self->sessions_path_.c_str(), contents, static_cast<gssize>(len),
                ↪  &gerr)) {
                self->load_sessions_file();
                self->rebuild_tree(self->capture_expanded(), !self->filter_.empty());
            } else {
                self->message("Not imported", gerr ? gerr->message : "unknown error");
            }
            g_clear_error(&gerr);
            g_free(contents);
        }
        g_free(path);
        g_object_unref(file);
    }
    g_object_unref(dialog);
}
```

Done when

Both menus open where you click; Close Tab asks first and Cancel leaves the tab open; Import Sessions refuses a malformed file with a message.



Figure 3: The tab menu, the row menu, and the confirm dialog.

In the recorded run, importing a file containing `port=abc` showed “Not imported: session ‘bad’: port is not a number” and left the tree as it was.

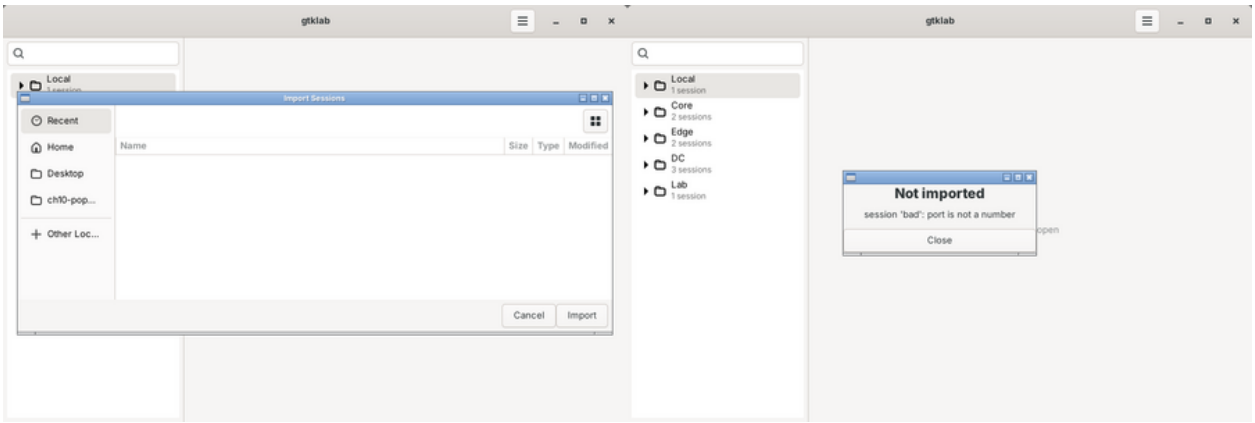


Figure 4: Import Sessions: the file chooser, and the refusal of a malformed file.

Debugger drill

Set a breakpoint in `show_popover` and compare `x, y` with the converted `px, py`. In the recorded run, a right-click at (258, 175) in the page was (258, 212) in the notebook, which is the page offset by the tab strip; a click at (91, 7) in a tree row was (104, 83) in the list view.

Break it

In `on_popover_closed`, call `gtk_widget_unparent` directly instead of on idle, and choose Copy from the tab menu. Nothing happens, and nothing is logged. The popover emits `closed` before it activates the chosen item’s action; unparenting it there destroys the menu that was about to activate it. With the idle version, the action runs.

Common mistakes

Parenting a context menu to the thing it can destroy. In testing, GTK 4.14 survived a popover whose parent tab was closed from the popover’s own menu. Parenting to the notebook means not depending on that.

Never unparenting. Every popover you create and parent for one use stays in the tree until you unparent it.

Writing dialog code as if it blocked. The line after showing the dialog runs before the user has answered. Act in `response`.

Holding a raw pointer across a dialog. Hold a reference to an object, or check that the thing still exists when the answer comes.

Read in tmssh

`show_terminal_menu`, `show_row_menu`, `confirm`, `message` and `act_import_file`.

Review

1. The tab menu is parented to the notebook. What could go wrong if it were parented to the page it was opened from, and which menu item causes it?
2. Why is the popover unparented on idle rather than in its `closed` handler? What would the user see otherwise?
3. A popover is parented to the list view, and the click arrived on a row. Whose coordinates does the gesture report, and what goes wrong without `gtk_widget_compute_point`?
4. The tab menu makes the clicked tab current; the row menu passes the session's name as an action target. Why the difference?
5. "Modal" and "blocking" are different things. What does a modal dialog stop, and what keeps running?
6. Why does the close confirmation hold a reference to the page rather than the `Tab *`, and what does it check when the answer comes?
7. The user presses Escape on the confirm dialog. What response arrives, and why does the code not need a case for it?
8. In the trace, why is there no call stack that contains both the right-click and the tab's removal?

Questions for deeper thought. A dialog that waits on the user is one kind of asynchronous operation; a connection that waits on a remote host is another. What does the "check again when the answer comes" rule look like for the second? When would you rather put a question in a popover or an inline banner than in a modal dialog? Your tool has to raise its version floor from 4.8 to 4.10 one day: which parts of this chapter's code would change, and how would you find them?

Exercises

1. **Copy only with a selection.** In `ch10-popovers/src/app_window.cpp`, make `show_terminal_menu` disable `win.copy` (with `g_simple_action_set_enabled`) when the current tab's text buffer has no selection, and enable it otherwise. Done when Copy is greyed out in the menu until you select text. tmssh does this in its own `show_terminal_menu`.
2. **Close Tabs to the Right.** Add a `win.close-tabs-right` action beside `act_close_other_tabs` and a menu item in `show_terminal_menu`. Done when it closes every tab after the clicked one and prints one `~Tab` per closed tab.
3. **A target instead of "current".** Add a `win.close-session-tabs` action with an `"s"` parameter that walks the notebook's pages and closes every tab whose `Tab` has that session name, and add it to `show_row_menu` with the session's name as the target. Done when right-clicking a session in the tree and choosing the item closes all of its tabs, whichever tab is current, with one `~Tab` per tab.
4. **Ask before replacing.** In `on_import_response`, after the file has parsed, call `confirm` with "Replace the sessions file?" and do the copy in its accept callback. Done when Cancel leaves the tree as it was and accepting imports the file. `confirm` takes a reference to the object it carries, so decide which GObject the callback needs (the chosen `GFile` is one).

Isolated lab

`labs/lab10-popover` puts the two mechanisms side by side with nothing else around them: a long-lived popover on a button, and a modal question whose answer arrives later. No menus or actions, so the parent and the response are the only things to watch.

The popover is created once and parented to its button, and unparented when the button is destroyed:

labs/lab10-popover/src/main.cpp

```
lab->button = gtk_button_new_with_label("Show popover");
gtk_box_append(GTK_BOX(box), lab->button);
lab->popover = gtk_popover_new();
GtkWidget *popover_label = gtk_label_new("This is contextual UI, not a new window.");
gtk_widget_set_margin_top(popover_label, 12);
gtk_widget_set_margin_bottom(popover_label, 12);
gtk_widget_set_margin_start(popover_label, 12);
gtk_widget_set_margin_end(popover_label, 12);
gtk_popover_set_child(GTK_POPOVER(lab->popover), popover_label);
// A popover is not a child the button knows how to drop: whoever calls
// gtk_widget_set_parent must call gtk_widget_unparent. Do it when the
// button is destroyed, or GTK warns "Finalizing GtkButton ... but it
// still has children left" at quit.
gtk_widget_set_parent(lab->popover, lab->button);
g_signal_connect(lab->button, "destroy", G_CALLBACK(+[](GtkWidget *, gpointer popover) {
                     gtk_widget_unparent(GTK_WIDGET(popover));
                 }),
                 lab->popover);
```

The dialog is shown and the function returns; the answer arrives in a separate handler:

labs/lab10-popover/src/main.cpp

```
static void on_confirm_clicked(GtkButton *, gpointer data) {
    auto *lab = static_cast<Lab *>(data);
    GtkWidget *dialog = gtk_message_dialog_new(
        GTK_WINDOW(lab->window), GTK_DIALOG_MODAL,
        GTK_MESSAGE_QUESTION, GTK_BUTTONS_NONE,
        "%s", "Apply the change?");
    gtk_message_dialog_format_secondary_text(
        GTK_MESSAGE_DIALOG(dialog), "%s", "The response will arrive asynchronously.");
    gtk_dialog_add_button(GTK_DIALOG(dialog), "Cancel", GTK_RESPONSE_CANCEL);
    gtk_dialog_add_button(GTK_DIALOG(dialog), "Apply", GTK_RESPONSE_ACCEPT);
    g_signal_connect(dialog, "response", G_CALLBACK(on_dialog_response), lab);
    gtk_window_present(GTK_WINDOW(dialog));
}
```

labs/lab10-popover/src/main.cpp

```
static void on_dialog_response(GtkDialog *dialog, int response, gpointer data) {
    auto *lab = static_cast<Lab *>(data);
    if (response == GTK_RESPONSE_ACCEPT) {
        lab->confirmations++;
        char message[80];
        g_snprintf(message, sizeof(message), "Confirmed %u time(s)", lab->confirmations);
        gtk_label_set_text(GTK_LABEL(lab->status), message);
    }
    gtk_window_destroy(GTK_WINDOW(dialog));
}
```

Build and run it:

```
cd labs/lab10-popover
meson setup build-debug --buildtype=debug
meson compile -C build-debug
./build-debug/lab
```

Or open the folder in VS Code and run the "lab" configuration.

Stop at	What you see
`on_menu_clicked`	The popover already has the button as its parent; this only pops it up
`on_popover_closed`	The popover is closed but still parented, ready to open again
`on_confirm_clicked`, last line	The dialog is presented; continue, and the function returns before anyone has answered
`on_dialog_response`	`response` is the button's id; `lab` is the same struct the click had
the button's `destroy` handler	At quit: the popover is unparented while the button still exists

Experiments:

1. Open the popover, then the dialog, and press Apply. The recorded run showed the popover under its button, a modal dialog, and then "Confirmed 1 time(s)".
2. Delete the `g_signal_connect(lab->button, "destroy", ...)` call and quit. With that code GTK warned: `Gtk-WARNING: Finalizing GtkButton ..., but it still has children left: - GtkPopover`.
3. Break in `on_dialog_response` and press Escape instead of a button. Read `response`, and compare it with `GTK_RESPONSE_CANCEL` and `GTK_RESPONSE_ACCEPT`.
4. Add a `g_timeout_add_seconds` in `on_activate` whose callback prints a counter with `g_print` and returns `G_SOURCE_CONTINUE`. Open the dialog and watch the terminal while it is up: that is "modal is not blocking".

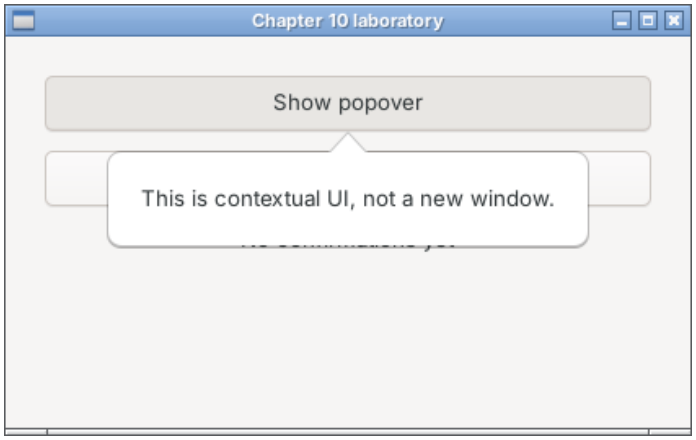


Figure 5: The popover, open under its button.

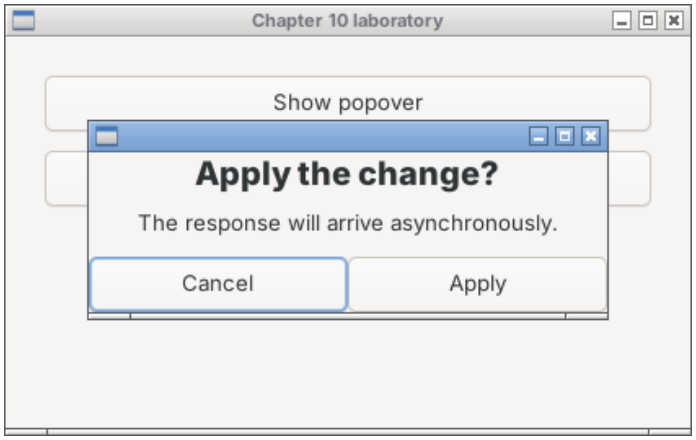


Figure 6: The modal question.

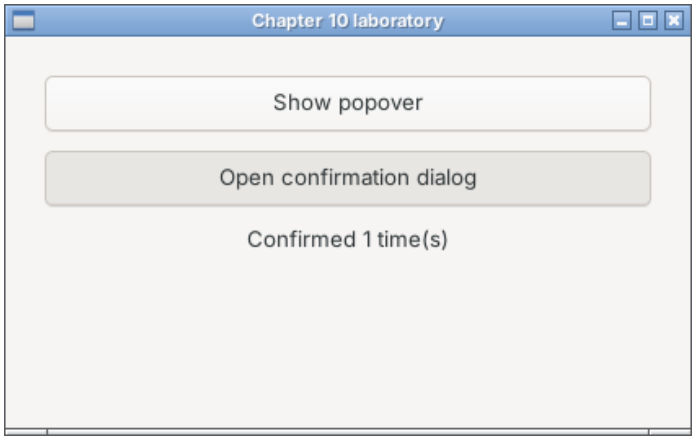


Figure 7: After Apply: the answer arrived in the response handler.

Checkpoint C — Tabs and menus

Spec. Chapter 10, plus Next Tab and Previous Tab (Ctrl+PageDown and Ctrl+PageUp, wrapping at the ends), and every shortcut reachable from the main menu.

Done when. None of Chapter 9's four input sequences has changed after adding the menus; and opening 50 tabs, choosing Close Other Tabs, then closing the last with Ctrl+Shift+W prints exactly 50 `~Tab` lines. Both were checked against the reference.

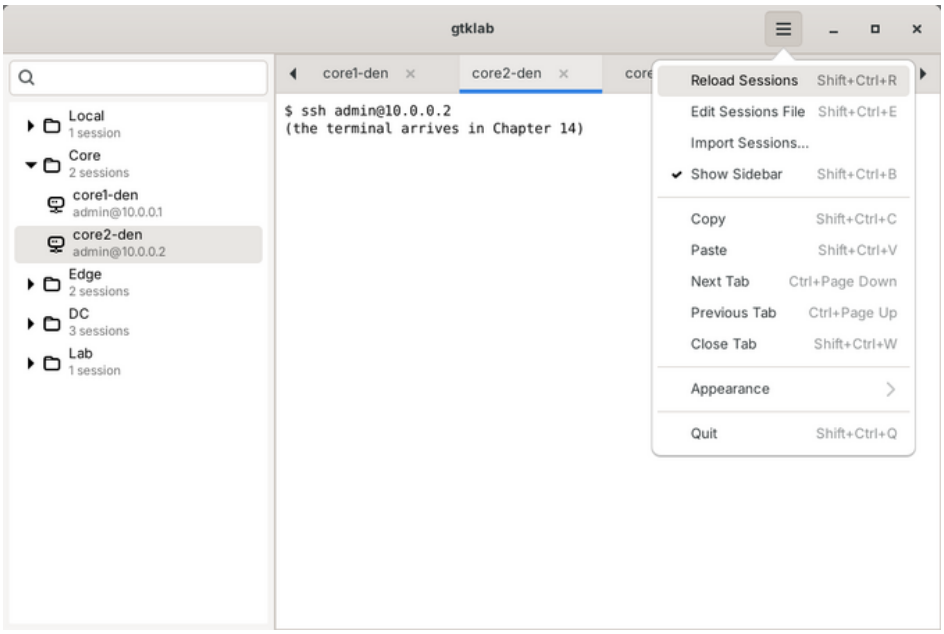


Figure 8: Checkpoint C: every shortcut in the main menu.

Hints. Next and previous are two more entries in the accelerator table and the action table; `(current + 1) % n` and `(current + n - 1) % n` wrap. GtkNotebook has its own Ctrl+PageDown binding, but it works only while focus is inside the notebook; the window's capture-phase shortcut works from anywhere, including the sidebar.

Reference. `ckC-tabs/`.

Part III: The Terminal

Part III gives each tab a live session: a child process on a pseudo-terminal, read on the main loop without freezing the window. It then builds a terminal-shaped widget from scratch, a character grid that scrolls, takes keys and selects text, and finally swaps in gtkpyte, a real terminal emulator with the same signals. Chapters 11 to 13 build a deliberately simple widget so that Chapter 14 is wiring rather than a wall of new code.

Part III also contains the book's most consequential finding: the recommended way to watch a file descriptor can stop a GTK window repainting, and the fix is one argument.

Chapter 11 — The main loop: file descriptors, idles, timers, monitors

Each tab now runs a real process on a pseudo-terminal: your shell for a local session, ssh for a remote one. Its output appears in the tab, a line typed under it goes to the process, and when the process exits the tab closes itself. Along the way the sessions file starts reloading by itself when it is edited. All of it is main loop programming, and this chapter is where the main loop stops being background and becomes the subject.

Recall

Why is a popover unparented on idle (Chapter 10)? What did the debounced settings save use (Chapter 7)? What does a tab's destructor clean up (Chapter 8)?

In this chapter

- Trace one iteration of the GLib main loop and predict which ready source it dispatches.
- Watch a file descriptor at a priority that leaves room for drawing, and bound the work each callback does.
- Run a child on a pseudo-terminal and explain what the kernel does between your bytes and the shell's.
- Remove every source that points at an object before the object goes, and defer work that would delete the caller.
- Turn a burst of file-change events into one reload.

Concepts

One thread, many sources

A GTK program runs on one thread. That thread spends its life in the GLib main loop, and the main loop does one thing over and over: find out which **sources** are ready, and call their callbacks. Everything your program reacts to is a source:

- a file descriptor with data to read: `g_unix_fd_add`;
- a child process that has exited: `g_child_watch_add`;
- a timer that has expired: `g_timeout_add`;
- "nothing else to do": `g_idle_add`;
- window system events, and GTK's own work of laying out and drawing the window, which GTK registers for you.

Signals are not on the list because they are not sources. A `GFileMonitor`'s `changed` signal or a button's `clicked` is a function call made from inside some source's callback, on the same thread, before that callback returns.

The single thread is a rule, not a choice: GTK widgets may only be touched from the thread that runs the main loop. You could read the pty on a worker thread, but every byte would then have to be handed back to the main thread before it reached a widget. A non-blocking descriptor watched by the main loop gets the same result with no locks and no hand-off.

One iteration

`g_main_context_iteration` runs the same five steps every time:

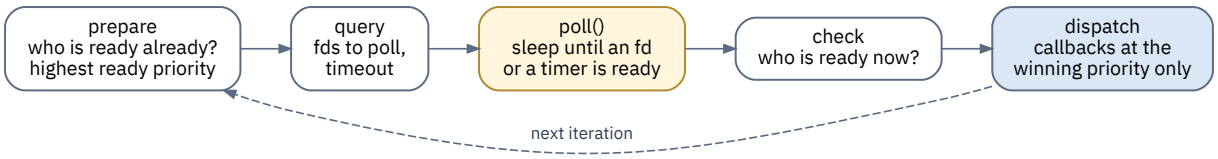


Figure 1: One iteration of the GLib main loop. Only the sources at the highest ready priority are dispatched; the rest wait for a later iteration.

1. **Prepare.** Each source is asked whether it is ready without waiting. An idle always is; a timer is if its time has passed; a descriptor cannot know yet. GLib notes the highest priority among the ready ones.
2. **Query.** GLib collects the descriptors to poll and a timeout: zero if something is already ready, the nearest timer otherwise, forever if nothing is pending.
3. **Poll.** The thread sleeps in `poll()` until a descriptor is ready or the timeout expires. An idle program spends nearly all its time here, using no CPU.
4. **Check.** Each source is asked again, now that the poll has answered.
5. **Dispatch.** The callbacks of the ready sources at the winning priority run, one after another. Each returns `G_SOURCE_CONTINUE` to stay or `G_SOURCE_REMOVE` to be destroyed.

Two rules follow, and the second is the one most GTK code gets wrong.

A callback must return quickly. While your callback runs, nothing else does: no input, no drawing. A callback that reads everything a busy process has written before returning freezes the window until the process stops. The fix is a **budget**: read up to a fixed amount, then return and let the loop come back for more.

Sources have priorities, and the highest wins. Every source has a priority, a number where lower means more urgent. On each iteration, GLib looks at all the sources that are ready, takes the highest priority among them, and dispatches only the sources at that priority. Lower-priority sources that are also ready wait for an iteration in which nothing more urgent is ready. A ten-line test program shows it plainly: with an idle at priority 0 that stays ready for 1,000 iterations and an idle at the default idle priority beside it, the second one first ran after the first had run 1,000 times and removed itself.

Priorities and the redraw

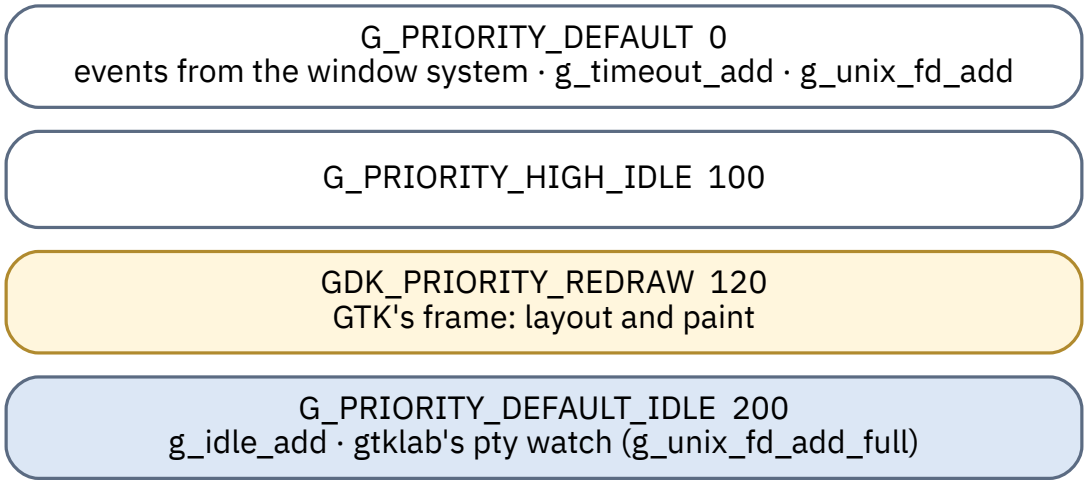


Figure 2: GLib priorities, most urgent at the top. A source that is always ready at the default priority wins every iteration, and GTK's redraw, below it, never gets a turn.

The numbers that matter, from the GLib and GDK headers and from asking each source for its priority with `g_source_get_priority`:

Source	Added with	Priority	gtklab uses it for
Window system events	(GTK)	0, `GDK_PRIORITY_EVENTS`	clicks, keys
Timer	`g_timeout_add`	0	debounced save (500 ms), sessions reload (300 ms)
Descriptor	`g_unix_fd_add`	0	(not used: see below)
Child exit	`g_child_watch_add`	0	closing the tab
GTK's frame	(GTK)	120, `GDK_PRIORITY_REDRAW`	layout and paint
Idle	`g_idle_add`	200, `G_PRIORITY_DEFAULT_IDLE`	focus, deferred close, popover cleanup
Descriptor, chosen	`g_unix_fd_add_full`	200	reading the pty

Here is the trap. `g_unix_fd_add` watches a descriptor at `G_PRIORITY_DEFAULT`, which is 0. GTK redraws at `GDK_PRIORITY_REDRAW`, which is 120, *less* urgent. While a process writes faster than you read it, which is exactly what `cat` of a large file does, the pty is ready on every iteration, the main loop dispatches it every time, and the window never repaints until the output stops. The read budget does not help: it makes the callback return, but the next iteration picks the pty again.

The fix is to watch the pty at idle priority, below the redraw, with `g_unix_fd_add_full(G_PRIORITY_DEFAULT_IDLE, ...)`. Reading still runs whenever nothing more urgent is waiting, which is nearly always, and the window gets a frame in between. Chapter 14 measures the difference: with a 67 MB `cat`, screenshots taken a second apart changed once in six seconds at the default priority and every second at idle priority, for about the same total time.

The budget and the priority do different jobs, and you need both. The budget bounds how long one callback holds the thread; the priority decides who gets the thread next.

A source points at you; you own the source

A source holds a function and a plain pointer, usually `this`. It takes no reference and has no idea when the object goes away. So the object must remove its sources before it goes, by the id each `g_*_add` returned, and must forget an id once the source has removed itself by returning `G_SOURCE_REMOVE`. That is the whole of `~PtySession`'s ordering rule, and the same rule behind the `save_source_` id of Chapter 7's debounced save.

The other half is the **deferred call**. Some work cannot happen inside the callback that wants it: closing the tab from the child-exit callback would delete the session whose callback is still running; unparenting a popover from its own `closed` handler (Chapter 10) would destroy it mid-signal; grabbing focus while a click is still being handled is undone when the click finishes. Each of these posts an idle that does the work after the current dispatch returns. An idle that touches a widget holds a reference to it, so it cannot vanish in between.

A **debounce** is the timer version of the same idea. Several events arrive close together; each one removes the pending timer and starts a new one; only the last one survives to fire.

A pseudo-terminal

A terminal program runs its child on a **pseudo-terminal** (pty): a pair of devices that behave like the two ends of a serial line. The child's end, the **slave** (`/dev/pts/N`), looks like a real terminal to it, so shells print prompts and programs like `vim` and `htop` draw full-screen. Your end, the **master**, is a file descriptor: you read what the child writes and write what the user types.

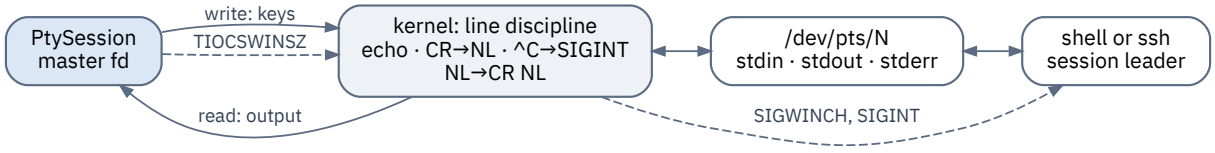


Figure 3: A pty. Between your descriptor and the child sits the kernel's line discipline, which echoes, translates line endings and turns control characters into signals.

Between the two ends sits the kernel's **line discipline**, and it does more than pass bytes along. With the default settings of a new pty, writing `hi\r` to the master delivered `hi\n` to the program on the slave (carriage return becomes newline on input) and `hi\r\n` came back out of the master (the input is echoed, and newline becomes carriage return plus newline on output). It also turns byte 3, Ctrl+C, into `SIGINT` for the program in the foreground. That is why a terminal never draws what you type and never sends signals: it sends bytes, and the echo and the signals come from the kernel or the shell.

`forkpty` does the setup in one call: it opens a pty, forks, and in the child starts a new session, makes the slave its controlling terminal, and makes it standard input, output and error. The parent gets the child's pid and the master descriptor. Window size travels separately: the `TIOCSWINSZ` ioctl on the master sets the size, and the kernel sends `SIGWINCH` to the foreground process group. A test child that printed its terminal size from a `SIGWINCH` handler reported 100 columns by 30 rows right after the parent set them.

When the child exits and the last copy of the slave closes, reading the master returns 0, or fails with `EIO` on Linux. That is the end of output; the exit status comes separately, from the child watch.

Build

PtySession

The interface:

ch11-mainloop/src/pty_session.h

```
// PtySession: a child process on a pseudo-terminal, driven by the GLib main
// loop. Output arrives through on_output; the session never touches a widget.
#pragma once

#include <glib.h>
#include <sys/types.h>

#include <functional>
#include <string>
#include <vector>

class PtySession {
public:
    using OutputHandler = std::function<void(const char *data, gsize len)>;
    using ExitHandler = std::function<void(int wait_status)>;

    PtySession() = default;
    ~PtySession();

    PtySession(const PtySession &) = delete;
    PtySession &operator=(const PtySession &) = delete;

    // Start argv[0] (searched on PATH) on a new pty of cols x rows. `term`
    // becomes $TERM in the child. Returns false and sets error() on failure.
    bool start(const std::vector<std::string> &argv, int cols, int rows,
               const std::string &term = "dumb");

    void set_output_handler(OutputHandler h) { on_output_ = std::move(h); }
    // Runs once, when the child has exited. It may delete this session.
    void set_exit_handler(ExitHandler h) { on_exit_ = std::move(h); }

    // Write bytes to the child. A short write drops the rest (Chapter 14's
    // gtkpyte session queues them instead).
    void write(const char *data, gsize len);
    void resize(int cols, int rows);

    const std::string &error() const { return error_; }
    pid_t pid() const { return pid_; }

    // Bytes read per wakeup before yielding to the main loop.
    static constexpr gsize kReadBudget = 512 * 1024;

private:
    static gboolean on_readable(gint fd, GIOCondition cond, gpointer self);
    static void on_child_exit(GPid pid, gint status, gpointer self);

    int fd_ = -1;
    pid_t pid_ = -1;
    guint read_watch_ = 0;
```

```
    guint child_watch_ = 0;
    std::string error_;
    OutputHandler on_output_;
    ExitHandler on_exit_;
};
```

`start` builds the child's argument list before forking, because after `fork` only async-signal-safe calls are allowed until `exec`. It then makes the master descriptor non-blocking and adds the two sources: the descriptor watch, at idle priority, and the child watch.

ch11-mainloop/src/pty_session.cpp

```
bool PtySession::start(const std::vector<std::string> &argv, int cols, int rows,
                       const std::string &term) {
    if (argv.empty()) {
        error_ = "no command";
        return false;
    }
    // Build the child's argv before forking: after fork, only async-signal-safe
    // calls are allowed until exec.
    std::vector<char *> cargv;
    for (const std::string &a : argv)
        cargv.push_back(const_cast<char *>(a.c_str()));
    cargv.push_back(nullptr);
    const std::string term_env = "TERM=" + term;

    struct winsize ws = {};
    ws.ws_col = static_cast<unsigned short>(cols);
    ws.ws_row = static_cast<unsigned short>(rows);

    int master = -1;
    pid_t pid = forkpty(&master, nullptr, nullptr, &ws);
    if (pid < 0) {
        error_ = std::string("forkpty: ") + std::strerror(errno);
        return false;
    }
    if (pid == 0) {  // child
        putenv(const_cast<char *>(term_env.c_str()));
        execvp(cargv[0], cargv.data());
        const char msg[] = "exec failed\r\n";
        ssize_t ignored = ::write(STDERR_FILENO, msg, sizeof msg - 1);
        (void)ignored;
        _exit(127);
    }

    fd_ = master;
    pid_ = pid;
    fcntl(fd_, F_SETFL, fcntl(fd_, F_GETFL) | O_NONBLOCK);
    fcntl(fd_, F_SETFD, FD_CLOEXEC);

    // Idle priority, below GTK's redraw (GDK_PRIORITY_REDRAW). GLib only
    // dispatches the highest-priority sources that are ready; at the default
    // priority a pty that is always readable (cat of a big file) would win
    // every iteration and the window would stop repainting until it drained.
    read_watch_ = g_unix_fd_add_full(G_PRIORITY_DEFAULT_IDLE, fd_,
                                     static_cast<GIOCondition>(G_IO_IN | G_IO_HUP | G_IO_ERR),
                                     on_readable, this, nullptr);
    child_watch_ = g_child_watch_add(pid_, on_child_exit, this);
    return true;
}
```

The read callback reads until the pty is drained or the budget is spent. Each `read` takes up to 16 KiB; the loop stops at 512 KiB, the same budget tmssh uses:

ch11-mainloop/src/pty_session.cpp

```
// Read until the pty is drained or the budget is spent, then give the main
// loop a turn. Without the budget, `cat` of a huge file would never return
// here and the window would stop redrawing and taking input.
gboolean PtySession::on_readable(gint, GIOCondition, gpointer data) {
    auto *self = static_cast<PtySession *>(data);
    char buf[16384];
    gsize total = 0;
    while (total < kReadBudget) {
        const ssize_t n = read(self->fd_, buf, sizeof buf);
        if (n > 0) {
            total += static_cast<gsize>(n);
            if (self->on_output_)
                self->on_output_(buf, static_cast<gsize>(n));
            continue;
        }
        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK))
            return G_SOURCE_CONTINUE;  // drained; wait for more
        if (n < 0 && errno == EINTR)
            continue;
        // 0, or EIO on Linux: the child side has closed. Stop watching.
        // Returning G_SOURCE_REMOVE destroys the source, so forget its id.
        self->read_watch_ = 0;
        return G_SOURCE_REMOVE;
    }
    g_debug("read budget spent after %zu bytes; yielding", total);
    return G_SOURCE_CONTINUE;
}
```

The destructor is short, and its order is the point of it. Every main loop source that holds `this` is removed before anything else happens; only then are the descriptor and the child released. The child is still reaped afterwards by a watch that holds nothing of the session's, or it would stay a zombie.

ch11-mainloop/src/pty_session.cpp

```
PtySession::~PtySession() {
    // Order matters: remove every main-loop source that holds `this` first,
    // then release the fd and the child.
    if (read_watch_)
        g_source_remove(read_watch_);
    if (child_watch_)
        g_source_remove(child_watch_);
    if (fd_ >= 0)
        close(fd_);
    if (pid_ > 0) {
        kill(pid_, SIGHUP);
        // Still reap it, or it stays a zombie. This watch holds nothing of ours.
        g_child_watch_add(pid_, [](GPid pid, gint, gpointer) { g_spawn_close_pid(pid); }, nullptr);
    }
    g_debug("~PtySession");
}
```

On Linux, `forkpty` lives in `libutil`. Add it in `meson.build`:

ch11-mainloop/meson.build

```
util = meson.get_compiler('cpp').find_library('util', required: false)  # forkpty on Linux
```

and add `util` to the executable's dependencies. On the Mac, `forkpty` is in the C library and declared in `<util.h>`, which `pty_session.cpp` includes there.

A tab that runs a session

The tab owns its session as a `std::unique_ptr<PtySession>`, so deleting the `Tab` (Chapter 8) ends the session. The output handler feeds the tab's view; the exit handler closes the tab, but never from inside the exit callback itself, because removing the page deletes the `Tab`, which deletes the `PtySession` whose callback is still running. It defers to an idle, holding a reference to the page so that it cannot disappear in between:

ch11-mainloop/src/app_window.cpp

```
void AppWindow::start_tab(Tab *tab) {
    tab->pty = std::make_unique<PtySession>();
    GtkTextView *view = GTK_TEXT_VIEW(tab->view);
    tab->pty->set_output_handler([view](const char *data, gsize len) { append_output(view, data, len); });

    // When the child exits, close the tab, but not from inside this
    // callback: removing the page deletes the Tab, which deletes the
    // PtySession whose callback is running. Defer to an idle, holding a
    // reference to the page so it cannot vanish in between.
    GtkWidget *page = tab->page;
    tab->pty->set_exit_handler([page](int status) {
        g_debug("child exited, status %d; closing tab on idle", status);
        g_idle_add(
            [](gpointer p) -> gboolean {
                GtkWidget *pg = GTK_WIDGET(p);
                if (GtkWidget *nb = gtk_widget_get_ancestor(pg, GTK_TYPE_NOTEBOOK)) {
                    const int n = gtk_notebook_page_num(GTK_NOTEBOOK(nb), pg);
                    if (n >= 0)
                        gtk_notebook_remove_page(GTK_NOTEBOOK(nb), n);
                }
                g_object_unref(pg);
                return G_SOURCE_REMOVE;
            },
            g_object_ref(page));
    });

    const char *shell = g_getenv("SHELL");
    if (!tab->pty->start(ssh_argv(tab->session, shell ? shell : ""), 80, 24)) {
        const std::string msg = "Could not start: " + tab->pty->error() + "\n";
        append_output(view, msg.c_str(), msg.size());
    }
}
```

The output view is still Chapter 8's text view, for one more chapter. Text views need valid UTF-8, and pty output is not guaranteed to be, so it goes through `g_utf8_make_valid`; the buffer is capped at 2,000 lines. The child runs with `TERM=dumb`, so the shell sends few escape sequences. A `GtkEntry` under the view sends a line to the session on Enter.

Focus, on idle

Testing found a bug here that is worth a section of its own. A tab opened by double-clicking a session in the tree was given focus by `add_tab`, and then lost it: the list view takes focus back when the click that activated the row finishes. Typed text went nowhere, and the next Enter re-activated the row and opened a second tab. The fix is to ask for focus on idle, after the click has finished:

ch11-mainloop/src/app_window.cpp

```
// Focus on idle. A tab opened by double-clicking the tree is created while
// that click is still being handled; the list view takes focus back when
// the click finishes, so grabbing focus now would be undone.
static void focus_later(GtkWidget *widget) {
    g_idle_add(
        [](gpointer p) -> gboolean {
            GtkWidget *w = GTK_WIDGET(p);
            if (gtk_widget_get_root(w))  // still in a window
                gtk_widget_grab_focus(w);
            g_object_unref(w);
            return G_SOURCE_REMOVE;
        },
        g_object_ref(widget));
}
```

The sessions file reloads itself

A `GFileMonitor` reports changes to a file. Editors save in several steps (write a temporary file, rename it, change its permissions), so one save arrives as several events; in the recorded run, one write produced four. Each event restarts a 300 ms timer, and only the last one reloads:

ch11-mainloop/src/app_window.cpp

```
// Editors save in several steps (write temp, rename, chmod). Each restarts
// a 300 ms timer, so the burst becomes one reload.
void AppWindow::on_sessions_changed(GFileMonitor *, GFile *, GFile *, GFileMonitorEvent event,
                                    gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    g_debug("sessions file event %d", event);
    if (self->reload_source_)
        g_source_remove(self->reload_source_);
    self->reload_source_ = g_timeout_add(300, on_sessions_settled, self);
}
```

ch11-mainloop/src/app_window.cpp

```
gboolean AppWindow::on_sessions_settled(gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    self->reload_source_ = 0;
    g_debug("sessions file settled; reloading");
    self->load_sessions_file();
    self->rebuild_tree(self->capture_expanded(), !self->filter_.empty());
    return G_SOURCE_REMOVE;
}
```

Done when

A local tab runs commands, `exit` closes it, and editing the sessions file while gtklab is running updates the tree without a reload key.

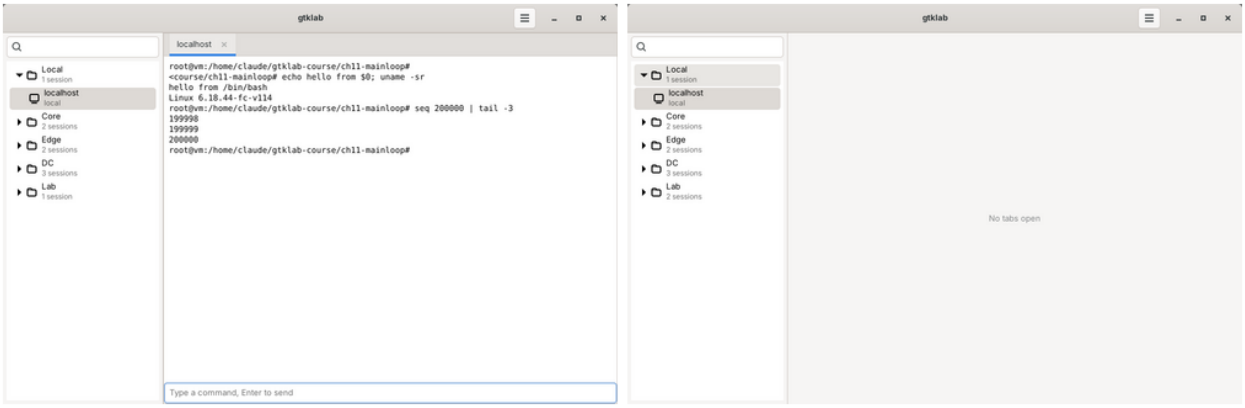


Figure 4: A local session running commands; after `exit` the tab has closed, and the Lab folder shows a session added to the file while gtklab was running.

Debugger drill

Set a breakpoint in `on_readable` and run `seq 3000000` in a local tab. Step through the loop and watch `total` grow until the budget is reached. The scratch config's log shows the same thing without stopping: `read budget spent after 528255 bytes; yielding`, over and over. While the debugger is stopped, try to resize the window: it does not repaint, because you stopped the only thread that draws it.

Break it

Remove `g_source_remove(read_watch_)` from `~PtySession`. Run `yes` in a tab and close the tab while it runs. The descriptor watch still exists and still holds the deleted session's address; the next time the pty is readable, it calls `on_readable` on freed memory. In a normal build this may crash, or may not, depending on what reused the memory. In the AddressSanitizer build of Chapter 16 it is reported on the first try as `heap-use-after-free`, with the stacks that allocated and freed the session both pointing into your code.

Common mistakes

Watching a pty at the default priority. The window stops repainting under heavy output. Use idle priority.

Reading until `EAGAIN` with no budget. The callback never returns while the child writes faster than you read.

Removing the tab from the exit callback. The callback belongs to the object you are about to delete. Defer to an idle.

Not clearing a source id when returning `G_SOURCE_REMOVE`. The source is gone; removing it again later is a critical warning.

Forgetting to reap the child. Every child must be waited for, or it stays in the process table as a zombie until gtklab exits.

Read in tmssh

`PtySession::on_readable` and `kReadBudget` in `gtkpyte/src/ptysession.cpp`. tmssh's session

waits for end of file and then collects the child's exit status on a short retry timer, rather than using a child watch, so the last output is never cut off. Then `start_tab` and its deferred removal, `focus_later`, `on_sessions_changed` and `watch_sessions_file`.
An exercise: tmssh watches its pty with `g_unix_fd_add`, at the default priority. Run `cat` of a 50 MB file in tmssh and take screenshots a second apart. Measured while writing this book, tmssh repainted once in six seconds. Try the idle priority in your own copy.

Review

1. In one iteration, a timer added with `g_timeout_add` is due, a descriptor watched with `g_unix_fd_add` is readable, and an idle added with `g_idle_add` is pending. Which callbacks run in this iteration, and which waits?
2. `on_readable` already stops after 512 KiB. Why is that not enough to keep the window repainting during `cat` of a large file at the default priority?
3. A program has one idle that always returns `G_SOURCE_CONTINUE`; another has only descriptor watches. Why does the first keep a CPU core busy while the second uses none until data arrives?
4. The input line sends `ls\n` to the session, and `ls` appears in the output view before the listing. gtklab never wrote it there. Where did it come from?
5. Byte 3 written to the pty stops `sleep 100` in the tab. Which part of the system turns that byte into a signal, and which processes receive it?
6. The exit handler closes the tab from an idle that holds a reference to the page. What goes wrong without the idle? Without the reference?
7. `on_readable` sets `read_watch_ = 0` before returning `G_SOURCE_REMOVE`. What would happen later if it did not?
8. Why does the sessions file reload restart its timer on every event, rather than reloading on the first event and ignoring the rest?

Questions for deeper thought. tmssh collects the exit status only after end of file on the pty. What does your own tool lose if "child exited" arrives before "output drained"? A tool that polls two hundred devices over ssh has two hundred descriptors at the same priority: what does the read budget do for fairness between them, and what does it not do? And which work in your tools would still belong on a worker thread, with only its result handed to the main loop?

Exercises

1. **Count the wakeups.** In `ch11-mainloop/src/pty_session.cpp`, give `PtySession` two counters, wakeups and bytes, updated in `PtySession::on_readable`, and print them with `g_debug` in `~PtySession`. Done when running `seq 3000000` and then closing the tab logs both numbers, at most `kReadBudget` plus one 16 KiB read per wakeup (the drill's log says 528255 bytes, not 524288).
2. **Change the budget.** With exercise 1 in place, set `kReadBudget` to 16 KiB and run `time cat` on a large file in a tab, then do the same at 512 KiB. Done when you have the wakeup count and the total time for both budgets written down.
3. **Keep a failed tab open.** In `AppWindow::start_tab` (`ch11-mainloop/src/app_window.cpp`), close the tab on idle only when the child exited with status 0; otherwise append `[exited with status N]` to the view and leave the tab open. Use `g_spawn_check_wait_status` to read the status. Done when `exit 3` leaves the tab with the message and `exit` closes it.

4. **Name the events.** In `AppWindow::on_sessions_changed`, log each event by name with `g_enum_to_string(G_TYPE_FILE_MONITOR_EVENT, event)`. Done when one save from your editor lists every event it caused, followed by exactly one `sessions file settled; reloading` line.

Chapter 12 — Writing a widget

The text view standing in for a terminal goes, and in its place comes a widget you write yourself: `GtklabGrid`, a fixed-pitch character grid that knows its size in rows and columns and tells the pty when it changes. It is not a terminal emulator; it handles printable text, carriage return, line feed, backspace and tab, and skips escape sequences. That is enough to learn how a GTK 4 widget is measured, sized and drawn, which is the point.

Recall

What does a factory's `bind` do (Chapter 5)? Name the macro pair that declares and defines a GObject type (Chapter 5). Which side of the split owns the model (Chapter 4)?

In this chapter

- Subclass `GtkWidget` and override `measure`, `size_allocate` and `snapshot`, knowing what question each one answers.
- Report a minimum and a natural size, and turn whatever size you are given into rows and columns.
- Change a widget's state and ask for a frame, instead of drawing.
- Measure and draw monospace text with Pango so that it stays on a grid.
- Give a widget a property and a signal, so that the code around it never needs to know its insides.

Concepts

A widget answers three questions

A custom widget is a GObject subclass of `GtkWidget` that overrides a few virtual functions in its class structure. Three of them carry the work:

Virtual function	The question	GtklabGrid's answer	Called
`measure`	How big would you like to be, in this orientation?	20 × 5 cells at least, 80 × 24 by preference	when a layout pass needs it
`size_allocate`	This is how big you are.	divide the pixels by the cell size	after the parent has decided
`snapshot`	What do you look like?	a background, a cursor, one text layout per row	when a frame needs a new recording

You never call these yourself. GTK calls them, in that order, when it lays out and paints the window. A widget with children also positions them in `size_allocate`; GtklabGrid has none.

The rest of the widget's life brackets these calls. `init` runs once, when the instance is created; `dispose` and `finalize` run at the end. Between them, the widget is **realized** when it is put on a display, and only then does its Pango context carry that display's font settings. gtkpyte measures its cell again in a `realize` handler for that reason: metrics taken in `init` can be off by a pixel.

Size is negotiated, top down

A widget does not choose its size; it states a range and the parent chooses. `measure` is called once per orientation and fills in two numbers: the **minimum**, below which the widget cannot work, and the **natural** size, which it would take if space were free. The parent adds up its children's answers, the window does the same for its child, and the window's own size decides how much there is to share. Then allocation runs back down: each parent calls `size_allocate` on its children with the space it gave them.

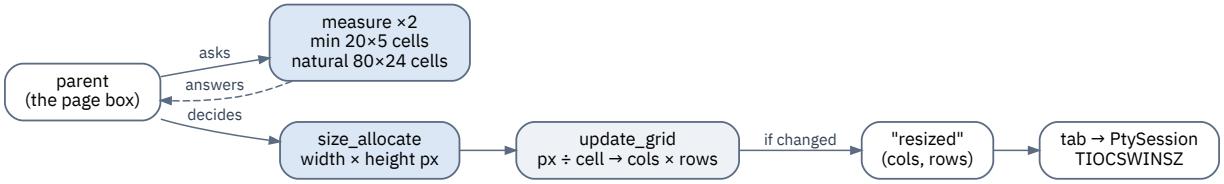


Figure 1: Size negotiation. The parent asks, the grid answers in pixels derived from cells, the parent decides, and the grid turns the pixels back into cells and tells the pty.

The consequence for a grid is that the allocation is almost never the natural size. It is larger when the window has room to spare, since GtklabGrid sets `hexpand` and `vexpand`, and smaller when the sidebar and the window edges take their share. So `size_allocate` must accept any size and make the best of it: integer division gives the whole cells, and the remainder is a margin at the right and bottom that the background fills. In the recorded run, 637 by 480 pixels became 70 columns by 26 rows of 9-by-18-pixel cells, ten columns short of the natural 80, with 7 pixels left over at the right and 12 at the bottom.

`measure` also receives a `for_size` argument, the size already fixed in the other orientation, and a widget can declare through `get_request_mode` that its height depends on its width. A wrapping label works that way: the narrower it is, the taller it gets. A grid has no such trade, so GtklabGrid ignores `for_size` and keeps the default request mode, which a test program reported as `GTK_SIZE_REQUEST_CONSTANT_SIZE`.

Queue, don't draw

When something changes the widget's size, you call `gtk_widget_queue_resize`; when something changes only its appearance, `gtk_widget_queue_draw`. GTK's **frame clock** then runs, at most once per frame, a layout pass that measures and allocates, and a paint pass that calls `snapshot` and renders the recorded operations. The frame runs from the main loop at `GDK_PRIORITY_REDRAW`, which is why Chapter 11's pty watch sits below it.

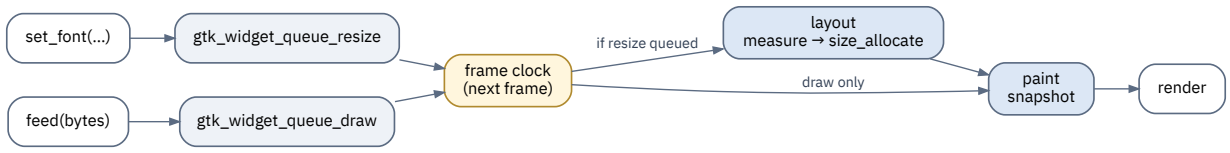


Figure 2: The two pipelines. Changing the font changes the size; new output changes only the appearance. Both end at the frame clock.

`snapshot` does not draw pixels. It records a list of operations, a tree of render nodes, and GTK keeps the recording. A widget that has not asked for a new frame is not asked to record again: in a test program, a widget beside a label whose text changed twenty times was snapshotted once, and one `gtk_widget_queue_draw` on it produced exactly one more snapshot. That is why you never draw from a signal handler and why forgetting `queue_draw` leaves stale content on screen: the old recording is still valid as far as GTK knows.

What changed	What you call	What runs on the next frame
The font, so the cell size	`gtk_widget_queue_resize`	`measure`, `size_allocate`, `snapshot`
New bytes in the cell model	`gtk_widget_queue_draw`	`snapshot`
Focus came or went (the cursor)	`gtk_widget_queue_draw`	`snapshot`
The window was resized	nothing: GTK queues it	`size_allocate`, `snapshot`

Several calls before the frame cost the same as one. `gtklab_grid_feed` queues a draw for every chunk the pty delivers, and the grid is still snapshotted at most once per frame.

Text on a grid

Drawing text means Pango, GTK's text layout library. A `PangoLayout` holds a string, a font and attributes, and knows how large it renders; `gtk_widget_create_pango_layout` makes one with the widget's font context, and `gtk_snapshot_append_layout` draws it.

A terminal grid needs every character in a column to start at the same pixel. The grid gets that by measuring one cell, the size of "M", and trusting that a monospace font's other glyphs are as wide. Anything that changes widths breaks the trust: ligatures, which the grid turns off, and characters that a monospace font draws two cells wide, which GtklabGrid does not handle and gtkpyte does.

Properties and signals: the widget's outward face

A widget can also have **properties** (named, typed values that can be set and watched, here `font`) and **signals** of its own (here `resized`). Both are declared in `class_init`, once per type.

The choice of what to put in them is a design decision. `resized` carries columns and rows, not pixels, because cells are what the pty needs and pixels are the widget's private business. And the grid emits a signal rather than calling the session, so it knows nothing about ptys; the tab connects the two. Chapter 13 sends keys out the same way, and Chapter 14 replaces the widget without touching the session.

Where the C++ lives

GObject allocates an instance with zeroed memory and runs no C++ constructors, so a `std::string` member is not a valid string until something builds it. There are two ways to deal with that:

	C++ members in the instance struct	One pointer to a C++ struct
Used by	GtklabGrid (and Chapter 5's item)	gtkpyte (`State *st`)
In `init`	placement `new` for each member	`new State()`
In `finalize`	an explicit destructor call for each member	`delete st`
Risk	forgetting one member in either list	one extra indirection

GtklabGrid uses the first so you see exactly what GObject does not do for you. gtkpyte uses the second because its state has many members with constructors; one `new` and one `delete` keep the problem in one place.

Build

The interface:

ch12-widget/src/gtklab_grid.h

```
// GtklabGrid: a character-cell widget. It shows text in a fixed-pitch grid
// and reports its size in cells. It is not a terminal emulator: it handles
// printable text, \r, \n, \b and \t, and skips escape sequences.
#pragma once

#include <gtk/gtk.h>

G_BEGIN_DECLS

#define GTKLAB_TYPE_GRID (gtklab_grid_get_type())
G_DECLARE_FINAL_TYPE(GtklabGrid, gtklab_grid, GTKLAB, GRID, GtkWidget)

GtkWidget *gtklab_grid_new(void);

// The "font" property: a Pango description string such as "Monospace 11".
void gtklab_grid_set_font(GtklabGrid *grid, const char *font);
const char *gtklab_grid_get_font(GtklabGrid *grid);

// Write UTF-8 text into the cell model at (row, col), clipped to the grid.
void gtklab_grid_put_text(GtklabGrid *grid, int row, int col, const char *utf8);

// Feed bytes from a pty: printable text at the cursor, with wrapping and
// scrolling. Partial UTF-8 sequences are kept until the rest arrives.
void gtklab_grid_feed(GtklabGrid *grid, const char *data, gsize len);

int gtklab_grid_get_columns(GtklabGrid *grid);
int gtklab_grid_get_rows(GtklabGrid *grid);

// Signals:
//   "resized" (GtklabGrid *, int columns, int rows): the grid changed shape.
```

```
G_END_DECLS
```

The instance and the class

The instance struct starts with the parent instance, as every GObject does, and holds C++ members that are built with placement `new` in `init` and destroyed in `finalize`, exactly as Chapter 5's item did:

ch12-widget/src/gtklab_grid.cpp

```
struct _GtklabGrid {
    GtkWidget parent_instance;

    // GObject zero-fills the instance and runs no C++ constructors. The C++
    // members are built with placement new in init and destroyed in finalize.
    std::string font;
    std::vector<Line> screen;  // rows x cols, ' ' for empty
    std::string pending;       // incomplete UTF-8 or escape sequence

    PangoFontDescription *font_desc;  // owned
    PangoAttrList *attrs;             // owned
    int cell_width;
    int cell_height;
    int cols;
    int rows;
    int cursor_row;
    int cursor_col;
};
```

`class_init` installs the virtual functions, the `font` property and the `resized` signal, and gives the widget a CSS name so that themes can style it:

ch12-widget/src/gtklab_grid.cpp

```
static void gtklab_grid_class_init(GtklabGridClass *klass) {
    GObjectClass *object_class = G_OBJECT_CLASS(klass);
    GtkWidgetClass *widget_class = GTK_WIDGET_CLASS(klass);

    object_class->set_property = gtklab_grid_set_property;
    object_class->get_property = gtklab_grid_get_property;
    object_class->dispose = gtklab_grid_dispose;
    object_class->finalize = gtklab_grid_finalize;

    widget_class->measure = gtklab_grid_measure;
    widget_class->size_allocate = gtklab_grid_size_allocate;
    widget_class->snapshot = gtklab_grid_snapshot;

    props[PROP_FONT] = g_param_spec_string("font", nullptr, nullptr, "Monospace 11",
                                           static_cast<GParamFlags>(G_PARAM_READWRITE |
                                                                    G_PARAM_EXPLICIT_NOTIFY |
                                                                    G_PARAM_STATIC_STRINGS));
    g_object_class_install_properties(object_class, N_PROPS, props);

    signals[SIGNAL_RESIZED] = g_signal_new("resized", G_TYPE_FROM_CLASS(klass), G_SIGNAL_RUN_LAST, 0,
                                           nullptr, nullptr, nullptr, G_TYPE_NONE, 2, G_TYPE_INT,
                                           G_TYPE_INT);

    gtk_widget_class_set_css_name(widget_class, "gtklabgrid");
}
```

`init` builds the C++ members, makes the widget focusable and expanding, redraws when focus comes and goes (the cursor depends on it), and sets the initial font and grid:

ch12-widget/src/gtklab_grid.cpp

```
static void gtklab_grid_init(GtklabGrid *self) {
    new (&self->font) std::string();
    new (&self->screen) std::vector<Line>();
    new (&self->pending) std::string();
    self->font_desc = nullptr;
    self->attrs = nullptr;
    self->cols = 0;
    self->rows = 0;
    self->cursor_row = 0;
    self->cursor_col = 0;

    gtk_widget_set_focusable(GTK_WIDGET(self), TRUE);
    gtk_widget_set_hexpand(GTK_WIDGET(self), TRUE);
    gtk_widget_set_vexpand(GTK_WIDGET(self), TRUE);
    gtk_widget_set_overflow(GTK_WIDGET(self), GTK_OVERFLOW_HIDDEN);

    // Focus changes the cursor, so redraw on focus in and out.
    GtkEventController *focus = gtk_event_controller_focus_new();
    g_signal_connect_swapped(focus, "enter", G_CALLBACK(gtk_widget_queue_draw), self);
    g_signal_connect_swapped(focus, "leave", G_CALLBACK(gtk_widget_queue_draw), self);
    gtk_widget_add_controller(GTK_WIDGET(self), focus);

    gtklab_grid_set_font(self, "Monospace 11");
    resize_screen(self, 80, 24);
}
```

`dispose` and `finalize` split cleanup in two. `dispose` drops references to other objects and may run more than once; `finalize` frees everything else and runs exactly once, last:

ch12-widget/src/gtklab_grid.cpp

```
// dispose: drop references to other objects (may run more than once).
static void gtklab_grid_dispose(GObject *object) {
    auto *self = GTKLAB_GRID(object);
    g_clear_pointer(&self->attrs, pango_attr_list_unref);
    G_OBJECT_CLASS(gtklab_grid_parent_class)->dispose(object);
}
```

ch12-widget/src/gtklab_grid.cpp

```
// finalize: free everything else (runs once, last).
static void gtklab_grid_finalize(GObject *object) {
    auto *self = GTKLAB_GRID(object);
    g_clear_pointer(&self->font_desc, pango_font_description_free);
    self->font.~basic_string();
    self->screen.~vector();
    self->pending.~basic_string();
    G_OBJECT_CLASS(gtklab_grid_parent_class)->finalize(object);
}
```

The cell size

A character grid needs a fixed cell. It is the size of "M" in the widget's font, measured with the same attributes every row will be drawn with. Those attributes turn ligatures off: with them on, programming fonts draw `->` or `==` as a single glyph whose width is not two cells, and every character after it lands off the grid.

ch12-widget/src/gtklab_grid.cpp

```
// Base attributes for every layout: the font, and no ligatures. With
// ligatures on, "->" or "==" can become one glyph wider or narrower than two
// cells, and every character after it lands off the grid.
static PangoAttrList *base_attrs(GtklabGrid *self) {
    PangoAttrList *attrs = pango_attr_list_new();
    pango_attr_list_insert(attrs, pango_attr_font_desc_new(self->font_desc));
    pango_attr_list_insert(attrs, pango_attr_font_features_new("liga 0, calt 0"));
    return attrs;
}
```

ch12-widget/src/gtklab_grid.cpp

```
// Cell size = the size of "M" in the widget's font.
static void measure_cells(GtklabGrid *self) {
    PangoLayout *layout = gtk_widget_create_pango_layout(GTK_WIDGET(self), "M");
    pango_layout_set_attributes(layout, self->attrs);
    int w = 0, h = 0;
    pango_layout_get_pixel_size(layout, &w, &h);
    g_object_unref(layout);
    self->cell_width = std::max(w, 1);
    self->cell_height = std::max(h, 1);
}
```

Setting the font re-measures the cell and asks for a new layout pass, which will allocate the widget again and so recompute the grid:

ch12-widget/src/gtklab_grid.cpp

```
void gtklab_grid_set_font(GtklabGrid *self, const char *font) {
    g_return_if_fail(GTKLAB_IS_GRID(self));
    if (!font || self->font == font)
        return;
    self->font = font;
    pango_font_description_free(self->font_desc);
    self->font_desc = pango_font_description_from_string(font);
    pango_attr_list_unref(self->attrs);
    self->attrs = base_attrs(self);
    measure_cells(self);
    // New cell size: re-measure, re-allocate (-> update_grid), redraw.
    gtk_widget_queue_resize(GTK_WIDGET(self));
    g_object_notify_by_pspec(G_OBJECT(self), props[PROP_FONT]);
}
```

Size

`measure` asks for twenty columns and five rows as a minimum, and eighty by twenty-four as natural:

ch12-widget/src/gtklab_grid.cpp

```
static void gtklab_grid_measure(GtkWidget *widget, GtkOrientation orientation, int,
                                int *minimum, int *natural, int *, int *) {
    auto *self = GTKLAB_GRID(widget);
    if (orientation == GTK_ORIENTATION_HORIZONTAL) {
        *minimum = self->cell_width * 20;
        *natural = self->cell_width * 80;
    } else {
        *minimum = self->cell_height * 5;
        *natural = self->cell_height * 24;
    }
}
```

`size_allocate` converts pixels to cells and emits `resized` only when the shape actually changes:

ch12-widget/src/gtklab_grid.cpp

```
// Pixels -> cells. Emits "resized" when the shape changes.
static void update_grid(GtklabGrid *self, int width, int height) {
    const int cols = std::max(width / self->cell_width, 1);
    const int rows = std::max(height / self->cell_height, 1);
    if (cols == self->cols && rows == self->rows)
        return;
    resize_screen(self, cols, rows);
    g_debug("update_grid: %dx%d px -> %d cols x %d rows", width, height, cols, rows);
    g_signal_emit(self, signals[SIGNAL_RESIZED], 0, cols, rows);
}
```

ch12-widget/src/gtklab_grid.cpp

```
static void gtklab_grid_size_allocate(GtkWidget *widget, int width, int height, int) {
    update_grid(GTKLAB_GRID(widget), width, height);
}
```

Drawing

`snapshot` fills the background, draws the cursor when the widget has focus, and draws each row as one Pango layout, translated to its row:

ch12-widget/src/gtklab_grid.cpp

```
static void paint_row(GtklabGrid *self, GtkSnapshot *snapshot, int row) {
    std::string text;
    for (gunichar c : self->screen[static_cast<size_t>(row)]) {
        char utf8[6];
        text.append(utf8, static_cast<size_t>(g_unichar_to_utf8(c, utf8)));
    }
    PangoLayout *layout = gtk_widget_create_pango_layout(GTK_WIDGET(self), text.c_str());
    pango_layout_set_attributes(layout, self->attrs);

    gtk_snapshot_save(snapshot);
    graphene_point_t at = GRAPHENE_POINT_INIT(0.f, static_cast<float>(row * self->cell_height));
    gtk_snapshot_translate(snapshot, &at);
    gtk_snapshot_append_layout(snapshot, layout, &kForeground);
    gtk_snapshot_restore(snapshot);
    g_object_unref(layout);
}
```

ch12-widget/src/gtklab_grid.cpp

```
// Records drawing operations; GTK renders them later. Never called directly:
// gtk_widget_queue_draw asks the frame clock to call it on the next frame.
static void gtklab_grid_snapshot(GtkWidget *widget, GtkSnapshot *snapshot) {
    auto *self = GTKLAB_GRID(widget);
    const int w = gtk_widget_get_width(widget);
    const int h = gtk_widget_get_height(widget);

    graphene_rect_t all = GRAPHENE_RECT_INIT(0.f, 0.f, static_cast<float>(w), static_cast<float>(h));
    gtk_snapshot_append_color(snapshot, &kBackground, &all);

    if (gtk_widget_has_focus(widget)) {
        graphene_rect_t cursor = GRAPHENE_RECT_INIT(
            static_cast<float>(std::min(self->cursor_col, self->cols - 1) * self->cell_width),
            static_cast<float>(self->cursor_row * self->cell_height),
            static_cast<float>(self->cell_width), static_cast<float>(self->cell_height));
        gtk_snapshot_append_color(snapshot, &kCursor, &cursor);
```

```
    }

    for (int row = 0; row < self->rows; row++)
        paint_row(self, snapshot, row);
}
```

Bytes in

`gtklab_grid_feed` takes bytes from the pty. It keeps an incomplete UTF-8 character or escape sequence until the rest arrives, because a read can end anywhere; it skips CSI and OSC escape sequences; and it ends by queueing a draw:

ch12-widget/src/gtklab_grid.cpp

```
void gtklab_grid_feed(GtklabGrid *self, const char *data, gsize len) {
    g_return_if_fail(GTKLAB_IS_GRID(self));
    std::string &buf = self->pending;
    buf.append(data, len);

    size_t i = 0;
    while (i < buf.size()) {
        const unsigned char c = static_cast<unsigned char>(buf[i]);
        if (c == 0x1b) {
            const size_t n = escape_length(buf, i);
            if (n == 0)
                break;  // wait for the rest
            i += n;
        } else if (c == '\r') {
            self->cursor_col = 0;
            i++;
        } else if (c == '\n') {
            newline(self);
            i++;
        } else if (c == '\b') {
            self->cursor_col = std::max(0, self->cursor_col - 1);
            i++;
        } else if (c == '\t') {
            const int next = std::min((self->cursor_col / 8 + 1) * 8, self->cols - 1);
            while (self->cursor_col < next)
                put_char(self, ' ');
            i++;
        } else if (c < 0x20 || c == 0x7f) {
            i++;  // other control characters: ignore
        } else {
            const gunichar u = g_utf8_get_char_validated(buf.data() + i, static_cast<gssize>(buf.size() -
            ↪  i));
            if (u == static_cast<gunichar>(-2))
                break;  // incomplete sequence: wait for more bytes
            if (u == static_cast<gunichar>(-1)) {
                put_char(self, 0xfffd);  // invalid byte
                i++;
                continue;
            }
            put_char(self, u);
            i = static_cast<size_t>(g_utf8_next_char(buf.data() + i) - buf.data());
        }
    }
    buf.erase(0, i);
    gtk_widget_queue_draw(GTK_WIDGET(self));
}
```

In the app

The grid replaces the text view. The pty's output goes to `gtklab_grid_feed`, and the grid's `resized` signal goes to the session, so the child learns its new size:

ch12-widget/src/app_window.cpp

```
GtklabGrid *grid = GTKLAB_GRID(tab->view);
tab->pty->set_output_handler([grid](const char *data, gsize len) { gtklab_grid_feed(grid, data, len); });

// The grid changed shape: tell the child (TIOCSWINSZ -> SIGWINCH).
g_signal_connect(tab->view, "resized",
                 G_CALLBACK(+[](GtklabGrid *, int cols, int rows, gpointer data) {
                     static_cast<Tab *>(data)->pty->resize(cols, rows);
                 }),
                 tab);
```

The session starts at the grid's current size rather than a fixed 80 by 24. The input line under the grid stays for one more chapter.

Done when

Output appears in the grid in a monospace font, and `stty size` in the shell follows the window as it is resized.

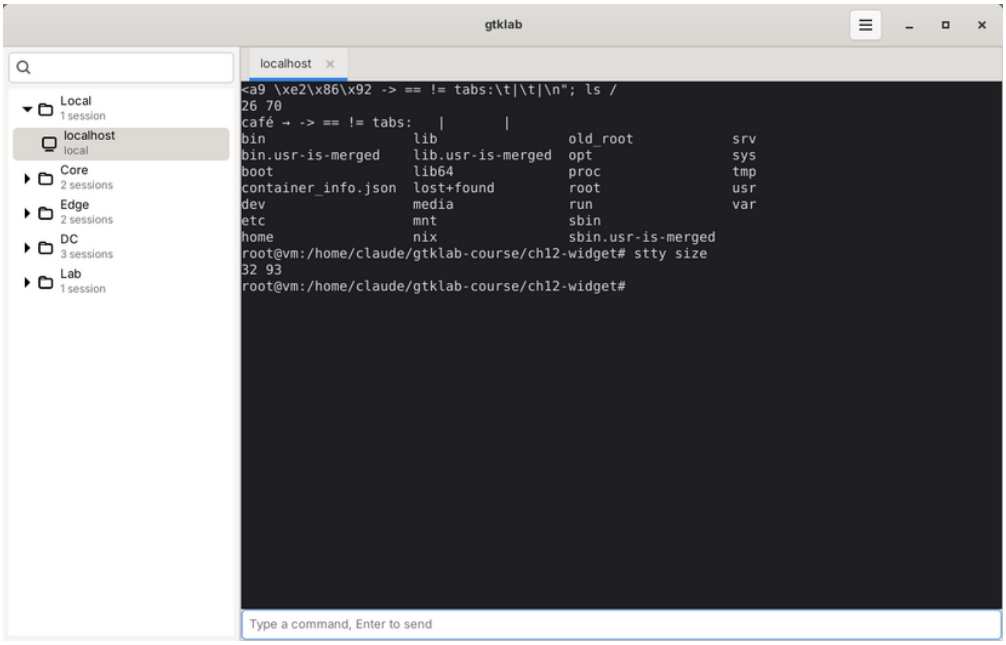


Figure 3: GtklabGrid. `stty size` reported 26 rows by 70 columns, then 32 by 93 after the window was enlarged.

Debugger drill

Set a breakpoint in `update_grid` and resize the window. In the recorded run the first allocation was `637x480 px -> 70 cols x 26 rows` and the resized one `837x580 px -> 93 cols x 32 rows`, each followed by the `resized` emission and, in the shell, a new `stty size`. Then set a breakpoint in `gtklab_grid_snapshot` with a hit count of 1, type a command and read the stack when it stops. It starts at the frame clock, not at your feed function: `queue_draw` only asked.

Break it

Delete the `gtk_widget_queue_draw` at the end of `gtklab_grid_feed`. Run `echo one; echo two`. The grid stays blank, prompt included. The text is all in the cell model; nothing asked for a frame. Resize the window, which does ask for one, and everything appears at once.



Figure 4: The Break it. Top: after the echo, nothing. Bottom: the same grid after a resize.

Common mistakes

Drawing in a signal handler. There is nothing to draw on outside `snapshot`. Change state, queue a draw, and let `snapshot` read the state.

`queue_draw` when the size changed. A new font needs a new layout pass: `queue_resize`, which also redraws.

Chaining up in the wrong place. `dispose` and `finalize` must call the parent class's version, at the end.

Ligatures in a grid. Turn them off for anything that must line up in columns.

Read in tmssh

gtkpyte's terminal in `gtkpyte/src/gtkpyte-terminal.cpp`: `gtkpyte_terminal_class_init`, `gtkpyte_terminal_size_allocate`, `update_grid`, `gtkpyte_terminal_snapshot`, `paint_row`, `measure_cells` and `base_attrs`. The names match on purpose; gtkpyte also draws colours, bold, underline, wide characters and a selection.

Review

1. The grid's natural width is 80 cells, and it is allocated 637 pixels with 9-pixel cells. What does `size_allocate` do with that, and what happens to the 7 pixels left over?
2. Setting the font calls `gtk_widget_queue_resize`; feeding bytes calls `gtk_widget_queue_draw`. What would go wrong if the font setter called only `queue_draw`?
3. In the Break it, the text is in the cell model but the grid stays blank. Why does GTK keep showing the old picture rather than asking the grid again?
4. Why does `resized` carry columns and rows rather than pixels, and why does `update_grid` emit it only when the shape changes?

5. `measure` receives a `for_size` that GtklabGrid ignores. What kind of widget would need it, and why does a character grid not?
6. Why must `measure_cells` lay out "M" with the same attribute list that `paint_row` uses?
7. The snapshot's stack in the debugger drill starts at the frame clock, not at `gtklab_grid_feed`. What does that tell you about when bytes read in `on_readable` reach the screen?
8. GtklabGrid builds its C++ members with placement `new` in `init`. What would happen to a `std::string` member if `init` did not?

Questions for deeper thought. gtkpyte measures its cell again when it is realized, because the font settings belong to the display. What else in a widget of yours depends on the display it lands on, such as the scale factor of a HiDPI screen, and where would you recompute it? If you wrote a widget for your own tools, say a grid of device states or a latency strip chart, which of its state would you expose as properties and which events as signals? `paint_row` builds a fresh Pango layout for every row on every snapshot: at what size of grid would you start caching, and what would the cache be keyed on?

Exercises

1. **A hollow cursor.** In `gtklab_grid_snapshot` (`ch12-widget/src/gtklab_grid.cpp`), draw the cursor as an outline with `gtk_snapshot_append_border` when the grid does not have focus, instead of not drawing it. Done when clicking the sidebar leaves a hollow box at the cursor and clicking the grid fills it again.
2. **Columns and rows as properties.** Add read-only `columns` and `rows` properties (`G_PARAM_READABLE | G_PARAM_EXPLICIT_NOTIFY`) and notify them from `update_grid`. In `ch12-widget/src/app_window.cpp`, connect `notify::columns` to a handler that logs the new value. Done when resizing the window logs a line only when the column count changes.
3. **Centre the grid.** Store the leftover pixels in `update_grid` and translate the whole drawing in `gtklab_grid_snapshot` by half of them in each direction. Done when the margins on the left and right differ by at most one pixel, checked with the GTK Inspector or a screenshot.
4. **Count the frames.** Add counters to `gtklab_grid_feed` and `gtklab_grid_snapshot` and log both every second from a `g_timeout_add_seconds` started in `gtklab_grid_init` (and removed in `gtklab_grid_dispose`). Run `seq 1000000` in a tab. Done when both counts are logged and you can explain, from the frame clock, why they differ.

Chapter 13 — Scrollback, keys and selection in your widget

The grid grows the three things that make a terminal usable: a scrollback you can wheel through, keys typed straight into it instead of into an input line, and text you can select, copy and paste. Each is a GTK mechanism you have used from the outside and now implement from the inside: a scrollable widget, a key controller, and a drag gesture with the clipboard.

Recall

Which phase do the page's right-click and zoom controllers run in, and why (Chapter 9)? When does `snapshot` run (Chapter 12)? At what priority does the pty watch run, and why (Chapter 11)?

In this chapter

- Implement `GtkScrollable` and keep a pair of adjustments in step with a model whose unit you choose.
- Decide when new output should move the view and when it must leave the user where they are.
- Turn key presses into the bytes a terminal sends, and say which bytes the kernel changes on the way to the shell.
- Select text with a drag gesture, and copy and paste through the asynchronous clipboard without outliving the widget.

Concepts

Two ways to scroll

A `GtkScrolledWindow` can scroll any child, but it scrolls a child in one of two ways. A child that does not implement **`GtkScrollable`** is wrapped in a `GtkViewport` (a test program that put a label in a scrolled window got a `GtkViewport` back from `gtk_scrolled_window_get_child`): the child is allocated its full size, drawn whole, and moved. That is wasteful for a terminal with thousands of lines of history. A child that implements `GtkScrollable` is given the scrolled window's two `GtkAdjustment` objects instead, one per direction, is allocated the visible area (or its minimum size, if that is larger), and draws only what the adjustment says is visible. A text view works this way, and from this chapter so does the grid.

The adjustment is a contract

An adjustment is a value with a range: `lower`, `upper`, `page_size` and the current `value`, plus the step and page increments for moving by a line or a page. It belongs to neither side. The scrolled window

draws its scrollbar from it and moves `value` when the user drags the bar or turns the wheel; the widget keeps the range current and redraws when `value` changes.

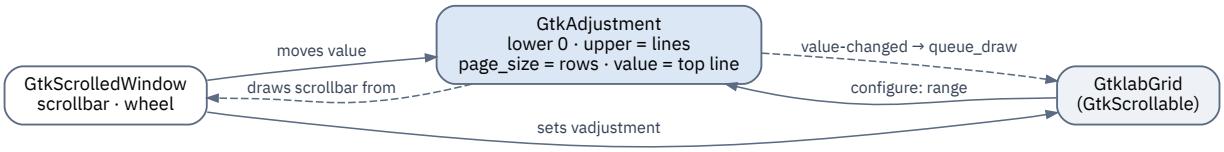


Figure 1: The GtkScrollable contract. The scrolled window hands the grid an adjustment; the grid sets the range; either side can move the value, and the grid redraws when it moves.

The units are the widget's choice, and the choice shapes everything after it. GtklabGrid uses lines:

Field	In GtklabGrid	Who changes it
`lower`	0, the oldest line kept	the grid
`upper`	lines held, scrollback and screen together	the grid, as lines arrive or are trimmed
`page_size`	rows on screen	the grid, on resize
`value`	the first line shown	the scrolled window (wheel, scrollbar), the grid (follow, typing)

Pixels would work too, and a text view uses them, because its lines are of different heights. A terminal's lines are all one cell high, and in lines the arithmetic stays whole: the first line drawn is `value`, a click at pixel `y` is on line `value + y / cell_height`, and a selection can be stored in absolute line numbers that do not change when the view scrolls.

The contract has one rule that is easy to miss: the scrolled window cannot know that the content grew. Every change to the number of lines must go back into the adjustment, or the scrollbar describes a document that no longer exists; this chapter's Break it shows exactly that. And because both sides can move `value`, the widget must decide when moving it is its business. New output follows the bottom only if the user was already there; a key press always returns to the bottom, as in every terminal.

Implementing an interface is declared in the type definition, with `G_DEFINE_FINAL_TYPE_WITH_CODE` and `G_IMPLEMENT_INTERFACE`. `GtkScrollable` declares four properties (`hadjustment`, `vadjustment`, `hscroll-policy`, `vscroll-policy`); an implementation overrides them in `class_init` and stores what it is given. The scrolled window sets them when the grid becomes its child.

Keys become bytes

A key press reaches a widget as a **keyval**, GDK's name for the symbol the key produces with the current layout and Shift already applied (`a`, `A`, `Return`, `Up`), together with the state of the modifiers. A terminal sends neither: it sends bytes, the ones a hardware terminal would have sent for that key. Printable keys become their UTF-8 text; Return becomes a carriage return; Ctrl with a letter becomes a control character, `a` as 1 through `z` as 26; keys with no character, like the arrows, become escape sequences.

The grid gets keys from a `GtkEventControllerKey` and hands the bytes out through its `commit` signal. The grid never writes to the pty itself; the tab connects `commit` to the session. Keeping the widget

ignorant of where its bytes go is what lets Chapter 14 swap it out, and what would let the same widget sit on an ssh channel or a test harness.

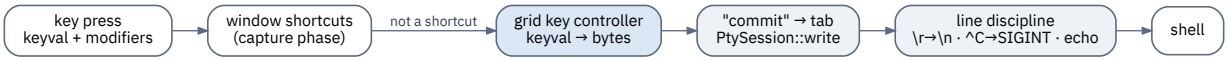


Figure 2: A key on its way to the shell. The window's shortcuts see it first; only a key that is not a shortcut becomes bytes, and the kernel changes some of those bytes again.

Chapter 11's line discipline does the rest, and it explains choices that look arbitrary from inside the widget. Return sends `\r`, because that is what a terminal's Return key sends. A program reading lines, such as `cat`, gets `\n`, because the line discipline turns carriage return into newline on input; bash at its prompt switches that off and reads the `\r` itself. Backspace sends byte 127, because that is the pty's erase character. Ctrl+C sends byte 3, and the kernel, not the grid, turns it into `SIGINT`. And the grid never draws what you type: the echo comes back through the pty as output, from the line discipline or the shell. A paste follows the same path, which is why the grid replaces each newline in pasted text with `\r`, the byte Return sends.

The mapping is where a toy and a real terminal part ways:

Key	GtklabGrid sends	gtkpyte sends
Return	`\r`	`\r`
Backspace	byte 127	byte 127; Ctrl+Backspace sends byte 8
Ctrl+C	byte 3	byte 3
Up	`ESC [ A`	`ESC [ A`, or `ESC O A` when the application asked for cursor-key mode; `ESC [ 1 ; 5 A` with Ctrl
Alt+b	`b` (Alt is ignored)	`ESC b`
Page Up, F5, Shift+Tab	nothing: the key is not handled	`ESC [ 5 ~`, `ESC [ 15 ~`, `ESC [ Z`

The last column depends on state the application set by sending escape sequences, which GtklabGrid skips and gtkpyte tracks. That is Chapter 14's subject.

Neither widget creates an input method context (`GtkIMContext`), which is how GTK delivers text composed from several keys, such as accented letters from a compose key or Chinese and Japanese input. Both see key presses only. For a tool aimed at network engineers typing ASCII commands, that is a reasonable cut; for a general-purpose terminal it is a gap.

Who sees a key first

Chapter 9's propagation decides which keys reach the grid at all. The window's shortcut controller from Chapter 7 runs in the capture phase, before the grid's key controller, so gtklab's own shortcuts never become bytes. That gives a terminal a policy question every terminal answers the same way: the plain Ctrl combinations belong to the program in the terminal (Ctrl+C interrupts, Ctrl+V is a literal-next in the shell), so the application's own commands live on Ctrl+Shift. Copy, Paste and Select All here are Ctrl+Shift+C, V and A.

Selection and the clipboard

A selection is model state like any other: two cell positions, where the drag started and where the pointer is now. `snapshot` reads it to paint a translucent band under the text, and copying reads the same cells back out as text. A `GtkGestureDrag` supplies the positions: a start point on `drag-begin` and an offset from it on each `drag-update`. Nothing about the selection is stored in pixels, so scrolling or a new frame cannot disturb it.

The clipboard is not yours. Reading it is asynchronous, because the content may belong to another program and getting it takes a round trip through the display server. Any asynchronous call raises the question Chapter 11's deferred calls raised: will the object that asked still exist when the answer comes?

There are two clipboards on Linux desktops. The **clipboard** is what Ctrl+Shift+C and Ctrl+Shift+V use. The **primary selection** holds whatever was last selected and is pasted with the middle button. GtklabGrid uses only the first; gtkpyte also puts every selection on the primary selection and pastes it on a middle-click.

Build

The interface grows selection, the clipboard and the `commit` signal:

ch13-input/src/gtklab_grid.h

```
// Selection and clipboard.
gboolean gtklab_grid_has_selection(GtklabGrid *grid);
char *gtklab_grid_get_selected_text(GtklabGrid *grid);  // free with g_free
void gtklab_grid_select_all(GtklabGrid *grid);
void gtklab_grid_copy_clipboard(GtklabGrid *grid);
void gtklab_grid_paste_clipboard(GtklabGrid *grid);  // emits "commit" when the text arrives

// The grid implements GtkScrollable (in lines): put it in a GtkScrolledWindow.
//
// Signals:
//   "resized" (GtklabGrid *, int columns, int rows): the grid changed shape.
//   "commit"  (GtklabGrid *, GBytes *data): bytes to send to the child
//             (keys and pastes). The grid never writes anywhere itself.
```

Scrollback and the adjustments

The cell model becomes a `std::deque<Line>` of scrollback followed by the screen, capped at 5,000 lines of history. The type definition declares the interface:

ch13-input/src/gtklab_grid.cpp

```
G_DEFINE_FINAL_TYPE_WITH_CODE(GtklabGrid, gtklab_grid, GTK_TYPE_WIDGET,
                              G_IMPLEMENT_INTERFACE(GTK_TYPE_SCROLLABLE, nullptr))
```

and `class_init` overrides the interface's properties:

ch13-input/src/gtklab_grid.cpp

```
// GtkScrollable declares these; an implementation overrides them.
g_object_class_override_property(object_class, PROP_HADJUSTMENT, "hadjustment");
g_object_class_override_property(object_class, PROP_VADJUSTMENT, "vadjustment");
g_object_class_override_property(object_class, PROP_HSCROLL_POLICY, "hscroll-policy");
g_object_class_override_property(object_class, PROP_VSCROLL_POLICY, "vscroll-policy");
```

Keeping the range current is one function, called whenever lines are added or the size changes. Its `follow` flag decides whether the view stays at the bottom: new output keeps the view at the newest line only if the user was already there, so someone scrolled back to read is not yanked down:

ch13-input/src/gtklab_grid.cpp

```
// Keep the adjustment's range in step with the line count. Units are lines:
// upper = every line, page_size = one screen.
static void configure_adjustment(GtklabGrid *self, gboolean follow) {
    if (!self->vadjustment)
        return;
    const double upper = static_cast<double>(self->lines.size());
    const double value = follow ? static_cast<double>(screen_start(self))
                                : std::min(gtk_adjustment_get_value(self->vadjustment),
                                           static_cast<double>(screen_start(self)));
    gtk_adjustment_configure(self->vadjustment, value, 0.0, upper, 1.0, self->rows, self->rows);
}
```

When the scrolled window hands the grid an adjustment, the grid takes a reference, listens for changes, and configures it:

ch13-input/src/gtklab_grid.cpp

```
static void set_vadjustment(GtklabGrid *self, GtkAdjustment *adj) {
    if (self->vadjustment == adj)
        return;
    if (self->vadjustment) {
        g_signal_handlers_disconnect_by_data(self->vadjustment, self);
        g_object_unref(self->vadjustment);
    }
    if (!adj)
        adj = gtk_adjustment_new(0, 0, 0, 0, 0, 0);
    self->vadjustment = GTK_ADJUSTMENT(g_object_ref_sink(adj));
    g_signal_connect(adj, "value-changed", G_CALLBACK(on_vadjustment_changed), self);
    configure_adjustment(self, TRUE);
}
```

`snapshot` now draws from `top_line`, the adjustment's value, instead of the first screen row.

Keys

ch13-input/src/gtklab_grid.cpp

```
// Keys to bytes for the child. Returning FALSE lets the key go on to other
// handlers. The app's own shortcuts never reach here: the window's
// capture-phase controller takes them first.
static gboolean on_key_pressed(GtkEventControllerKey *, guint keyval, guint, GdkModifierType state,
                               gpointer data) {
    auto *self = GTKLAB_GRID(data);
    std::string out;
    const bool ctrl = state & GDK_CONTROL_MASK;
    const bool shift = state & GDK_SHIFT_MASK;

    switch (keyval) {
    case GDK_KEY_Return:
    case GDK_KEY_KP_Enter: out = "\r"; break;
    case GDK_KEY_BackSpace: out = "\x7f"; break;
    case GDK_KEY_Tab: out = "\t"; break;
    case GDK_KEY_Escape: out = "\x1b"; break;
    case GDK_KEY_Up: out = "\x1b[A"; break;
    case GDK_KEY_Down: out = "\x1b[B"; break;
    case GDK_KEY_Right: out = "\x1b[C"; break;
```

```
    case GDK_KEY_Left: out = "\x1b[D"; break;
    case GDK_KEY_Home: out = "\x1b[H"; break;
    case GDK_KEY_End: out = "\x1b[F"; break;
    case GDK_KEY_Delete: out = "\x1b[3~"; break;
    default:
        if (ctrl && !shift && keyval >= GDK_KEY_a && keyval <= GDK_KEY_z) {
            out = std::string(1, static_cast<char>(keyval - GDK_KEY_a + 1));  // Ctrl+C = 0x03
        } else if (ctrl) {
            return FALSE;  // not ours
        } else {
            const gunichar u = gdk_keyval_to_unicode(keyval);
            if (u == 0)
                return FALSE;  // a modifier on its own, a function key...
            char utf8[6];
            out.assign(utf8, static_cast<size_t>(g_unichar_to_utf8(u, utf8)));
        }
    }
    g_debug("grid key: %zu bytes", out.size());
    // Typing jumps back to the bottom, like every terminal.
    if (self->vadjustment)
        gtk_adjustment_set_value(self->vadjustment, static_cast<double>(screen_start(self)));
    emit_commit(self, out.data(), out.size());
    return TRUE;
}
```

Selection and the clipboard

A `GtkGestureDrag` records the cell under the press and the cell under the pointer as it moves, in absolute line numbers so that a selection survives scrolling. Copying joins the selected lines and trims trailing blanks, which are padding, not content:

ch13-input/src/gtklab_grid.cpp

```
char *gtklab_grid_get_selected_text(GtklabGrid *self) {
    if (!self->has_selection)
        return nullptr;
    CellPos start, end;
    selection_bounds(self, &start, &end);
    std::string out;
    for (long l = start.line; l <= end.line; l++) {
        const Line &line = self->lines[static_cast<size_t>(l)];
        const int from = (l == start.line) ? start.col : 0;
        const int to = (l == end.line) ? end.col : self->cols;
        std::string text;
        for (int c = from; c < to; c++) {
            char utf8[6];
            text.append(utf8, static_cast<size_t>(g_unichar_to_utf8(line[static_cast<size_t>(c)], utf8)));
        }
        while (!text.empty() && text.back() == ' ')  // trailing blanks are padding
            text.pop_back();
        out += text;
        if (l != end.line)
            out += '\n';
    }
    return g_strdup(out.c_str());
}
```

Pasting is asynchronous: the clipboard may belong to another program, and reading it takes a round trip. The grid could be destroyed before the text arrives, so the request holds a reference to it until the callback runs:

ch13-input/src/gtklab_grid.cpp

```
static void on_paste_ready(GObject *source, GAsyncResult *result, gpointer data) {
    auto *self = GTKLAB_GRID(data);  // we hold a reference: see paste_clipboard
    char *text = gdk_clipboard_read_text_finish(GDK_CLIPBOARD(source), result, nullptr);
    if (text) {
        std::string s = text;
        std::replace(s.begin(), s.end(), '\n', '\r');  // Enter sends \r
        emit_commit(self, s.data(), s.size());
        g_free(text);
    }
    g_object_unref(self);
}
```

ch13-input/src/gtklab_grid.cpp

```
void gtklab_grid_paste_clipboard(GtklabGrid *self) {
    // Asynchronous: the grid could be destroyed before the text arrives, so
    // the callback keeps it alive with a reference.
    gdk_clipboard_read_text_async(gtk_widget_get_clipboard(GTK_WIDGET(self)), nullptr, on_paste_ready,
                                  g_object_ref(self));
}
```

In the app

The grid goes inside a `GtkScrolledWindow`, which becomes the notebook page; the input line is deleted; and the grid's `commit` goes to the session:

ch13-input/src/app_window.cpp

```
// Keys and pastes from the grid go to the child.
g_signal_connect(tab->view, "commit",
                 G_CALLBACK(+[](GtklabGrid *, GBytes *bytes, gpointer data) {
                     gsize len = 0;
                     const char *p = static_cast<const char *>(g_bytes_get_data(bytes, &len));
                     static_cast<Tab *>(data)->pty->write(p, len);
                 }),
                 tab);
```

Copy, Paste and Select All (Ctrl+Shift+C, V and A) become actions that call the grid.

One more focus fix, found by the same scripted testing as Chapter 11's. When a popover menu closes, focus goes to the notebook's tab strip, so the next keys typed go nowhere. `on_popover_closed` now hands focus back to the current tab's view, on idle:

ch13-input/src/app_window.cpp

```
// "closed" fires before the chosen item's action runs. Unparenting here
// would destroy the menu under its own feet, so do it on idle.
void AppWindow::on_popover_closed(GtkPopover *popover, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    g_idle_add(
        [](gpointer p) -> gboolean {
            GtkWidget *w = GTK_WIDGET(p);
            if (gtk_widget_get_parent(w))
                gtk_widget_unparent(w);
            g_object_unref(w);
            return G_SOURCE_REMOVE;
        },
        g_object_ref(popover));
    // Without this, focus lands on the notebook's tab strip when the menu
    // closes, and the next keys typed go nowhere.
    if (Tab *tab = self->current_tab())
```

```
        focus_later(tab->view);
}
```

Done when

The wheel scrolls back through 300 lines of `seq 1 300`, a drag selects across lines, and Ctrl+Shift+C followed by Ctrl+Shift+V pastes the selection at the prompt.

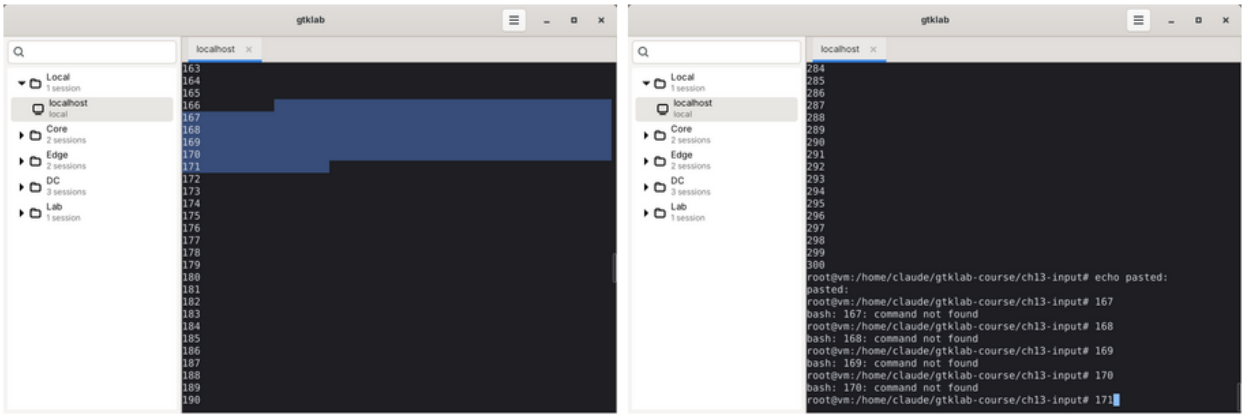


Figure 3: Left: scrolled back, with lines 166 to 171 selected. Right: after Ctrl+Shift+C, typing `echo pasted:` and Ctrl+Shift+V.

Look closely at what the paste did. Each pasted line ran as a command, because the paste turned each line break into Enter. That is how a paste into a terminal behaves unless both sides agree otherwise, and it is why gtkpyte supports **bracketed paste**: when the program in the terminal asks for it, the paste is wrapped in markers so the shell inserts it as text instead of running it.

Debugger drill

Set a breakpoint in `on_key_pressed`, focus a grid, and press Ctrl+Shift+Q. gtklab quits and the breakpoint is never hit: a capture-phase shortcut on the window (Chapter 7 installs two) handled the key before the grid saw it. The scratch config's log shows the same thing: no `grid key` line after the keypress.

Break it

Remove the call to `configure_adjustment` at the end of `gtklab_grid_feed`. Run `seq 1 300` and scroll down as far as you like. The view stays on lines 1 to 27. The scrolled window still believes there is one screen of lines, because nothing told it otherwise.

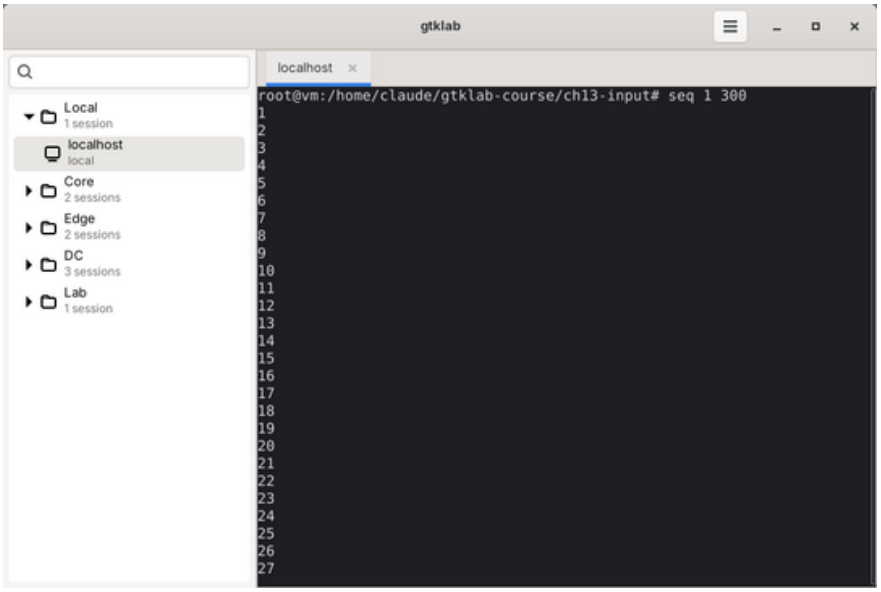


Figure 4: The Break it: 300 lines printed, and the view stuck at the top.

Common mistakes

Scrolling by pixels in a terminal. Lines are the natural unit, and they make the selection and the redraw arithmetic simple.

Always following the output. A user who scrolled back to read something is pulled to the bottom by every new line. Follow only when already at the bottom.

Writing to the pty from the widget. The widget then knows about ptys, and cannot be reused for ssh channels, recorded sessions or tests. Emit a signal.

A paste callback that assumes its widget is still alive. It may not be. Hold a reference for the round trip.

Read in tmssh

In gtkpyte: `on_click_pressed` (selection, including double- and triple-click), `on_scroll`, `on_key_pressed`, and the adjustment properties in `gtkpyte_terminal_class_init`.

Review

1. The grid holds 5,000 lines of scrollback above a screen of 27 rows, and the view is at the bottom. What are the adjustment's `lower`, `upper`, `page_size` and `value`?
2. In the Break it, the view stays on lines 1 to 27 however far you scroll. Which side of the adjustment contract was broken, and why can the scrolled window not notice by itself?
3. New output follows the bottom only if the view was already there, but a key press always returns to the bottom. Why the difference?
4. Why are lines a better unit than pixels for a terminal's adjustment, and why is a text view's adjustment in pixels?
5. You press Return in the grid. Which byte does the grid send, which byte does `cat` receive, and what changed it? Why does bash at its prompt see something different?

6. With a grid focused, Ctrl+Shift+Q quits gtklab and the grid's key handler never runs. Why, and why do terminals put their own commands on Ctrl+Shift rather than plain Ctrl?
7. A user pastes and closes the tab before the clipboard answers. What does the reference held by the paste request prevent?
8. Pasting three lines ran three commands. Which code made that happen, and what does bracketed paste change?

Questions for deeper thought. If one of your tools showed a 100,000-line device log, which unit would you choose for its adjustment, and what would the choice make easy or hard? Neither widget uses an input method: who uses your tools, and does that matter for them? The `commit` signal can have more than one listener; what could a second one do in a tool for network work, such as recording a session for replay or typing the same command into several tabs, and what would you have to guard against?

Exercises

1. **Page through history.** In `on_key_pressed` (`ch13-input/src/gtklab_grid.cpp`), make Shift+Page Up and Shift+Page Down move the vertical adjustment by `rows` lines and return `TRUE` without emitting `commit`. Done when, after `seq 1 300`, Shift+Page Up shows earlier lines and the scratch config's log shows no `grid key` line for those presses.
2. **Alt as Escape.** In `on_key_pressed`, prefix the bytes with ESC when Alt is held, as gtkpyte's `encode_key` does. Done when Alt+B at a bash prompt moves the cursor back one word.
3. **Double-click a word.** In `gtklab_grid_init`, add a `GtkGestureClick` for the primary button; on a press with `n_press == 2`, select the run of non-blank cells under the pointer. Done when a double-click highlights exactly one word and Ctrl+Shift+C copies just that word.
4. **The primary selection.** When a drag ends with a selection, put its text on `gtk_widget_get_primary_clipboard`; on a middle-button press, read the primary selection asynchronously, holding a reference as `gtklab_grid_paste_clipboard` does, and emit it through `commit`. Done when text selected in one tab is pasted into another by a middle-click.

Chapter 14 — Swapping in gtkpyte

GtklabGrid has taught its lesson. This chapter replaces it with gtkpyte, a real terminal emulator for GTK 4 built on a C++ port of the pyte emulation core. Colour, full-screen programs, mouse reporting, wide characters, bracketed paste: all of it arrives at once. The chapter also adds font zoom and per-tab session logging, and it is where the main-loop priority from Chapter 11 is measured.

Recall

What are the three phases an event passes through (Chapter 9)? Which two things stop a large `cat` from freezing the window (Chapter 11)? Which signal tells the pty that the window changed size (Chapter 12)?

In this chapter

- Replace one widget with another behind the same signals, without touching the session that feeds it.
- Explain what a terminal emulator does with the bytes that GtklabGrid skipped: parsing, modes, replies, a second screen.
- Predict where a mouse press or a wheel step goes, from the modes the program in the terminal has set.
- Vendor a third-party library into a Meson build and keep its headers from shadowing yours.
- Tap a byte stream for logging without disturbing it, and measure what a source priority costs.

Concepts

The same shape, a real engine

gtkpyte's terminal widget owns a screen and nothing else. It has no pty, no shell and no ssh: bytes go in through `gtkpyte_terminal_feed`, and bytes come out through its `commit` signal. Its signals match GtklabGrid's on purpose:

ch14-gtkpyte/gtkpyte/src/gtkpyte-terminal.h

```
* Signals
*
*   "commit"  (GtkpyteTerminal *, GBytes *data)
*       Bytes to send upstream: keystrokes, pastes, and replies the emulator
*       owes to Device Status Report and friends. Drop it and applications
*       that query the terminal will hang waiting.
*
*   "resized" (GtkpyteTerminal *, int columns, int rows)
*       The grid changed shape. Whatever is producing the bytes needs to know
*       (TIOCSWINSZ for a pty, window-change for SSH).
*
*   "selection-changed" (GtkpyteTerminal *)
*       A selection appeared or went away -- for enabling a Copy menu item.
```

So the swap is wiring: the same `PtySession` from Chapter 11 feeds the new widget, and the same two signals go back the other way. That is the payoff of Chapters 12 and 13 refusing to let the widget know about ptys.

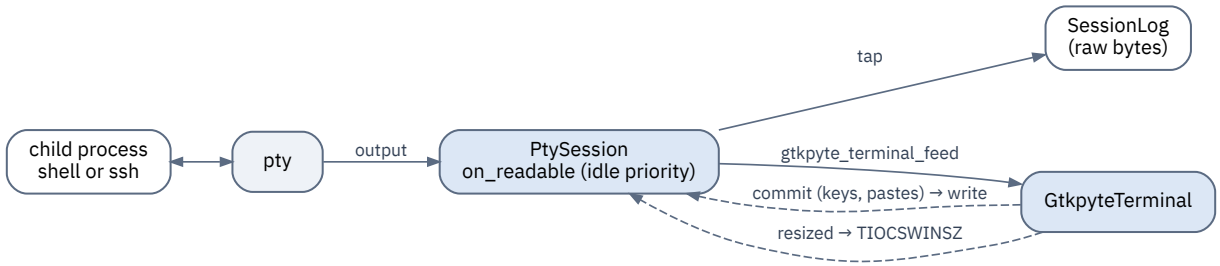


Figure 1: Data flow for one tab. Bytes from the child go through the session's log tap into the terminal; keys and pastes leave as `commit`; a new size leaves as `resized`.

What an emulator does that the grid did not

GtklabGrid recognised escape sequences only to throw them away. A program running under `TERM=xterm-256color` relies on them for everything beyond plain text: the terminfo entry for `xterm-256color` says that the sequence to reset attributes is `ESC ( B ESC [ m` and that entering a full-screen program sends `ESC [ ? 1049 h`. An emulator has to act on all of that.

It parses with a state machine. pyte's `Stream` reads one byte at a time and remembers which state it is in between calls, so a sequence split across two pty reads needs no special handling: the next `feed` simply continues where the last one stopped. GtklabGrid's approach, buffering an incomplete sequence and scanning it again, works only for the shapes it knows. Its rule that any other `ESC x` is two bytes would print the `B` of the three-byte `ESC ( B` above.

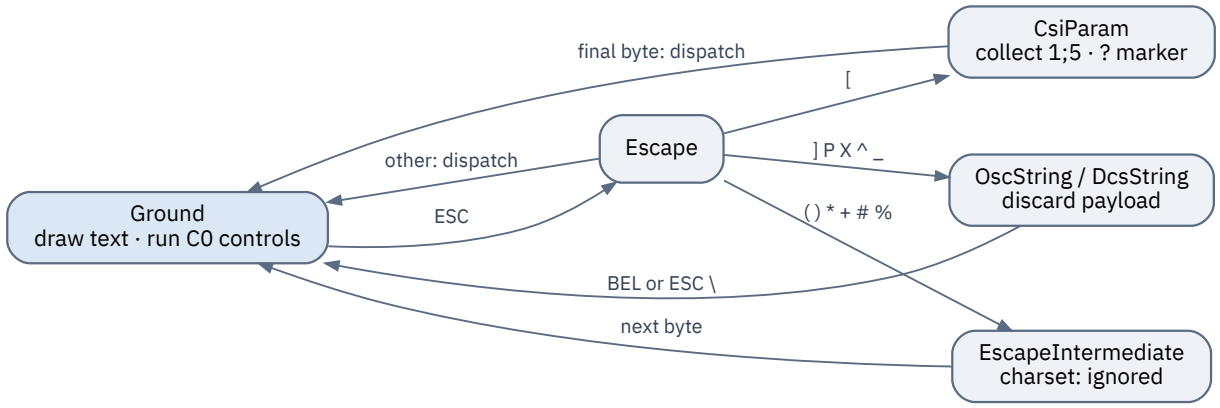


Figure 2: pyte's parser states. Every byte moves the machine; only the Ground state draws text. A CSI sequence collects numeric parameters until a final byte names the command.

It keeps a screen of cells, not characters. Each cell has a character and attributes: colours, bold, underline. The `CSI` commands move the cursor, erase, insert and delete lines and characters, set a scrolling region, and change attributes (`CSI ... m`). The widget paints from those cells.

It answers. Some sequences are questions. A program can ask where the cursor is (`CSI 6 n`) or what kind of terminal this is (`CSI c`), and wait for the reply. pyte produces the reply, and gtkpyte sends it out

through the same `commit` signal as keystrokes. Its header puts the consequence plainly: drop `commit`, and applications that query the terminal will hang waiting.

It keeps state the program sets. Some sequences change how the terminal behaves rather than what it shows. They are the private modes, set with `CSI ? n h` and reset with `CSI ? n l`, and pyte reports each one to the widget:

Mode	Name	What it changes	Kept in
1	cursor keys (DECCKM)	arrows send `ESC O A` instead of `ESC [ A`	`KeyModes::application_cursor`
25	cursor visible	whether the cursor is drawn	`State::cursor_visible`
47, 1047, 1049	alternate screen	which buffer is shown	pyte's `Screen`
1000, 1002, 1003	mouse tracking	which mouse events are reported	`MouseModes::tracking`
1006	SGR mouse encoding	report format, no coordinate limit	`MouseModes::sgr`
1007	alternate scroll	the wheel sends arrows on the alternate screen	`MouseModes::alternate_scroll`
2004	bracketed paste	pastes are wrapped in `ESC [200~` and `ESC [201~`	`State::bracketed_paste`

This is why Chapter 13's key table had a column that depended on state: the terminfo entry gives the Up key as `ESC O A`, the cursor-key-mode form, and a program that turns that mode on expects it.

Two screens

A shell writes line after line, and what scrolls off the top goes into scrollback. A full-screen program such as `vim` or `htop` instead redraws a fixed screen in place, and when it exits, the shell's screen should come back as it was. Mode 1049 provides that: on entry, pyte saves the cursor, switches to a second buffer and clears it; on exit, it switches back and restores the cursor. The alternate buffer keeps no scrollback, because a redrawn screen has no history worth keeping.

The second screen changes what the wheel means. gtkpyte decides on every scroll event:

Situation	The wheel
The program asked for mouse reports, no Shift	is sent to it as button presses (64 up, 65 down) at the pointer's cell
Alternate screen, no mouse reports, mode 1007 on (the default)	sends Up or Down arrow keys, so `less` and `man` scroll
Normal screen	scrolls the scrollback, three lines per notch

Who gets the mouse

Mouse reporting raises Chapter 9's question again: two controllers on one widget want the same press. Selection wants it to start a drag; a program that asked for the mouse wants it as a report. gtkpyte settles

it with propagation. A drag gesture for every button runs in the capture phase, ahead of the selection gestures. When the program has asked for the mouse and Shift is not held, it claims the sequence: the press, and the motion and release when the mode reports them, go to the program as report bytes through `commit`. Otherwise it denies the sequence, and the press continues to selection untouched.

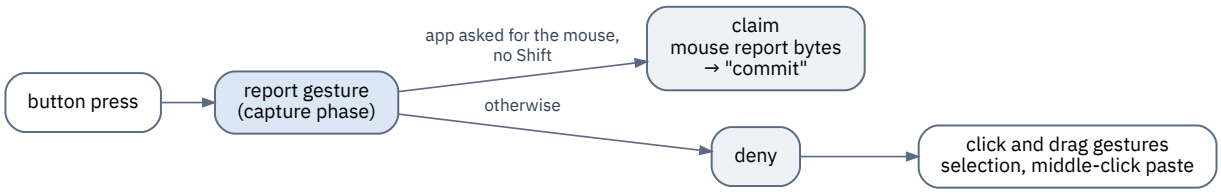


Figure 3: One press, two possible owners. The capture-phase report gesture claims the press when the program has asked for the mouse; otherwise it denies it and selection gets it.

Shift as the override is xterm's convention: it lets the user select text even inside a program that has taken the mouse. The widget also changes its pointer from the text I-beam to an arrow while reporting is on, so the user can see who owns the mouse.

Vendoring a widget

gtkpyte arrives as source copied into the tree, not as a system package. That is **vendoring**, and it trades one set of problems for another. You get a fixed version that builds the same everywhere, with no package to install; you take on keeping it up to date. The `VENDORED.md` that comes with the copy records the upstream commit the copy came from, lists which files came from where, and sets the rule that nothing in the copy is edited in place: fixes go upstream first and are copied down. Two more costs come with it. The licence travels with the code, so gtklab becomes GPL. And the library's include directory becomes yours: a header it ships can shadow one of the system's, as the Build section finds out with `pty.h`.

Build

Vendoring gtkpyte

Copy tmssh's `gtkpyte/` directory (its sources, its `pyte/` core, its `meson.build` and `VENDORED.md`) into gtklab, and copy tmssh's `subprojects/utf8proc.wrap` and `subprojects/packagefiles/utf8proc/`. The first time the build is configured, Meson fetches utf8proc 2.11.3 from GitHub; after that it builds offline.

gtkpyte is licensed GPL-3.0-or-later, so from this chapter on, gtklab is too.

meson.build

gtkpyte's `meson.build` expects a variable called `gtk_dep` and provides `gtkpyte_dep`:

ch14-gtkpyte/meson.build

```
# The terminal widget, vendored from gtkpyte (GPL-3.0; see gtkpyte/VENDORED.md).
# Its meson.build expects gtk_dep and provides gtkpyte_dep.
gtk_dep = gtk
subdir('gtkpyte')
```

Then a problem that only shows up in a real build. gtkpyte's source directory, which is on the include path of anything that uses `gtkpyte_dep`, contains its own `pty.h`. With it on the path, `#include <pty.h>` in `pty_session.cpp` finds gtkpyte's file instead of the system's, and the build fails with `'forkpty' was not declared in this scope`. The fix is to build the pty code as its own small library, which does not use `gtkpyte_dep`:

ch14-gtkpyte/meson.build

```
# Our pty code is built on its own: gtkpyte's include directory has its own
# pty.h, which would shadow the system <pty.h> (and its forkpty) here.
pty_lib = static_library('gtklab_pty', 'src/pty_session.cpp', dependencies: [glib, util])

app_sources = [
  'src/main.cpp',
  'src/app_window.cpp',
  'src/session_item.cpp',
]
executable('gtklab', app_sources, link_with: pty_lib,
  dependencies: [gtk, core_dep, util, gtkpyte_dep])
```

The wiring

ch14-gtkpyte/src/app_window.cpp

```
void AppWindow::start_tab(Tab *tab) {
    tab->pty = std::make_unique<PtySession>();
    GtkpyteTerminal *term = GTKPYTE_TERMINAL(tab->view);
    // The log tap sees raw bytes before the terminal does.
    tab->pty->set_output_handler([tab, term](const char *data, gsize len) {
        if (tab->log)
            tab->log->write(data, len);
        gtkpyte_terminal_feed(term, data, len);
    });

    // Keys and pastes from the grid go to the child.
    g_signal_connect(tab->view, "commit",
                     G_CALLBACK(+[](GtkpyteTerminal *, GBytes *bytes, gpointer data) {
                         gsize len = 0;
                         const char *p = static_cast<const char *>(g_bytes_get_data(bytes, &len));
                         static_cast<Tab *>(data)->pty->write(p, len);
                     }),
                     tab);

    // The grid changed shape: tell the child (TIOCSWINSZ -> SIGWINCH).
    g_signal_connect(tab->view, "resized",
                     G_CALLBACK(+[](GtkpyteTerminal *, int cols, int rows, gpointer data) {
                         static_cast<Tab *>(data)->pty->resize(cols, rows);
                     }),
                     tab);

    // When the child exits, close the tab, but not from inside this
    // callback: removing the page deletes the Tab, which deletes the
    // PtySession whose callback is running. Defer to an idle, holding a
    // reference to the page so it cannot vanish in between.
    GtkWidget *page = tab->page;
    tab->pty->set_exit_handler([page](int status) {
```

```
        g_debug("child exited, status %d; closing tab on idle", status);
        g_idle_add(
            [](gpointer p) -> gboolean {
                GtkWidget *pg = GTK_WIDGET(p);
                if (GtkWidget *nb = gtk_widget_get_ancestor(pg, GTK_TYPE_NOTEBOOK)) {
                    const int n = gtk_notebook_page_num(GTK_NOTEBOOK(nb), pg);
                    if (n >= 0)
                        gtk_notebook_remove_page(GTK_NOTEBOOK(nb), n);
                }
                g_object_unref(pg);
                return G_SOURCE_REMOVE;
            },
            g_object_ref(page));
    });

    const char *shell = g_getenv("SHELL");
    // A real emulator now: tell the child so.
    if (!tab->pty->start(ssh_argv(tab->session, shell ? shell : ""), gtkpyte_terminal_get_columns(term),
                         gtkpyte_terminal_get_rows(term), "xterm-256color")) {
        const std::string msg = "Could not start: " + tab->pty->error() + "\r\n";
        gtkpyte_terminal_feed(term, msg.c_str(), msg.size());
    }
}
```

The child now runs with `TERM=xterm-256color`, because the terminal really is one. The log tap sits in the output handler, so the log sees the raw bytes before the terminal does.

Zoom

The zoom from Chapter 9 becomes one font size for every tab, from Ctrl+=, Ctrl+- and Ctrl+0, and from Ctrl+wheel, saved through the debounced save:

ch14-gtkpyte/src/app_window.cpp

```
// One font size for every tab, saved with the debounced save.
void AppWindow::set_font_size(int size) {
    size = CLAMP(size, 6, 48);
    if (size == settings_.font_size)
        return;
    settings_.font_size = size;
    g_debug("font size %d", size);
    for (int i = 0; i < gtk_notebook_get_n_pages(GTK_NOTEBOOK(notebook_)); i++) {
        GtkWidget *page = gtk_notebook_get_nth_page(GTK_NOTEBOOK(notebook_), i);
        if (auto *tab = static_cast<Tab *>(g_object_get_data(G_OBJECT(page), "tab")))
            apply_font(tab);
    }
    save_settings_soon();
}
```

ch14-gtkpyte/src/app_window.cpp

```
void AppWindow::apply_font(Tab *tab) {
    const std::string font = "Monospace " + std::to_string(settings_.font_size);
    gtkpyte_terminal_set_font(GTKPYTE_TERMINAL(tab->view), font.c_str());
}
```

Session logs

A `SessionLog` opens a file under `$XDG_DATA_HOME/gtklab/logs/`, appends raw bytes, and writes a footer when it is destroyed:

ch14-gtkpyte/src/session_log.h

```
// SessionLog: raw pty output appended to a file, one per tab while logging
// is on. The file closes (with a footer) when the SessionLog is destroyed.
#pragma once

#include <glib.h>
#include <glib/gstdio.h>

#include <cstdio>
#include <string>

class SessionLog {
public:
    // $XDG_DATA_HOME/gtklab/logs/<session>-<YYYYmmdd-HHMMSS>.log
    explicit SessionLog(const std::string &session_name) {
        char *dir = g_build_filename(g_get_user_data_dir(), "gtklab", "logs", nullptr);
        g_mkdir_with_parents(dir, 0700);
        GDateTime *now = g_date_time_new_now_local();
        char *stamp = g_date_time_format(now, "%Y%m%d-%H%M%S");
        const std::string name = session_name + "-" + stamp + ".log";
        char *path = g_build_filename(dir, name.c_str(), nullptr);
        path_ = path;
        file_ = g_fopen(path, "ab");
        if (file_) {
            char *header = g_date_time_format(now, "--- log started %F %T ---\n");
            std::fputs(header, file_);
            g_free(header);
        }
        g_free(path);
        g_free(stamp);
        g_date_time_unref(now);
        g_free(dir);
    }

    ~SessionLog() {
        if (!file_)
            return;
        GDateTime *now = g_date_time_new_now_local();
        char *footer = g_date_time_format(now, "\n--- log closed %F %T ---\n");
        std::fputs(footer, file_);
        g_free(footer);
        g_date_time_unref(now);
        std::fclose(file_);
        g_debug("~SessionLog %s", path_.c_str());
    }

    SessionLog(const SessionLog &) = delete;
    SessionLog &operator=(const SessionLog &) = delete;

    bool ok() const { return file_ != nullptr; }
    const std::string &path() const { return path_; }

    // Raw bytes, escape sequences and all. tmssh cleans these into plain
    // text (its LogCleaner); gtklab keeps them as received.
    void write(const char *data, gsize len) {
        if (file_)
            std::fwrite(data, 1, len, file_);
```

```
    }

private:
    std::FILE *file_ = nullptr;
    std::string path_;
};
```

The tab owns its log as a `std::unique_ptr<SessionLog>`, declared after the session, so that the log closes first when a tab is destroyed. Logging is a stateful action, `win.log` (Ctrl+Shift+L), whose check mark must follow the current tab. The action creates or drops the current tab's log and shows a dot in its tab label:

ch14-gtkpyte/src/app_window.cpp

```
void AppWindow::act_log(GSimpleAction *action, GVariant *value, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    Tab *tab = self->current_tab();
    if (!tab)
        return;
    if (g_variant_get_boolean(value)) {
        tab->log = std::make_unique<SessionLog>(tab->session.name);
        if (!tab->log->ok()) {
            self->message("Logging failed", "Could not open " + tab->log->path());
            tab->log.reset();
        } else {
            g_debug("logging %s to %s", tab->session.name.c_str(), tab->log->path().c_str());
        }
    } else {
        tab->log.reset();  // writes the footer and closes the file
    }
    gtk_widget_set_visible(tab->log_dot, tab->log != nullptr);
    g_simple_action_set_state(action, g_variant_new_boolean(tab->log != nullptr));
}
```

and switching tabs moves the check mark:

ch14-gtkpyte/src/app_window.cpp

```
// The check mark follows the current tab.
void AppWindow::sync_log_action() {
    Tab *tab = current_tab();
    GAction *a = g_action_map_lookup_action(G_ACTION_MAP(window_), "log");
    g_simple_action_set_enabled(G_SIMPLE_ACTION(a), tab != nullptr);
    g_simple_action_set_state(G_SIMPLE_ACTION(a), g_variant_new_boolean(tab && tab->log));
}
```

Copy, Paste and Select All now call `gtkpyte_terminal_copy_clipboard`, `gtkpyte_terminal_paste_clipboard` and `gtkpyte_terminal_select_all`.

Done when

Colour output, `htop` and `vim` work in a local tab; zoom changes the font in every tab and survives a restart; Ctrl+Shift+L shows the log dot and writes a log file.

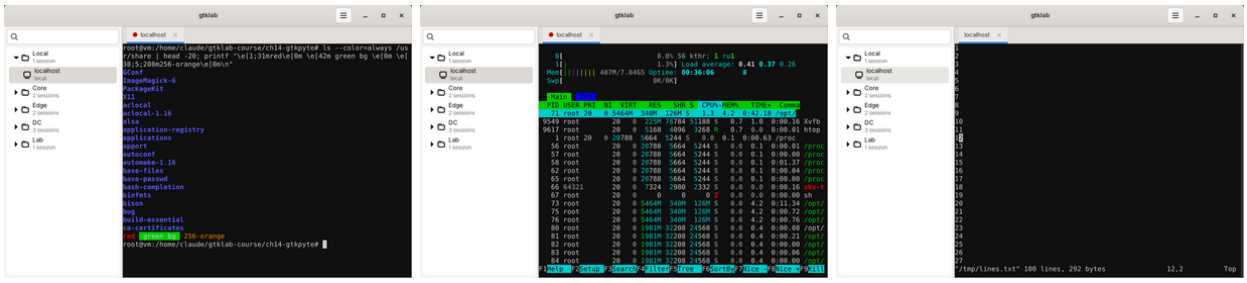


Figure 4: Colour with the log dot in the tab label; `htop`; `vim` after a mouse click on line 12.

Debugger drill

Run `vi -u NONE -c "set mouse=a ruler"` on a file of a hundred lines and click a line. The ruler jumps to it; in the recorded run, `12,2`. Now set a breakpoint in gtkpyte's `on_report_begin` and click again: the gesture claims the press and sends it to vim as a mouse report. Quit vim and click at the shell prompt: the same gesture denies the press, and selection gets it instead.

Break it

In `pty_session.cpp`, go back to `g_unix_fd_add`, which watches at the default priority, and `cat` a large file. This was measured with a 67 MB file, taking a screenshot every second and comparing them:

Read watch priority	Screens that changed, 6 one-second samples	Time for the whole `cat`
`G_PRIORITY_DEFAULT` (`g_unix_fd_add`)	1 change in 6 seconds	17.9 s
`G_PRIORITY_DEFAULT_IDLE`	changed every second	16.5 s

At the default priority the terminal is frozen on one screen for seconds at a time while the bytes pour through. At idle priority it repaints throughout, and the whole run takes about the same time. The priority costs nothing; it only decides whether the redraw gets a turn. The measurements were made under Xvfb with software rendering, where a frame is expensive; the mechanism is the same on a real GPU.

Common mistakes

Linking both `pty.hs`. Any library with a `pty.h` on its public include path shadows the system header for everything that uses it. Keep your pty code out of that target.

Feeding the terminal outside the read budget. Feed it inside `on_readable`, chunk by chunk; the budget is what keeps the loop moving.

Logging after the terminal. Tap the bytes before `gtkpyte_terminal_feed`, so the log has what the child sent, not what the screen became.

Read in tmssh

`start_tab`, `act_log`, `sync_log_action`, and `LogCleaner` in `src/session_log.h`: tmssh strips escape sequences out of its logs so they read as plain text, where gtklab keeps raw bytes. Then

`gtkpyte::PtySession`, which is Chapter 11's `PtySession` with one important addition: a write queue, so that a long paste into a slow child is never cut short.

Review

1. Why does swapping GtklabGrid for gtkpyte not change `PtySession` at all?
2. A pty read ends in the middle of `ESC [ 3 8 ; 5`. What does pyte do with the rest when the next read arrives, and why is that simpler than GtklabGrid's approach?
3. A program sends `CSI 6 n` and waits. What does it get back, by which path, and what happens if nothing is connected to `commit`?
4. While `vim` runs, there is no scrollback to wheel through. Why? What does the wheel do in `vim` with `set mouse=a`, and what without it?
5. Inside `vim` with the mouse on, how does a user still select text, and which code makes that possible?
6. Why does the pty code build as its own library?
7. Moving the read watch from `G_PRIORITY_DEFAULT` to `G_PRIORITY_DEFAULT_IDLE`: what did it cost, and what did it buy?
8. The log tap runs before `gtkpyte_terminal_feed`. What does the log file therefore contain, and why is that different from what the screen shows?

Questions for deeper thought. For a tool you hand to colleagues, would you vendor a widget like gtkpyte or depend on a system package, and what would you write down either way? gtklab's `PtySession::write` drops what the kernel does not accept, where gtkpyte's session queues it: when would that difference bite a network engineer, for example pasting a long configuration into a slow console? For an audit of changes made on network devices, do you want raw logs, cleaned logs, or both?

Exercises

1. **Copy only when there is something to copy.** In `ch14-gtkpyte/src/app_window.cpp`, connect each terminal's `selection-changed` signal in `AppWindow::start_tab` to a function that enables `win.copy` only when the current tab's terminal has a selection, and call the same function where `on_switch_page` syncs the log action. Done when the Copy menu item is greyed out until text is selected, and follows the current tab.
2. **A colour scheme setting.** Add a `color_scheme` field to `Settings` (`ch14-gtkpyte/src/core/settings.h` and `settings.cpp`, with a doctest case), apply it next to the font in `AppWindow::apply_font` with `gtkpyte_terminal_set_color_scheme`, and offer the names from `gtkpyte_terminal_list_color_schemes` as a stateful action in the menu. Done when a chosen scheme applies to every tab and survives a restart.
3. **A write queue.** In `PtySession::write` (`ch14-gtkpyte/src/pty_session.cpp`), keep the bytes the kernel does not accept and write them from a `G_IO_OUT` watch, removed in the destructor like the others; gtkpyte's `PtySession::send` and `drain` show one way. Done when pasting a 1 MB text file with Unix line endings into `wc -c` (then Ctrl+D) reports the same count as `wc -c` on the file.
4. **Readable logs.** Give `SessionLog` an option to strip escape sequences before writing, with a small state machine that keeps its state between calls, as in the parser figure. Done when the log of a session that ran `ls --color` reads as plain text in `less`, and a sequence split across two writes leaves no stray bytes. tmssh's `LogCleaner` is a reference.

Checkpoint D — A usable terminal

Spec. Real local and ssh tabs; zoom persisted; per-tab logging with Ctrl+Shift+L and a dot in the tab label; a tab that closes itself when its session exits.

Done when. `cat` of a 50 MB file leaves the window repainting; `htop` and `vim` render and take the mouse; and every gtklab shortcut works with a terminal focused. All three were checked against the Chapter 14 code, which is the reference for this checkpoint; the 67 MB `cat` was also logged, and the log file held all of it.

Reference. `ch14-gtkpyte/`.

Part IV: Production

Part IV turns a working program into one you can hand to someone: themes, styles and an icon compiled into the binary; the lifetime bugs proven gone with the right tool for each; and a build checked against the oldest GTK the book supports, with install rules for the desktop. It ends with a capstone: a second tool, from a blank repository, using only what the book has taught.

Chapter 15 — Resources, CSS, themes and icons

Until now gtklab's colours have been written in code and its only icons have come from the system theme. This chapter adds four terminal colour themes, a stylesheet and an icon of its own, all compiled into the binary so that the program needs nothing installed beside it, and a menu that picks a terminal theme independently of the light or dark application theme.

Recall

What does `gtk_widget_queue_draw` do, and what does it not do (Chapter 12)? What goes in the core library, and what does not (Chapter 4)? Which gtkpyte calls set colours (its header, Chapter 14)?

In this chapter

- Compile a stylesheet, an icon and data files into the binary as a `GResource` bundle, and read each one back by path.
- Predict which CSS rule wins, from provider priority first and selector specificity second.
- Show state with style classes and the theme's named colours, and know when a colour belongs in data instead.
- Ship a symbolic icon that GTK finds by name and recolours from CSS.
- Parse colour themes in the core, with tests, and apply them to every open terminal from a stateful action.

Concepts

A bundle is a file system inside the binary

A **`GResource` bundle** is a set of files compiled into the program and read back by path, such as `/io/github/gtklab/style.css`. Nothing is installed beside the binary, so nothing can go missing, be the wrong version, or be found in the wrong prefix.

It happens in two halves. At build time, an XML manifest lists the files; Meson's `gnome.compile_resources` runs `glib-compile-resources`, which packs them into one blob and writes it out as C: a byte array in a section of its own, and a constructor function. At run time that constructor runs before `main`, and registers the blob with GLib. From then on any code in the process can open `/io/github/gtklab/...` as if it were a read-only file system.

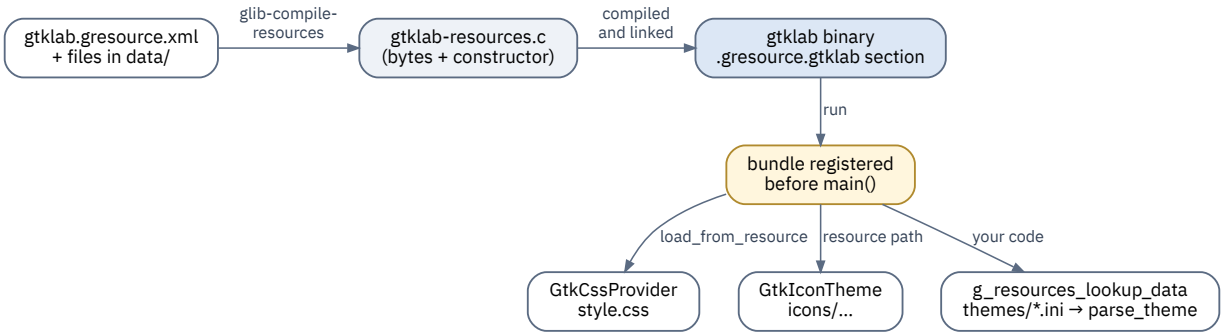


Figure 1: A bundle at build time and at run time. Three readers take files out of it.

Two consequences are worth knowing. The build tracks the files in the manifest: edit `style.css` and Meson regenerates the C, recompiles it and relinks gtklab, and nothing else. And the bundle is inspectable in the built program: `gresource list build-debug/gtklab` prints every path in it, which is the quickest way to answer "did my file make it in?".

The manifest's `prefix` is the application id written as a path. That is not only tidy. `GtkApplication` derives a **resource base path** from its id, `/io/github/gtklab` here, and looks under it for things it uses automatically: it adds `icons/` beneath it to the icon theme, and loads `gtk/menus.ui` if there is one. gtklab's `install_style` adds the icon path again, explicitly; the call is redundant but harmless, and it keeps the dependency visible in gtklab's own code.

Three readers

GTK reads two kinds of file straight out of a bundle: a `GtkCssProvider` loads a stylesheet from a resource path, and the icon theme searches a resource path for icons. (`GtkBuilder` can also load UI definitions from one; gtklab builds its UI in code.) Anything else, such as a colour theme, is just bytes: `g_resources_lookup_data` returns a `GBytes` and your code parses it. gtklab's theme files take that third route, and the parser is in the core (Chapter 4), so it is tested without a display.

The CSS cascade: priority first, then specificity

GTK styles widgets with CSS, matched against each widget's **CSS node**: its node name (`label`, `image`, `button`), its style classes, its name, and its state. Rules come from several **style providers**, each added with a priority:

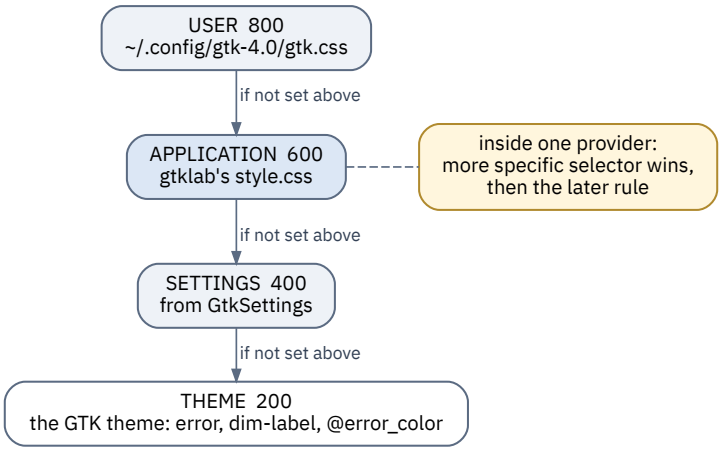


Figure 2: Style providers, highest priority first. A property comes from the highest provider that sets it.

The order of the two rules matters, and it is not the one web CSS teaches. Priority is decided first: for each property, the highest-priority provider that sets it wins, however specific a lower provider's selector is. Specificity only breaks ties inside one provider: an id beats a class, a class beats a node name, two classes beat one, and between equals the later rule wins. Measured on GTK 4.8 and 4.14 with two providers of our own: a `label.error` rule at theme priority lost to a bare `label` rule at application priority, on a label with the `error` class.

That is why `style.css` uses narrow selectors (`image.log-indicator`, `label.error.banner`). gtklab's stylesheet sits at `GTK_STYLE_PROVIDER_PRIORITY_APPLICATION`, above the theme; a broad rule there such as `label { color: ... }` would override the theme on every label in the program, error labels and dim labels included. And it sits below `USER`, so the user's own `gtk.css` still has the last word.

Classes, named colours, and colours in data

There are four places a colour can come from, and they do not behave the same when the user switches to dark or high contrast:

Where the colour comes from	Follows light and dark	Use it for
A theme class: `error`, `dim-label`, `caption`	yes	common states the theme already styles
Your class, a named colour: `@error_color`	yes	your own element in a theme colour
Your class, a literal: `#c01c28`	no	almost nothing
A widget API, from data: `apply_theme`	by choice	content colours the user picks

The first two are the idiom: code decides *what state a widget is in*, by adding or removing a class; CSS decides *what that looks like*. The sessions-file banner from Checkpoint B does exactly this, with `error` from the theme and `banner` from `style.css`, whose background is `alpha(@error_color, 0.12)`: a tint of whatever red the current theme uses.

The terminal palette is the exception, and deliberately so. Sixteen ANSI colours, a background and a cursor are content the user chooses, not chrome, and a terminal theme is independent of the application's light or dark variant. They are data: parsed in the core, handed to gtkpyte through its own API, which queues a redraw (Chapter 12).

Symbolic icons

A **symbolic** icon is a single-colour SVG whose name ends in `-symbolic`. GTK does not draw its colours as written. It renders the SVG with a stylesheet of its own that forces every `rect`, `circle` and `path` to the widget's CSS `color`; shapes with the class `warning`, `error` or `success` get the theme's colours for those. gtklab's log dot is a circle filled `#222222` in the file, and it is drawn in `@error_color` because `style.css` sets `color` on `image.log-indicator`. Change the rule and the icon follows; switch the application to dark and it follows the dark red.

How GTK finds an icon by name

`gtk_image_new_from_icon_name("gtklab-log-symbolic")` names an icon; it does not name a file. The icon theme searches for it:

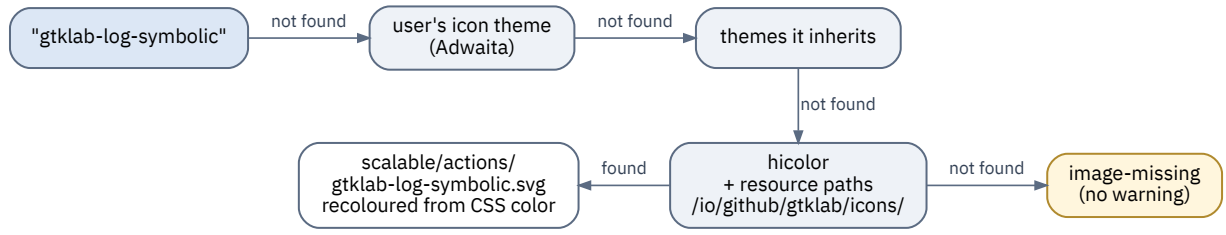


Figure 3: Icon lookup. Resource paths are searched as part of hicolor, the last theme in the chain.

The user's theme (Adwaita, say) comes first, then the themes it inherits, then **hicolor**, the fallback every theme ends in. Resource paths are treated as part of hicolor, and must use its layout: `<path>/scalable/actions/name.svg` or `<path>/16x16/actions/name.png`. Two consequences follow. A system icon with the same name wins over yours, which is why gtklab prefixes its icon names with `gtklab-`. And a name that is found nowhere is not an error: measured, the lookup returns GTK's `image-missing` icon with no warning at all. `gtk_icon_theme_has_icon` answers the question directly when you need to know.

Where this sits in the book

The theme parser is Chapter 4's boundary again: GTK only reads bytes from the bundle and applies colours; everything that can be wrong with a theme file is decided, and tested, in the core. The theme menu is Chapter 7's stateful radio action with a string state. And the application's light or dark variant is still Chapter 7's `gtk-application-prefer-dark-theme`; the terminal theme is a second, independent setting.

Build

The bundle

ch15-resources/data/gtklab.gresource.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Files compiled into the gtklab binary. Add a line when a file is added. -->
<gresources>
  <gresource prefix="/io/github/gtklab">
    <file>style.css</file>
    <file>themes/dark.ini</file>
    <file>themes/light.ini</file>
    <file>themes/nord.ini</file>
    <file>themes/solarized-dark.ini</file>
    <file>icons/scalable/actions/gtklab-log-symbolic.svg</file>
  </gresource>
</gresources>
```

`gnome.compile_resources` generates C, so the project must now list C as a language. Without it, configuring fails with `No host machine compiler for 'gtklab-resources.c'`:

ch15-resources/meson.build

```
project('gtklab', ['c', 'cpp'],  # c: compile_resources generates C
  version: '0.1.0',
  default_options: ['cpp_std=c++17', 'warning_level=2'])
```

ch15-resources/meson.build

```
# Files compiled into the binary: themes, CSS, icons.
gnome = import('gnome')
resources = gnome.compile_resources('gtklab-resources', 'data/gtklab.gresource.xml',
  source_dir: 'data')
```

The generated source goes into the executable's sources with `src/theme_apply.cpp`; `src/core/theme.cpp` joins the core, and `tests/test_theme.cpp` the tests.

Themes: parsed in the core

A theme file is a key file with a label, a type, and twenty colours:

ch15-resources/data/themes/nord.ini

```
[theme]
label=Nord
type=dark

[colors]
background=#2e3440
foreground=#d8dee9
cursor=#d8dee9
selection=#434c5e
black=#3b4252
red=#bf616a
green=#a3be8c
yellow=#ebcb8b
blue=#81a1c1
magenta=#b48ead
cyan=#88c0d0
white=#e5e9f0
```

```
bright_black=#4c566a
bright_red=#bf616a
bright_green=#a3be8c
bright_yellow=#ebcb8b
bright_blue=#81a1c1
bright_magenta=#b48ead
bright_cyan=#8fbcbb
bright_white=#eceff4
```

Parsing it needs no GTK, so it lives in the core with tests, and rejects a bad colour by naming the key:

ch15-resources/src/core/theme.h

```
// Terminal colour themes: parsing only, no GTK.
#pragma once

#include <array>
#include <string>

struct Theme {
    std::string name;   // stable id, from the file name
    std::string label;  // shown in menus
    bool dark = true;
    std::string background, foreground, cursor, selection;  // "#rrggbb"
    std::array<std::string, 16> palette;  // ANSI 0-15
};

// Parse a theme file (GKeyFile format):
//
//   [theme]
//   label=Nord
//   type=dark
//   [colors]
//   background=#2e3440
//   ...            (foreground, cursor, selection, and the 16 below)
//   black=...  red=...  green=...  yellow=...  blue=...  magenta=...  cyan=...  white=...
//   bright_black=...  ...  bright_white=...
//
// Every colour must be #rrggbb. Returns false and sets `error` otherwise.
bool parse_theme(const std::string &name, const std::string &text, Theme &out, std::string &error);

// The 16 key names in palette order.
const std::array<const char *, 16> &theme_palette_keys();
```

ch15-resources/tests/test_theme.cpp

```
TEST_CASE("theme: bad colours are rejected with the key named") {
    Theme t;
    std::string err;
    CHECK_FALSE(parse_theme("test", full_theme("blue"), t, err));
    CHECK(err.find("background") != std::string::npos);
    CHECK_FALSE(parse_theme("test", "[theme]\nlabel=x\n", t, err));
}
```

Themes: read from the bundle, applied to terminals

Only reading the bundle and applying colours need GTK. `load_themes` lists the bundle's theme directory, parses each file and reports any that fail:

ch15-resources/src/theme_apply.cpp

```
std::vector<Theme> load_themes(std::string &error) {
    std::vector<Theme> themes;
    GError *err = nullptr;
    char **names = g_resources_enumerate_children(kThemeDir, G_RESOURCE_LOOKUP_FLAGS_NONE, &err);
    if (!names) {
        error = std::string("themes: ") + err->message;
        g_error_free(err);
        return themes;
    }
    for (char **n = names; *n; n++) {
        if (!g_str_has_suffix(*n, ".ini"))
            continue;
        const std::string file = *n;
        const std::string path = std::string(kThemeDir) + "/" + file;
        GBytes *bytes = g_resources_lookup_data(path.c_str(), G_RESOURCE_LOOKUP_FLAGS_NONE, nullptr);
        if (!bytes)
            continue;
        gsize len = 0;
        const char *data = static_cast<const char *>(g_bytes_get_data(bytes, &len));
        Theme t;
        std::string perr;
        if (parse_theme(file.substr(0, file.size() - 4), std::string(data, len), t, perr))
            themes.push_back(t);
        else
            error += file + ": " + perr + "\n";
        g_bytes_unref(bytes);
    }
    g_strfreev(names);
    // Case-insensitive, so labels sort the way a reader expects.
    std::sort(themes.begin(), themes.end(), [](const Theme &a, const Theme &b) {
        char *fa = g_utf8_casefold(a.label.c_str(), -1);
        char *fb = g_utf8_casefold(b.label.c_str(), -1);
        const bool less = g_strcmp0(fa, fb) < 0;
        g_free(fa);
        g_free(fb);
        return less;
    });
    return themes;
}
```

`apply_theme` hands the colours to gtkpyte, the selection with some transparency because it is painted over the text:

ch15-resources/src/theme_apply.cpp

```
void apply_theme(GtkpyteTerminal *terminal, const Theme &theme) {
    GdkRGBA fg = rgba(theme.foreground);
    GdkRGBA bg = rgba(theme.background);
    GdkRGBA palette[16];
    for (size_t i = 0; i < 16; i++)
        palette[i] = rgba(theme.palette[i]);
    gtkpyte_terminal_set_colors(terminal, &fg, &bg, palette, 16);
    GdkRGBA cursor = rgba(theme.cursor);
    gtkpyte_terminal_set_color_cursor(terminal, &cursor);
    GdkRGBA sel = rgba(theme.selection);
    sel.alpha = 0.6f;  // painted over the text
    gtkpyte_terminal_set_color_selection(terminal, &sel);
}
```

Style and icons

The stylesheet uses the theme's named colours, so it works in light and dark:

ch15-resources/data/style.css

```
/* gtklab's own styles. Classes from the theme (error, dim-label) need nothing here. */

/* The log indicator: a symbolic icon takes its colour from CSS. */
image.log-indicator {
  color: @error_color;
  -gtk-icon-size: 10px;
}

/* The sessions-file error banner. */
label.error.banner {
  padding: 4px 6px;
  border-radius: 6px;
  background-color: alpha(@error_color, 0.12);
}
```

`install_style` loads it for the whole display, and adds the bundle's icon directory to the icon theme. GTK then finds `icons/scalable/actions/gtklab-log-symbolic.svg` by the name `gtklab-log-symbolic`, exactly like a system icon:

ch15-resources/src/app_window.cpp

```
void AppWindow::install_style() {
    GdkDisplay *display = gdk_display_get_default();

    GtkCssProvider *css = gtk_css_provider_new();
    gtk_css_provider_load_from_resource(css, "/io/github/gtklab/style.css");
    gtk_style_context_add_provider_for_display(display, GTK_STYLE_PROVIDER(css),
                                               GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
    g_object_unref(css);

    // Icons under <path>/scalable/actions/ become part of the icon theme,
    // so gtk_image_new_from_icon_name finds them with no icons installed.
    gtk_icon_theme_add_resource_path(gtk_icon_theme_get_for_display(display), "/io/github/gtklab/icons");
}
```

The log dot from Chapter 14 becomes that icon, with the `log-indicator` class.

The theme menu

The Terminal Theme submenu is built from the loaded themes, one radio item per theme on a `win.terminal-theme` action with a string state, the same pattern as Chapter 7's light and dark choice:

ch15-resources/src/app_window.cpp

```
// The theme list is data: one radio item per theme found in the resources.
GMenu *theme_menu = g_menu_new();
for (const Theme &t : themes_) {
    GMenuItem *item = g_menu_item_new(t.label.c_str(), nullptr);
    g_menu_item_set_action_and_target_value(item, "win.terminal-theme",
                                            g_variant_new_string(t.name.c_str()));
    g_menu_append_item(theme_menu, item);
    g_object_unref(item);
}
g_menu_append_submenu(appearance_section, "Terminal Theme", G_MENU_MODEL(theme_menu));
g_object_unref(theme_menu);
```

ch15-resources/src/app_window.cpp

```
void AppWindow::act_terminal_theme(GSimpleAction *action, GVariant *value, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    g_simple_action_set_state(action, value);
    self->settings_.terminal_theme = g_variant_get_string(value, nullptr);
    for (int i = 0; i < gtk_notebook_get_n_pages(GTK_NOTEBOOK(self->notebook_)); i++) {
        GtkWidget *page = gtk_notebook_get_nth_page(GTK_NOTEBOOK(self->notebook_), i);
        if (auto *tab = static_cast<Tab *>(g_object_get_data(G_OBJECT(page), "tab")))
            self->apply_terminal_theme(tab);
    }
    self->save_settings_soon();
}
```

The application's light and dark variant (Chapter 7) and the terminal theme are independent settings; a dark application with a light terminal is a legitimate choice.

Done when

Choosing a terminal theme recolours every open tab, the application's light or dark choice is independent of it, and the log icon renders. The core tests pass, now with the theme tests.

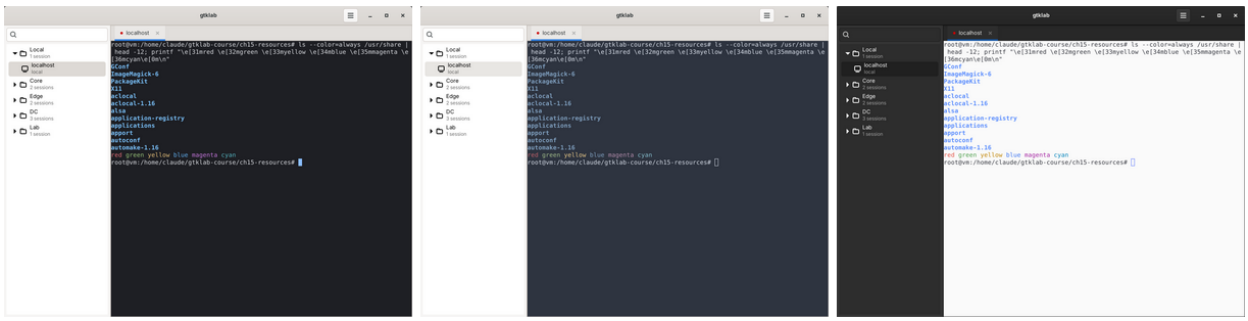


Figure 4: gtklab Dark; Nord; and a dark application with the light terminal theme. The log icon is in each tab label.

Debugger drill

Run **gtklab + GTK Inspector**. On the CSS tab, type `image.log-indicator { color: #00c000; }` and watch the log dot turn green as you type. Then open the Resources tab and find `/io/github/gtklab`, with the four themes, the stylesheet and the icon inside it.

Break it

In `theme_apply.cpp`, misspell the theme directory as `/io/github/gtklab/theme`. `g_resources_enumerate_children` returns `NULL` and sets its `GError`. gtklab puts the message in the sidebar's error banner rather than starting with no themes and no explanation.

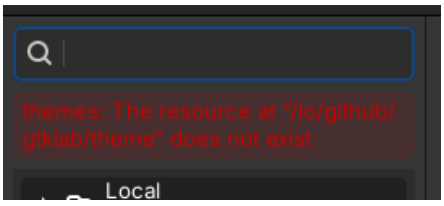


Figure 5: The Break it: the resource error, shown rather than swallowed.

Common mistakes

A project without C. `compile_resources` output is C; list it in `project()`.

Colours in code. They ignore the dark theme and the user's accessibility settings. Add a class; let CSS decide.

Forgetting `-symbolic`. Only icons whose names end in `-symbolic` are recoloured by CSS.

Icons at the wrong path. An icon theme path expects the theme layout underneath it, `scalable/actions/` and the like. An icon in the wrong directory is silently not found.

Read in tmssh

`data/tmssh.gresource.xml`, which bundles thirty YAML terminal themes; `load_themes` in `src/theme_apply.cpp`; `set_app_variant`; and `release_appimage_theme_pin`, which drops an AppImage's bundled `GTK_THEME` so that switching between light and dark works there. tmssh bundles themes only; its icons come from the system.

Review

1. gtklab has no installed data files at all. What does the program contain that makes `/io/github/gtklab/style.css` readable, and when is it registered?
2. You edit `nord.ini` and rebuild. Which build steps run, and why does the core library not need to be recompiled?
3. A rule `label.error { color: blue; }` at theme priority and a rule `label { color: red; }` at application priority both match an error label. Which colour does it get, and why does that argue for narrow selectors in `style.css`?
4. Why is the terminal palette passed through `apply_theme` rather than written as CSS like the rest of gtklab's colours?
5. The log icon's SVG fills its circle with `#222222`. Why does it draw in red, and what would make it draw in green?
6. You rename the icon file to `gtklab-log.svg`, keeping it in the bundle, and change the name in code to match. What changes on screen, and why?
7. A system icon theme happens to ship an icon called `log-symbolic`. What happens if gtklab bundles its own icon under that name?
8. Why can a theme file's bad colour be reported with its key name, without any GTK running?

Questions for deeper thought. A tool you write is used by people with high-contrast themes and large fonts. Which of your colours would you let the theme decide, and which are genuinely data the user owns? If you bundled thirty terminal themes, as tmssh does, would you still read them from the bundle at start-up, or let users add their own from a directory, and what would the error path look like for a bad user file? What is lost when everything is in the binary?

Exercises

1. **A fifth theme.** In `ch15-resources`, add `data/themes/gruvbox.ini` (any twenty valid colours) and list it in `data/gtklab.gresource.xml`. Done when `gresource list build-debug/gtklab` shows it and it appears in the Terminal Theme submenu in its sorted place, with no change to `load_themes`.

2. **A broken theme, reported.** Give `gruvbox.ini` a colour of `#12345`. Done when gtklab starts with the other four themes and the banner names the file and the key, and a new case in `tests/test_theme.cpp` proves `parse_theme` rejects a five-digit colour.
3. **A user stylesheet.** In `AppWindow::install_style`, also load `style.css` from gtklab's configuration directory (`config_path` gives the path) when it exists, at `GTK_STYLE_PROVIDER_PRIORITY_APPLICATION + 1`. Done when `image.log-indicator { color: green; }` in that file turns the log dot green, a missing file is silent, and a syntax error reaches the banner through the provider's `parsing-error` signal.
4. **A warning, not an error.** Theme-file problems are not fatal; show them in the banner with the class `warning` instead of `error`, and add a `label.warning.banner` rule to `style.css` using `@warning_color`. Done when a bad theme file shows an amber banner, a bad sessions file still shows a red one, and no colour literal appears in C++.

Chapter 16 — Lifetime, proven

Since Chapter 8, gtklab has quit with a warning you were told to keep. This chapter explains it, fixes it the way tmssh does, and shows why the usual memory tool could never have found it. Along the way it adds an AddressSanitizer build, which finds the other kind of lifetime bug on the first try.

Recall

Which warning did you keep in Chapter 8, and when does it appear? What does `g_signal_connect_object` do that `g_signal_connect` does not (Chapter 2)? What must `~PtySession` do before anything else (Chapter 11)?

In this chapter

- Tell the two kinds of lifetime bug apart, and pick the tool that finds each.
- Say what happens in `dispose` and in `finalize`, and which one your cleanup should hang on.
- Build gtklab with AddressSanitizer and read its three stacks.
- Turn GTK criticals into debugger stops with `G_DEBUG=fatal-criticals`.
- Audit any object that hands out `this`: every handler, source and reference it needs gone, or kept, before it goes.

Concepts

Two kinds of lifetime bug

Every lifetime bug in a GTK program is code running on an object that should no longer be used. They come in two kinds, and each needs its own tool.

Use after free. The object has been deleted and its memory returned to the allocator; something still holds its address and reads through it. Usually nothing visible happens, because the freed memory still holds what it held a moment ago. Sometimes it crashes, somewhere else, later. **AddressSanitizer** (ASan) makes this kind of bug deterministic.

Use during destruction. The object is being destroyed but has not been freed yet. Its destructor or `dispose` is running, and something it does calls back into code that assumes the object is whole. The memory is still allocated, so ASan has nothing to report. The members it reads may point at objects that *have* been freed, but those belong to GTK and GLib, which are not built with ASan. The evidence is a GTK `CRITICAL` from a failed type check, and the tool is `G_DEBUG=fatal-criticals`.

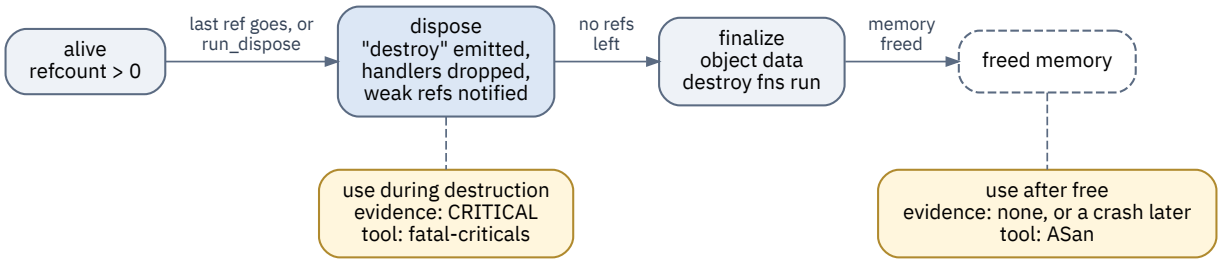


Figure 1: A GObject's life, and where each kind of bug lives.

The two are often found together, which is why the ASan launch configuration sets both.

Dispose, then finalize

A GObject dies in two steps, and the gap between them is where the second kind of bug lives.

	`dispose`	`finalize`
Runs when	the last reference goes, or `g_object_run_dispose` forces it	after dispose, once the last reference has gone
Job	drop references to other objects, break cycles	free the object's own memory
Runs	possibly more than once	exactly once
What fires	a widget's `destroy` signal; then the object's signal handlers are dropped; then weak-reference notifies	destroy functions of `g_object_set_data_full`
The object afterwards	still valid memory, still its type	gone

Measured with a label holding one extra reference: `g_object_run_dispose` printed the `destroy` handler and the weak-reference notify, and the label was still a `GtkLabel` afterwards; only the final `g_object_unref` ran the object-data destroy function. A disposed object can be referenced and even called into. It is just no longer whole.

That table is the whole argument of this chapter's fix. Since Chapter 8 the `AppWindow` was deleted by an object-data destroy function, the last thing that happens to the window. Its handlers on the window's children stayed connected through everything before it.

What a window's teardown promises

Very little. The window is disposed when it is destroyed; its `dispose` releases its child, and the child's subtree goes with it as references drop; at some point in that process the `destroy` signal is emitted. In GTK 4.8's source, `GtkWindow` releases its child before it chains up to `GtkWidget`'s dispose, which is what emits `destroy`. That is an implementation detail, not API, and tmssh's comments say not to rely on either order. What a `destroy` handler can assume is this: it runs when the window is disposed, and some of the window's children may already be gone.

So the `AppWindow` cannot trust the widgets it built, with one exception: a widget it holds its own reference to. A reference keeps the object alive, undisposed, whatever its parent did. gtklab holds the notebook, because the notebook is the one object the destructor must touch: its handlers carry `this`, and its pages own the tabs.

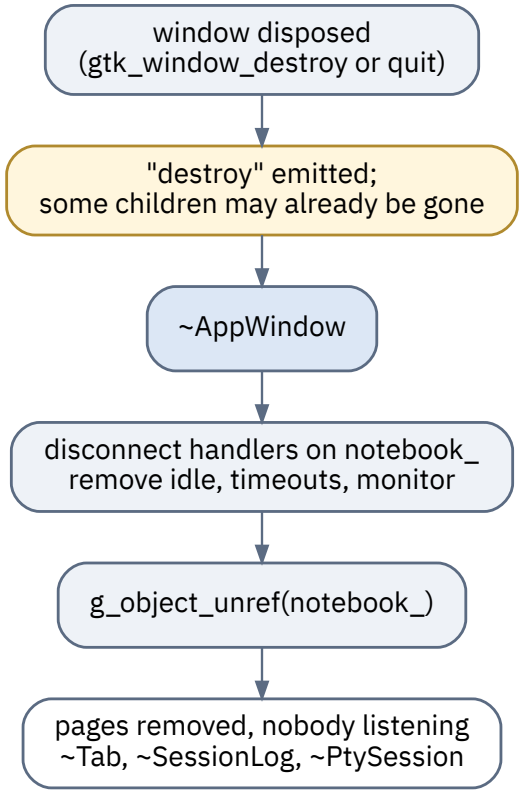


Figure 2: gtklab's teardown, in the order the code guarantees.

What ASan does, in enough detail to trust it

ASan is a compiler pass plus a runtime library. The pass inserts a check before every load and store in the code it compiles. The runtime keeps **shadow memory**: one byte for every eight bytes of the process's memory, saying how many of those eight are addressable. A check is a shift, an add and a compare against the shadow byte.

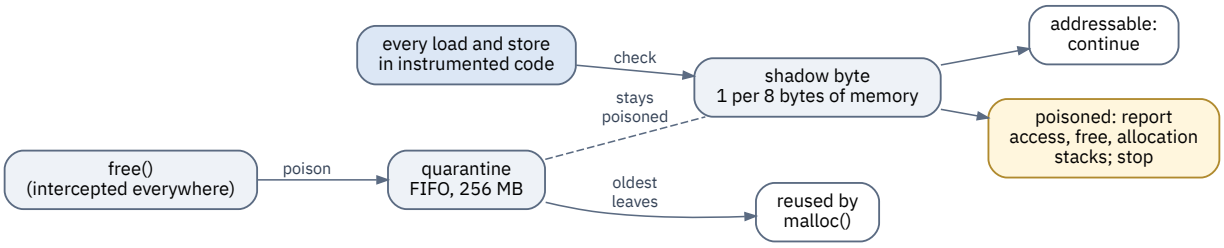


Figure 3: ASan's shadow check and quarantine.

The runtime also replaces `malloc` and `free` for the whole process, including GTK's and GLib's calls. Each allocation gets **redzones**, poisoned bytes on either side, so an overrun lands on poison. A free poisons the whole block and puts it in **quarantine**, a first-in-first-out queue (256 MB by default here) that keeps freed memory from being handed out again for a while. Without quarantine, a freed `Tab` would soon be reused by the next allocation, and a stale read would see valid-looking data.

What ASan cannot see follows from that design. It checks only accesses in instrumented code: yours. A stale pointer passed into GTK and dereferenced there goes unchecked. And it knows nothing about object lifetimes, only memory: a disposed object, or an `AppWindow` halfway through its destructor, is allocated, addressable and clean.

`detect_leaks=0` in the launch configuration turns off LeakSanitizer, which otherwise reports every allocation still alive at exit; GTK and its drivers keep many alive on purpose.

What fatal-criticals does

GTK and GLib check their arguments. A cast macro such as `GTK_STACK(w)` checks the instance's type; a `g_return_if_fail` checks a precondition. A failure logs a `CRITICAL` and returns, and the program carries on with the bug behind it. `G_DEBUG=fatal-criticals` makes criticals fatal: GLib prints the message, then raises `SIGTRAP`. Under a debugger that is a stop, with the stack on screen at the exact frame that made the bad call. Without one the process dies; measured, the shell reported `Trace/breakpoint trap` and exit status 133.

Warnings stay non-fatal under `fatal-criticals`. The Chapter 8 message is a `Gtk-WARNING`, so stopping on it needs `G_DEBUG=fatal-warnings`, which makes warnings and criticals both fatal.

A checklist for any object that hands out `this`

`AppWindow` is not a GObject; GTK cannot see the raw pointers it gives away. Every one must be accounted for:

What holds `this`	Undo it in the destructor	In gtklab
a signal handler on an object that can outlive you	`g_signal_handlers_disconnect_by_data`	`notebook_`, `sessions_monitor_`
a main-loop source	keep its id; `g_source_remove`	`sync_idle_`, `save_source_`, `reload_source_`
an object you must touch while dying	hold a reference; release it last	`notebook_`
whoever deletes you	hang it on the start of teardown, not the end	the window's `destroy`

When the receiver is itself a GObject, Chapter 2's `g_signal_connect_object` does the first row for you. For a plain C++ object, the destructor is the list.

Build

An ASan build

A second build directory, beside `build-debug`:

```
meson setup build-asan --buildtype=debug -Db_sanitize=address
meson compile -C build-asan
```

The `build-asan` task in `tasks.json` does the same, and the **gtklab (ASan)** launch configuration runs the result:

ch16-lifetime/.vscode/launch.json

```
"name": "gtklab (ASan)",
"type": "lldb",
"request": "launch",
"program": "${workspaceFolder}/build-asan/gtklab",
"cwd": "${workspaceFolder}",
"env": {
  "ASAN_OPTIONS": "detect_leaks=0",
  "G_DEBUG": "fatal-criticals"
},
"preLaunchTask": "build-asan"
```

`detect_leaks=0` matters. GTK, fontconfig and the Mesa drivers keep allocations alive until exit by design, and at exit LeakSanitizer reports every one of them, hundreds of lines that bury the one report you want. Leaks are not lifetime bugs; turn their detector off here and look for leaks separately if you need to.

Replay Chapter 11's Break it in this build: remove `g_source_remove(read_watch_)` from `~PtySession`, run `yes` in a tab and close the tab. ASan stops on the first try with `heap-use-after-free`: the bad read is in `on_readable`, the allocation in `start_tab`, and the free in the tab's destruction, all three in your own code. Put the line back.

Who deletes the AppWindow

Since Chapter 8 the window has owned its `AppWindow` through object data, deleted by the destroy function of `g_object_set_data_full`. Object data is cleared when the window is finalized, at the very end of its life, and its children have been torn down before that. The `AppWindow`'s handlers are connected to those children, and they keep running while the children go.

The Chapter 8 warning is exactly that. Tearing down the window drops the stack's pages, "empty" among them, and then the notebook removes its own pages one by one, emitting `page-removed` each time. `on_page_removed`, which holds `this`, asks the half-dismantled stack to show "empty".

The order in which GTK takes a window apart is not part of its API, and tmssh's comments say so: do not depend on it. Its answer, which gtklab now copies, has three parts. First, delete the `AppWindow` on the window's `destroy` signal, emitted when the window is disposed, rather than waiting for finalization, which comes only when the last reference to the window goes:

ch16-lifetime/src/app_window.cpp

```
// Delete this AppWindow on the window's "destroy", emitted when the window
// is disposed. Object data would wait for finalization, which comes only
// when the last reference goes. Either way some children may already be
// gone, so build_ui holds the notebook for ~AppWindow to rely on.
g_signal_connect(window_, "destroy",
                 G_CALLBACK(+[](GtkWidget *, gpointer self) { delete static_cast<AppWindow *>(self); }),
                 this);
```

Second, hold a reference to the notebook, taken in `build_ui` once its signals are connected. Whatever order the window's children go in, the notebook, with its pages still in it, is alive when the destructor runs:

ch16-lifetime/src/app_window.cpp

```
// Our own reference: the notebook must outlive ~AppWindow's disconnect.
g_object_ref(notebook_);
```

Third, the destructor disconnects every handler that carries `this` before letting the notebook go, then removes every main-loop source that carries `this`:

ch16-lifetime/src/app_window.cpp

```
AppWindow::~AppWindow() {
    g_debug("~AppWindow");
    // The notebook is held (see build_ui), so it is still alive here with its
    // pages in it. Handlers holding `this` go first; then let go, and the
    // pages are removed with nobody listening. Either teardown order is safe.
    if (notebook_) {
        g_signal_handlers_disconnect_by_data(notebook_, this);
        g_object_unref(notebook_);
    }
    // Pending sources hold `this`. Remove them, or they fire on freed memory.
    if (sync_idle_)
        g_source_remove(sync_idle_);
    if (save_source_)
        g_source_remove(save_source_);
    if (reload_source_)
        g_source_remove(reload_source_);
    if (sessions_monitor_) {
        g_signal_handlers_disconnect_by_data(sessions_monitor_, this);
        g_file_monitor_cancel(sessions_monitor_);
        g_clear_object(&sessions_monitor_);
    }
    g_clear_object(&tree_model_);
    g_clear_object(&root_);
}
```

`g_signal_handlers_disconnect_by_data` removes every handler on an instance whose user data is the given pointer: one call for all the notebook's handlers. After it, the pages can be removed in any order with nobody listening.

One source was missing from that list until now. `switch-page` schedules an idle to move the log check mark (Chapter 14), and that idle holds `this`. Quit in the moment between a tab switch and the next idle, and it runs on a deleted `AppWindow`. Its id is now kept:

ch16-lifetime/src/app_window.cpp

```
void AppWindow::on_switch_page(GtkNotebook *, GtkWidget *, guint, gpointer data) {
    auto *self = static_cast<AppWindow *>(data);
    // switch-page runs before the notebook's current page changes; sync on
    // idle. The idle holds `this`, so keep its id for ~AppWindow.
    if (self->sync_idle_)
        return;
    self->sync_idle_ = g_idle_add(
```

```
        [](gpointer p) -> gboolean {
            auto *w = static_cast<AppWindow *>(p);
            w->sync_idle_ = 0;
            w->sync_log_action();
            return G_SOURCE_REMOVE;
        },
        self);
}
```

The rule the destructor follows is the one that runs through the book: anything that holds a raw pointer to you, a signal handler or a main-loop source, must be gone before you are.

Done when

Open several tabs, start a log in one, and quit, in the ASan build with `G_DEBUG=fatal-criticals`. There is no ASan report, no `CRITICAL` and no warning, and the log file ends with its footer. This was checked three runs in a row; each run's debug log ended the same way, the two ssh tabs having closed themselves earlier because their hosts did not exist:

```
gtklab-DEBUG: ~AppWindow
~Tab localhost
gtklab-DEBUG: ~SessionLog .../gtklab/logs/localhost-20261012-072632.log
gtklab-DEBUG: ~PtySession
```

`~AppWindow` runs first; the tabs go afterwards, with nobody listening; each tab's log closes with it, then its session. (`~Tab` prints with `g_print`, so it has no `gtklab-DEBUG` prefix; timestamps are trimmed.)

Debugger drill

Set breakpoints in `~AppWindow`, `on_page_removed` and `~Tab`, open two tabs and quit. `~AppWindow` stops first, called from the window's `destroy` emission. Nothing in the destructor touches `stack_`, and the Break it below shows why it must not. Continue: `on_page_removed` is never hit, because its handler was disconnected; `~Tab` is hit once per tab, from `g_object_unref(notebook_)`.

Break it

Keep the notebook reference but delete the `g_signal_handlers_disconnect_by_data(notebook_, this)` line. Quit with a tab open in the ASan build. ASan reports nothing, yet GLib prints:

```
GLib-GObject-CRITICAL **: invalid unclassed pointer in cast to 'GtkStack'
```

Run it under **stop on GTK criticals** and read the stack (trimmed):

```
#2  g_type_check_instance_cast
#3  AppWindow::on_page_removed at ../src/app_window.cpp:482
#4  ??? at libgtk-4.so.1
     ... g_signal_emit ...
#10 g_object_unref at libgobject-2.0.so.0
#11 AppWindow::~AppWindow at ../src/app_window.cpp:46
#12 operator() at ../src/app_window.cpp:24
```

The handler ran *inside* `~AppWindow`, from `g_object_unref(notebook_)`: the notebook's last reference went, it removed its pages, and `page-removed` called into an `AppWindow` that was being destroyed but not yet freed. Its `stack_` pointed at a stack the window had already destroyed. That is the Chapter

8 warning, now a critical because the order changed. ASan was silent because the `AppWindow` itself was never freed memory; the stack was, but it was GTK's. This is why the book keeps both tools.

Common mistakes

Running with leak detection on. The bug you want is somewhere in four hundred lines of GTK's intentional exit-time allocations.

Concluding "ASan is clean, so it is fine". ASan sees only use after free in instrumented code. Use during destruction, and anything inside GLib or GTK, needs fatal criticals.

Relying on teardown order. GTK does not promise which child goes first. Hold what your destructor needs, and disconnect before you let go.

Removing sources but not handlers, or the reverse. Both hold `this`. List them in the destructor, one per member that holds a source id or an instance with your handlers on it.

Read in tmssh

`AppWindow::~AppWindow`; the `destroy` connection in the constructor; and `g_object_ref(notebook_)` further down, with the comment explaining why neither teardown order is safe without it. tmssh's destructor removes its two timers, the reload and the save, the same way.

Review

1. A tab's `PtySession` is deleted but its read watch is still installed, and output arrives. Which kind of lifetime bug is this, what does a normal build usually show, and what does the ASan build show?
2. Why could ASan never have reported the Chapter 8 warning? And in this chapter's Break it, where the stack really was freed, why was it still silent?
3. Since Chapter 8 the `AppWindow` was deleted by an object-data destroy function. Using the difference between `dispose` and `finalize`, explain why that was too late.
4. A `destroy` handler on the window wants to read the text of a label inside the window. Is that safe? What would make it safe?
5. What does the notebook reference taken in `build_ui` guarantee to `~AppWindow`, and why is the reference released only after the handlers are disconnected?
6. Why does ASan keep freed memory in quarantine instead of returning it to the allocator at once?
7. You run gtklab with `G_DEBUG=fatal-criticals` outside a debugger and it dies with `Trace/breakpoint trap`. What happened, and what is the next step?
8. Why did the idle scheduled from `switch-page` need its id stored, when it removes itself after one run?

Questions for deeper thought. `AppWindow` is a plain C++ object that gives `this` to GTK in a dozen places. What would change if it were a GObject subclass with its own `dispose`, and what would not? Your tool runs a background thread that posts results back with `g_idle_add`; what does the checklist in this chapter say about the window closing while a result is in flight? Is a test suite that runs every build under ASan and `fatal-criticals` worth its cost for a tool of a few thousand lines?

Exercises

1. **The audit.** In `ch16-lifetime/src/app_window.cpp`, find every call that gives `this` (or `self`) to GTK or GLib: `g_signal_connect`, `g_idle_add`, `g_timeout_add`, action entries, dialog responses. For each, write down the line in `~AppWindow` that undoes it, or why none is needed. Done when every row has one or the other, and any row that has neither has been fixed.
2. **Teardown in the debug log.** `~Tab` in `src/tab.h` prints with `g_print`, unlike the other destructors. Change it to `g_debug`. Done when, with `G_MESSAGES_DEBUG=gtklab`, quitting with a logging tab prints `~AppWindow`, then for each tab `~Tab` and `~PtySession`, with `~SessionLog` between them for the logging tab, all with the `gtklab-DEBUG` prefix, and nothing at all without the variable.
3. **Catch a use after free on purpose.** In `AppWindow::close_tab`, keep the `Tab *` in a static and schedule an idle that prints `tab->session.name`. Done when the ASan build reports `heap-use-after-free` and you can match each of its three stacks to a function of your own: the idle, `close_tab`'s page removal, and `add_tab`. Then remove it.
4. **The original warning, stopped.** Run the `ch08-tabs` build under the debugger with `G_DEBUG=fatal-warnings`, open a tab and quit. Done when the debugger stops on `Child name 'empty' not found` and you can point to `on_page_removed` in the stack and the GTK frames below it that were tearing the window down.

Chapter 17 — Holding the floor, and shipping

Every chapter has compiled against the GTK 4.8 API, because the version macros from Chapter 0 make anything newer a compile error. That proves less than it seems. This chapter builds gtklab against a real GTK 4.8, adds the install rules a desktop needs, and finishes with the book's last exercise: a second tool of your own.

Recall

Which macros hold the code to the GTK 4.8 API (Chapter 0)? Why does Chapter 10 use `GtkFileChooserNative` rather than `GtkFileDialog`? What did the ASan build need turned off, and why (Chapter 16)?

In this chapter

- Explain what the GTK and GLib version macros check, and what only a build on the floor can.
- Build and run gtklab against a real GTK 4.8, in Debian 12 or from a prefix.
- Choose a build type for what you are about to hand out.
- Install gtklab with a desktop entry whose name, and window, match the application id.

Concepts

The floor is a machine, not a macro

The **floor** is the oldest platform you promise to support; for this book, Debian 12, with GTK 4.8 and GLib 2.74. Since Chapter 0 every build has had two pairs of macros that hold the code to it:

Macro	Effect
`GDK_VERSION_MAX_ALLOWED=GDK_VERSION_4_8`	anything marked newer than 4.8 in GTK's headers warns: `Not available before 4.10`
`GDK_VERSION_MIN_REQUIRED=GDK_VERSION_4_8`	deprecations from after 4.8 stay quiet, so 4.8's API is usable as is
`GLIB_VERSION_MAX_ALLOWED`, `GLIB_VERSION_MIN_REQUIRED`	the same for GLib, at 2.74
`-Werror=deprecated-declarations`	turns those warnings into errors

The mechanism is plain C. GTK's headers annotate each function with the version that added it, `GDK_AVAILABLE_IN_4_10` for example. With `MAX_ALLOWED` below 4.10, that annotation expands

to a deprecation attribute, and the compiler does the rest. `MIN_REQUIRED` works the other way: `GtkFileChooserNative` is marked deprecated in 4.10, but with the minimum at 4.8 that marking expands to nothing, which is why Chapter 10's dialog compiles clean on a newer GTK under `-Werror`.

That is a compile-time check of names against annotations, on your own machine, with your own GTK. It cannot catch:

- a change in **behaviour** between 4.8 and the GTK you develop on, such as a widget that measures differently or a signal emitted in a different order;
- a **library the macros do not cover**: gtkpyte calls Pango and cairo directly, and gtklab sets no version macros for either, so a newer Pango function compiles on your desk and fails on the floor;
- a **bug** in 4.8 that newer versions fixed.

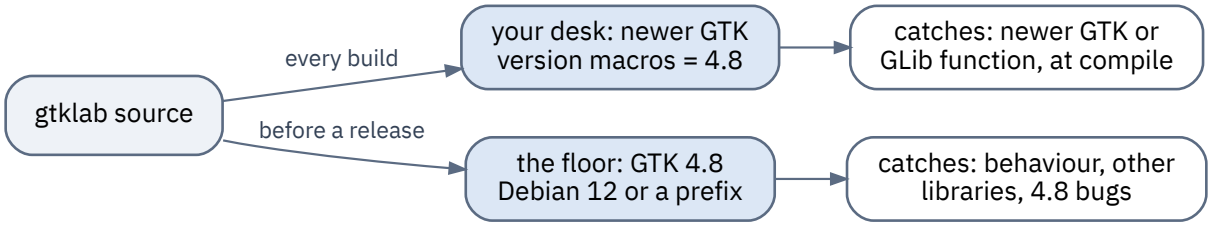


Figure 1: Two checks, two places. The macros run on every build; the floor runs before a release.

Only building and running on the floor itself catches these. Build there before every release, and run the program, not just the tests: behaviour is the one thing tests of a GTK-free core cannot see.

Two ways to stand on the floor

	Debian 12 (machine or container)	GTK 4.8 in a prefix
GTK	4.8, as Debian ships it	4.8.3, built by you
GLib, Pango, cairo	Debian 12's, the real pairing	your machine's, newer
Setup	one `docker run`	a GTK build, a few minutes
Trust	what a Debian 12 user runs	GTK's own code at 4.8, on newer neighbours

The container is the truer test, because the floor is a whole platform, not one library. The prefix is what you can do anywhere, and it is what this chapter ran; the caveat in *Done when* below is exactly the cost of its mixed pairing.

Build types

Meson's `buildtype` picks optimisation and debug information together. Measured on gtklab's own configuration:

`--buildtype`	Compiler flags	gtklab binary
`debug`	`-O0 -g`	2.5 MB
`debugoptimized`	`-O2 -g`	3.9 MB
`release`	`-O3`, no debug info	703 KB

None of them defines `NDEBUG`: Meson's `b_ndebug` is `false` by default for every build type, so `assert` stays active in a release build unless you ask otherwise. The release build of gtklab compiled with no warnings, like the debug one. Hand people `release`; keep `debugoptimized` for a build you expect to debug on someone else's machine, since a stack from it still has names and lines.

The application id, the window and the desktop entry

`gtk_application_new("io.github.gtklab", ...)` names the application in a form the desktop can use. The id is reverse DNS, by convention under a domain you control; `io.github.<user>` is the usual choice for a project hosted on GitHub. Several things are derived from it:

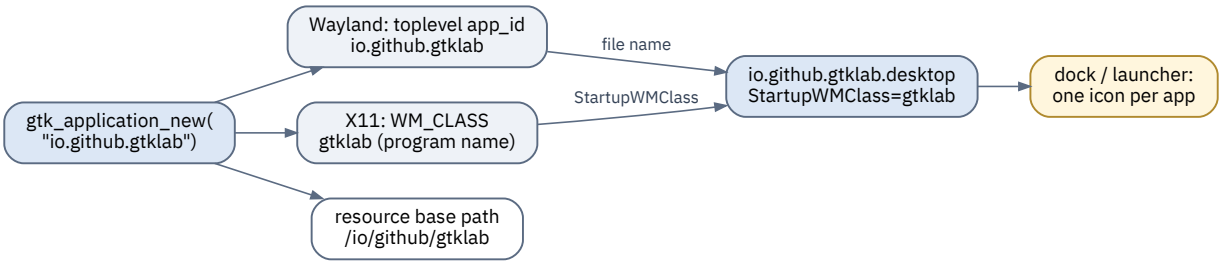


Figure 2: What the application id reaches, and how a launcher ties a window to its entry.

- **The window's identity.** On Wayland, GTK gives the window the application id as its app id (when the program is on the session bus, as every desktop session provides). On X11 the window's `WM_CLASS` is the program name instead: measured, gtklab's window matched the class `gtklab` and not `io.github.gtklab`.
- **The resource base path**, `/io/github/gtklab`, which Chapter 15 used.
- **Single-instance negotiation**, on D-Bus. gtklab passes `G_APPLICATION_NON_UNIQUE`, which turns it off: every launch is its own process with its own window, as a terminal usually is.

A **desktop entry** is how a launcher, a menu and a dock know about the program: its name, the command to run (`Exec`), an icon, categories. Its file name should be the application id plus `.desktop`, because that is what a Wayland desktop matches the running window's app id against. `StartupWMClass=gtklab` covers X11, by naming the class the window really has. Get either wrong and the dock shows the launcher's icon and a second, generic icon for the running window.

`desktop-file-validate` checks the entry's syntax and keys; it passes for gtklab's. It cannot check that the name matches the id: that is on you.

Build

On Debian 12

GTK 4.8 is Debian 12's system GTK, so a Debian 12 machine or container builds gtklab with no special steps. From a checkout, with Docker installed, one command does it:

```
docker run --rm -v "$PWD":/src -w /src debian:12 bash -c "apt-get update && apt-get install -y build-essenti
```

(This line was not run for the book: the machine it was written on had no Docker daemon. The next route is what was run.)

GTK 4.8 in a prefix

Without Debian 12, build GTK 4.8.3 into a directory of its own. The options turn off everything gtklab does not use, which keeps the build to a few minutes and its dependencies to the ones a GTK 4 development machine already has:

```
git clone --depth 1 --branch 4.8.3 https://github.com/GNOME/gtk.git gtk-4.8.3
meson setup gtk-4.8.3/build gtk-4.8.3 --prefix=$HOME/floor/gtk48 --buildtype=release -Dx11-backend=true -
ninja -C gtk-4.8.3/build install
```

Then build gtklab against it. The script points `pkg-config` and the dynamic linker at the prefix, builds into `build-floor`, and runs the tests:

ch17-shipping/scripts/floor-build.sh

```
#!/bin/bash
# Build and test gtklab against the oldest GTK it supports.
#
# usage: scripts/floor-build.sh <prefix>
#   <prefix> is where a GTK 4.8 build is installed (e.g. ~/floor/gtk48).
# On a Debian 12 machine or container, GTK 4.8 is the system GTK: run
# `meson setup build-floor` directly instead.
set -euo pipefail
PREFIX="$1"
PCDIR=$(find "$PREFIX" -name gtk4.pc -printf '%h\n' | head -1)
export PKG_CONFIG_PATH="$PCDIR"
export LD_LIBRARY_PATH="$(dirname "$PCDIR")"
pkg-config --modversion gtk4
rm -rf build-floor
meson setup build-floor --buildtype=debug
meson compile -C build-floor
meson test -C build-floor
echo "Built against GTK $(pkg-config --modversion gtk4). Run: LD_LIBRARY_PATH=$LD_LIBRARY_PATH
↪  build-floor/gtklab"
```

```
scripts/floor-build.sh $HOME/floor/gtk48
```

Install rules

`meson install` needs to know what to install. The executable gets `install: true`, and a desktop entry goes into `share/applications`:

ch17-shipping/meson.build

```
executable('gtklab', app_sources, link_with: pty_lib,
  dependencies: [gtk, core_dep, util, gtkpyte_dep],
  install: true)

# The .desktop file's name matches the application id, so desktops match
# the running window to its launcher.
install_data('data/io.github.gtklab.desktop',
  install_dir: get_option('datadir') / 'applications')
```

ch17-shipping/data/io.github.gtklab.desktop

```
[Desktop Entry]
Type=Application
Name=gtklab
Comment=Tiny session-manager terminal built while learning GTK 4
Exec=gtklab
Icon=utilities-terminal
Terminal=false
Categories=System;TerminalEmulator;GTK;
StartupWMClass=gtklab
```

The file's name is the application id from `main.cpp`, `io.github.gtklab`. Desktops use that match, and `StartupWMClass` on X11, to tie a running window to its launcher, so the dock shows one icon rather than two. Check the entry with `desktop-file-validate`, then:

```
meson setup build-release --buildtype=release --prefix=$HOME/.local
meson install -C build-release
```

Done when

On GTK 4.8.3, gtklab builds with no warnings, the core tests pass, and the program runs: tabs, colours, the context menu, logging, zoom, and a 5 MB `cat` that keeps repainting.

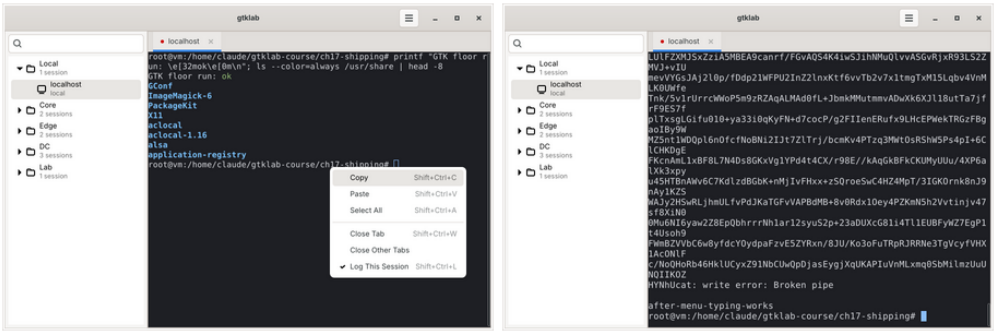


Figure 3: gtklab running against GTK 4.8.3, built from the prefix.

One caveat, measured on the machine the book was built on. GTK 4.8.3 built against that machine's GLib 2.80 logs `Attempting to freeze the notification queue for object GtkImage` criticals from

inside GTK whenever an image is finalized, the first from `gtk_message_dialog_new`. That is GTK 4.8 and GLib 2.80 disagreeing, not gtklab; Debian 12 pairs GTK 4.8 with GLib 2.74 and does not log them. On a mixed build like this one, run floor checks without `fatal-criticals`, and read any critical that does appear for a frame of your own code.

Debugger drill

Debug the floor build in VS Code: copy the **gtklab** launch configuration, point `program` at `build-floor/gtklab`, and add `LD_LIBRARY_PATH` to its `env`, with the value the script printed. Set a breakpoint in `start_tab` and open a tab. In the Debug Console, evaluate `gtk_get_minor_version()`: it returns 8. A wrong `LD_LIBRARY_PATH` builds against 4.8 and runs against your system GTK, and this is the quickest way to know which one you are testing.

Break it

Add a call to `gtk_file_dialog_new()` anywhere in `app_window.cpp`. With the version macros in place, the normal build fails:

```
error: 'GtkFileDialog* gtk_file_dialog_new()' is deprecated: Not available before 4.10
```

Now remove `-DGDK_VERSION_MAX_ALLOWED=GDK_VERSION_4_8` from `gtk_floor_args` and build again. The normal build succeeds; the floor build fails, because the function does not exist in GTK 4.8:

```
error: 'gtk_file_dialog_new' was not declared in this scope; did you mean 'gtk_file_filter_new'?
```

The macros move that failure from the floor machine to your desk. Put both back.

Common mistakes

Testing only on your own machine. It runs the newest GTK you have, which is the one your users are least likely to have.

Trusting the macros alone. They cover GTK and GLib functions, nothing else, and say nothing about behaviour.

A desktop file named after the program, not the application id. The dock shows a second, generic icon for the running window.

Shipping a debug build. `--buildtype=release` for anything you hand to someone.

Read in tmssh

`scripts/build.sh`, which configures and builds each flavour; and `scripts/appimage.sh`, which bundles tmssh with its libraries into one file that runs on most distributions. Then `release_appimage_theme_pin` in `src/theme_apply.cpp`, called from `main`: the GTK plugin of the AppImage tool exports `GTK_THEME=Adwaita:<variant>` at start-up, which locks the window to one variant and breaks the light and dark switch from Chapter 7. tmssh unsets it and pins Adwaita itself.

Review

1. With the version macros at 4.8, a call to `gtk_file_dialog_new` fails to compile on a GTK 4.14 machine, yet the function exists there. What turns its presence in the header into an error?
2. Why does `GtkFileChooserNative`, deprecated in GTK 4.10, compile without a warning under `-Werror=deprecated-declarations`?
3. gtklab builds cleanly with the macros in place. Name a change you could make that would still compile on your desk and fail, or misbehave, on Debian 12.
4. The prefix build logged criticals that Debian 12 does not. Why, and what does that say about which floor to trust?
5. `floor-build.sh` succeeds, but the program you run afterwards reports GTK 4.14 from `gtk_get_minor_version()`. What went wrong?
6. Why is the desktop file named `io.github.gtklab.desktop` and not `gtklab.desktop`, and why does it also need `StartupWMClass=gtklab`?
7. Is `assert` active in gtklab's release build? Why?
8. When would you hand someone a `debugoptimized` build rather than a `release` one?

Questions for deeper thought. Where would you put your own floor for a tool your team runs on a mix of laptops and jump hosts, and how would you find out what those hosts actually have? Which of your tool's behaviours would you check by hand on the floor before a release, given that the core tests cannot see any of them? tmssh ships an AppImage that carries its libraries with it; what does that change about the floor, and what does it not?

Exercises

1. **An icon of its own.** In `ch17-shipping`, add `data/icons/hicolor/scalable/apps/io.github.gtklab.svg`, install it to `share/icons/hicolor/scalable/apps` with `install_data` in `meson.build`, and set `Icon=io.github.gtklab` in the desktop file. Done when `desktop-file-validate` passes and, after `meson install -C build-release`, the launcher shows that icon.
2. **Validate in the tests.** In `meson.build`, find `desktop-file-validate` with `find_program(..., required: false)` and, when it is found, add it as a test on the desktop file. Done when `meson test` lists and passes it, and misspelling a key makes it fail.
3. **One version number.** Pass `meson.project_version()` to the compiler as `GTKLAB_VERSION`, and add a `win.about` action to `AppWindow::install_actions` with a menu entry in `build_menu` that calls `gtk_show_about_dialog`. Done when About shows `0.1.0`, and changing `version:` in `project()` changes it after a rebuild.
4. **The floor under ASan.** Give `scripts/floor-build.sh` an optional second argument, `asan`, that configures `build-floor` with `-Db_sanitize=address`. Done when the floor build runs gtklab through opening tabs, logging and quitting with no ASan report.

Capstone — A second tool

gtklab is done. The last exercise is to prove the book's ideas are yours and not gtklab's, by building something that is not a terminal, from an empty repository, with no reference solution.

The tool. A log viewer, `logpeek`:

- It opens a file and **follows** it as it grows, like `tail -f`, using a `GFileMonitor` and the read-budget pattern from Chapter 11, at idle priority.
- It shows every line in a `GtkListView` and stays smooth at a **million lines**, which only factory reuse (Chapter 5) makes possible.
- A search entry **filters** the lines, debounced (Chapter 7) so that typing does not re-filter on every key.
- The header bar holds a **sparkline** of lines per second over the last minute, a widget of your own with `measure` and `snapshot` (Chapter 12).
- Every command (open, follow on and off, clear the filter, jump to the end) is an **action** with a shortcut and a menu entry (Chapter 7).
- Line parsing and filtering live in a **core library with tests** (Chapter 4).

Done when.

1. It runs clean in an ASan build with `G_DEBUG=fatal-criticals` through opening a file, following it while another program appends to it, filtering, and quitting with follow still on.
2. It builds and runs on the floor (this chapter).
3. You can explain each of the four strands from *How to use this book*, lifetime, the main loop, model and widget, and input propagation, using your own code, without opening this book.

Hints, if you need them. The list's model can be a `GListStore` of line objects; filtering it is a `GtkFilterListModel` with a `GtkCustomFilter`, and `gtk_filter_changed` re-runs it. Measure your follow loop with `seq 1 1000000 >> file` while the window is open: if the sparkline stops moving, check the priority of your watch first.

Appendices

Appendix A — GTK 3 to GTK 4

Most GTK material online, and most answers to most questions, are about GTK 3. Translate with this table before copying anything. The full guide is *Migrating from GTK 3.x to GTK 4* in GTK's documentation.

GTK 3	GTK 4	Chapter
Widgets hidden until `gtk_widget_show_all`	Widgets visible by default; `gtk_widget_set_visible`	1, 3
`gtk_container_add`, `GtkContainer`	Per-container calls: `gtk_window_set_child`, `gtk_box_append`, `gtk_paned_set_start_child`	3
`gtk_widget_destroy` on any widget	Remove it from its parent; `gtk_window_destroy` for windows	2, 3
`GtkTreeView` with `GtkListStore`	`GListModel` with `GtkListView` or `GtkColumnView` and a factory	5
`GtkTreeView` with `GtkTreeStore`	`GtkTreeListModel` with `GtkTreeExpander`	6
`GtkMenu`, `GtkMenuItem`, `GtkUIManager`	`GMenu` model, `GAction`, `GtkPopoverMenu`	7, 10
`gtk_widget_add_accelerator`, `GtkAccelGroup`	`GtkShortcutController`, or accelerators on actions	7
`button-press-event`, `key-press-event` signals	Event controllers and gestures, with propagation phases	9
`gtk_menu_popup_at_pointer`	A `GtkPopoverMenu` parented to a widget, positioned with `set_pointing_to`	10
`gtk_dialog_run`	The `response` signal; dialogs are asynchronous	10
`GtkFileChooserDialog`	`GtkFileChooserNative` at the 4.8 floor; `GtkFileDialog` from 4.10	10
`draw` signal with cairo	`snapshot` virtual function with `GtkSnapshot`	12
`get_preferred_width` / `_height`	`measure`, called once per orientation	12
`size-allocate` signal, `gtk_widget_get_allocation`	`size_allocate` virtual function; `gtk_widget_get_width` / `_height`	12
`GtkStyleContext` for adding classes	`gtk_widget_add_css_class` / `remove_css_class`	15

GTK 3	GTK 4	Chapter
`gtk_widget_translate_coordinates`	`gtk_widget_compute_point` (both exist in 4.8; the first is deprecated from 4.12)	10

Appendix B — Debugger recipes

Each recipe uses one of the launch configurations from Chapter 0. The chapter column is where the symptom first appears in the book.

Symptom	Recipe	Chapter
A `CRITICAL` scrolls past in the terminal	Run **stop on GTK criticals** (`G_DEBUG=fatal-criticals`); go up the stack to the first frame of your own code	0, 2
A widget is the wrong size, or collapsed to nothing	**gtklab + GTK Inspector**: select it with the picker, check `hexpand`, `vexpand` and its allocation; check its parent's shrink settings	3, 5
`GtkGizmo ... reported min height -3`	A homogeneous `GtkStack` measuring a hidden notebook with no tabs; make the stack non-homogeneous	3
A handler never runs	Inspector, Controllers tab: check the controller's phase, and whether something earlier claims the event or returns `TRUE`	9
A menu item is grey	The action name's prefix (`win.` or `app.`) and its parameter type must match exactly	7
A menu item does nothing, and nothing is logged	The popover was unparented inside its `closed` handler; unparent on idle	10
Typed keys go nowhere	The widget that was clicked, or the closing popover, took focus back; move focus with `focus_later`	11, 13
The window stops repainting under heavy output	The descriptor watch's priority first, then the read budget	11, 14
Nothing appears until the window is resized	A model change without `gtk_widget_queue_draw`	12
A crash that comes and goes	The ASan build, **gtklab (ASan)**; read the free and allocation stacks	16

Symptom	Recipe	Chapter
A `CRITICAL` at quit, and ASan is silent	A handler running inside a destructor; read the fatal-criticals stack for your destructor, then disconnect by data before letting go	16
You want to step into GTK itself	The **gtklab (gdb)** configuration, with `DEBUGINFOD_URLS` exported before VS Code starts	0
A bug that only your users see	Build against the floor; check `gtk_get_minor_version()` in the Debug Console	17

Appendix C — Recall answer key

Chapter 5. The core library owns the session data; the window only displays it. A new widget starts with a floating reference, which the first container it is added to sinks and keeps, so the container owns it.

Chapter 6. `setup` creates the row's widgets, once per row widget; `bind` only fills them, and runs every time a row widget is reused. Because `GListModel` holds GObjects and nothing else. `src/core/session.h` and `session.cpp`, and they do not include GTK.

Chapter 7. For every visible folder at startup, so that the expander knows whether to draw a triangle, and again for several folders on each expand and collapse; so it must be cheap. Setup, then bind, then unbind, with bind and unbind repeating as the row widget is reused. Which folders were open: `capture_expanded` records their names before the rebuild, and the rebuild expands them again.

Chapter 8. That its destroy function runs when the object is finalized, or when the data is replaced. To coalesce a burst of changes into one write. That the action belongs to the window, not the application.

Chapter 9. `page-removed`, while the page still exists, then `~Tab` as it is destroyed. The capture phase, so that the shortcut runs before the focused widget sees the key.

Chapter 10. It takes the whole sequence: no other gesture sees it. The event carries on to the next controller, so the scrolled window scrolls. The page, through object data.

Chapter 11. `closed` is emitted before the menu activates the chosen item; unparenting there destroys the menu, and the action with it. A timeout from `g_timeout_add`. In Chapter 8, only a debug line; from Chapter 11 it destroys the session, which closes the pty.

Chapter 12. Fills a row widget, which may be reused, with an item's data. `G_DECLARE_FINAL_TYPE` and `G_DEFINE_FINAL_TYPE`. The core.

Chapter 13. The capture phase, so that the page gets first refusal before the widget inside it. At the next frame after a draw is queued, from the frame clock. At idle priority, below GTK's redraw, so that a busy pty cannot stop the window repainting.

Chapter 14. Capture, target, bubble. The read budget, and the idle priority of the descriptor watch. `resized`.

Chapter 15. It asks for a frame; it does not draw, and it does not lay the widget out again. Code that needs no display: parsing, the session and settings models, now the theme parser; not anything that needs a widget. `gtkpyte_terminal_set_colors`, with `set_color_cursor` and `set_color_selection`.

Chapter 16. `Child name 'empty' not found in GtkStack`, when quitting with tabs open. It disconnects the handler automatically when the object passed with it goes away. Remove every main-loop source that holds `this`, the descriptor watch and the child watch, before releasing anything else.

Chapter 17. `GDK_VERSION_MIN_REQUIRED` and `GDK_VERSION_MAX_ALLOWED`, with the GLib pair. `GtkFileDialog` arrived in 4.10, above the 4.8 floor. Leak detection, because GTK and its libraries keep allocations until exit by design, and the reports bury the one you want.

Appendix D — GObject in one page

The pieces of GObject this book uses, in the order it uses them.

You want	Write	Chapter
A reference you own	`g_object_ref`; give it back with `g_object_unref`	2
To keep an object that may still be floating	`g_object_ref_sink`	13
A C++ object owned by a GObject	`g_object_set_data_full(obj, key, ptr, destroy)`	2, 8
A handler that cannot outlive a second object	`g_signal_connect_object`	2
Every handler with your pointer, gone	`g_signal_handlers_disconnect_by_data(instance, this)`	16
A new final type	`G_DECLARE_FINAL_TYPE` in the header, `G_DEFINE_FINAL_TYPE` in the source	5, 12
A type that implements an interface	`G_DEFINE_FINAL_TYPE_WITH_CODE(..., G_IMPLEMENT_INTERFACE(...))`	13
C++ members in a GObject	Placement `new` in `init`, explicit destructor call in `finalize`	5, 12
Drop references to others	`dispose`; may run more than once; chain up last	12
Free everything else	`finalize`; runs once; chain up last	12
A property	`g_param_spec_*` in `class_init`, `g_object_class_install_properties`	12
A signal	`g_signal_new` in `class_init`	12
An interface's properties	`g_object_class_override_property`	13
Release and clear a member	`g_clear_object(&member)`	5

Appendix E — Source map

Each chapter's code is a complete program in its own directory. The tmssh column is where the production version of that chapter's idea lives.

Chapter	Directory	Start reading tmssh at
1	`ch01-window`	`main` and `on_activate` in `src/main.cpp`
2	`ch02-gobject`	`add_tab` in `src/app_window.cpp`
3	`ch03-layout`	`build_ui`, `build_sidebar`
4	`ch04-core`	`tmssh_core` in `meson.build`; `tests/`
5	`ch05-listview`	`on_factory_setup`, `on_factory_bind`, `on_factory_unbind`
6	`ch06-tree`	`create_child_model`, `rebuild_tree`, `capture_expanded`
7	`ch07-actions`	`install_actions`, `build_menu`, `save_settings_soon`
8	`ch08-tabs`	`add_tab` and its `g_object_set_data_full` deleter
9	`ch09-input`	the page controllers in `add_tab`; gtkpyte's `on_report_begin`
10	`ch10-popovers`	`show_terminal_menu`, `confirm`, `act_import_file`
11	`ch11-mainloop`	`PtySession::on_readable` in `gtkpyte/src/ptysession.cpp`
12	`ch12-widget`	`gtkpyte/src/gtkpyte-terminal.cpp`
13	`ch13-input`	gtkpyte's `on_key_pressed`, `on_click_pressed`, `on_scroll`
14	`ch14-gtkpyte`	`start_tab`, `act_log`, `sync_log_action`
15	`ch15-resources`	`data/tmssh.gresource.xml`, `src/theme_apply.cpp`
16	`ch16-lifetime`	`AppWindow::~AppWindow`
17	`ch17-shipping`	`scripts/build.sh`, `scripts/appimage.sh`

Appendix F — Review answers

Short answers to each chapter's Review questions. Check your own answer against the chapter first; these say what a good answer must contain.

Chapter 0

1. pkg-config failed: Meson finds GTK by asking pkg-config, which reads the `.pc` file that the development package installs. Install `libgtk-4-dev` (or `gtk4` with Homebrew); `meson.build` is not the problem.
2. The compiler gets its include paths and `-D` flags from the command Ninja runs; the editor has no such command unless it reads `compile_commands.json`. With it, IntelliSense sees the same GTK headers and the same version-floor macros as the compiler.
3. `GDK_VERSION_MAX_ALLOWED=GDK_VERSION_4_8`, made fatal by `-Werror=deprecated-declarations`. It reports any function newer than 4.8 as "not available", so the tool cannot quietly depend on API that GTK 4.8 machines lack.
4. `MIN_REQUIRED` stops the headers flagging functions that were deprecated after 4.8, because on 4.8 they are the normal API. Without it, it defaults to the installed version (4.14), which is above `MAX_ALLOWED`, so the headers stop the build at once with `#error "GDK_VERSION_MAX_ALLOWED must be >= GDK_VERSION_MIN_REQUIRED"`. With a matching `MAX_ALLOWED` removed too, a `gtk_widget_show` call (deprecated in 4.10) would become an error under `-Werror=deprecated-declarations`.
5. No. The packaged GTK ships without debug information, so its internal functions have no names; exported ones such as `g_object_unref` still do. Use the **gtklab (gdb)** configuration with `DEBUGINFOD_URLS` set when you need to step into GTK; for bugs in your own frames it is not needed.
6. **gtklab: stop on GTK criticals** (`G_DEBUG=fatal-criticals`). It turns the warning into an abort, so the debugger stops with the offending call, and your line, on the stack instead of letting the program carry on in a bad state.
7. Each build directory holds one configuration (build type, sanitizer, which GTK). Keeping them separate means switching between them costs nothing and none of them is ever rebuilt from scratch because another was configured differently.

Chapter 1

1. `main` runs before the main loop and before GTK has opened the display, so nothing that draws can be built there. `activate` is emitted from inside `g_application_run` once the application is ready to show windows.
2. The main loop inside `g_application_run`, kept running because the window is registered with the application. When the last registered window closes, the application quits and `g_application_run` returns.
3. With `G_APPLICATION_NON_UNIQUE` the second launch is an independent process with its own `startup` and `activate` and its own window. Without it, the second launch finds the first process through its application id on D-Bus, asks it to activate, and exits; the first process opens a second window.
4. The default flags (`G_APPLICATION_DEFAULT_FLAGS`). `on_activate` then runs more than once in the same process, so it must create a new window each time and must not assume it is the first or only window.

5. `close-request` asks a question whose answer changes what GTK does next; `clicked` only notifies. Returning `FALSE` lets the close go ahead: GTK destroys the window, and if it was the last one the application quits.
6. A frozen window that does not redraw or respond. The handler runs inside the main loop, and the loop cannot process input or draw a frame until the handler returns.
7. No. `g_application_run` returns only after the main loop ends, and the loop ends when the last window has closed.

Chapter 2

1. A new widget's reference is floating, and `gtk_box_append` sinks it, so the box owns the only reference. An extra `g_object_unref` would destroy the button while it is still in the box, and the crash would come later, somewhere else.
2. It needs the box to call `gtk_box_remove`, and after the call the box's reference, the only one, is gone: the button is destroyed inside the call, so `count` points at freed memory.
3. GTK keeps every window in its list of toplevel windows; a new window is not floating. Closing the window (or `gtk_window_destroy`) releases it.
4. The click emits `clicked` on Remove, which runs `on_remove_clicked`; it calls `gtk_box_remove`, the box drops the last reference, the count reaches zero, GObject disposes and then finalizes the button, finalize clears the button's attached data, and the destroy function (the lambda) deletes the `Counter`.
5. The handler finds its state on the emitting object, and GObject keeps the emitter alive for the whole emission, so there is no pointer that can outlive what it points at.
6. The `user_data` object, the counter button: the handler is disconnected when it is destroyed. It does nothing for any other pointer the handler might reach, such as a C++ object it looks up elsewhere.
7. GObject's checked cast (`GTK_WIDGET(user_data)`) found that the memory no longer held a valid instance. ASan could not, because the freed memory belonged to GLib's allocations in libraries not built with ASan.
8. Take a reference with `g_object_ref` before removing it, so removal drops the count to one instead of zero. After appending it to the new box (which takes its own reference), call `g_object_unref` once.

Chapter 3

1. A container's size depends on its children's sizes, so measurement has to start at the leaves; a child's rectangle depends on how its parent divided the space, so allocation has to start at the top.
2. Its slot in the box gets a share of the spare height, because it expands; `START` then gives the label only its natural height, placed at the top of that slot. With `FILL` it would be stretched over the slot.
3. Expansion propagates upward. The scrolled window inside the box has `vexpand` set, so the box (and the paned above it) computes that it wants to expand vertically too, because neither has `vexpand` set explicitly.
4. Its natural height is not its content's; it reports a small natural size because its content is meant to scroll. Give it `vexpand` so it receives the box's spare height.
5. The stack keeps both pages alive and only changes which one is visible, so nothing is destroyed and rebuilt, and the notebook keeps its state; with gtklab's settings the stack also measures only the visible page.
6. A homogeneous stack is as large as its largest page, so it is allocated the hidden page's minimum width; the visible page is laid out in that wider space and the pane shows only part of it, so it appears off centre.
7. `resize` says whether the child grows when the paned grows; `shrink` says whether the divider may squeeze it below its minimum size. Both default to on, so a sidebar would widen with the window and could be dragged out of existence.

8. A size request only raises the widget's minimum. With `shrink` on, the paned is allowed to give the child less than its minimum, so the request never comes into play.

Chapter 4

1. Formatting `user@host:port` is core: it needs only the session's fields and can be tested as a string. Setting the tooltip on a widget is the GTK layer: it needs the widget.
2. The core target in `meson.build` is not given the GTK dependency, so the compiler cannot find `gtk/gtk.h` and the build fails. A build rule is checked on every compile; a comment is checked only when someone remembers it.
3. At the display. GLib is a portable C runtime (key files, file I/O, path and UTF-8 helpers) that needs no window; GTK needs a display, and code that needs one cannot be tested without one.
4. The tests would depend on the shell of whoever runs them, so a test that passes on one machine could fail on another. Passing the shell in makes the input visible and fixed.
5. The core's results end up in GTK callbacks, which are called from C. An exception escaping a callback unwinds through C frames not built for it; an error value can be checked and shown by the GTK layer.
6. The label would become the source of truth. The count would depend on the label's exact wording, could not be tested without a window, and would break as soon as the text changed; the model exists so the label is only output.
7. A core test answers whether the behaviour is right (does port 2222 give `-p 2222`?) quickly and on any machine. Only a check through the window answers whether GTK is wired to that behaviour (does the click start the right session?).
8. So the core stays plain C++ values that are copied, compared and tested with no GObject machinery. The wrapper is an adapter at the list level: it holds a copy of a core value so a GTK list model can display it.

Chapter 5

1. `bind`, then `unbind`. The name and the address come from the same item in the same `bind`, so a mismatch means a row kept text from a previous item: a label `bind` did not set, or one `unbind` did not clear. The model holds whole sessions and cannot mix two of them.
2. `setup` creates widgets; `bind` only fills them. Rows are reused, so building in `bind` adds a new set of widgets every time a row is re-bound: slow, and a leak of the widgets it replaces.
3. Anything tied to the old item: a signal handler connected to the item, a timer started for it, a reference to it stored on the row. Left in place, it keeps acting on, or keeping alive, a session the row no longer shows.
4. `GListModel` is a list of GObjects, so it can reference-count them and check their type. gtklab wraps `Session` so the core stays plain C++ with no GTK or GObject, testable on its own (Chapter 4); the wrapper exists only at the GTK boundary.
5. The selection takes ownership of the window's only reference. When the window's destructor unrefs `store_` as well, the store is released one time too many when the window closes: a use after free.
6. The store took its own reference in `g_list_store_append`. `unref` gives back the caller's reference only; the item is finalized when its last reference, the store's, is gone.
7. `session` would be an unconstructed `Session` in zero-filled memory; the first assignment to it is undefined behaviour. The compiler cannot see it because GObject allocates the struct with C calls and never runs a C++ constructor.
8. For a short list whose rows differ from each other or carry their own controls, such as a settings page or a few dozen entries. A list that grows with the user's data should be a `GtkListView`.

Chapter 6

1. In the data. Each folder item carries its sessions (gtklab's `GtklabSessionItem` holds a `Folder`); the tree list model only asks for a child model when it needs one, and inserts rows only while the folder is expanded.
2. A `GtkTreeListRow`. `gtk_tree_list_row_get_item` returns your `GtklabSessionItem` inside it, as a new reference that must be released.
3. The expander has to know whether to draw a triangle, so binding a row asks `gtk_tree_list_row_is_expandable`, which calls the child function. It therefore runs for every row shown, at startup and again on expand, and must be quick and safe to call repeatedly.
4. The expander would keep the old row: a recycled widget could show the previous row's indent and triangle, and the expander would hold that row until the next bind.
5. A filter over the flattened rows only sees rows that exist, so sessions in closed folders cannot match and non-matching folders hide their children. Filtering in the core can express exactly the rule wanted and is already tested without GTK; rebuilding the root from its result is simple.
6. The splice replaces every folder item with a new one, and new rows start collapsed (`autoexpand` is `FALSE`). Positions change whenever a folder above opens or a filter drops folders, so only a stable key such as the folder's name identifies the same folder before and after.
7. Two folders to re-open, the first with sessions in it. Expanding the first inserts its sessions after it, which pushes the second folder past the count read before the loop, so the loop ends before reaching it.
8. The folders that were open when the search began. `user_expanded_` is captured on the first keystroke and restored on clear; what happened to the filtered tree in between is thrown away by the rebuild.

Chapter 7

1. The parameter says what to do this time and arrives with each activation; the state is what is true now and stays on the action. `win.toggle-sidebar` has only a boolean state; `win.app-variant` has a string parameter and a string state.
2. The prefix (`win.` or `app.`) and the spelling of the name against the action map; then whether the action was added to the map the control can reach (the window, not some other object); then whether it is disabled. None of these prints a warning.
3. The first press asks for `false`: the sidebar hides, but the state stays `true`, so the check mark stays on. The second press asks for `false` again, and nothing changes: the sidebar cannot be shown from the action any more.
4. The state is parsed as GVariant text, and a string in that format needs its own quotes. Without them, `light` is not a valid GVariant value.
5. Find the window it should act on, since its user data could not be one `AppWindow`; with several windows it would have to decide whether to close one or all of them.
6. `gtk_application_set_accels_for_action` makes the menu show the key, and it also runs the action from the window's capture phase; the window's capture-phase `GtkShortcutController` runs it too. Whichever handles the key first stops it, so the action runs once.
7. A pending timeout holds `this`; if the window is destroyed first, it would fire on freed memory. Returning `G_SOURCE_REMOVE` destroys the source, so the id is stale; zeroing it stops a later `g_source_remove` on a source that no longer exists, which is a critical warning.
8. The second window's `activate` adds a new `refresh` to the application, replacing the first one, so Refresh from either window updates only the second window's label. Window state belongs in a `win.` action.

Chapter 8

1. Reloading replaces the window's `folders_` vector and rebuilds the tree's items, so a pointer into either would point at freed memory. The tab's own copy stays valid and keeps the host and user it was opened with.

2. Nothing. The notebook is the only list of tabs, and each page carries its own `Tab`, so the new order is simply the notebook's order. A registry beside the notebook would now be out of order.
3. Because a stored pointer goes stale when that tab is closed or the pages are reordered, and every path would have to remember to update it. Asking the notebook for its current page when the command runs is always right.
4. Not the new page: in the measured runs it returned the old page's number, or -1 while the current page was being removed. Use the `page` argument the signal passes.
5. The close button, Close Tab and Close Other Tabs (Chapter 10), the process exiting (Chapter 11), and the window closing with tabs open. Every one of them ends with the page leaving the notebook and being finalized, so `~Tab` is the one place every path reaches.
6. `page-removed` runs at `remove_page`, and the page is left with no parent and its `Tab` still attached. `~Tab` runs only when the extra reference is dropped and the page is finalized.
7. By the time the page is finalized and its data cleared, its child view has already been finalized; the measured order was view finalized, page destroy, `~Tab`. `tab->view` is a dangling pointer in the destructor.
8. If the page went first by some other path while the button could still be clicked, the handler would be called with a pointer to a dead page. `g_signal_connect_object` disconnects the handler when the page goes.

Chapter 9

1. First the page's capture-phase click gesture, which denies button 1 (`page capture: button 1 denied`); then, with the press still unclaimed, the text view's own gestures place the cursor and the bubble-phase logging gesture on the view prints `text view (bubble): button 1`.
2. The capture phase runs from the window down before the target's controllers, so the page claims the sequence before the text view's menu gesture gets it. In the bubble phase the text view, as the target, sees the press first and handles it, so the page's gesture never gets the chance.
3. The plain wheel would no longer scroll the text view: the scroll would be reported as handled and swallowed before reaching the scrolled window.
4. The focused widget in the tree is the target, and propagation only visits the target and its ancestors. The notebook is not an ancestor of anything in the sidebar, so its controllers are never on the path; a shortcut that must work everywhere goes on the window, in the capture phase.
5. No. A target-phase controller runs only when its own widget is the target; a click on the text view targets the text view. The test program's box never ran its target-phase gesture for a click on its child.
6. The controllers belong to the page and the page owns the `Tab`; they are torn down together, so no handler can run with a deleted `Tab`.
7. `gtk_event_controller_get_current_event_state` gives the modifiers recorded in the event being handled. A separate tracker is extra state that can get out of step with the event actually being handled, for no gain.

Chapter 10

1. Close Tab removes that page, and a popover dies with its parent, so the menu would be destroyed by the very action it was dispatching. Parenting it to the notebook, which outlives every tab, means not depending on how GTK copes with that.
2. The popover emits `closed` before it activates the chosen item's action. Unparenting in `closed` destroys the menu first, so the chosen item (Copy in the Break it) silently does nothing.
3. The gesture is attached to the row's widget, so `x, y` are in the row's coordinates. Without conversion the menu is placed as if the row's origin were the list view's, near the top of the list instead of at the pointer.
4. Actions on tabs already resolve "the current tab", and making the clicked tab current is what a user expects. A row is not current in any sense the actions know, and rows are recycled, so the row menu names its session explicitly as the action's target.

5. A modal dialog stops input to its parent window. The main loop keeps running: timers, redraws and, from Chapter 11, output to other tabs all continue while the question is up.
6. The tab may be closed another way while the dialog is up; a `Tab *` could then point at freed memory, while the reference keeps the page (and so its `Tab`) alive. When the answer comes it checks that the page still has a parent, that is, it is still in a notebook, before closing it.
7. `GTK_RESPONSE_DELETE_EVENT` (-4) in the recorded run. `confirm` acts only on `GTK_RESPONSE_ACCEPT` and treats everything else as no, so Escape, Cancel and closing the dialog need no cases.
8. Each layer runs in its own callback from the main loop: the right-click, the later click on the menu item that runs the action, and the dialog's response even later. That is why each log line carries the target name.

Chapter 11

1. The timer and the descriptor callbacks run, because both are at priority 0 and that is the highest ready priority. The idle, at 200, waits for an iteration in which nothing more urgent is ready.
2. The budget makes the callback return, but at the default priority the pty is ready again on the next iteration and still outranks the redraw at 120, so it is dispatched again and the frame never runs. Only a priority below the redraw lets the frame in.
3. An idle is always ready, so the loop's poll timeout is zero and the loop calls it again immediately, forever. Descriptor watches are ready only after `poll()` reports data, so the thread sleeps in `poll()` until then.
4. From the other side of the pty: the line discipline echoes input, or the shell's line editor does, and the echo comes back through the master like any other output.
5. The kernel's line discipline: with `ISIG` set, the interrupt character (byte 3) becomes `SIGINT` for the foreground process group of that terminal, here `sleep`. gtklab is not in that session and receives nothing.
6. Without the idle, removing the page deletes the `Tab` and its `PtySession` while the session's exit callback is still running, a use-after-free. Without the reference, the page could be destroyed (say, by the user closing the tab) before the idle runs, and the idle would use a dangling pointer.
7. The source is already destroyed; `~PtySession` would call `g_source_remove` with a stale id, which GLib reports as a critical warning.
8. One save arrives as several events (write, rename, permission change). Reloading on the first would read the file before the editor finished writing it; restarting the timer means the reload happens once, 300 ms after the last event, when the file has settled.

Chapter 12

1. `update_grid` divides the pixels by the cell size: 637 / 9 gives 70 whole columns, and the grid resizes its cell model to that and emits `resized`. The 7 pixels left over are a margin at the right that `snapshot` fills with the background colour.
2. The cell size would change but no layout pass would run, so `size_allocate` would not be called, `update_grid` would keep the old column and row counts, and the pty would not learn the new size; the grid would draw new-size cells into an old-size model.
3. GTK keeps each widget's last recording and asks for a new one only after `queue_draw` (or a new size). Nothing queued a draw, so the old recording was still valid as far as GTK knew; the resize asked for a new one.
4. Cells are what the pty needs (`TIOCSWINSZ` is in rows and columns), and pixels are the widget's private business. Allocations happen for many reasons that do not change the shape, and only a shape change is news to the child.
5. A widget whose size in one direction depends on the other, such as a wrapping label that gets taller as it gets narrower. A grid's minimum and natural sizes are fixed by its cell, whatever the other dimension is.

6. The cell is the size of one character as it will be drawn. If the measurement used different attributes (a different font, ligatures on), the cell would not match the glyphs `paint_row` draws and text would drift off the grid.
7. They do not reach it during `on_readable`: feeding only changes the cell model and queues a draw. The text appears on the next frame, which runs after `on_readable` returns and the main loop lets the frame clock in.
8. GObject zero-fills the instance and runs no constructors, so the member would be zeroed bytes rather than a valid string; the first assignment to it would be undefined behaviour.

Chapter 13

1. `lower` is 0, `upper` is 5,027 (every line held), `page_size` is 27 (the rows on screen), and `value` is 5,000, the first line of the screen.
2. The widget's side: the grid must put every change in its line count into the adjustment. The scrolled window only knows what the adjustment tells it, so it went on describing one screen of lines and had nothing to scroll to.
3. Output arrives on its own, and a user who scrolled back to read something should not be pulled away from it. A key press is the user acting at the prompt, so showing the prompt is what they want.
4. Every terminal line is one cell high, so in lines the first line drawn is simply `value` and a selection can be stored as absolute line numbers. A text view's lines differ in height, so only pixels describe its content exactly.
5. The grid sends `\r`; `cat` receives `\n`, because the pty's line discipline turns carriage return into newline on input (the `ICRNL` flag). bash's line editor turns that flag off while it waits at the prompt, so it reads the `\r` and treats it as Enter.
6. The window's shortcut controller runs in the capture phase, before the grid's key controller, and handles the key, so it never reaches the grid. Plain Ctrl combinations belong to the program in the terminal (Ctrl+C interrupts, Ctrl+V is a literal-next in the shell), so a terminal that took them would break the programs it runs.
7. Without the reference, closing the tab would destroy the grid and the callback would then run on freed memory. With it, the grid stays alive until the callback has run and dropped the reference.
8. `on_paste_ready` replaces each newline with `\r`, the byte Return sends, so each line reaches the shell as if Enter were pressed. With bracketed paste, the terminal wraps the paste in `ESC [200~` and `ESC [201~` when the program has asked for it, and the program inserts the text instead of running it.

Chapter 14

1. The session never touches a widget: it hands output to a handler the tab sets, and the tab connects the widget's `commit` and `resized` to its `write` and `resize`. gtkpyte has a feed function and the same two signals, so only the wiring in `start_tab` changes.
2. Nothing special: pyte's parser keeps its state (here, collecting CSI parameters) between calls, so the next `feed` continues from that state. GtklabGrid had to buffer the incomplete sequence and scan it again, which works only for the sequence shapes it recognises.
3. pyte builds a cursor position report, `ESC [ row ; col R`, and gtkpyte emits it through `commit`, which the tab writes to the pty like a keystroke. With nothing connected, the reply is lost and the program waits for an answer that never comes.
4. `vim` runs on the alternate screen, which keeps no scrollback. With `set mouse=a`, `vim` has asked for mouse reports, so each wheel notch is sent to it as a button 64 or 65 press. Without it, alternate scroll (mode 1007, on by default) turns the wheel into Up and Down arrow keys.
5. By holding Shift. The capture-phase report gesture's `on_report_begin` denies the sequence when Shift is held, so the press continues to the selection gestures.

6. gtkpyte's source directory, on the include path of anything that uses `gtkpyte_dep`, contains its own `pty.h`, which would shadow the system `<pty.h>` and hide `forkpty`. Built separately without `gtkpyte_dep`, the pty code sees the system header.
7. It cost nothing measurable: the 67 MB `cat` took about the same time (16.5 s against 17.9 s). It bought a terminal that repaints every second during the flood instead of once in six seconds, because the redraw at priority 120 now outranks the pty watch.
8. The raw bytes the child sent, escape sequences included. The screen shows the result of interpreting them (colours, cursor movement, a full-screen program redrawing in place), which is not the same as the stream.

Chapter 15

1. The bundle itself: `glib-compile-resources` turned the manifest's files into a byte array in the binary's `.gresource.gtklab` section, plus a constructor. The constructor runs before `main` and registers the bundle, so any code can then read `/io/github/gtklab/...` by path.
2. Meson regenerates `gtklab-resources.c` from the manifest, compiles that one file and relinks gtklab. The core is untouched because it contains no theme data; it only parses whatever bytes it is given at run time.
3. Red. Provider priority is decided before specificity, so the application-priority rule wins over the more specific theme rule; a broad selector in `style.css` would therefore override the theme on every matching widget.
4. The terminal colours are content the user chooses, independent of the application's light or dark variant, not chrome the theme should decide. They are data, parsed and tested in the core, and handed to gtkpyte's own colour API, which queues a redraw.
5. Its name ends in `-symbolic`, so GTK forces every `rect`, `circle` and `path` to the widget's CSS `color`, which `style.css` sets to `@error_color` on `image.log-indicator`. Setting `color` to a green in that rule (or in the Inspector) makes it green.
6. It is no longer treated as symbolic, because only names ending in `-symbolic` are recoloured, so it draws as written: a dark grey dot, in light and dark alike.
7. The system theme is searched before hicolor, where resource paths live, so the system's icon is shown and gtklab's is never used. That is why gtklab prefixes its names with `gtklab-`.
8. Parsing is in the core: `parse_theme` takes a string and reads the key file with GLib and checks each colour against `#rrggbb` itself, so it can name the failing key in its error and be tested by doctest with no display.

Chapter 16

1. Use after free: the watch's callback reads a `PtySession` whose memory has been returned. A normal build usually shows nothing, or crashes later somewhere else; the ASan build stops at the first read with `heap-use-after-free` and the access, free and allocation stacks, all in gtklab's code.
2. In Chapter 8 nothing that was read had been freed: the `AppWindow` lived until the window's finalization, and the stack was being dismantled but was still allocated, which is use during destruction. In the Break it, the freed stack was dereferenced only inside GLib's type check, which is not instrumented, and the `AppWindow` itself was valid memory mid-destructor; GLib's check produced the critical instead.
3. Object-data destroy functions run in `finalize`, the very last step. By then `dispose` had already released the window's children, and their teardown emitted `page-removed` into an `AppWindow` that was still connected and still believed the widgets were whole.
4. Not safe: `destroy` is emitted when the window is disposed, and GTK does not promise that its children are still there. It becomes safe if the code holding the handler took its own reference to the label beforehand and releases it afterwards.

5. That the notebook is alive, undisposed and still holding its pages when the destructor runs, whatever order GTK tore the rest down in. Disconnecting first means that when the last reference goes and the notebook removes its pages, no handler holding `this` hears it.
6. If freed memory were reused at once, the next allocation would refill it with valid-looking data and a stale read would see nothing wrong. Quarantine keeps it poisoned for a while, so the read hits poison and is reported.
7. A GTK or GLib `CRITICAL` was logged, and `fatal-criticals` made GLib raise `SIGTRAP` after printing it; with no debugger attached, that killed the process. Run the same configuration under the debugger (**stop on GTK criticals**) and read the stack at the stop.
8. The idle holds `this` until it runs. If the window is destroyed between the tab switch and the next idle, the callback would run on a deleted `AppWindow`; with the id stored, the destructor can remove it.

Chapter 17

1. GTK's header marks it `GDK_AVAILABLE_IN_4_10`; with `GDK_VERSION_MAX_ALLOWED` at 4.8 that expands to a deprecation attribute ("Not available before 4.10"), and `-Werror=deprecated-declarations` makes the warning an error.
2. `GDK_VERSION_MIN_REQUIRED` is 4.8, so deprecations made after 4.8 expand to nothing. The API is valid on the floor, and the macros keep it quiet on newer GTK.
3. For example, a call to a Pango or cairo function newer than Debian 12's, made from gtkpyte, which no version macro guards; or code that relies on how a newer GTK measures or orders its signals. Both compile on your desk.
4. The prefix pairs GTK 4.8.3 with the build machine's GLib 2.80, a combination Debian never ships, and the two disagree inside GTK. Debian 12 pairs GTK 4.8 with GLib 2.74, so it is the truer floor; the prefix is a convenient approximation.
5. The program ran against the system GTK: `LD_LIBRARY_PATH` was set only inside the script, so the run needs the value the script printed, otherwise the dynamic linker picks the system library.
6. A Wayland desktop matches the window's app id, which is the application id, against desktop-file names. On X11 the window's `WM_CLASS` is the program name, `gtklab`, so `StartupWMClass` tells the desktop which class belongs to this entry. Without the match the dock shows a second, generic icon.
7. Yes. Meson's `b_ndebug` defaults to `false` for every build type, so `NDEBUG` is not defined in a release build unless you set `b_ndebug`.
8. When you expect to debug a problem on that person's machine: `debugoptimized` is optimised but keeps debug information, so a stack from it still has function names and lines.